Neovim user guide

Leader key: Space. Keys, commands and workflows of this config.

A hands-on manual for one specific Neovim setup (Catppuccin Mocha theme, plugins managed by lazy.nvim). It explains how to work in this editor without touching the mouse: what to press, what happens, and why.

Who it is for. People new to Neovim or to this config, and the author when a key has been forgotten. No Vim knowledge is assumed: modes and the verb + noun idea are explained first (section 1). Every key listed here is taken from the real config, so it works as written.

What it covers.

Area	Sections
Basics: modes, cheat sheet, saving, recovering, getting help	1, 2, 69 to 70, 72 to 74
Moving around	3, 22, 23, 37, 50
Editing: selection, text objects, registers, macros, the dot command	4 to 6, 16, 17, 24, 25, 29, 60 to 62, 64, 65, 68, 71, 76
Search, replace, fuzzy finding, file tree, quickfix, files	10 to 12, 26, 30, 51, 57, 63, 67
Windows, buffers, terminal, sessions, statusline and UI	7, 8, 32, 33, 58
Code: LSP, completion, snippets, folding, treesitter, running, debugging, docs, workflows	13 to 15, 18, 19, 21, 35, 36, 43 to 47, 52 to 56, 59, 75
Git	20, 48
Claude Code, Markdown, LaTeX/Typst, spelling, URLs	9, 27, 28, 31, 38, 49
Everything else: custom commands, configuration, automatic behaviors, shell commands, Neovide, other plugins	34, 39 to 42, 66, 77
Languages: Java, Python, LaTeX, Markdown, Typst, C++, Vim, Nix	78 to 82, 84 to 86
Catalog of every plugin, each linked to its section	83

How it is organized. Every topic is a numbered section, and the text "see section N" always means that number. Section 2 is a one-page cheat sheet of the daily keys, with pointers to the full sections. Sections with "in depth" in the title go further than the short section on the same topic. Start with section 1, then section 2; use the table of contents (or the plugin catalog, section 83) to find the rest.

Conventions.

Notation	Meaning
`<Space>ff`	Press the leader key (Space), then `f`, then `f`
`<Ctrl-o>`, `<Alt-j>`, `<Esc>`, `<CR>`, `<BS>`	A key with a modifier, or a named key (`<CR>` is Enter, `<BS>` is Backspace); all key names: section 2, "Key names"
`:Z word`	A command: type it after `:` in Normal mode and press Enter
`n`, `x`, `o`, `i`, `s`, `t`, `cmd` in a Mode column	Normal, Visual (also `v`), Operator-pending, Insert, Select, Terminal, Command-line; several letters (e.g. `n, x`) mean the key works in each of those modes
N/A in a table cell	Not applicable: nothing is produced or to enter (cells are never left empty)
see section N	Go to that section number (not the PDF's "Chapter" number, which counts files)

Reading and rebuilding. In Neovim press `<Space>?` (or use the dashboard item) to open the PDF at the contents page. The `.md` files are the source: after editing run `user-guide/build-pdf.py`, then `user-guide/build-pdf.py --check` (details under Contents). This edition: 86 numbered sections in 20 chapters, 127 plugins in the catalog.

Contents

Chapter 1: Day-to-day cheat sheet (section 2) 17
Key names (how keys are written) 18
Reading the guide PDF (zathura keys) 20
Navigation 20
Copy, delete and move 22
Indented blocks 22
Text objects (what to delete, change or copy) 23
Macros 23
Bulk rename and replace 23
Undo, redo and repeat 24
Visual block (many lines at once) 24
Comments and surrounding pairs (vim-commentary, vim-sandwich) 24
Search in a file 24
Buffers, splits, tabs 25
Save and quit 25
Sessions, dashboard and zoxide 26
Code intelligence (LSP) 26
Completion and snippets 26
Git day to day 27
Mouse 28
Language guides (tools that exist only for one language) 28
Language devShells (`:DevEnv`) 28
Neovide (Neovim in its own window) 28
Lists with a search bar (pickers) 29
Quickfix list (search results, errors) 29
Spell checking 29
Run code and terminal 30
Python debugger keys (pdb through nvim-gdb) 30
Java debugger keys (nvim-dap through nvim-java) 30
Keyboards without function keys, Insert or Page keys 31
Folding (nvim-ufo) 31
Text tricks 31
Claude and fuzzy finders 32
Other most-used keys (from the former quick reference) 32
Chapter 2: Basics: modes, saving, recovering, getting help 33
1. Understanding modes 33
Entering insert mode 33
Leaving insert mode 33
Visual mode variants 34
69. Tips for Vim beginners 34
The most important habits 34
Common mistakes and how to fix them 35
Keyboard layouts with dead keys (for example US International) 35
Learning path 36
70. The verb + noun system (how Vim commands work) 36

Operators (verbs) 37
Motions (nouns) 37
Text objects (structured nouns) 38
Combining verbs and nouns 38
Using counts 39
Why this matters 40
72. Saving, quitting, and file state 40
Saving 40
Quitting 40
Closing buffers (without quitting Neovim) 41
73. Recovering from mistakes 41
Undo and redo 41
Undo tree (builtin) 41
Time-based undo 42
If you accidentally deleted a file 42
74. Discovering keymaps and getting help 42
See available keybindings with which-key.nvim 42
Browse all keymaps 43
Getting help 43
Checking system health 43
Chapter 3: Navigation 44
3. Core navigation (moving without the mouse) 44
Basic cursor movement 44
Moving within a line 44
Moving by word 45
Moving by line/screen 46
Jumping to matching brackets/parentheses 46
Jumping to specific characters 47
Jump navigation with hop.nvim (plugin) 47
Jump history 47
Word references (vim-illuminate) 47
Marks (bookmarks) 48
22. Jump navigation (`hop.nvim`) 48
23. Search lens (`nvim-hlslens`) 49
37. Symbol outline (`aerial.nvim`) 49
50. Code navigation strategies 49
Finding files 50
Finding code 50
Understanding code 50
Refactoring code 51
Chapter 4: Editing, text objects, macros and tricks 52
4. Editing 52
Entering insert mode for editing 52
Deleting text 52
Copying (yanking) text 53
Pasting text 53
Changing (delete + enter insert) 54

Replacing text . . . 54
Undo / redo . . . 55
Repeating actions . . . 55
Line operations . . . 55
Split and join code (treesj) . . . 56
Indentation . . . 57
Insert mode shortcuts . . . 57
Miscellaneous editing . . . 58
5. Selection (visual mode) . . . 58
Selecting characters . . . 58
Selecting lines . . . 59
Selecting blocks (columns) . . . 59
Selecting inside/around delimiters (text objects) . . . 59
Parentheses, brackets, braces . . . 60
Quotes . . . 60
Words, lines, paragraphs . . . 60
Tags (HTML/XML) . . . 61
Example for the text objects above . . . 61
Using with operators (d, c, y) . . . 61
Select a whole block, from its first line to the closing brace (method, function, if, class) . . . 62
Treesitter node selection (builtin) . . . 63
More text objects . . . 64
Markdown code block text objects . . . 64
Precision selection: from the cursor to an exact spot . . . 65
Quick lookup . . . 65
End of line: `L` . . . 65
`t` and `T`: stop just before a character . . . 66
Counting: `v2tX` is not `2vtX` . . . 66
`)` and `(` are sentence motions, not parentheses . . . 66
Search as a selection motion (crosses lines) . . . 67
hop.nvim: select to something you can see (no counting) . . . 68
Target on another line, count unknown: use the relative numbers . . . 69
Typing before or after: `i` `a` `I` `A` . . . 69
Non-contiguous lines (for example line 3 and line 10 together) . . . 70
Line range yanking (command mode) . . . 71
6. Working with parentheses, quotes, and brackets . . . 71
Jumping to matching pair . . . 71
Selecting inside/around pairs . . . 71
Changing text inside pairs . . . 72
Deleting text inside pairs . . . 72
Adding surrounding pairs (vim-sandwich plugin) . . . 72
Removing surrounding pairs (vim-sandwich plugin) . . . 73
Replacing surrounding pairs (vim-sandwich plugin) . . . 73
Wrap a whole list or part of it . . . 74
Auto-pairing (nvim-autopairs plugin) . . . 74
Tabbing out of pairs (custom, `lua/tabout.lua`) . . . 74

16. Code commenting 76
vim-commentary (plugin) 76
Smart commenting (custom) 77
17. Surrounding pairs (`vim-sandwich` + `nvim-autopairs`) 78
24. Yank history (`yanky.nvim`) 78
25. Undo history 79
29. Registers & macros 79
Registers 79
Macros 79
60. Macros in depth 79
Recording a macro 80
Playing a macro 80
Scenario: add semicolons to the end of 20 lines 80
Scenario: wrap each line in double quotes 80
Scenario: append the same text to a block of lines 80
Scenario: turn `// name` lines into `names.add("name");` calls 81
Scenario: convert a list of variables to assignments 81
Scenario: turn CSV into SQL VALUES 82
Tips for writing macros 82
Visual mode macros 82
61. The dot command (`.`) – repeating actions 83
What counts as a "change" 83
Scenario: change a variable name one-by-one 83
Scenario: add a prefix to multiple lines 83
Scenario: delete the first word on several lines 84
Scenario: indent multiple blocks 84
Combining `.` with counts 84
62. Visual block editing (multi-cursor-like) 84
Scenario: add a prefix to multiple lines at once 84
Scenario: append text to multiple lines 84
Scenario: delete a column 85
Scenario: replace a column 85
64. Everyday editing scenarios 85
Swap two lines 85
Swap two words (vim-swap) 85
Duplicate a line 86
Duplicate a block of code 86
Fix indentation of entire file 86
Remove all blank lines 86
Sort lines 86
Convert tabs to spaces (or vice versa) 87
Wrap a selection in a tag/function 87
65. Swapping function arguments (`vim-swap`) 87
68. Useful Vim tricks 88
Run a normal-mode command on every line 88
Execute a command on a range 88
Increment/decrement numbers 88

Letter sequences (a, b, c...) instead of numbers 90
Open the file under cursor 91
Change case 91
Align text with tabular 91
Command abbreviations 92
71. The global command (`:g`) 92
Common uses 93
Everyday scenarios 93
Delete all print/debug statements 93
Keep only lines matching a pattern 93
Extract all function signatures 94
Add prefix/suffix to matching lines 94
Move lines matching a pattern to the top 94
76. Common editing power combos 94
Changing text 94
Deleting text 95
Copying text 95
Selecting text 95
Quick transformations 96
The most powerful patterns 97
Chapter 5: Completion and snippets 98
14. Autocompletion (`nvim-cmp`) 98
15. Snippets (`UltiSnips`) 98
Java snippets 99
Other snippets 102
45. Autocompletion in depth (`nvim-cmp`) 104
How it works 104
The smart tab behavior 104
Completion keymaps 105
Visual indicators 105
Completion sources and helpers 105
52. Snippets for developers (`UltiSnips`) 106
What snippets are 106
Ready-made snippets (`vim-snippets`) 107
How to use snippets 107
The snippet gallery 107
Custom snippets 108
Creating your own snippets 109
Chapter 6: Searching, replacing, files and the file tree 111
10. Searching, replacing, and refactoring text 111
Searching in the current file 111
Search modifiers 111
Search examples 112
Substitution (find & replace in current file) 112
Understanding the range (where to replace) 112
Understanding the flags (how to replace) 113
Flag combinations 113

Confirmation mode (`c` flag) controls ... 115
Changing the delimiter ... 115
Special replacement patterns ... 115
Substitution examples ... 116
Searching in the current buffer with `*` and `#` ... 116
Using `ciw` + `.` for selective single-file replacement ... 117
67. Multi-file search and replace (complete guide) ... 117
Quick decision guide: which method to use ... 117
Method 1: LSP rename (best for code symbols) ... 117
LSP rename versus `:grep` + `:cfdo`: can the fast text rename replace a slow `<Space>rn`? ... 118
Method 2: `:grep` + `:cfdo` (best for plain text) ... 119
Step-by-step: replace all without confirmation ... 120
Step-by-step: replace with confirmation for each occurrence ... 120
Step-by-step: review results before replacing ... 120
Using regex with `:grep` ... 120
Limiting to specific file types ... 121
Method 3: `:vimgrep` + `:cfdo` (built-in, slower but portable) ... 121
Method 4: `<Space>fg` for finding (no replace) ... 121
Complete examples ... 121
Example 1: rename an API endpoint across the project ... 121
Example 2: replace a deprecated function name (with confirmation) ... 122
Example 3: delete all `console.log` statements in JavaScript files ... 122
Example 4: add a comment before every TODO ... 122
Example 5: replace only in Python files in the src/ directory ... 122
Example 6: case-insensitive project-wide replace ... 122
Understanding `:cdo` vs `:cfdo` vs `:bufdo` ... 122
Undoing a multi-file replace ... 124
12. Fuzzy finding & project-wide search (`fzf-lua`) ... 124
Moving inside any picker (lists with a search bar) ... 124
Keymaps ... 125
Inside the fzf-lua popup ... 126
`<Space>fg` – Live grep (project-wide text search) in depth ... 126
What it does ... 126
Plain text search ... 126
Regex search ... 126
Use cases for live grep ... 127
`<Space>ff` – Find files (file name search) ... 127
The difference between search methods ... 128
26. Quickfix & location list ... 128
Commands ... 128
Inside the quickfix window (nvim-bqf, quicker.nvim) ... 128
51. Quickfix workflows for developers ... 129
What populates the quickfix list ... 129
Navigating the quickfix list ... 130
Batch operations on quickfix items ... 130
trouble.nvim: better quickfix UI ... 130

11. File explorer (`nvim-tree`) 130
File operations and language servers (nvim-lsp-file-operations) 132
30. Working with directories 133
Path modifiers (for use in commands) 133
57. File management for developers 133
File operations 133
Project structure navigation 134
63. Working with multiple files 134
Opening several files 134
Comparing two files side by side 134
Copying between files 135
Running the same edit across multiple files 135
Closing files you're done with 136
Chapter 7: Windows, buffers, terminal, sessions and the interface ... 137
7. Windows, splits, and buffers 137
Key concepts 137
Creating splits 137
Put two chosen buffers side by side or one above the other 137
Navigating between windows 139
Resizing windows 139
Moving windows around 140
Buffer tabs (the top line, bufferline.nvim) 140
Changing the order of the tabs 141
Closing windows 141
Buffer management 142
Reading the `:ls` flags 142
Tabs 143
Closing floating windows 143
8. Terminal integration 144
Opening a terminal 144
Navigating in and out of terminal 144
Closing a terminal 145
58. Session and productivity 145
Auto-save (auto-save.nvim) 145
Session management (persistence.nvim, vim-obsession) 145
Collaborative editing (instant.nvim) 145
32. Statusline (`lualine.nvim`) 146
Long-line warning 146
33. UI features 148
Centred command line (tiny-cmdline.nvim) 148
Breadcrumb bar (dropbar.nvim) 148
LSP progress messages (fidget.nvim) 149
Colorschemes 149
Which theme is active at startup 149
Switching and listing themes 149
The 19 themes of the random list 151
Line number column (`statuscol.nvim`) 152

Icons and UI libraries 152
Input popups and big files (`snacks.nvim`) 153
Dashboard (start screen, dashboard-nvim) 153
Help keys: the user guide and Claude (any buffer, and the dashboard) . 154
Chapter 8: Code: LSP, running, debugging, folding, treesitter 156
13. LSP: language server protocol 156
Configured language servers 156
LSP keymaps 157
Built-in Neovim LSP and diagnostic keys 158
LSP commands 158
glance.nvim: peek without jumping 158
Diagnostics (errors, warnings) 159
44. Language server protocol (LSP) in depth 159
What LSP is 159
How LSP is managed (nvim-lspconfig) 160
Configured servers and what they provide 161
How language servers are registered (lua/config/lsp.lua) 163
Shared capabilities 163
The server table 163
When is a server enabled 165
R: the one-time probe 166
Lua: lua_ls and stylua 167
typos_lsp: where it attaches 168
LSP keymaps (all languages) 168
Formatting (conform.nvim) 169
Format on save 170
Keys and commands 171
Peeking without jumping (glance.nvim) 171
Diagnostics in depth 171
The lightbulb (nvim-lightbulb) 172
Editing the Neovim config in Lua (lazydev.nvim) 172
18. Code folding (`nvim-ufo`) 173
47. Code folding in depth (`nvim-ufo`) 174
What it is 174
How it works 174
Folding keymaps 174
21. Treesitter & text objects 175
nvim-treesitter (plugin) 175
targets.vim (plugin) 175
vim-matchup (plugin) 176
46. Treesitter in depth (`nvim-treesitter`) 176
What treesitter is 176
Installed parsers 176
19. Code running 176
Filetype-specific 177
55. Code running in depth 177
The universal runner 177

Language-specific details 178
Filetype-specific runners 178
36. Debugging 179
56. Debugging in depth 179
Debug adapter protocol (DAP, nvim-dap) 179
GDB integration (nvim-gdb) 179
43. How the development toolchain fits together 180
Enter a language devShell from a running Neovim (`:DevEnv`) 181
53. Documentation lookup 184
devdocs.nvim (plugin) 184
devdocs.nvim: a set is missing from `:DevDocs get` 184
Hover documentation (LSP) 185
59. Useful developer commands 186
Part III: everyday scenarios & recipes 186
75. Real-world developer workflows 186
Workflow: investigating a bug 186
Workflow: code review (reviewing your own changes) 187
Workflow: refactoring a function name across the project 187
Workflow: adding a feature in a new branch 187
Workflow: quickly editing a config file 187
Workflow: working with JSON 187
Workflow: writing documentation (Markdown) 188
Workflow: pair programming with split views 188
35. Java development (`nvim-java`) 188
Build & run 189
Testing 189
Refactoring 189
54. Java development in depth 189
How it works 190
Build & run 190
Testing 190
Debugging 190
Refactoring 191
Chapter 9: Git 192
20. Git integration 192
vim-fugitive (plugin) 192
gitsigns.nvim (plugin) 194
gitlinker.nvim (plugin) 194
Neogit (plugin) 195
vim-flog (plugin) 195
diffs.nvim (plugin) 196
codediff.nvim (plugin) 196
Other Git tools 197
Resolving merge conflicts (diffview.nvim) 197
48. Git workflow in depth 198
The Git plugin ecosystem 198
Daily Git workflow 198

Understanding gitsigns.nvim 199
Chapter 10: Claude, Markdown, LaTeX/Typst, spelling, URLs 200
9. AI assistant window (Claude Code, `claude-code.nvim`) 200
Claude Code keys (claude-code.nvim) 200
49. AI-assisted development in depth 201
Claude Code in depth (claude-code.nvim) 201
27. Markdown support 202
Preview (mdkite.nvim) 202
Truth tables and Boolean algebra (truth-table.nvim) 202
Footnotes (vim-markdownfootnotes) 202
Text objects & operators (Markdown only) 202
Other Markdown plugins 203
28. LaTeX and Typst support 203
LaTeX (`vimtex`) 203
Typst (`typst.vim`, tinymist) 203
31. Spell checking 204
38. URL & Unicode (`gx.nvim`, `vim-highlighturl`, `unicode.vim`) 205
Chapter 11: Various: custom commands, other plugins, configuration, Neovide 206
34. Custom commands 206
Plugin manager shortcuts (lazy.nvim) 206
39. Other plugins 208
telescope.nvim (second picker) 210
Copying over SSH (vim-oscyank) 210
Keyboard layout switching (vim-xkbswitch) 210
Neovim in the browser (firenvim) 211
Live preview of :norm (live-command.nvim) 211
Vim-script debugging (vim-scriptease) 211
WhatsApp in Neovim (whatsapp.nvim) 212
Practice game (vim-be-good) 213
Libraries and dependencies 213
SQL databases (nvim-dbee, vim-dadbod-ui) 214
40. Configuration management 215
41. Filetype-specific settings 216
Filetype syntax plugins (vim-tmux, vim-toml) 216
42. Automatic behaviors 216
Part II: developer guide 218
66. Shell commands from inside Neovim 218
Running a shell command 218
Inserting command output into the buffer 218
Filtering a selection through a command 218
The asyncrun.vim plugin 219
77. Neovide (graphical Neovim) 219
Starting it 219
What this configuration sets for Neovide 219
Keys that only exist in a GUI 220
Chapter 12: Plugin catalog 221

83. Plugin catalog 221
LSP and code intelligence 221
Language servers, diagnostics and navigation 221
Java 222
Debugging and running code 222
Completion and snippets 223
Completion 223
Snippets and pairs 223
Editing and motions 223
Text objects and surroundings 223
Moving around and searching in the buffer 224
Editing helpers 224
Search and files 225
Sessions and saving 225
Git 226
Everyday Git in the buffer 226
Git interfaces and diffs 226
Databases 226
User interface 227
Bars, messages and start screen 227
Visual helpers 227
Icons and UI libraries 228
Colorschemes 228
Writing: Markdown, LaTeX, Typst 229
AI and collaboration 230
Plugin manager, libraries and other tools 230
Plugin manager and libraries 230
Terminal, browser and file types 230
Chapter 13: Java 231
78. Java (nvim-java, jdtls, tests, debugging) 231
1. What you get and why 232
How nvim-java is loaded 232
spring-boot (Spring Boot tools) 233
The Java devShell (`use_dev_env java`) 234
Outside the devShell 235
2. Quick start 235
3. Project layout rules 235
The root folder 235
First start and where files appear 236
4. Keys and commands 236
Build (`<Space>jb`) 236
Runner (`<Space>jr`) 237
Test (`<Space>jt`) 237
Refactor / extract (`<Space>je`) 237
Debug (`<Space>jh`, `<Space>jp`, `<Space>jP`, `<Space>jx`) 238
Settings and debugger setup 238
The `:Java*` commands 238

The runner window 239
Profiles window (`<Space>jrp`) 239
Clean workspace (`<Space>jbc`) 240
Change runtime (`<Space>jj`) 240
Compact source files (Java 25, `<Space>jrc`) 240
5. Language server features in a Java buffer 240
Go to definition, back, hover, references 242
When a rename does nothing 242
Rename: `:grep` + `:cfdo` instead of `<Space>rn`? 243
Add a missing import (auto-import) 244
Generate getters, setters and constructors 244
6. Tests (JUnit) 245
Why and how 245
Using it 245
`cursor is not on a test method` 246
The test terminal window does not come back after you close it 247
Needs 248
7. Debugging 248
Why and how 248
Debug workflow 250
Learning exercise (try it) 250
Learning exercise: debug a test 250
The Configuration picker 251
8. Refactoring (extract) 252
9. Snippets 254
10. Windows while running, testing and debugging 267
11. Troubleshooting 268
Related sections 270
Chapter 14: Python 271
79. Python (pyright, ruff, uv, running and debugging) 271
What you get 272
Quick start 274
Why each tool exists 275
Devshell and tools 276
How Python is set up (the real code) 277
The server entries (lua/config/lsp.lua) 277
pyright settings (after/lsp/pyright.lua) 278
ruff settings (after/lsp/ruff.lua) 279
Which environment: get_py_env (lua/utils.lua) 279
Run keys (after/ftplugin/python.lua) 280
AsyncRun (lua/plugin_specs.lua) 280
pdb through nvim-gdb (plugin spec and ftplugin) 281
Running code 282
When `<Space>rf` / `<F9>` uses uv 283
Stopping a running program 284
Formatting and linting 284
Code actions from ruff 285

Linting from the command line (`:compiler ruff`) 286
LSP keys 286
Debugging with pdb (nvim-gdb) 288
Virtual environments 290
Testing 291
Navigation and text objects 291
Snippets 292
Jupyter notebooks (nvjup) 292
Requirements 294
Quick start 294
Keys (notebook buffers only) 295
What differs from the rest of the config 297
Troubleshooting 298
Related sections 300
Chapter 15: LaTeX 301
80. LaTeX (vimtex, texlab, ltex, PDF viewer) 301
The big picture 301
What it needs (the devShell) 301
How vimtex is set up (the real code) 303
Quick start 305
Compiling 305
Errors 306
Several files (a project with chapters) 307
The viewer 307
Table of contents 307
Moving and editing (vimtex) 308
Motions 308
Text objects (use after `d`, `y`, `c`, `v`) 309
Change, delete, toggle 310
Language server (texlab) 312
Citations (bibliography) 312
Grammar and spelling (ltex_plus) 313
Snippets 314
Math symbols 315
Troubleshooting 324
Related sections 324
Chapter 16: Markdown 325
81. Markdown (writing, preview, footnotes, PDF) 325
The tools and why they exist 326
Quick start 327
Requirements 327
Try the keys: an example file 328
Keys 328
Text objects and operators 330
Footnotes (vim-markdownfootnotes) 330
Reference links and tables 331
Preview (mdkite.nvim) 332

Truth tables and Boolean algebra (truth-table.nvim) 333
The expression language 333
Commands 335
Keys 337
Typing the symbols 337
Worked lesson: a half adder and a proof 338
Limits and open points 339
Rendering inside the buffer (render-markdown.nvim) 339
Formatting with prettier 340
PDF export (`:ToPDF`) 341
Snippets 342
Math symbols 345
Writing quality 353
ltex_plus (grammar, LanguageTool) 353
Vim spell checking 354
typos_lsp 354
Troubleshooting 355
Related sections 357
Chapter 17: Typst 358
82. Typst (typst.vim, tinymist, watch and preview) 358
What you get 358
Requirements: the Typst devShell 358
Outside the devShell 359
Quick start 359
Keys and commands 360
The language server: tinymist 361
Watch, compile and the PDF 362
Prose checking in Typst 364
Filetype settings 365
Example document 365
What to try 366
Snippets 366
Starting flow 366
Where they work and limits 367
Math delimiters 367
Math symbols 368
Troubleshooting 372
Related sections 372
Chapter 18: C++ 373
84. C++ (snippets, compile and run) 373
What you get 373
Snippets 373
Related sections 377
Chapter 19: Vimscript 378
85. Vimscript (snippets, source and settings) 378
What you get 378
Snippets 378

Related sections 379
Chapter 20: Nix 380
86. Nix (snippets and delib modules) 380
What you get 380
The Nix devShell 381
What delib is 381
Snippets 382
Related sections 387

Chapter 1: Day-to-day cheat sheet (section 2)

The things you do all day, in one place. Details are in the sections named at the end of each table.

Key names (how keys are written)

Written as	Key to press	Notes
`<Ctrl-w>h`, `<C-w>h`	Hold **Control**, press `w`, release both, then press `h`	`<Ctrl-x>`, `<C-x>` and `<c-x>` are spellings of the same thing; the key after it is a separate press
`<Alt-j>`, `<A-j>`, `<M-j>`	Hold **Alt**, press `j`	Alt is also called Meta; all three spellings are the same
`<S-Tab>`	Hold **Shift**, press `Tab`	A capital letter such as `G` or `V` is Shift + the letter; `g` is the plain letter
`<Esc>`	**Escape**	Leaves Insert, Visual and other modes
`<CR>`, `<Enter>`, `<Return>`	**Enter**	All the same key
`<BS>`	**Backspace**	Deletes the character before the cursor
`<Del>`	**Delete**	Deletes the character under the cursor
`<Tab>`	**Tab**	
`<Space>`	The **space bar**	In this config it is also the **leader** key
`<leader>`, `<Leader>`	The leader key, which is `<Space>` here	`<leader>rf` is the same as `<Space>rf`
`<Up>`, `<Down>`, `<Left>`, `<Right>`	The arrow keys	
`<Home>`, `<End>`	The Home and End keys	
`<PageUp>`, `<PageDown>`, `<Insert>`, `<F1>` to `<F12>`	Page, Insert and function keys	The guide avoids needing them (see "Keyboards without function keys, Insert or Page keys")
`gg`, `dd`, `<Space>ff`	Press the keys one after another, quickly	After `<Space>` a popup (which-key) lists what can follow
`;`	Same as `:` in Normal and Visual mode	`;w<Enter>` saves, `;qa!<Enter>` quits everything without asking
`<file>`, `<name>`, `<path>`, `<word>`, `<char>`, `<url>`	Not a key: replace it with your own value	Any word in `<...>` that is not a key name is a placeholder
`<Plug>(...)`, `<Cmd>`, `<C-U>` (in code)	Not keys you press	Internal names inside the quoted Lua and Vim script of a mapping
`:'<,'>`	Not typed by you	Vim writes it after `:` when you press `:` in Visual mode; it means "the selected lines"
`<p>`, `<script>`, `<style>`, `<b>` in examples	Not keys	Literal HTML tags in the text being edited

Reading the guide PDF (zathura keys)

`<Space>?` opens this guide in zathura, always in light colors whatever your own zathura theme is. This holds for every zathura Neovim starts (this guide, LaTeX, Typst, `:!zathura`): `lua/zathura_light.lua` puts a small `zathura` shim first on Neovim's PATH (`~/.cache/nvim/zathura-shim`) that adds a light config on top of yours. Zathura started outside Neovim is unchanged. Zathura has its own keys (the defaults, from its man page), not Neovim's:

Keys	What it does
`J` / `K`	Next page / previous page (the same as `<PageDown>` / `<PageUp>`)
`j` `k`, `h` `l`	Scroll down / up, left / right
`<Ctrl-d>` / `<Ctrl-u>`, `<Space>`	Scroll half a page down / up, a full page down
`gg`, `G`, `42G`	First page, last page, page 42
`Tab`	Show the table of contents (index): `j` `k` move, `l` / `h` expand / collapse an entry, `<Enter>` opens it, `Tab` or `<Esc>` hides the index again
`/word<Enter>`, `n` / `N`	Search; next / previous match (`n` is NOT "next page")
`f`	Show the clickable links of the page as numbers; type a number to follow it. `<Ctrl-o>` goes back
`a` / `s`	Fit the whole page / the page width in the window
`R`	Reload the file (after rebuilding the PDF)
`r`	Rotate the page clockwise by 90 degrees (press again for each further 90 degrees)
`<Ctrl-r>`	Switch recolor (inverted colors) on / off; the guide starts with it off, i.e. light
`q`	Close zathura

Navigation

Move one step at a time (a number before the key repeats it: `5j` is five lines down):

Keys	What it does
`h` / `l`	One character left / right
`j` / `k`	One line down / up

Move by word (a "word" stops at punctuation; a "WORD", the capital letters, only at spaces):

Keys	What it does
`w` / `b`	Start of the next / previous word
`e` / `ge`	End of the next / previous word
`W` / `B` / `E`	The same with WORDs (`foo.bar` counts as one)

Move inside a line:

Keys	What it does
`H`	First non-blank character of the line (same as `^`)
`L`	Last non-blank character of the line (same as `g_`)
`0` / `$`	Very first column / very end of the line

Move through the file:

Keys	What it does
`gg` / `G`	First / last line of the file
`42G` or `:42`	Line 42
`{` / `}`	Previous / next blank line (paragraph)
`<Ctrl-d>` / `<Ctrl-u>`	Half a screen down / up
`<Ctrl-f>` / `<Ctrl-b>`	A full screen down / up
`zz` / `zt` / `zb`	Put the cursor line in the middle / at the top / at the bottom of the window

Jump far in one go:

Keys	What it does
`f` + two characters	Hop to any matching spot on screen (then press the label letter)
`%`	Jump to the matching bracket
`<Ctrl-o>` / `<Ctrl-i>`	Back / forward through your jump history (after `gd`, searches, `gg` …)
`ma`, then `` `a ``	Set mark `a`, jump back to it later
`gd`	Go to definition (use `<Ctrl-o>` to come back)

Move between files and views:

Keys	What it does
`<Space>ff` / `<Space>fg` / `<Space>fr`	Find a file / search text in the project / recent files
`<Space>s`	File tree (follows `:cd`, `:Z`, `:z`)
`gb` / `gB`	Next / previous buffer
`<Space>bp`	Pick a buffer by letter
`<Space>t`	Symbol outline of the file
`:Z <word>`	Change the working folder with zoxide

More: sections 3, 7, 11, 12, 22.

Copy, delete and move

Keys	What it does
`yy` then `p`	Copy a line, paste it below
`dd` then `p`	Cut a line, paste it below
`diw` / `ciw` / `yiw`	Delete / change / copy the word under the cursor (`c` does not overwrite your paste register)
`5dd` / `5yy`	Delete / copy 5 lines
`<Alt-j>` / `<Alt-k>`	Move the line (or selection) down / up
`;3m10<CR>` / `;.m+2<CR>`	Move line 3 below line 10 / move the current line below the line 2 further down (`;` is `:`; `m` puts the line BELOW the target, `m0` = top)
`;.t+2<CR>`	Copy the current line below the line 2 further down (`t` copies, `m` moves)
`d` / `dd` / `D` / `x` then `p`	Delete is cut: there is no separate cut key. `c` / `C` / `cc` do not fill the register
`<Space>y`	Copy the whole buffer
`<Space>p` / `<Space>P`	Paste on a new line below / above
`p` in Visual mode	Replace the selection with what you copied
`[y` / `]y`	After a paste, step back / forward through the yank history
`:Rename <name>` / `:Move <path>` / `:Duplicate <name>` / `:Delete`	Rename, move, copy or delete the current FILE
tree: `a` `d` `r` `c` `x` `p`	Create, delete, rename, copy, cut, paste files in the file tree

More (moving lines by number, a selection, part of a line: Non-contiguous lines): sections 4, 11, 24, 57.

Indented blocks

Keys	What it does
`dii` / `yii` / `cii` / `vii`	Delete / copy / change / select the code block at the cursor's indent level
`dai` / `yai` / `cai` / `vai`	Same, including the lines that open and close the block
`[i` / `]i`	Jump to the top / bottom of the block

More: section 5 ("ii / ai").

Text objects (what to delete, change or copy)

Keys	What it does
`diw` / `daw`	The word / the word with its space
`das` / `dis` / `cis`	The sentence with / without the space after it; change the sentence
`dip` / `dap`	The paragraph (without / with the blank line after it)
`di(` / `da(`	The text inside the parentheses / with the parentheses
`diS(` / `daS(`	Sandwich: empty the surrounding `( )` / remove them with their content
`ii` / `ai`	The indented block (see above)
`$V%`	Select a whole `{ }` block from its first line to the closing `}` (cursor on the line that ends with `{`; then `y`, `d`, `<Alt-j>` / `<Alt-k>` to copy, delete, move; details)

The same objects work after `c`, `y` and `v`. More: sections 5, 6, 17 and 70.

Macros

Keys	What it does
`Qa`	Start recording into register `a` (`qa` works too)
`q`	Stop recording
`@a` / `5@a` / `@@`	Play it once / 5 times / repeat the last one (in Markdown buffers `@@` returns from a footnote instead: use `@a` again)
`:%normal @a`	Play it on every line of the file

More: sections 29 and 60.

Bulk rename and replace

Goal	What to do
Rename a code symbol everywhere in the project	`<Space>rn`, type the new name, `<Enter>` (LSP)
Replace text in the current file	`:%s/old/new/g` (add `c` to confirm each one)
Replace text in every file of the project	`:grep "old"` then `:cfdo %s/old/new/g | update`
Replace one by one, deciding each time	`*`, `ciw` + new word + `<Esc>`, then `n` to skip or `.` to replace
Rename or move one file	`:Rename <name>` / `:Move <path>`, or `r` in the file tree
Undo a multi-file replace	See section 67

Files are renamed one at a time with the commands above. More: sections 10 and 67.

Undo, redo and repeat

Keys	What it does
`u` / `<Ctrl-r>`	Undo / redo
`<Space>u`	Show the undo tree in a left panel
`.`	Repeat the last change (e.g. `ciw` + word, then `n` and `.` on the next match)
`[y` / `]y`	After a paste, replace the pasted text with an earlier / later yank (yank two lines, `p`, then `[y`)

More: sections 24, 25, 61.

Visual block (many lines at once)

Keys	What it does
`<Ctrl-v>` then `j` / `k`	Select a rectangle of text
`I` then text then `<Esc>`	Insert the text at the start of every selected line
`A` then text then `<Esc>`	Append the text at the end of every selected line
`d` / `c`	Delete / change the selected block
`g<Ctrl-a>`	Count up 1, 2, 3 ... down the selected numbers
`<Alt-j>` / `<Alt-k>`	Move the selected lines down / up

More: section 62.

Comments and surrounding pairs (vim-commentary, vim-sandwich)

Keys	What it does
`gcc` / `gc` + motion / `gc` in Visual	Toggle a comment on a line / a motion / the selection (`gcip` = paragraph)
`gcss` / `gcs` + motion, `gcr` + motion	Comment / uncomment exactly the rows given (`gcsip`, `gcr200j`)
`gcu`	Uncomment the adjacent commented lines
`:1,3Commentary`	Toggle comments on a line range
`dgc` / `ygc`	Delete / copy the comment block under the cursor
`saiw"`	Surround the word with `"`
`sd"`	Delete the surrounding `"`
`sr"'`	Change the surrounding `"` into `'`

More: sections 16 and 17.

Search in a file

Keys	What it does
`/text` then `<Enter>`, `n` / `N`	Search forward, next / previous match
`*` / `#`	Search the word under the cursor forward / backward
`;noh<Enter>`	Clear the search highlight
`:%s/old/new/gc`	Replace with a question for each match

More: sections 10 and 23.

Buffers, splits, tabs

Keys	What it does
`gb` / `gB`, `<Space>bp`	Next / previous buffer, pick one by letter
`<Ctrl-^>` (in Neovide also `<Ctrl-6>`)	Jump to the previous buffer and back again, like Alt-Tab for files
`\d` / `\D`	Close this buffer / close all other buffers
`<Space>-` / `<Space>|`	Split below / to the right (same file)
`:b <name|number>` then `:vert sbuffer <name|number>`	Two chosen open buffers side by side (`:sbuffer` without `vert`: one above the other)
`<Ctrl-w>h` `j` `k` `l`	Move between windows
`<Left>` `<Down>` `<Up>` `<Right>`	The same, with the arrow keys (normal mode)
From a terminal window (run output, `:term`)	`<Esc>`, then `<Ctrl-w>h` (or an arrow key) to go to the code; `i` to type in the terminal again
From the Claude panel	`<Ctrl-h>` goes back to the code on the left (`<Esc>` is sent to Claude); `<Ctrl-w>l` or `<Right>` from the code goes back in
`<Ctrl-w>=` / `<Ctrl-w>o`	Make windows equal / keep only this window
`<Ctrl-w>x` / `<Ctrl-w>r`	Swap this window with the next one (the file on the right goes to the left) / rotate all windows of the row
`:BufferLineMovePrev` / `:BufferLineMoveNext`	Move the current tab one place left / right in the top bar (no key for it: type the command; section 7)
`gt` / `gT`, `\t` / `\T`	Next / previous tab, close this tab / the other tabs
`:sp <file>` / `:vs <file>`	Open a file in a new horizontal / vertical split (`;` works like `:`, so `;vs <file>` too)

More: section 7.

Save and quit

Keys	What it does
`<Space>w`	Save (only writes if changed)
`<Space>q`	Save and close this window
`<Space>Q`	Quit nvim, asks first (default No), discards unsaved work
`:wa` / `:q!`	Save all buffers / close this window and discard changes
(automatic)	Auto-save: a changed file saves itself when you switch buffer or leave the nvim window; a small popup "AutoSave: saved at ..." appears and the file is not formatted (not for unnamed or read-only buffers, terminals, LaTeX and Typst files)

More: section 72.

Sessions, dashboard and zoxide

Keys	What it does
`\h` / `\H`	Open the dashboard / return to the previous buffer
`r` / `L` on the dashboard	Restore the session of this folder / of the last folder
`<Space>?`	Open the user guide as a PDF in zathura next to Neovim (any buffer and the dashboard; clickable table of contents; `q` closes it)
`<Space>a`	Ask Claude "how do I do X in Neovim": split on the right (own tab on the dashboard), skill loaded, always Sonnet; `/exit` closes it
`:Z <word>` / `:z <word>`	Jump to the best zoxide match (the file tree follows)
`:Obsession`, `nvim -S Session.vim`	Keep a `Session.vim` up to date, restore it later
`<Space>sv`	Restart nvim (writes all files first)

More: sections 30 and 58.

Code intelligence (LSP)

Keys	What it does
`gd` / `K`	Go to definition / hover documentation
`<Space>gd` / `<Space>gr` / `<Space>gi`	Peek definitions / references / implementations
`<Space>rn` / `<Space>ca`	Rename symbol / code actions
`<Space>fm`	Format the file (Visual mode: the selection)
`<Space>fo`	Toggle format on save (on by default for `:w`)
`]d` / `[d`	Next / previous diagnostic
`<Space>dd`	Show the diagnostic under the cursor

More: sections 13 and 44.

Completion and snippets

The completion menu opens by itself while you type in Insert mode (from 1 character; buffer words from 2).

Keys	What it does
`<Tab>`	Menu open: select the next item. Menu closed with the cursor before a closing `)` `]` `}` `"` `'` or `` ` ``: jump out past it (press twice to leave `("ab")`). Otherwise: insert a normal tab
`<S-Tab>` (Shift + Tab)	Jump back to before the opening bracket or quote of the pair you are in. With the menu open it does NOT select the previous item: use `<Ctrl-p>`
`<CR>`	Confirm the item you picked with `<Tab>`; with nothing picked it is a plain newline
`<Ctrl-e>` / `<Esc>`	Close the menu (without it: end of line / leave Insert mode)
`<Ctrl-d>` / `<Ctrl-f>`	Scroll the documentation window up / down (menu open)
`<Ctrl-j>` / `<Ctrl-k>`	Expand a snippet or jump to the next placeholder / jump back
`<Space>fs`	Gallery: fuzzy-search only your own snippets (`my_snippets/`) for this filetype and insert one
`<Alt-s>` (Insert mode)	Same gallery, inserting at the cursor

Menu order: language server, then snippets, then paths, then buffer words (LaTeX files start with vimtex). In the `/` search line the menu offers buffer words; in `:` it offers paths and commands.

More: sections 14, 15, 45 and 52 (the gallery is in section 52, "The snippet gallery").

Git day to day

Keys	What it does
`<Space>gs`	Git status
`<Space>gw` / `<Space>ga`	Add the current file / add all changes
`<Space>gu`	Unstage the current file
`<Space>gv`	Vertical diff of the file against the index
`<Space>gc` / `<Space>gA`	Commit / amend the last commit
`<Space>gp` / `<Space>gP`	Pull / push
`<Space>gB`	Branch menu: pick a branch to switch to (same as clicking the branch in the statusline)
`]c` / `[c`	Next / previous changed hunk
`<Space>hp` / `<Space>hb`	Preview the hunk / blame the line
`<Space>hs` / `<Space>hr`	Stage / reset the hunk (reset asks Yes/No; works on selected lines in visual mode)
`<Space>gl`	Copy a permalink for the line
`:Neogit` / `:NeogitLogCurrent`	Open the Neogit status window / the log of the current file (`q` closes)

More: sections 20 and 48.

Mouse

Action	What it does
Click a tab in the top line	Switch to that buffer
Click the `x` / `●` of a tab	Close the buffer (a changed file asks Save / Discard / Cancel)
Click the branch name in the statusline	Branch menu (also `<Space>gB`)
Click the language-server name in the statusline	Popup with the attached language servers (also `:LspAttached`)
Right-click in the text	Neovim's menu: `Paste` pastes the clipboard, `Select All` selects the whole file, `Inspect` shows which highlight and syntax group is under the click, `How-to disable mouse` opens the help for it

More: sections 7 and 32.

Language guides (tools that exist only for one language)

Language	What the guide covers	Section
Java	nvim-java, jdtls, running, JUnit tests, debugging, refactoring, profiles	78
Python	pyright, ruff, uv, `<Space>rf` / `<F9>` and `<Space>rr`, pdb debugging	79
LaTeX	vimtex, texlab, ltex, compiling, the PDF viewer	80
Markdown	marksman, rendering, preview, footnotes, `:ToPDF`	81
Typst	tinymist, `<Space>tw` watch, the PDF viewer	82

Language devShells (`:DevEnv`)

Command	What it does
`:DevEnv <lang>`	Enter a language devShell (`java`, `latex`, `typst`, `python`, ...) from the running Neovim, without restarting: adds its programs to `PATH`, enables the language servers and plugins that were waiting. `<Tab>` lists the names. First call about 10 s, then cached. Example: `:DevEnv java`

Full list of devShells and details: section 43.

Neovide (Neovim in its own window)

Command	What it does
`neovide`	Open Neovim in a normal desktop window instead of the terminal, with smooth cursor animation. Same config, same keys.

Full explanation: section 77.

Lists with a search bar (pickers)

Keys	What it does
`<Ctrl-n>` / `<Ctrl-p>` or `<Down>` / `<Up>`	Next / previous item (branch menu, code actions, `<Space>ff`, `<Space>fg`, Telescope …)
`<Ctrl-j>` / `<Ctrl-k>`	Same, except in Telescope
`<Tab>`	Move down (marks the item too in fzf-lua and Telescope)
`<Enter>` / `<Esc>`	Choose / close (in snacks and Telescope the first `<Esc>` only leaves the search bar)

`j` and `k` type letters into the search bar. More: section 12.

Quickfix list (search results, errors)

Keys	What it does
`:copen` / `:cclose`	Open / close the list
`:cnext` / `:cprev`	Next / previous item
`\x`	Close the quickfix and location lists
`p` / `zf` in the list	Preview the item / filter the list with fzf
`<Tab>` in the list	Mark the item
`<Ctrl-x>` / `<Ctrl-v>` in the list	Open the item in a horizontal / vertical split
`:cfdo %s/old/new/g | update`	Replace in every file of the list (fill the list first with `:grep`; other file sets: `:bufdo`, `:argdo`, see do-commands)

More: sections 26 and 51.

Spell checking

Keys	What it does
`<Space>cz`	Spell checking on / off (`:set spell` / `:set nospell` too)
`]s` / `[s`	Next / previous misspelled word
`z=`	Show suggestions
`zg` / `zw`	Add the word to your allowed list / mark it as wrong (`2zg` Italian, `3zg` German, `4zg` French)
`zug`	Undo the last `zg`
`:e ~/.config/nvim/spell/en.utf-8.add`	Open the list of allowed English words (one per line: delete a line to forbid the word again)

`:set nospell` only turns off the built-in checker. Typo underlines that come from the language servers (`typos_lsp` in code, `ltex_plus` in prose) stay; they are diagnostics. More: section 31.

Run code and terminal

Keys	What it does
`<Space>rr`	Run the current file in a terminal on the left
`<Space>rf` / `<F9>`	Run or compile the file the editor's own way (Lua, Vim script, Python, C++, LaTeX; one warning in other file types)
`:term`	Open a terminal in this window
`<Esc>` / `i`	Leave terminal mode / go back into it (`<Esc>` works differently in the Claude panel, see sections 8 and 9)

More: sections 8 and 19. `<Space>rf` in any other file type shows one warning (`no editor-run for this file type (use <Space>rr for a terminal run)`).

Python debugger keys (pdb through nvim-gdb)

Start with `<Space>dp` in a Python buffer (pdb through nvim-gdb); the keys work only during that session (otherwise one warning `pdb: no debug session here (start one with <Space>dp)`). nvim-gdb's own F-keys still work too.

Keys	What it does
`<Space>dp`	Start the pdb debugger on this file
`<Space>dB` / `<F8>`	Toggle a breakpoint on this line
`<Space>dc` / `<F5>`	Continue to the next breakpoint
`<Space>dn` / `<F10>`	Next line (step over)
`<Space>ds` / `<F11>`	Step into the call
`<Space>df` / `<F12>`	Finish (run until the function returns)
`<Space>du` / `<F4>`	Run until this line
`<Space>dv`	Evaluate the word under the cursor (Visual: the selection)
`:GdbDebugStop`	Quit the debugger

More: section 79.

Java debugger keys (nvim-dap through nvim-java)

Global keys; all four (`<Space>jp`, `<Space>jh`, `<Space>jP`, `<Space>jx`). Stepping and continuing have no keys: type the commands.

Keys	What it does
`<Space>jp`	Toggle a breakpoint on this line (put it on a line with code)
`:DapContinue`	Start the debugger on a `main` class (pick an entry in the `Configuration` list) or run on while paused
`:DapStepOver` / `:DapStepInto` / `:DapStepOut`	Next line / into the call / back to the caller
`<Space>jh`	While paused: show the value of the variable under the cursor
`<Space>jx`	Stop the debug session
`<Space>jP`	Remove ALL breakpoints in all files at once (capital P; no undo)
`:DapClearBreakpoints`	Same as `<Space>jP` (`<Space>jp` removes only the one on the line)

More: section 78.

Keyboards without function keys, Insert or Page keys

Nearly every key in this guide has a version without function keys. `<F9>` is `<Space>rf` (run the file the editor's own way); the Python debugger has the `<Space>d` keys above. `<S-Insert>` (paste in the GUI, ginit.vim) is replaced by `<Ctrl-r>` then `+` in Insert mode or on the command line (built-in Vim, works everywhere); `<Ctrl-d>` / `<Ctrl-u>` scroll instead of the Page keys. Unicode `<F4>` is replaced by `<Ctrl-k>` + two letters (section 38). The only keys left are three vimtex defaults in section 80 (`<F6>`, `<F7>`, `<F8>`).

Folding (nvim-ufo)

Keys	What it does
`za`	Toggle the fold under the cursor
`zR` / `zM`	Open / close all folds
`zr` / `zm`	Open / close one more fold level
`<Space>K`	Preview the folded lines

More: section 18.

Text tricks

Keys	What it does
`gUiw` / `guiw`	Uppercase / lowercase the word
`<Ctrl-a>` / `<Ctrl-x>`	Add 1 to / subtract 1 from the number under the cursor (`10<Ctrl-a>` adds 10)
`:%!sort`	Sort the whole buffer
`:Tabularize /=`	Align the `=` signs
`:g/pattern/d`	Delete every line that matches
`:g/pattern/normal @a`	Run macro `a` on every matching line
`:!cmd` / `:read !cmd`	Run a shell command / insert its output

More: sections 66, 68 and 71.

Claude and fuzzy finders

Keys	What it does
`<Space>cc`	Toggle the Claude Code window
`<Space>a`	Ask Claude how to do something in Neovim (vertical split, skill loaded, always Sonnet)
`<Space>?`	Open the user guide PDF
`<Space>ff` / `<Space>fg`	Find a file / search text in the project
`<Space>fb` / `<Space>fr` / `<Space>fh`	Open buffers / recent files / help tags
`<Space>fs` (`<Alt-s>` in insert mode)	Fuzzy-search your own snippets and insert one (section 52)

More: sections 9 and 12.

Other most-used keys (from the former quick reference)

Keymap	Action
`]<Space>` / `[<Space>`	Insert a blank line below / above (cursor stays; `3]<Space>` inserts 3)
`gS`	Split / join the list, arguments or block under the cursor (treesj)

Chapter 2: Basics: modes, saving, recovering, getting help

1. Understanding modes

Neovim is a modal editor. You are always in one of these modes:

Mode	How to enter	What it does
Normal	`<Esc>` from any mode	Navigate, delete, copy, paste, run commands. This is your "home base".
Insert	`i`, `a`, `o`, `O`, `c` from Normal (`s` is disabled here: it is the vim-sandwich prefix)	Type text into the file.
Visual	`v`, `V`, `<Ctrl-v>` from Normal	Select text (character, line, or block).
Command	`;` or `:` from Normal	Type commands at the bottom of the screen (e.g., `:w` to save).
Terminal	When inside a terminal buffer	Interact with a shell. Press `<Esc>` to go to Normal mode (in a Claude terminal `<Esc>` goes to Claude: `<Ctrl-w>h/j/k/l` still move to another window, `<Ctrl-q>` or `<Ctrl-\><Ctrl-n>` leaves terminal mode).

Entering insert mode

Keymap	Description
`i`	Insert before cursor
`I`	Insert at beginning of line
`a`	Insert after cursor
`A`	Insert at end of line
`o`	Open new line below and insert
`O`	Open new line above and insert

Leaving insert mode

Keymap	Description
`<Esc>`	Return to Normal mode
`jk` (typed quickly)	Return to Normal mode (via better-escape.vim plugin, 200ms window)

Visual mode variants

Keymap	Description
`v`	Character-wise visual (select individual characters)
`V`	Line-wise visual (select entire lines)
`<Ctrl-v>`	Block visual (select a rectangular block of text)

What each one selects, on `alpha beta gamma` with the cursor on the `b` of `beta` (the selection is shown in `[ ]`):

```
alpha beta gamma        cursor on the b of beta
v e         ->  alpha [beta] gamma         characters from the cursor to the end of the word
V           ->  [alpha beta gamma]         the whole line
<Ctrl-v> j  ->  a rectangle: the same columns on this line and on the next one
```

---

69. Tips for Vim beginners

The most important habits

1. **Stay in Normal mode**. Only enter Insert mode to type, then immediately `<Esc>` back. Normal mode is where all the power lives.
2. **Think in verbs + nouns**. `d` (delete) + `iw` (inner word) = delete word. `c` (change) + `i"` (inside quotes) = change quoted text. `y` (yank) + `ap` (around paragraph) = copy paragraph.
3. **Use `.` aggressively**. Make one change, then `.` to repeat it everywhere.
4. **Use `*` and `n`**. Search for a word with `*`, jump through occurrences with `n`.
5. **Use text objects**. `ciw`, `di(`, `va"` are faster than selecting character-by-character.
6. **Press `<Space>` and wait**. The which-key popup shows you all available keybindings.

Example for habit 2: on `x = compute("old", 5)` with the cursor anywhere inside `"old"`, press `ci"`, type `new` and press `<Esc>`: the line becomes `x = compute("new", 5)`.

Common mistakes and how to fix them

Problem	Cause	Fix
Typing random commands instead of text	You're in Normal mode	Press `i` to enter Insert mode first
Text won't stop appearing	You're in Insert mode	Press `<Esc>` to go back to Normal
Screen looks weird / frozen	You pressed `<Ctrl-s>` (terminal freeze)	Press `<Ctrl-q>` to unfreeze. (In kitty + tmux with this config `<Ctrl-s>` did not freeze the screen. In Insert/Select mode `<Ctrl-s>` is LSP signature help.)
Can't exit Neovim	`:q` only closes the current window, and with unsaved changes it asks "Save changes?" instead of quitting (the `confirm` option is on)	Type `<Space>Q` and answer `y` to the confirmation, or `;qa!<Enter>` (no confirmation)
Pasted text looks wrong	Paste from outside with `<Ctrl-v>` in terminal mode	Use `"+p` in Normal mode, or the terminal paste key
Search highlight won't go away	Yellow boxes left over from a search or `*`	Type `;noh<Enter>`. `<Esc>` does **not** clear it in this config
Accidentally opened a macro	Pressed `Q`	Press `q` to stop recording
A key like `"` does nothing until you press another key, or an accented letter appears (`ë`, `è`)	Your keyboard layout uses **dead keys** (see below)	Press `<Space>` right after the key

Keyboard layouts with dead keys (for example US International)

Some layouts (for example US International, used on the main machine) treat certain keys as **dead keys**: the key does not type anything by itself, it waits for the next key to decide. `"` followed by `e` gives `ë`; `"` followed by `<Space>` gives a plain `"`. Neovim only receives the character after that decision, so commands that need `"` look like they do nothing.

- **Symptom (with `"`)**: `vt"l` selected nothing, because `t` kept waiting for its character. `vt"<Space>l` works. In a search, `v/"<Enter>` worked because `<Enter>` also ends the wait.
- **The rule**: after a dead key, press `<Space>` before continuing (`vt"<Space>l`).
- **Keys that may be affected** (depends on the layout): `'` and `` ` `` (marks such as `` `a `` and `'a`), `"` (registers such as `"ay`, text objects such as `ci"`), `~` (toggle case) and `^` (start of line). Without the space, a dead key followed by a letter can become an accented letter (`"a` -> `ä`, `` `a `` -> `à`) and the Vim command never runs.
- **This is not Neovim**: it happens in any application with that layout. A different PC may have a different layout, so if a key does nothing or types a strange character, check the layout before suspecting the config.

- **Quick test in Insert mode**: type the key, then a letter (`"e`); if you get an accented letter, it is a dead key on your layout.

Learning path

1. First week: `h j k l`, `i`, `<Esc>`, `:w`, `:q`, `dd`, `yy`, `p`, `u`
2. Second week: `w`, `b`, `e`, `0`, `$`, `gg`, `G`, `/search`, `n`, `N`
3. Third week: `ciw`, `di(`, `vi"`, `V`, `>`, `<`, `.`
4. Fourth week: `<Space>ff`, `<Space>fg`, `gd`, `K`, `<Space>ca`, `gcc`
5. After that: Macros, quickfix, text objects, splits, registers

---

70. The verb + noun system (how Vim commands work)

This is the single most important mental model for understanding Vim. Almost every command follows this pattern:

`[count] operator motion` or `[count] operator text-object`

- **Operator** (verb): What you want to do (`d` delete, `c` change, `y` yank, `>` indent, `gU` uppercase, etc.)
- **Motion** (noun): Where to do it (`w` word, `$` end of line, `gg` top of file, `}` next paragraph, etc.)
- **Text object** (noun): A structural unit to act on (`iw` inner word, `i(` inside parentheses, `at` around an HTML tag, etc.)
- **Count**: How many times (optional)

Operators (verbs)

Operator	What it does	Example
`d`	**Delete** (and cut to register)	`dw` on `foo bar` -> `bar`
`c`	**Change** (delete and enter insert mode)	`cw`, type `x` on `foo bar` -> `x bar`
`y`	**Yank** (copy)	`yw` on `foo bar` copies `foo`; `p` pastes it
`>`	**Indent** right	`>>` on `foo` -> `  foo` (2 spaces: this config's shiftwidth)
`<`	**Indent** left	`<<` on `    foo` -> `  foo`
`=`	**Auto-indent** (fix indentation)	`=ip` on the Lua lines `if a then` / `x` / `end` -> the `x` line gets 2 spaces
`gU`	Convert to **UPPERCASE**	`gUw` on `foo bar` -> `FOO bar`
`gu`	Convert to **lowercase**	`guw` on `FOO BAR` -> `foo BAR`
`~`	**Toggle case** (in this config `~` is an operator because `tildeop` is set)	`~w` on `foo bar` -> `FOO bar`
`gc`	**Toggle comment** (vim-commentary)	`gcc` on `x = 1` in a Lua file -> `-- x = 1`
`gcs` / `gcr`	**Comment** / **uncomment** whole rows (smart commenting)	`gcsip` on the Lua lines `a = 1` / `b = 2` -> `-- a = 1` / `-- b = 2`; `gcrip` removes the markers again
`gq`	**Format/wrap** text	`gqq` on one very long line re-wraps it into several lines of at most 79 columns

Motions (nouns)

Motion	What it means
`w`	To start of next word
`b`	To start of previous word
`e`	To end of current/next word
`$`	To end of line
`0`	To start of line
`^`	To first non-whitespace
`gg`	To top of file
`G`	To bottom of file
`}`	To next blank line
`{`	To previous blank line
`%`	To matching bracket
`j`	Down one line
`k`	Up one line

Where the cursor ends up, on the line `  total = price * 3;` (two leading spaces) with the cursor on the `t` of `total`:

Keys	Cursor lands on
`w`	the `=` (start of the next word)
`e`	the last `l` of `total` (end of the word)
`$`	the `;` (end of the line)
`0`	the first space (column 1)
`^`	the `t` of `total` (first non-blank character)

Text objects (structured nouns)

Text Object	What it selects	Example (with `d`)
`iw` / `aw`	Inner word / a word (with white-space)	`diw` on `foo bar` (cursor in `foo`) -> ` bar`; `daw` -> `bar`
`iW` / `aW`	Inner WORD / a WORD	`diW` on `a.b c` (cursor on `a`) -> ` c`; plain `diw` would only delete `a`
`is` / `as`	Inner sentence / a sentence (with trailing whitespace)	`das` on `One. Two. Three.` (cursor in `Two.`) -> `One. Three.`; `dis` -> `One. Three.`
`iS` / `aS`	vim-sandwich "query" object: type the surrounding character after it, e.g. `diS(`, `caS"`, `viS[` (inside / around that surrounding pair)	`diS(` on `f(old)` (cursor in `old`) -> `f()`
`ip` / `ap`	Inner paragraph / a paragraph	`dip` on the lines `a` / `b` / (blank) / `c` -> (blank) / `c`; `dap` -> `c`
`i(` / `a(`	Inside / around parentheses	`di(` on `f(old)` -> `f()`; `da(` on `f(old) x` -> `f x`
`i{` / `a{`	Inside / around braces	`di{` on `{old}` -> `{}`; `da{` on `x {old} y` -> `x  y`
`i[` / `a[`	Inside / around brackets	`di[` on `x [old] y` -> `x [] y`
`i"` / `a"`	Inside / around double quotes	`di"` on `x "y" z` -> `x "" z`; `da"` -> `x  z`
`i'` / `a'`	Inside / around single quotes	`di'` on `x 'old' y` -> `x '' y`
`it` / `at`	Inside / around HTML tags	`dit` on `<p>old</p>` -> `<p></p>`; `dat` on `a <b>old</b> c` -> `a  c`

Combining verbs and nouns

Every operator works with every motion and every text object. This creates hundreds of commands from a small set of building blocks:

Command	Verb	Noun	What it does
`dw`	delete	word forward	Delete from cursor to next word
`diw`	delete	inner word	Delete the word under cursor
`daw`	delete	a word	Delete word + surrounding whitespace
`di(`	delete	inside parentheses	Delete everything inside `(...)`
`da(`	delete	around parentheses	Delete everything including the `( )`
`di"`	delete	inside quotes	Delete everything inside `"..."`
`d$`	delete	to end of line	Delete from cursor to line end
`dG`	delete	to end of file	Delete from here to bottom of file
`d}`	delete	to next paragraph	Delete to next blank line
`ciw`	change	inner word	Delete word, enter insert mode
`ci(`	change	inside parens	Delete contents of parens, enter insert
`ci"`	change	inside quotes	Delete quoted text, enter insert
`cit`	change	inside HTML tag	Delete tag content, enter insert
`yiw`	yank	inner word	Copy the word under cursor
`yi{`	yank	inside braces	Copy contents of `{...}`
`yap`	yank	a paragraph	Copy the paragraph
`>ip`	indent	inner paragraph	Indent the current paragraph
`=i{`	auto-indent	inside braces	Fix indentation inside `{...}`
`gUiw`	uppercase	inner word	Uppercase the entire word
`guap`	lowercase	a paragraph	Lowercase the entire paragraph
`gcip`	comment	inner paragraph	Comment out the paragraph
`gc3j`	comment	3 lines down	Comment out 3 lines

Worked example on the line `call(foo, "bar baz", [1, 2])`; the cursor is on the word named in the first column:

Cursor	Keys	After
on `foo`	`di(`	`call()`
on `foo`	`da(`	`call`
on `bar`	`di"`	`call(foo, "", [1, 2])`
on `bar`	`da"`	`call(foo, , [1, 2])`
on `bar`	`ci"` + `X` + `<Esc>`	`call(foo, "X", [1, 2])`
on `foo`	`dt,`	`call(, "bar baz", [1, 2])`
`<p>old</p>`, on `old`	`cit` + `new` + `<Esc>`	`<p>new</p>`
`<p>old</p>`, on `old`	`dit`	`<p></p>`
`a` / `b` / (blank) / `c`, on `a`	`>ip`	`  a` / `  b` / (blank) / `c`
`a` / `b` / `c` / `d` in a Lua file, on `a`	`gc2j`	`-- a` / `-- b` / `-- c` / `d`

Using counts

Counts multiply the action:

Command	What it does	Example
`3dw`	Delete 3 words	`3dw` on `a b c d e` -> `d e`
`5dd`	Delete 5 lines	`5dd` on 7 lines deletes the first 5; lines 6 and 7 remain
`2yy`	Yank 2 lines	`2yy` on the lines `1`, `2`, `3` copies `1` and `2`
`3>>`	Indent 3 lines	`3>>` on the lines `a`, `b`, `c`, `d` indents `a`, `b`, `c`; `d` is unchanged
`10j`	Move down 10 lines	`10j` from line 1 puts the cursor on line 11

Why this matters

Once you learn a few operators and a few motions/text-objects, you can combine them freely. Learning one new operator (e.g., `gU` for uppercase) instantly gives you dozens of new commands (`gUiw`, `gUi"`, `gU$`, `gUap`, etc.) without memorizing anything extra.

---

72. Saving, quitting, and file state

All the ways to save and quit, consolidated in one place.

Saving

Keymap / Command	What it does
`<Space>w`	Save the current buffer (`:update` – only writes if modified)
`:w`	Save the current buffer
`:w filename.txt`	Save as a new file (original stays open)
`:wall` or `:wa`	Save ALL open buffers
`:saveas filename.txt`	Save as new file AND switch to it

Auto-save is also active: files save on `FocusLost` (switching to another app) and `BufLeave` (switching buffers). Auto-saves do not format the file; saving yourself with `:w` does (see Formatting (conform.nvim)).

Quitting

Keymap / Command	What it does
`<Space>q`	Save and quit the current window (`:x`)
`<Space>Q`	Force quit all windows (`:qa!`) after a Yes/No confirmation
`:q`	Quit current window (fails if unsaved changes)
`:q!`	Quit current window, discard unsaved changes
`:qa`	Quit all windows (fails if any unsaved)
`:qa!`	Quit all windows, discard all unsaved changes
`:wq`	Save and quit current window
`:wqa`	Save all and quit all
`ZZ`	Save and quit (same as `:wq`)
`ZQ`	Quit without saving (same as `:q!`)

Closing buffers (without quitting Neovim)

Keymap	What it does
`\d`	Close current buffer, keep the window (shows previous buffer)
`\D`	Close all other buffers; buffers with unsaved changes and running terminals (Claude panel, `:terminal`) are kept, with one warning

---

73. Recovering from mistakes

Undo and redo

Keymap	What it does
`u`	Undo the last change
`<Ctrl-r>`	Redo (undo the undo)
`U`	Undo all changes on the current line (rarely used)

Example: on the line `one two three`, `ciw` + `A` + `<Esc>`, `w`, `ciw` + `B` + `<Esc>` give `A B three`. Then:

Keys	Result
`u`	`A two three` (only the last change is undone)
`uu`	`one two three`
`uu` then `<Ctrl-r>`	`A two three` (the first change is redone)
`U`	`one two three` (the whole line is back in one step)
`U` then `u`	`A B three` (`U` is itself a change, so `u` undoes the `U`)

Undo tree (builtin)

Vim's undo history is a tree, not a linear stack. If you undo several times and then make a new edit, the old states aren't lost – they become branches. Neovim's builtin undo tree (`nvim.undotree`) shows this tree.

Keymap	What it does
`<Space>u`	Toggle a 30-column undo tree panel on the far left

Inside the panel just move the cursor (`j`/`k`): the buffer switches to that undo state. Close it with `<Space>u` again or `:q`.

Example of a branch (including the panel, in a real terminal):

1. In an empty buffer press `ione<Esc>`, then `ccTWO<Esc>`: the buffer is `TWO`.
2. Press `u`: the buffer is `one` again.
3. Now `ccTHREE<Esc>`: the buffer is `THREE`. `TWO` is not on the undo path any more (`u` from here goes back to `one`), but it is not lost: it is a side branch of the tree.
4. `g-` (go back in time, builtin) shows `TWO` again. In the panel the fork is drawn like this, and the buffer text follows the row the cursor is on (cursor on the `| *` row: `TWO`; one row up: `one`; on the last row: `THREE`):

```
*    0    (origin)
*    1    (9 seconds ago)
|\
| *    2    (7 seconds ago)      <- TWO, the side branch
*    3    (3 seconds ago)       <- THREE, where you are
```

Time-based undo

Command	What it does
`:earlier 5m`	Restore the file to how it was **5 minutes ago**
`:earlier 1h`	Restore to **1 hour ago**
`:earlier 10`	Undo 10 changes
`:later 5m`	Go forward 5 minutes (redo)
`:earlier 1f`	Go back to the state before the last file save

This works because Neovim stores persistent undo history (the `undofile` option is enabled). Even if you close and reopen a file, you can still undo.

If you accidentally deleted a file

The `auto-save.nvim` plugin saves when you leave a buffer or Neovim loses focus, and Neovim creates backups in `~/.local/share/nvim/backup/`. You may be able to recover from there.

---

74. Discovering keymaps and getting help

See available keybindings with which-key.nvim

Press `<Space>` (the leader key) and **wait about 200ms**. A popup appears showing every available `<Space>+...` keybinding organized by category. A single key is shown with its description, a key that starts several keymaps is shown as `+` and a count. Below is a simplified sketch (the real popup lists many more keys, draws a small arrow glyph instead of `->` and its counts change whenever keymaps are added):

```
w -> save buffer
u -> toggle undo tree
f -> +N keymaps
g -> +N keymaps
j -> +Java
```

You can also press any partial key sequence and wait:

- `g` then wait – shows all `g...` keybindings
- `z` then wait – shows all `z...` keybindings (folding, spelling, etc.)
- `<Ctrl-w>` then wait – shows all window management keybindings
- `"` then wait – shows all registers

Browse all keymaps

Command	What it does
`:Telescope keymaps`	Searchable list of all defined keymaps
`m` on the dashboard	Search keymaps (fzf-lua)
`:map`	Show all mappings (raw output)
`:nmap`	Show normal-mode mappings
`:imap`	Show insert-mode mappings
`:vmap`	Show visual-mode mappings
`:verbose nmap <Space>fg`	Show exactly where a specific mapping was defined (the file; for a mapping written in Lua there is no line number unless Neovim runs with `-V1`)

Real output of `:verbose nmap <Space>fg`:

```
n  <Space>fg   * <Cmd>FzfLua live_grep<CR>
                 Fuzzy grep files
        Last set from .../nvim/init.lua (run Nvim with -V1 for more details)
```

The first line is the mode and the key with what it runs, the second line is its description, the last line is the file that defined it.

Getting help

Command	What it does
`:help keyword`	Open Neovim's built-in help for any topic
`:help ciw`	Help on the `ciw` motion
`:help :substitute`	Help on the substitute command
`<Space>fh`	Fuzzy search help tags
`<Space>?`	Open this user guide as a PDF next to Neovim
`<Space>a`	Ask Claude how to do something in Neovim (vertical split with the skill loaded, always Sonnet)
`K` (on a symbol)	LSP hover documentation

Checking system health

Command	What it does
`:checkhealth`	Diagnose installation issues (LSP servers, providers, etc.)
`:LspInfo`	LSP status: opens `:checkhealth vim.lsp` (clients and configuration)
`:LspAttached`	Popup with the LSP servers attached to this buffer
`:Lazy`	Open the plugin manager
`:messages`	Show recent notification messages

Chapter 3: Navigation

3. Core navigation (moving without the mouse)

All navigation happens in **Normal mode**. Press `<Esc>` first if you are in Insert mode.

Basic cursor movement

Keymap	Description
`h`	Move cursor one character **left**
`l`	Move cursor one character **right**
`j`	Move cursor one line **down** (follows visual/wrapped lines when no count is given)
`k`	Move cursor one line **up** (follows visual/wrapped lines when no count is given)
`5j`	Move 5 lines down (with a count, moves by actual lines, not wrapped lines)
`12k`	Move 12 lines up
`3l`	Move 3 characters right (same as pressing `l` three times)
`4h`	Move 4 characters left (same as pressing `h` four times)

General rule: a number typed before `h`, `j`, `k` or `l` repeats that key that many times. `Nh` moves N characters left, `Nl` N characters right, `Nj` N lines down, `Nk` N lines up.

- It works the same in Visual mode (`v3l` extends the selection by 3 characters, `V3j` selects the current line plus 3 below) and after an operator (`d3l` deletes 3 characters, `d3j` deletes the current line and the 3 below).
- With relative line numbers on, the number shown next to a line is exactly the `N` to use with `Nj` / `Nk` to reach it.

Moving within a line

Keymap	Description
`H`	Jump to **first non-whitespace character** of the line (custom, mapped to `^`)
`L`	Jump to **last non-whitespace character** of the line (custom, mapped to `g_`)
`0`	Jump to the **very first column** (column 0) of the line
`^`	Jump to **first non-whitespace** character (same as `H` in this config)
`$`	Jump to the **end of the line**
`g_`	Jump to the last non-blank character of the line

Where the cursor lands, on the line `    return foo(bar);  ` (4 leading spaces, 2 trailing spaces, cursor on `foo`):

Keys	Cursor lands on
`H` (or `^`)	the `r` of `return` (first non-blank)
`L` (or `g_`)	the `;` (last non-blank, NOT the trailing spaces)
`$`	the last trailing space
`0`	column 1 (a leading space)

With an operator, `dg_` on `foo` deletes up to and including the `;` and leaves `    return`.

Moving by word

Keymap	Description
`w`	Move **forward** to the start of the next word
`b`	Move **backward** to the start of the previous word
`e`	Move **forward** to the end of the current/next word
`ge`	Move **backward** to the end of the previous word
`W`	Move forward to the next WORD (separated by whitespace only, ignores punctuation)
`B`	Move backward to the previous WORD
`E`	Move forward to the end of the current/next WORD

word vs WORD: A "word" stops at punctuation (e.g., `foo.bar` is 3 words: `foo`, `.`, `bar`). A "WORD" only stops at whitespace (e.g., `foo.bar` is 1 WORD).

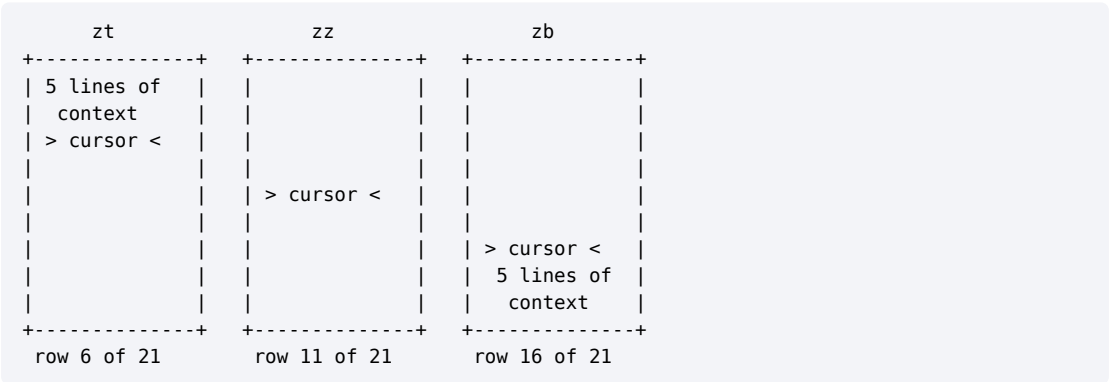
Example on `foo.bar baz` with the cursor on the first `f`:

Keys	Cursor lands on
`w`	the `.` (the next word is the punctuation)
`ww`	the `b` of `bar`
`W`	the `b` of `baz` (the whole `foo.bar` is one WORD)
`e`	the second `o` of `foo` (end of the word)
`E`	the `r` of `bar` (end of the WORD `foo.bar`)

Moving by line/screen

Keymap	Description
`gg`	Jump to the **first line** of the file
`G`	Jump to the **last line** of the file
`42G` or `:42`	Jump to **line 42**
`<Ctrl-d>`	Scroll **half a screen down**
`<Ctrl-u>`	Scroll **half a screen up**
`<Ctrl-f>`	Scroll **one full screen down** (forward)
`<Ctrl-b>`	Scroll **one full screen up** (backward)
`<Ctrl-e>`	Scroll screen down by one line (cursor stays)
`<Ctrl-y>`	Scroll screen up by one line (cursor stays)
`zz`	Center the current line on screen
`zt`	Move current line to the **top** of the screen
`zb`	Move current line to the **bottom** of the screen
`{`	Jump to the previous **blank line** (previous paragraph)
`}`	Jump to the next **blank line** (next paragraph)
`(`	Jump to the beginning of the previous sentence
`)`	Jump to the beginning of the next sentence

Where the cursor line ends up on the screen (a window 21 lines high; `scrolloff` is 5 here, so `zt` and `zb` leave 5 lines of context instead of putting the line on the very edge):



Jumping to matching brackets/parentheses

Keymap	Description
`%`	Jump to the **matching bracket/parenthesis/brace**. If your cursor is on `(`, pressing `%` jumps to the matching `)`, and vice versa. Works with `()`, `[]`, `{}`, and also language keywords like `if`/`endif` (via vim-matchup plugin).

The `matchpairs` option also includes: `<>`, and several CJK bracket pairs.

Examples: on `if (a && (b || c)) {` with the cursor on the first `(`, `%` jumps to the last `)` of the outer pair (the one before `{`). With an operator, `d%` on `x(a, b)y` with the cursor on the `(` gives `xy`.

Jumping to specific characters

Note: The built-in `f` motion has been replaced by the hop.nvim plugin (see Jump Navigation section). The following built-in motions still work:

Keymap	Description
`t<char>`	Jump forward **to just before** the next occurrence of `<char>` on the current line
`T<char>`	Jump backward **to just after** the previous occurrence of `<char>`

`;` is the command key in this config, so it does **not** repeat `t`/`T`. `,` still repeats the last `t`/`T` in the opposite direction.

Jump navigation with hop.nvim (plugin)

Keymap	Mode	Description
`f`	n, x, o	Press `f`, then type 2 characters. All matches on screen get labeled. Press the label letter to jump there instantly. Case insensitive. Press `<Esc>` to cancel.

Example (in a real terminal): the visible text is `the first foo and the second foo end` and the cursor is on the `t` of `the`.

```
visible text:         the first foo and the second foo end
press f, type fo ->   the first aoo and the second soo end     (a label replaces the first
letter of each match)
press s          ->   the cursor jumps to the second foo
```

The label letters depend on the screen (here `a` and `s`), so read them from the screen. After an operator the jump is the range: `d` + `f` + `fo` + `s` on the same text deletes everything from the cursor up to and including the `f` of the second `foo` and leaves `oo end`; the range is not limited to the line (see the note on `f` after an operator in the editing chapter).

Jump history

Keymap	Description
`<Ctrl-o>`	Jump **back** to the previous location in the jump list
`<Ctrl-i>`	Jump **forward** to the next location in the jump list

Every time you use a jump command (like `gg`, `G`, `/search`, `gd`, etc.), your position is saved. You can then go back and forth through your history with these keys.

Example: on line 10 of a 12-line file, press `gg` (line 1), then `G` (last line), then `<Ctrl-o>`: you are back on line 1. `<Ctrl-o>` again: back on line 10, where you started. `<Ctrl-i>`: line 1 again.

Word references (vim-illuminate)

Other uses of the word under the cursor are highlighted when there are at least 2 (from the LSP server, else from Treesitter). It only runs in these file types: bash,

c, cpp, go, java, javascript, json, lua, markdown, nix, python, rust, sh, tex (also plain TeX), toml, typescript, typst, yaml (and the React variants of javascript and typescript). In `.nix` files only the identical word is highlighted (text matching): the nix language server would mark every package of a `with pkgs; [...]` list.

Keymap	Mode	Description
`<Alt-n>`	n	Jump to the next reference of the word under the cursor
`<Alt-p>`	n	Jump to the previous reference
`<Alt-i>`	x, o	Text object: the reference under the cursor (e.g. `d<Alt-i>`)

Example (`<Alt-n>` in a Lua file): with `local Config = {}` on line 6 and `function Config.get(key)` on line 8, put the cursor on `Config` in line 8 and press `<Alt-n>`: the cursor jumps to the `Config` on line 6. Pressed again it goes back to line 8 (it wraps around at the end). Both `Config` words carry the plugin's highlight (read from its extmarks: `IlluminatedWordWrite` on the line 6 one, `IlluminatedWordRead` on the line 8 one).

These three keys are plugin defaults of vim-illuminate: the plugin sets them only when nothing else uses the key, and the config does not define them itself.

Commands: `:IlluminateToggle`, `:IlluminatePause`, `:IlluminateResume`.

Marks (bookmarks)

Keymap	Description
`ma`	Set mark `a` at current cursor position
`` `a ``	Jump to the exact position of mark `a`
`'a`	Jump to the line of mark `a` (first non-whitespace)
`` `" ``	Jump to position where you last edited the file
`` `. ``	Jump to position of last change
`:marks`	List all marks

Marks `a-z` are local to the file. Marks `A-Z` are global (across files).

Example: the file has the lines `a`, `  b c d`, `c`, `d`, `e` and the cursor is on the `c` of line 2 (column 5). Press `ma`, then `5G` (last line). `` `a `` returns to line 2, column 5 (the exact spot); `'a` returns to line 2 but on the first non-blank, the `b` (column 3). Marks also work with an operator: on the lines `a`, `b`, `c` with the cursor on `a`, `ma`, `jj`, then `d`a` deletes from the mark to the cursor and leaves `c`.

---

22. Jump navigation (`hop.nvim`)

Keymap	Mode	Description
`f`	n, x, o	Type `f` then 2 characters: all matches highlight with jump labels. Press the label letter to jump. Case insensitive. `<Esc>` to cancel.

Note: Replaces Vim's built-in `f` motion (`F` is unchanged). `t{char}` jumps forward to just before a character on the current line, `T{char}` backward to just after it. `;` is mapped to `:`, so it does not repeat these motions; use `,` (opposite direction).

---

23. Search lens (`nvim-hlslens`)

Keymap	Description
`n`	Next match with `[x/y]` count overlay
`N`	Previous match with count overlay
`*`	Search the word under the cursor forward as a whole word (the cursor stays on the word; with a count, e.g. `3*`, it jumps 3 matches forward from the cursor, like `3n`)
`#`	Same, backward

Example (in a real terminal): after `/count<Enter>` and `n` in a file where `count` appears 4 times, the line with the second match ends with a small virtual text:

```
local count = 0
count = count + 1  [2/4]        <- the match you are on is the 2nd of 4
return count
```

---

37. Symbol outline (`aerial.nvim`)

The outline comes from Treesitter or the LSP server (no ctags needed).

Keymap	Description
`<Space>t`	Toggle the symbol outline sidebar (functions, classes, methods; the cursor stays in your code)
`[t` / `]t`	Previous / next symbol (only in buffers where aerial is active; there they replace Vim's `:tprevious` / `:tnext` keys)

Commands: `:AerialToggle`, `:AerialOpen`, `:AerialNavToggle`.

What it looks like (in a real terminal, on a Lua file; the sidebar opens on the right and each symbol has a small icon in front of it that depends on the font):

```
code window                              Outline sidebar
local M = {}                           |  (icon) M.load
function M.load(path)                  |  (icon) Config.get
  return path
end
local Config = {}
function Config.get(key)
  ...
```

---

50. Code navigation strategies

This section covers how developers typically navigate code in this setup.

Finding files

Method	Keymap	Best for
Fuzzy file search	`<Space>ff`	When you know part of the filename
Recent files	`<Space>fr`	Files you've worked on recently
File explorer	`<Space>s`	Browsing project structure visually
Buffer search	`<Space>fb`	Switching between already-open files
Buffer pick	`<Space>bp`	Quick switch when you can see the buffer tab
Buffer cycle	`gb` / `gB`	Cycling through open files linearly (`{N}gb` = buffer number N)

Finding code

Method	Keymap	Best for
Project-wide grep	`<Space>fg`	Searching for a string/pattern across all files
Word under cursor	`*` or `#`	Finding all uses of the current word in the file
Go to definition	`gd`	Jumping to where something is defined
Peek definition	`<Space>gd`	Checking a definition without leaving context
Peek references	`<Space>gr`	Seeing everywhere something is used
Buffer tags	`<Space>ft`	Jumping to a function/class in the current file (uses ctags)
Symbol outline	`<Space>t`	Sidebar (aerial) with all symbols in the file; focus stays in the code

Understanding code

Method	Keymap	What you learn
Hover docs	`K`	Function signature, parameters, return type, docstring
Peek references	`<Space>gr`	How and where this symbol is used
Git blame	`<Space>hb`	Who wrote this line, when, and why (commit message)
Diagnostics	`<Space>dd`	What's wrong with this line and where the error comes from
Code outline	`<Space>t`	The structure of the file (classes, functions, methods)
Breadcrumb bar	(automatic)	Current location in the code shown at the top (dropbar)

Refactoring code

Method	Keymap	What it does
Rename	`<Space>rn`	Rename a symbol across the entire project
Code action	`<Space>ca`	Auto-import, extract variable, fix lint issue, etc.
Format	`<Space>fm`	Format the file (Visual mode: the selection) with the formatter of the file type (stylua, prettier, ruff, …); saving with `:w` formats too, `<Space>fo` toggles that. See Formatting (conform.nvim)
Comment/uncomment	`gcc` / `gc` (toggle), `gcs` / `gcr` (comment / uncomment explicitly; `gcss` / `gcrr` for the current line)	Toggle or set comments
Surround	`sa` / `sd` / `sr`	Add/delete/replace quotes, brackets, etc.
Change inside	`ci(` / `ci"` / `ci{`	Change text inside delimiters
Multiple replace	`*` then `ciw` then `n` `.`	Find-and-replace one at a time with control

---

Chapter 4: Editing, text objects, macros and tricks

4. Editing

Entering insert mode for editing

Keymap	Description
`i`	Insert before cursor
`I`	Insert at beginning of line (first non-whitespace)
`a`	Insert after cursor
`A`	Insert at end of line
`o`	Open new line below and enter insert mode
`O`	Open new line above and enter insert mode
`s`	**Note**: `s` is disabled (used by vim-sandwich). Use `cl` instead (delete char + insert).
`S`	Delete entire line content and enter insert mode
`C`	Delete from cursor to end of line and enter insert mode (without polluting register)
`cc`	Delete entire line and enter insert mode (without polluting register)

Deleting text

There is no separate cut key: **delete is cut**. All delete operations (`d`, `dd`, `D`, `x`, `diw`, …) put the text into a register, so you can paste it with `p` (after) or `P` (before). Because `clipboard` gets `unnamedplus` in `lua/options.lua` when a clipboard provider exists, cut text also reaches the system clipboard. Not to be confused with `x` in the file tree (nvim-tree), which cuts a FILE. Exception: `c`, `C`, `cc` (also `c` on a selection) in this config send to the black hole register (they do NOT save to paste register). `S` is NOT redirected: it fills the paste register.

Keymap	Mode	Description
`x`	n	Delete the character under the cursor
`X`	n	Delete the character before the cursor (like backspace)
`dd`	n	Delete (cut) the entire current line
`D`	n	Delete from cursor to end of line
`dw`	n	Delete from cursor to the start of the next word
`db`	n	Delete backward to the start of the previous word
`diw`	n	Delete the word under the cursor (inner word)
`daw`	n	Delete the word under the cursor + surrounding whitespace
`d$`	n	Delete from cursor to end of line
`d0`	n	Delete from cursor to beginning of line
`dG`	n	Delete from current line to end of file
`dgg`	n	Delete from current line to start of file
`5dd`	n	Delete 5 lines starting from current

Example on `The quick brown fox` with the cursor on the `u` of `quick`:

Keys	Result
`x`	`The qick brown fox`
`X`	`The uick brown fox`
`dw`	`The qbrown fox` (from the cursor to the start of the next word)
`db`	`The uick brown fox` (from the start of the word to just before the cursor)
`diw`	`The  brown fox` (both spaces stay)
`daw`	`The brown fox` (the space after the word goes too)
`d$` or `D`	`The q`
`d0`	`uick brown fox`

Copying (yanking) text

Keymap	Mode	Description
`yy`	n	Yank (copy) the current line
`yw`	n	Yank from cursor to start of next word
`yiw`	n	Yank the word under cursor
`y$`	n	Yank from cursor to end of line
`y0`	n	Yank from cursor to beginning of line
`5yy`	n	Yank 5 lines
`<Space>y`	n	Yank the entire buffer (custom)

Note: The clipboard is set to `unnamedplus` whenever a clipboard tool (wl-clipboard, xclip, …) is installed, so yanking also copies to the system clipboard.

Pasting text

Keymap	Mode	Description
`p`	n	Paste after the cursor
`P`	n	Paste before the cursor
`p`	x	Replace the selection with the register (yanky's paste: the replaced text goes into the register, so a second `p` pastes what was replaced; use `"0p` to paste the last yank again)
`<Space>p`	n	Paste on a new line below (custom)
`<Space>P`	n	Paste on a new line above (custom)
`[y`	n	After pasting, cycle to previous yank history entry (needs a paste first)
`]y`	n	After pasting, cycle to next yank history entry (needs a paste first)

yanky.nvim loads right after the first screen (VeryLazy), so every yank of the session is recorded in its history. `p`/`P` (Normal and Visual) are then yanky's paste with a 300 ms highlight of the pasted text, and `[y`/`]y` exist (after a paste).

Example: the Visual `p` swaps (text `one two`):

1. `yiw` on `one` copies `one`.

2. `w`, `viw`, `p` replaces `two` with `one`: the text is now `one one`, and the register now holds `two` (the replaced word).
3. `$p` pastes it: `one onetwo`.

Example: `<Space>p` and `<Space>y`. With the cursor on `foo` in `foo bar`, `yiw` then `<Space>p` puts `foo` on a new line below (the text is now two lines, `foo bar` and `foo`; the cursor stays on line 1). `<Space>y` copies the whole buffer, so on a two-line buffer `<Space>y` then `G p` pastes both lines again below the last one (4 lines).

Changing (delete + enter insert)

`c` (change) deletes text and puts you into insert mode. In this config, `c`/`C`/`cc` do NOT save the deleted text to the paste register (they use the black hole register).

Keymap	Description	Example
`cw`	Change from cursor to end of word	`foo bar` with the cursor on `f` -> `cw`, type `x` -> `x bar`
`ciw`	Change the entire word under cursor	`foo bar` with the cursor on `f` -> `ciw`, type `x` -> `x bar`
`caw`	Change the word + surrounding white-space	`foo bar baz` with the cursor on `b` of `bar` -> `caw`, type `x` -> `foo xbaz` (the space after `bar` is gone too)
`cc`	Change the entire line	`old line` -> `cc`, type `new` -> `new`
`C`	Change from cursor to end of line	`foo bar` with the cursor on `b` -> `C`, type `baz` -> `foo baz`
`c$`	Same as `C`	`foo bar` with the cursor on `b` -> `c$`, type `baz` -> `foo baz`
`c0`	Change from cursor to beginning of line	`foo bar` with the cursor on `b` -> `c0`, type `baz` -> `bazbar`

Example: what `p` pastes after a change. Text `alpha beta`: `yiw` copies `alpha`, then `w`, `ciw`, type `gamma`, `<Esc>`, `$p` gives `alpha gammaalpha`. The `ciw` did not touch the register, so `p` pastes the yanked `alpha`, not the `beta` that was removed. With `dw` instead of `ciw`, the register would hold `beta` and `p` would paste that.

Replacing text

Keymap	Description	Example
`r<char>`	Replace the single character under cursor with `<char>`	`cat` with the cursor on `c` -> `rx` -> `xat`
`R`	Enter **Replace mode** (overtype mode): every character you type replaces the existing one	`cat` with the cursor on `c` -> `R`, type `ab`, `<Esc>` -> `abt`

Undo / redo

Keymap	Description
`u`	Undo the last change
`<Ctrl-r>`	Redo (undo the undo)
`<Space>u`	Toggle Neovim's builtin undo tree in a 30-column panel on the left

Undo breakpoints: Typing `,` `.` `!` `?` `;` `:` in insert mode creates undo checkpoints. This means pressing `u` after a long insert session will undo in smaller chunks instead of reverting everything at once.

Repeating actions

Keymap	Description
`.`	Repeat the last change. Works with most editing commands. Extremely powerful: e.g., `ciw` + type new word + `<Esc>`, then move to another word and press `.` to repeat.

Example: `foo x foo`: `ciw`, type `bar`, `<Esc>`, `ww` (to the second `foo`), `.` -> `bar x bar`. More scenarios are in section 61 (the dot command).

Line operations

Keymap	Mode	Description
`<Alt-j>`	n	Move current line down one position (`3<Alt-j>` moves it 3 lines; re-indented; nothing happens at the last line)
`<Alt-k>`	n	Move current line up one position (count works the same)
`<Alt-j>`	x	Move selected lines down (count works; re-indented; the selection stays)
`<Alt-k>`	x	Move selected lines up
`]<Space>`	n	Insert a blank line below (cursor stays in place; `3]<Space>` inserts 3)
`[<Space>`	n	Insert a blank line above (cursor stays in place)
`J`	n	Join the current line with the next line (cursor stays in place; `3J` joins 3 lines)
`gJ`	n	Join lines without inserting a space (cursor stays in place)
`gS`	n	Toggle split / join of the list, argument list, table, dict or block under the cursor (treesj, Treesitter based)

Examples:

Before	Keys	After
lines `one`, `two`, `three`, cursor on `one`	`<Alt-j>`	`two`, `one`, `three` (the cursor follows the moved line)
lines `1` to `5`, cursor on `1`	`3<Alt-j>`	`2`, `3`, `4`, `1`, `5`
lines `1` to `4`, `Vj` on `1` and `2`	`<Alt-j>`	`3`, `1`, `2`, `4` (the selection stays)
lines `a`, `b`, cursor on `a`	`]<Space>`	`a`, an empty line, `b` (the cursor stays on `a`)
lines `a`, `b`, cursor on `b`	`3]<Space>`	`a`, `b`, then three empty lines
`hello` / `  world`	`J`	`hello world`
`hello` / `  world`	`gJ`	`hello  world` (the indent is kept, no space is inserted)
lines `a`, `b`, `c`, `d`	`3J`	`a b c` and `d`

Split and join code (treesj)

Plugin: **treesj**. Put the cursor inside a list, the arguments of a call, a table, a dict or a block and press `gS`. If the construct is on one line it is split with one item per line; if it is already split over several lines it is joined back to one line (one key toggles both ways). A normal `u` undoes either change.

Key	Mode	Effect	Example
`gS`	n	Toggle split / join of the construct under the cursor	Lua `foo(a, b, c)` with the cursor on `a`: `gS` -> `foo(` / `  a,` / `  b,` / `  c` / `)` (one item per line); `gS` again joins it back to `foo(a, b, c)`

```
foo(a, b, c)   --gS-->   foo(
                           a,
                           b,
                           c
                         )          (the cursor lands on line 2)
```

Join it back with `gS` from any of the lines between the brackets or from the closing `)`; it does not join when the cursor is on the opening `foo(` line.

- It works from the Treesitter syntax tree, so it needs a parser for the file type, and treesj needs a rule ("preset") for that language: it ships rules for many languages (for example Lua, Python, Java, JavaScript / TypeScript, JSON, Nix, Rust, C / C++, YAML, TOML), not for every file type.
- The plugin's own default keys are switched off in `lua/config/treesj.lua`; `gS` is the only one.
- Capital `gS` is treesj; lowercase `gs` is vim-swap's interactive swap (section 65). It loads the first time `gS` is pressed.

Indentation

Keymap	Mode	Description	Example
`>>`	n	Indent the current line to the right	`foo` -> `>>` -> `  foo` (2 spaces: shiftwidth is 2)
`<<`	n	Indent the current line to the left	`  foo` -> `<<` -> `foo`
`>`	x	Indent selection right (stays in visual mode so you can press `>` again)	lines `a` and `b`: `Vj>` -> `  a` / `  b`, still selected
`<`	x	Indent selection left (stays in visual mode)	`  a`: `V<` -> `a`, still selected
`=`	n, x	Auto-indent: fix indentation of the current line or selection	Lua `if a then` / `x` / `end` with the cursor on `x`: `==` -> `x` becomes `  x`
`gg=G`	n	Auto-indent the entire file	the same three Lua lines: `gg=G` -> `x` becomes `  x`

Insert mode shortcuts

Keymap	Description
`<Ctrl-u>`	Convert the current word to UPPERCASE (the word under or right before the cursor; if only spaces separate the cursor from the previous word, that word). You stay in Insert mode.
`<Ctrl-t>`	TOGGLE the case of the FIRST letter of that same word (`foo` -> `Foo`, press again -> `foo`); letters whose case does not round-trip (`ß`, `ı`) and non-letters are left alone; the cursor keeps its place
`<Alt-;>`	Insert a semicolon at the end of the line (without moving cursor)
`<Ctrl-a>`	Jump to the beginning of the line
`<Ctrl-a>` on the `:` command line	Jump to the start of the command line (the same key as in insert mode)
`<Ctrl-e>`	Jump to the end of the line (while the completion menu is open it closes the menu instead)
`<Ctrl-d>`	Delete the character to the right of the cursor
`<Ctrl-w>`	Delete the word before the cursor
`<Ctrl-h>`	Delete the character before the cursor (like backspace)
`<Ctrl-s>`	Show the LSP signature help of the function call you are typing

Examples (all in Insert mode):

Text, cursor position	Keys	After
`say hello`, cursor at the end	`<Ctrl-u>`	`say HELLO` (you stay in Insert mode)
`foo bar`, cursor at the end	`<Ctrl-t>`	`foo Bar`
`foo bar`, cursor right after `foo`	`<Ctrl-t>`	`Foo bar` (the cursor keeps its place)
`x = 1`, cursor anywhere	`<Alt-;>`	`x = 1;` (the semicolon goes to the end of the line; the cursor stays where it was)

Miscellaneous editing

Keymap	Mode	Description
`<Space><Space>`	n	Remove all trailing whitespace from the file (not in Markdown: there it only shows the warning "markdown: trailing spaces are hard line breaks, not stripped")
`<Space>v`	n	Reselect the text that was just pasted
`<Space>cl`	n	Toggle a vertical cursor column highlight
`<Space>cb`	n	Blink the cursor (helps find it on screen)
`~`	n	Toggle case of character(s) (tilde is set as operator, so use `~w` for word, `~e`, etc.)

Example: `<Space><Space>` on the lines `a` plus three trailing spaces and `b` plus a trailing Tab gives `a` and `b`. `~w` on `Hello World` gives `hELLO World`.

---

5. Selection (visual mode)

Press `v`, `V`, or `<Ctrl-v>` to enter visual mode, then use any motion to extend the selection. Once selected, you can act on the selection with `d` (delete), `y` (yank), `c` (change), `>` (indent), `<` (deindent), etc.

Selecting characters

Keymap	Description
`v`	Start character-wise visual selection at cursor
`v` + `h` / `l`	Extend selection left / right by character
`v` + `w`	Extend selection to the next word
`v` + `b`	Extend selection backward to previous word
`v` + `e`	Extend selection to end of current word
`v` + `$`	Extend selection to end of line
`v` + `0`	Extend selection to beginning of line
`v` + `G`	Extend selection to end of file
`v` + `gg`	Extend selection to start of file
`v` + `}`	Extend selection to next blank line

Example: cursor on the `b` of `beta`, then `v` `e` selects the rest of the word (the selection is shown in brackets):

```
alpha beta gamma      alpha [beta] gamma
```

Selecting lines

Keymap	Description
`V`	Start line-wise visual (selects the entire current line)
`V` + `j` / `k`	Extend selection down / up by lines
`V` + `5j`	Select current line + 5 lines below
`ggVG`	Select the entire file

Example: `V` `j` with the cursor anywhere on the first line selects both whole lines:

```
[alpha beta gamma]
[delta zeta omega]
```

Selecting blocks (columns)

Keymap	Description
`<Ctrl-v>`	Start block visual (rectangular selection)
`<Ctrl-v>` + `j` / `k` / `h` / `l`	Extend the block in any direction
`<Ctrl-v>` + `I`	Insert text at the start of every selected line (press `<Esc>` to apply)
`<Ctrl-v>` + `A`	Append text at the end of every selected line
`<Ctrl-v>` + `d`	Delete the selected block
`<Ctrl-v>` + `c`	Change the selected block

This is extremely useful for editing columns of text, adding prefixes to multiple lines, etc.

Example: cursor on the `b` of `beta`, then `<Ctrl-v>` `j` `l` selects a 2x2 rectangle (two characters on each of two lines):

```
alpha [be]ta gamma
delta [ze]ta omega
```

Selecting inside/around delimiters (text objects)

These are the most powerful selection commands. They work with `v` (select), `d` (delete), `c` (change), `y` (yank), and any other operator.

Parentheses, brackets, braces

Keymap	Description
`vi(`	Select everything **inside** `(...)`
`va(`	Select everything **including** `(...)` (the parentheses themselves)
`vib` / `vab`	Select inside / around the nearest enclosing pair of **any** of `()`, `[]`, `{}` (targets.vim; on `{ [ x ] }` with the cursor on `x`, `vib` selects `x`)
`vi[`	Select everything inside `[...]`
`va[`	Select everything including `[...]`
`vi{` or `viB`	Select everything inside `{...}`
`va{` or `vaB`	Select everything including `{...}`
`vi<`	Select everything inside `<...>`
`va<`	Select everything including `<...>`

Quotes

Keymap	Description
`vi"`	Select everything inside `"..."`
`va"`	Select everything including the `"` characters
`vi'`	Select everything inside `'...'`
`va'`	Select everything including the `'` characters
`vi\``	Select everything inside `` `...` ``
`va\``	Select everything including the `` ` `` characters

Words, lines, paragraphs

Keymap	Description
`viw`	Select the word under cursor
`vaw`	Select the word + surrounding whitespace
`viW`	Select the WORD under cursor (delimited by whitespace only)
`vaW`	Select the WORD + surrounding whitespace
`vis`	Select the sentence (inner: without the trailing whitespace)
`vas`	Select the sentence + trailing whitespace (`das`, `dis`, `cis` work the same; the sentence motions are `(` / `)`)
`viS(` / `vaS(`	vim-sandwich "query" objects: type the surrounding character after them (`viS(` selects inside the nearest `(...)`, `vaS"` around the nearest `"..."`; `viS` alone just waits for that character; `diS(`, `daS"`, `ciS[` work in the same way). `ib` / `ab` stay targets.vim's any-bracket objects
`vip`	Select the paragraph (block of non-empty lines)
`vap`	Select the paragraph + surrounding blank lines

Tags (HTML/XML)

Keymap	Description
`vit`	Select everything inside the nearest HTML/XML tag pair
`vat`	Select the entire tag pair including the tags

Example for the text objects above

One sample line, `call(foo, "bar baz", [1, 2])`, and what is selected (and so what `y` would copy):

Cursor on	Keys	Selected text
`foo`	`vi(`	`foo, "bar baz", [1, 2]`
`foo`	`va(`	`(foo, "bar baz", [1, 2])`
`bar`	`vi"`	`bar baz`
`bar`	`va"`	`"bar baz"`
`1`	`vi[`	`1, 2`
`1`	`vab`	`[1, 2]` (the nearest bracket pair of any kind)

Using with operators (d, c, y)

All the `vi` and `va` patterns above work with any operator, not just `v`:

Keymap	Description
`di(`	**Delete** everything inside parentheses
`da(`	Delete everything including the parentheses
`ci"`	**Change** text inside double quotes (deletes it and enters insert mode)
`ca"`	Change including the quotes themselves
`yi{`	**Yank** (copy) everything inside curly braces
`ya{`	Yank including the braces
`di[`	Delete everything inside square brackets
`dit`	Delete everything inside HTML tags
`ci'`	Change text inside single quotes
`dip`	Delete the entire paragraph

On the sample line `call(foo, "bar baz", [1, 2])`:

Cursor on	Keys	Result
`foo`	`di(`	`call()`
`foo`	`da(`	`call`
`bar`	`ci"` + type `X` + `<Esc>`	`call(foo, "X", [1, 2])`
lines `if x {` / `    y` / `}`, cursor on line 1	`yi{`, then `G` `p`	`    y` is copied and pasted below the last line
`old` in `<p>old</p>`	`dit`	`<p></p>`

Select a whole block, from its first line to the closing brace (method, function, if, class)

Cursor anywhere on the line that ends with the opening `{` (for example `int foo(int a, int b) {`):

Keys	Mode	What happens
`$`	Normal	Jump to the last character of the line, the `{`
`V`	Normal	Start line-wise Visual mode
`%`	Visual	Jump to the matching `}` (vim-matchup): every line from the signature line to the closing `}` is selected

Example: a Java class, block of 4 lines on lines 10 to 13, cursor at column 0 of the signature line.

Keys	Selected
`$V%`	lines 10 to 13: the whole method, signature line included
`V%`	only line 10: without `$`, `%` finds the first `(` on the line and jumps to its `)` on the same line
`va{`	lines 3 to 27, the WHOLE CLASS: the cursor is before the method's own `{`, so the nearest `{ }` pair around it is the outer one
`$va{`	lines 10 to 13, but character-wise from the `{`: the signature text before the `{` is not selected
`$V%ok`	lines 9 to 13: `o` jumps to the other end of the selection, `k` extends it one line up (use it to include an annotation line such as `@Test`; a doc comment needs several `k`)

Notes:

- In Normal mode `$` is the normal end-of-line. Only in Visual mode `$` is remapped to `g_` (last non-blank character); here `$` is pressed before `V`, so it is the Normal one. `%` is provided by vim-matchup.
- Works for any `{ }` block (function, class, `if`, `for`, a JSON or CSS block). For a `( )` or `[ ]` block that opens at the end of a line, the same idea works with that bracket.
- If the line has trailing spaces after the `{`, `$` lands on a space and `%` does not find the bracket: remove the spaces or use `g_` instead of `$`.
- Without braces (Python): use the indent objects `ii` / `ai` in More text objects. `V%` on `if` … `end` keywords works through vim-matchup.
- Cursor INSIDE the block: `va{` / `vaB` selects the braces of that level only (character-wise, not the signature line). `[{` jumps to the enclosing opening `{`; from there continue with `V%` because you are already on the `{`.

What to do with the selected block (the selection from `$V%` stays active until you press one of these):

Keys	Effect
`y`	Copy the block; move the cursor, then `p` pastes it BELOW the cursor line (after `$V%y<Esc>`, `G`, `k`, `p` the 4 lines appeared a second time, 27 to 31 lines)
`d`	Delete the block (27 to 23 lines)
`<Alt-j>` / `<Alt-k>`	Move the whole block down / up ONE line per press; the selection stays, so press again
`5<Alt-j>`	Move the block 5 lines: type the count BEFORE the key, see Line operations
`>` / `<`	Indent / outdent the block (the line count is unchanged)
`gc`	Comment the whole block out, see vim-commentary
`=`	Re-indent the block
`J`	JOINS all lines of the block into ONE line (4 lines to 1, 27 to 24): usually not what you want, a trap while the selection is still active
`<Esc>`	Cancel the selection

Treesitter node selection (builtin)

Neovim 0.12 can grow and shrink a selection along the syntax tree (needs a Treesitter parser; otherwise it uses the LSP selection range):

Keymap	Mode	Description
`an`	x, o	Select the parent (outer) node: press `v`, then `an` repeatedly to grow the selection
`in`	x, o	Select the child (inner) node: shrinks the selection again
`]n` / `[n`	x	Select the next / previous node
`]N` / `[N`	x	Select the next / previous sibling node

Example: Lua line `local x = foo(a, b)`, cursor on `a`, then `v` and `an` pressed repeatedly (the selection after each press):

Keys	Selected
`v`	`a`
`an`	`(a, b)`
`an`	`foo(a, b)`
`an`	`x = foo(a, b)`
`an`	`local x = foo(a, b)`

`in` goes back one step (after three `an` presses, `in` shows `foo(a, b)` again).

More text objects

Keymap	Mode	Description
`ii` / `ai`	x, o	The current indent scope (mini.indentscope); `ai` includes its border lines. `[i` / `]i` jump to the top / bottom of the scope
`i%` / `a%`	x, o	Inside / around a matching pair, also keywords like `if` … `end` (vim-matchup)
`ia` / `aa`	x, o	Inside / around a function argument (targets.vim: on `b` in `f(a, b, c)`, `daa` gives `f(a, c)`)
`iq` / `aq`	x, o	Inside / around the nearest quotes of any kind (targets.vim)
`i,` / `a,`	x, o	Between separators such as `,` `;` `:` `+` `-` `=` `/` `|` `&` (targets.vim)
`<Space>iB`	x, o	The whole buffer (`y<Space>iB` copies everything)
`<Space>iu`	x, o	The URL under the cursor (`d<Space>iu`)
`<Alt-i>`	x, o	The LSP/Treesitter reference under the cursor (vim-illuminate; the key is the plugin's default, not set in this config)

Examples:

Before	Keys	After
`f(a, bb, c)`, cursor on `bb`	`daa`	`f(a, c)`
`say 'hi there' now`, cursor on `say`	`ciq` + type `X` + `<Esc>`	`say 'X' now`
`x(a, b, c)`, cursor on `b`	`di,`	`x(a,, c)` (the item between the separators goes, with the space before it)
lines `x`, `y`	`y<Space>iB`, then `G` `p`	the whole buffer is pasted again below (4 lines)
`see https://x.org/a now`, cursor on the URL	`d<Space>iu`	`see  now`
Python `def f():` / `    a = 1` / `    b = 2` / `x = 3`, cursor on `a = 1`	`dii`	`def f():` / `x = 3` (only the indented body goes)
the same lines	`dai`	the buffer is empty: the border lines count, so the `def f():` line and the first line below the block (`x = 3`) go too

Careful with `cia`: it also takes the space in front of the argument (`f(a, bb, c)` with the cursor on `bb`: `ciaX` gives `f(a,X, c)`), so type the space again or use `ciw` for a plain word.

Markdown code block text objects

In markdown files only:

Keymap	Description
`vic`	Select inside a fenced code block (the object is `ic`, so `dic`, `yic` and `cic` work too)
`vac`	Select the code block including the fences (the object is `ac`: `dac`, `yac`, `cac`)

Worked examples: Text objects and operators in the Markdown chapter.

Precision selection: from the cursor to an exact spot

How to select (or delete/copy/change) from the cursor to a specific character, word, or place, including on another line.

Golden rules

- The character **under the cursor is always included** in a visual selection, both at the start and at the end.
- Everything below works after `v`. Most of it also works after an operator (`d`, `y`, `c`) in place of `v`: for example `vt)` -> `dt)`, `yt)`, `ct)`. Exceptions are noted below.
- A *selection* can only be one continuous area. You cannot select two separate places at once (see "Non-contiguous lines" below).

Quick lookup

I want to select from the cursor to…	Keys
the end of the line	`vL` (or `v$`)
just **before** the next `X` on this line	`vtX`
including the next `X` on this line	`vtXl`
just before the 2nd `X` on this line	`v2tX`
the next `X` anywhere (crosses lines)	`v/X<Enter>` (lands on the start of the match)
the **end** of the next word/text `foo`	`v/foo/e<Enter>`
the 2nd `foo` after the cursor, anywhere in the file	`v2/foo/e<Enter>`
something visible on screen, no counting	`v` `f` `xx` `<label>`, then adjust with `l`/`h`/`e`
a target N lines below, count unknown	`Nj`, `L`, `?foo<Enter>`, `viw`

End of line: `L`

- `L` is mapped to `g_` (last **non-blank** character of the line) in normal and visual mode; `H` is the first non-blank.
- `vL` selects from the cursor through the last character of the line (for example a trailing `;`). `v$` does the same (`$` is remapped to `g_` in visual mode only, so the newline is not selected).
- The mapping covers normal and visual mode only, **not operator-pending**. After an operator, `L` is the built-in "bottom of the screen" motion and works on whole lines. **Do not use `dL` / `yL` / `cL`**: `dL` deleted whole lines from the current one to the bottom of the window. Use `dg_` / `yg_` / `cg_` (`dg_` deletes from the cursor through the last non-blank character), or `vL` then `d` / `y`.
- With a count, `2L` goes to the end of the line below.

`t` and `T`: stop just before a character

- `t<char>` jumps forward to the character **just before** the next `<char>`. `T<char>` does the same backward and stops just **after** it.
- It only searches the **current line**. If `<char>` is not on the line, nothing happens.
- It finds the next occurrence, not the last. The character under the cursor is not counted as a match.
- To **include** the target, add `l` after it: `vt:l` (up to and including the next `:`).
- `f` cannot be used for this, because `f` is hop.nvim here (it asks for 2 characters and shows labels). `vf:` does not do what you expect. After an operator `f` is still hop, but it is **not limited to the current line**: see the `df,` note below.
- Example: `a, b, c` with the cursor on `a`: `dt,` -> `, b, c` (everything up to, not including, the first comma).
- Works with operators: `dt)` deletes up to before `)`, `yt"` copies up to before `"`, `ct)` changes up to before `)`. To include the target character with an operator, use `v` first: `vt)l` then `d`.
- **`f` after an operator (`df,`, `cf,`) is hop, not the built-in `f`**: `df,` deleted from the cursor through a comma several lines below (it was the only comma in the visible text), so it is **not limited to the current line** and the target character is **deleted too**. `cf,` deleted the same kind of range (with one match on screen: `cf,` deleted the same range as `df,` and left you in Insert mode; with several matches you pick the label first). With several matches on screen, hop shows labels and you choose the target, so the deleted range depends on the label you press; check what you typed before pressing `d`/`c` on a big range (`u` undoes it).

Counting: `v2tX` is not `2vtX`

- Put the count **after** `v`, directly before `t`: `v2to` stops just before the 2nd `o` on the line. `v3t,` stops before the 3rd `,`.
- `2vt` is a **different** thing: a count typed before `v` reselects a region of that size, it does not mean "2nd occurrence".
- Counts also work with operators: `d2to`, `y2t)`, `c3t,`.
- `T` counts backward: `v2T"`.
- If the line has fewer occurrences than the count, nothing is selected or moved.

`)` and `(` are sentence motions, not parentheses

- `)` jumps to the **start of the next sentence**, and `(` to the start of the current/previous one. They do **not** mean "closing parenthesis". The search is not limited to the current line, so they can cross many lines.

What counts as the end of a sentence (standard Vim rules):

- A `.`, `!` or `?` that is followed by a **space, a tab or the end of the line**. Closing characters `)`, `]`, `"` or `'` may sit between the `.`/`!`/`?` and that space (for example `done.") Next` still ends a sentence).
- A **blank line** also separates sentences (it is a paragraph boundary).
- A `.` with **no space after it does not count**: `obj.method()` or `file.txt` do not end a sentence. In code, only a `.`/`!`/`?` followed by a space or at the end of a line counts (for example inside a string or a comment).

Where the cursor lands: on the **first non-blank character of the next sentence**, which is the first character after the terminator and the spaces that follow it.

Where a selection ends: in Visual mode the character under the cursor is included, so `v)` selects from the cursor **through that first letter of the next sentence** (the spaces before it are selected too). With an operator (`d)`, `y)`) the motion stops just before that letter and does not include it (with `d)`: with the cursor on the `t` of `there`, it deleted `there. ` and the cursor ended on the `H` of `How`; `y)` yanks `there. `).

Concrete example. Text on one line, cursor on the `t` of `there`:

```
Hello there. How are you? Fine!
```

Keys	What is selected / where the cursor ends
`v)`	`there. H`: ends on the `H` of `How`, the first letter of the next sentence (the `. ` after `there` is the boundary)
`v))`	`there. How are you? F`: ends on the `F` of `Fine`
`v(`	from the start of `Hello` up to the `t`, because `(` goes back to the start of the current sentence

On real code: starting inside a line of code, `v)` ran across several lines and stopped on the first letter after a `. ` that was inside a string literal (`"... text. Next ..."`, it stopped on the `N`), because that `. ` counts as a sentence boundary even in code. All three rows of the table above gave exactly the results listed with the example text. Also, on a line like `see file.txt now. Next one.` with the cursor on `see`, `v)` selected `see file.txt now. N`, so the `.` in `file.txt` (no space after it) did not stop it.

Tip when testing these: type the sequence in one go (`v))`, not `v` and then `)` as separate steps). Pressing `v` again while already in Visual mode leaves Visual mode, and the which-key popup that appears after `v` is only a help list.

- To reach a closing parenthesis: `vt)l` (up to and including it), or `v/)<Enter>`.
- `%` jumps between a bracket and its match; it will not take you to a closing bracket from a plain character (it first finds the next bracket on the line and jumps to its partner, which can be backward).
- `va(` / `vi(` select the whole group / its inside, but always starting from the opening `(`, never from the cursor. Thanks to targets.vim the cursor may also be in front of the pair on the same line (`vi(` on `foo (bar) baz` with the cursor on `f` selects `bar`).

Search as a selection motion (crosses lines)
Searching with `/` (forward) or `?` (backward) works as a motion after `v`:

Keys	The selection ends at
`v/foo<Enter>`	the **first** character of the next `foo` (included)
`v/foo/e<Enter>`	the **last** character of `foo` (included)
`v/foo/e-1<Enter>`	one character before the end of `foo`
`v/foo/e+1<Enter>`	one character after the end of `foo`
`v2/foo<Enter>`	the start of the 2nd `foo` after the cursor
`v2/foo/e<Enter>`	the end of the 2nd `foo` after the cursor

- The text after the second `/` (`e`, `e-1`, `e+1`) is a search **offset**; `e` means "end of the match".
- **The count counts matches, not lines.** `2/foo` means "the 2nd `foo` after the cursor, wherever it is". It does not care on which line a match is. If a line you expected has no `foo`, the count simply moves on to the next match.
- Because it counts matches, it also works **inside a single line**: if a line contains `foo` twice, `v2/foo/e<Enter>` selects up to the end of the second `foo` on that same line (on a line with `Nome` and `nome`, the selection ended on the last letter of the second match, `nome`). (Unlike `t`, which also works on one line, search lets you target a whole word or phrase, not just one character.)
- If there are fewer matches than the count, the search **wraps** to the top of the file (`wrapscan` is on by default and this config does not change it) and keeps counting, so you can end up before the cursor. Check the hlslens `[n/total]` overlay.
- `ignorecase smartcase` is on, so a lowercase pattern (`foo`) also matches `Foo`. Type a capital letter to make it case-sensitive. (This is for `/` and `?`; `:s` and `:g` always ignore case here, see the Substitution section.)
- `/` is not remapped in this config.
- With operators, `d/foo<Enter>` deletes up to (not including) the match; `d/foo/e<Enter>` includes the last letter of the match (with the cursor on the start of `two words`, `d/words/e<Enter>` deleted everything up to and including the last letter of `words`).

hop.nvim: select to something you can see (no counting)

1. `v` starts the selection at the cursor.
2. `f` then type **2 characters**. Every matching place on screen gets a label (case insensitive).
3. Press the label letter. The cursor jumps there, and the selection now ends **on the first character of that match**, included.
4. Extend or trim with a normal motion:

Key	Effect
`l`	extend the selection one character **to the right** (`3l` = three characters)
`h`	move the end one character **to the left**: use it to pull back if you went one too far, or to extend leftward if the target is before the start
`e`	extend to the end of the word (no need to count its letters)
`b`	move back to the start of the word

Example: to select from the cursor to the end of a 4-letter word you can see: `v`, `f`, the word's first 2 letters, the label, then `lll` (first letter + 3 more) or simply `e`.

- Hop only reaches text that is **visible on screen**.
- Two-letter patterns match many places (for example `no` in `non`, `nome`, `Nome`), so look at the labels and pick the right one.

Target on another line, count unknown: use the relative numbers
With relative line numbers the gutter shows how far each line is from the cursor (the current line shows its absolute number).

```
  3   foo
  2   bar
  1   baz
 12   the cursor line shows its absolute number
  1   target
```

Here `1j` (or just `j`) moves onto `target`, and `3k` would move onto `foo`.

1. `Nj` moves down `N` lines, where `N` is the number shown next to the target line (with a count, `j` moves real lines, not wrapped ones). Use `Nk` to go up.
2. `L` goes to the end of that line.
3. `?foo<Enter>` searches **backward** from the end of the line, which finds the **last** `foo` on that line (in a real terminal). In general `?foo` goes to the start of the `foo` the cursor is inside; from the first letter of a `foo` it goes to the previous `foo`. (Searching forward from the middle of the line could hit an earlier, unwanted match or a capitalised one.)
4. `viw` selects the word, or `ve` selects from the match start to the word end.

Related: `V3j` selects the current line and 3 below; `d3j` deletes 4 lines; `10G` or `:10` (Enter) jumps to absolute line 10.

Typing before or after: `i` `a` `I` `A`

Key	Where the typed text goes
`i`	**before** the character under the cursor
`a`	**after** the character under the cursor
`I`	before the **first non-whitespace** character of the line (not column 0: on an indented line it goes after the indentation; for the true start use `0` then `i`)
`A`	after the **last character** of the line (the real end, even past trailing spaces)

Typing at the absolute start of an indented line (before the indentation):

Keys	Where you start typing
`I`	after the indentation, before the first non-blank character
`0` then `i`	at column 0, before the indentation (**preferred**)
`gI`	the same as `0` then `i`, in one key (standard Vim; not remapped in this config)

- `0` is remapped to `g0` (start of the *screen* line), but `set nowrap` is on, so lines never wrap and it is the same as the real column 0.
- `gI` and `0` then `i` both start typing at column 0, before the indentation.
- These only insert from **Normal mode**. In Visual mode `i` and `a` do not insert: they start a text object (`iw`, `i(`, `aw`…). Press `<Esc>` first.

- **Typing after a selection that ends at the end of the line** (for example after `v$` or `vL`): press `<Esc>` (the cursor stays on the last character), then `a`. Using `i` would put the text *before* that last character (for example before a final `;`).
- **Shortest way to type at the end of the line**: `A`, from anywhere on the line, with no selection needed.
- `L` and `g_` stop on the last **non-blank** character, so `<Esc>` + `a` after `vL` types right after the last visible character, before any trailing spaces. `A` goes after the trailing spaces.

Non-contiguous lines (for example line 3 and line 10 together)
Not possible. Vim has no selection of separate pieces, and this config has no multi-cursor plugin (`vim-visual-multi` is commented out in `lua/plugin_specs.lua`). Do it in steps:

- **Repeat with `.`**: do the edit on one line, jump to the other, press `.`. Mind that deleting a line shifts the numbers below it.
- **Ex commands with line numbers**: `;3d` then `;9d`, or `;3,10d` for the whole range 3 to 10 (`;3,10d` deleted lines 3 to 10 **inclusive**, 8 lines).
- **Bring them together**: `;3m10` moves line 3 **below** line 10, `;3t10` copies line 3 below line 10. Then select both with `V`. Example: `;4m11` moved line 4 (an `import` line) to just below line 11, and `;4t11` copied line 4 below line 11 and kept the original in place. The line numbers refer to the file **before** the move, and afterwards the cursor sits on the moved/copied line.
- **The numbers are absolute file line numbers**, not relative to the cursor, and it does not matter where the cursor is. With relative numbers on, only the cursor line shows its absolute number in the gutter; move onto a line to read it. Line 1 counts even if it is blank (a file that starts with an empty line has its first real line at number 2).
- **Relative addresses**: `.` is the current line and `+N` / `-N` are N lines after/before it, so they match the relative numbers in the gutter. Example: `;.t.` duplicated the current line and left the cursor on the new copy. Also: `;.m+2` moves the current line to below the line 2 lines further down, and `;.,+3d` deletes the current line and the next 3 (4 lines in total).
- **Where `m` puts the line (careful with `-N`)**: `m` always puts the line **below** the target line. So `;.m-3` puts the current line **below** the line 3 above it, not above it. To put a line **above** line X use the target line number minus 1 (above line 7: `;.m6`; with a relative address `;.m-4` goes above the line 3 above), and `m0` puts it at the very top of the file. The rule works as described, including `m0` for the top of the file and target number minus 1 to land above a line.
- **Copy with a relative address**: `;.t+2` copies the current line below the line 2 lines further down (`t` copies, `m` moves).
- **Move a selection of whole lines**: `V`, then `j` / `k` to extend the selection, then `:` (the command line fills with `:'<,'>`), type `m<target>` (for example `m10`), then `<CR>`. The whole selection goes below the target line, keeping its order; the cursor ends on the last moved line. Use an **absolute** target number. A relative target such as `m+2` after a range is counted from the cursor line (the end of the selection where the cursor stood), which is easy to misjudge, so prefer the absolute number.

- `m` **only moves whole lines.** For part of a line: select it (`v` + a motion, for example `viw` for a word), `d` (cut), move to the new spot (`5j`, or hop with `f`), then `p` (paste after the cursor) or `P` (before). See "Deleting text" for why `d` is a cut.
- **Blank lines hide the effect**: moving or copying a blank line next to another blank line changes nothing you can see. Use a line with text to check `m` and `t`.
- **Same spot on adjacent lines**: `<Ctrl-v>` block mode, but only for adjacent lines (see the Visual Block Editing section).
- **Matching by content instead of number**: the `:g` command (see its section).

Line range yanking (command mode)

Command	Description
`:-5,+10yank`	Yank from 5 lines before to 10 lines after cursor
`:2,10yank`	Yank from line 2 to line 10 (absolute)

---

6. Working with parentheses, quotes, and brackets

This section covers everything about matching, jumping to, selecting inside, changing, adding, and removing surrounding characters.

Jumping to matching pair

Keymap	Description
`%`	Jump between matching `()`, `[]`, `{}`, `<>`, and language keywords. The vim-matchup plugin extends this to work with `if`/`else`/`end`, `do`/`while`, `try`/`catch`, etc. If the match is offscreen, a popup shows the matching line.

Selecting inside/around pairs

See the full table in the Selection section above (with a sample line and the selected text for each key, under "Example for the text objects above"). Quick summary:

Pattern	Inside	Around (including delimiters)
Parentheses `()`	`vi(`	`va(`
Braces `{}`	`vi{` or `viB`	`va{` or `vaB`
Brackets `[]`	`vi[`	`va[`
Angle brackets `<>`	`vi<`	`va<`
Double quotes `""`	`vi"`	`va"`
Single quotes `''`	`vi'`	`va'`
Backticks ``` `` ```	`vi\``	`va\``
HTML/XML tags	`vit`	`vat`
Nearest of `()` `[]` `{}` (targets.vim)	`vib`	`vab`

Changing text inside pairs

Keymap	Description	Example
`ci(`	Delete everything inside `()` and enter insert mode to type replacement	`f(old)` with the cursor in `old` -> `ci(`, type `new` -> `f(new)`
`ci"`	Delete everything inside `""` and enter insert mode	`a "old" b` with the cursor in `old` -> `ci"`, type `new` -> `a "new" b`
`ci{`	Delete everything inside `{}` and enter insert mode	`{old}` -> `ci{`, type `new` -> `{new}`
`ci[`	Delete everything inside `[]` and enter insert mode	`[old]` -> `ci[`, type `new` -> `[new]`
`ci'`	Delete everything inside `''` and enter insert mode	`a 'old' b` with the cursor in `old` -> `ci'`, type `new` -> `a 'new' b`
`cit`	Delete everything inside an HTML tag and enter insert mode	`<p>old</p>` with the cursor in `old` -> `cit`, type `new` -> `<p>new</p>`

Deleting text inside pairs

Keymap	Description	Example
`di(`	Delete everything inside `()` (parentheses remain empty)	`f(old)` with the cursor in `old` -> `f()`
`di"`	Delete everything inside `""`	`x "y" z` with the cursor on `y` -> `x "" z`
`di{`	Delete everything inside `{}`	`{old}` -> `{}`
`da(`	Delete everything including the `()` themselves	`f(old) x` with the cursor in `old` -> `f x`
`da"`	Delete everything including the `""` themselves	`x "y" z` with the cursor on `y` -> `x z`

Adding surrounding pairs (vim-sandwich plugin)

The `sa` command adds surrounding characters. `s` key alone is disabled (use `cl` instead).

Keymap	Description	Example
`saiw"`	Add double quotes around the current word	`hello` becomes `"hello"`
`saiw(`	Add parentheses around the current word	`hello` becomes `(hello)`
`saiw{`	Add curly braces around the current word	`hello` becomes `{hello}`
`saiw[`	Add square brackets around the current word	`hello` becomes `[hello]`
`saiw'`	Add single quotes around the current word	`hello` becomes `'hello'`
`sa$"`	Add quotes from cursor to end of line	`hello world` with the cursor on `w` becomes `hello "world"`
(visual) `sa"`	First select text with `v`, then `sa"` adds quotes around selection	`foo bar`: `vee` selects both words, then `sa"` gives `"foo bar"`

Removing surrounding pairs (vim-sandwich plugin)

Keymap	Description	Example
`sd"`	Delete surrounding double quotes	`"hello"` becomes `hello`
`sd'`	Delete surrounding single quotes	`'hello'` becomes `hello`
`sd(`	Delete surrounding parentheses	`(hello)` becomes `hello`
`sd{`	Delete surrounding curly braces	`{hello}` becomes `hello`
`sdb`	Delete the surrounding pair without naming it (`()`, `[]`, `{}` or quotes). With the cursor on a letter it takes the INNERMOST pair; with the cursor ON a bracket or quote character it can pick the outer pair (see the note below)	`[hello]` becomes `hello`
`sd[`	Delete surrounding square brackets	`[hello]` becomes `hello`

Replacing surrounding pairs (vim-sandwich plugin)

Keymap	Description	Example
`sr"'`	Replace `"` with `'`	`"hello"` becomes `'hello'`
`sr({`	Replace `()` with `{}`	`(hello)` becomes `{hello}`
`sr{[`	Replace `{}` with `[]`	`{hello}` becomes `[hello]`
`sr'(`	Replace `'` with `()`	`'hello'` becomes `(hello)`
`srb'`	Replace the surrounding pair without naming it (same pair choice as `sdb`)	`"hello"` becomes `'hello'`

Nested pairs, `b` and exactness (Java buffer, line `names.add("apple");`):

- Cursor on a letter of `apple`: `sd"` gives `names.add(apple);`, `sd(` gives `names.add"apple";`, `sdb` gives `names.add(apple);` (the innermost pair, the quotes), `srb[` gives `names.add([apple]);` (the quotes became brackets).

- Cursor ON a quote character or on a parenthesis: `sdb` gave `names.add"apple";` (the PARENTHESES were removed, not the quotes). The same happens with the cursor on the closing `)`.
- A hands-on session reported the parentheses removed with the cursor "on `apple`". That was not reproduced with the cursor on a letter of `apple` (every column of `apple` removed the quotes), so check the exact cursor column first.
- With nested pairs `b` is therefore not a safe guess: name the pair (`sd"`, `sd(`, `sr"'`, `sr({`) when it matters. `b` is fine for a single pair.

More round trips on `names.add("apple");` with the cursor on `apple`: `sd"` then `saiw"` restores `"apple"`; `sr"'` gives `'apple'` and `sr'"` goes back; `sr({` gives `names.add{"apple"};` and `sr{(` goes back.

Wrap a whole list or part of it

Line `String.join(", ", "second", "first", "third")` with the cursor on `second`:

Keys	Result
`vi(` then `sa(`	`String.join((", ", "second", "first", "third"))`: the whole argument list is wrapped in a second pair of parentheses
`sai((`	the same result in one go
Select exactly `"second", "first"` in Visual mode (`v` on the first quote, move to the last quote of `"first"`), then `sa(`	`String.join(", ", ("second", "first"), "third")`

Check that the Visual highlight covers the LAST character you want inside the pair: a selection that stops one character short (before the comma) gave `("second", "first",) "third"`, and one that also takes the space after the last comma gave `("second", "first", )"third"`.

Auto-pairing (nvim-autopairs plugin)

When typing in insert mode, opening characters automatically insert their closing pair:

- Type `(` and `)` appears: `(|)` (cursor between them)
- Type `"` and closing `"` appears: `"|"`
- Type `{` and `}` appears: `{|}`
- Type `[` and `]` appears: `[|]`
- Pressing `<BS>` right after typing `(` removes both brackets (`(` `<BS>` `x` gives `x`).

Tabbing out of pairs (custom, `lua/tabout.lua`)

In Insert mode `<Tab>` jumps the cursor out past the closing character of the pair you are in, so you do not have to leave Insert mode or reach for the arrow keys. You stay in Insert mode and keep typing. It reads the text of the current line only (no language parser needed), so a pair has to open and close on the same line.

Keys (Insert mode)	What it does	Example (`|` is the cursor)
`<Tab>`	Jump past the closing character of the innermost pair around the cursor	`foo("ab|")`: `<Tab>` gives `foo("ab"|)`, a second `<Tab>` gives `foo("ab")|`
`<S-Tab>` (Shift + Tab)	Jump back to before the opening character of the innermost pair around the cursor (or the one you just left)	`foo("ab")|`: `<S-Tab>` gives `foo|("ab")`
`<Tab>` with nothing to jump out of	Inserts a normal tab / indent as before	an empty line: `<Tab>` indents

Pairs known in every file type: `( )`, `[ ]`, `{ }`, `" "`, `' '` and `` ` ` ``. An apostrophe inside a word (`don't`) is not treated as a quote. A backslash protects the next character (`"a\"b"` is one string).

Extra pairs per file type:

File type	Extra pairs
Lua	`[[ ]]`, `[=[ ]=]`
Python	`""" """`, `''' '''`; `{ }` inside a string (f-string fields)
Nix	`'' ''` (indented string), `${ }` also inside strings
Shell (bash, zsh)	`${ }`, `$( )`, `$(( ))`, `[[ ]]`
fish	`$( )`
LaTeX	`$ $`, `$$ $$`, `\( \)`, `\[ \]`, `\left( \right)`, `\left[ \right]`
Markdown	`** **`, `~~ ~~`, `* *`, `_ _` (the marker must touch the text: `snake_case` and `2 * 3` are left alone)
Typst	the Markdown ones plus `$ $`
C++ and Java	`< >` for templates, generics and `#include <x.h>`, but not for a comparison like `a < b`

Good to know:

- With the completion menu open, `<Tab>` selects the next menu item instead (section 14). Close the menu with `<Esc>` or `<Ctrl-e>`, then press `<Tab>`.
- `<S-Tab>` does not select the previous menu item; use `<Ctrl-p>` for that.
- In Select mode (a snippet placeholder is highlighted) `<Tab>` and `<S-Tab>` do nothing; jump between placeholders with `<Ctrl-j>` / `<Ctrl-k>` (section 15).
- Normal mode `<Tab>` is not changed.
- A pair left open on the line (`foo(a`) has nothing to jump out of, so `<Tab>` inserts a tab.

---

16. Code commenting

vim-commentary (plugin)

Keymap	Mode	Description	Example
`gcc`	n	Toggle comment on current line	Lua `x = 1` -> `gcc` -> `-- x = 1`; `gcc` again removes the marker
`gc` + motion	n	Toggle comment on a motion (e.g., `gcip` comments a paragraph)	Lua lines `a` / `b` with the cursor on `a`: `gcip` -> `-- a` / `-- b`
`gc`	v	Toggle comment on selected lines	Lua lines `a`, `b`, `c`: `Vjgc` -> `-- a`, `-- b`, `c`
`gc`	o	Comment text object: `dgc` deletes the comment block under the cursor, `ygc` yanks it	Lua lines `-- a`, `-- b`, `c` with the cursor on `-- a`: `dgc` -> `c`; `ygc` copies the two comment lines
`gcu`	n	Uncomment the adjacent commented lines	Lua lines `-- a`, `-- b`, `c` with the cursor on `-- a`: `gcu` -> `a`, `b`, `c`
`:[range]Commentary`	cmd	Toggle comment on a range (`:2,3Commentary`)	Lua lines `a`, `b`, `c`: `:2,3Commentary` -> `a`, `-- b`, `-- c`

vim-commentary loads right after the first screen (VeryLazy), so these commands and the `gc` text object exist from then on.

More examples (Python buffers; `gcc` and `gc` are plain vim-commentary: they use the filetype's comment marker and do not look inside embedded languages, so use the smart `gcs` / `gcr` below there):

Before	Keys	After
`a = 1` / `b = 2`	`gcc` on line 1	`# a = 1` / `b = 2`
`a = 1` / `b = 2` / blank / `c = 3`	`gcip`	`# a = 1` / `# b = 2` / blank / `c = 3`
`# a = 1` / `# b = 2` / blank / `c = 3`, cursor on line 1	`gcu`	`a = 1` / `b = 2` / blank / `c = 3`
the same	`dgc`	`c = 3` only (the comment block is deleted; the blank line goes with it)

Java example (in a Java buffer; the Java marker is `//`):

Keys	Result
`gcc`	the current line becomes `// ...`; `gcc` again removes it
`Vjjjgc` (`V`, then `3j`, then `gc`)	4 lines are toggled to `// ...` and Visual mode ends
`gcu` with the cursor on the first commented line	the adjacent commented lines are uncommented
`:22,25Commentary`	lines 22 to 25 are toggled; the same command again toggles them back

These keys only toggle. A range that mixes commented and uncommented lines was not tried here and may surprise you, so check the lines afterwards (or use `gcs` / `gcr` below, which add and remove explicitly).

Smart commenting (custom)

String-aware, multi-line-capable comment add/remove (`lua/smart_comment/`).

Keymap	Mode	Description	Example
`gcs` + motion	n	Comment the rows a motion covers (`gcsip`, `gcs3j`, `gcsG`); `.` repeats	Lua lines `a`, `b`, `c`, `d`, `e`: `gcs3j` -> `-- a`, `-- b`, `-- c`, `-- d`, `e`
`{count}gcs`	n	Comment count rows from the cursor down (`200gcs`)	Lua lines `a`, `b`, `c`: `2gcs` -> `-- a`, `-- b`, `c`
`gcss`	n	Comment current line(s) (`3gcss` = 3 rows); `.` repeats	Lua lines `a`, `b`: `gcss` -> `-- a`, `b`
`gcs`	x	Comment the selected rows	Lua lines `a`, `b`, `c`: `Vjgcs` -> `-- a`, `-- b`, `c`
`gcr` + motion	n	Uncomment the rows a motion covers (`gcrip`, `gcr200j`); `.` repeats	Lua lines `-- a`, `-- b`: `gcrip` -> `a`, `b`
`{count}gcr`	n	Uncomment count rows from the cursor down (`200gcr`)	Lua lines `-- a`, `-- b`, `-- c`: `2gcr` -> `a`, `b`, `-- c`
`gcrr`	n	Uncomment current line(s) (`3gcrr` = 3 rows); `.` repeats	Lua lines `-- a`, `b`: `gcrr` -> `a`, `b`
`gcr`	x	Uncomment the selected rows	Lua lines `-- a`, `-- b`, `-- c`: `Vjgcr` -> `a`, `b`, `-- c`

`gcc` against `gcs` on the same lines:

Before	Keys	After
Python `# # a`	`gcrr`	`a` (every marker level is removed)
Python `color = "#ff0000"` / `x = 1  # keep`	`gcss` on line 1	`# color = "#ff0000"` (the `#` inside the string is not mistaken for a comment; line 2 is untouched)
HTML: `<script>` / `let a = 1` / `</script>`, cursor on the middle line	`gcc`	`<!-- let a = 1 -->` (vim-commentary only knows the HTML marker)
the same	`gcss`	`// let a = 1` (smart comment finds the JavaScript inside `<script>`; inside `<style>` it writes `/* ... */`)

Where smart comment does not follow the embedded language: in a `.tsx` file a JSX line gets `//` (not `{/* */}`), and in a `.vue` file `<!-- -->` is used everywhere (no Treesitter parser attached).

Rule: `gcs` adds one marker per row and never nests (redundant markers already in the rows are removed); `gcr` removes every marker level (`# # a` -> `a`). Only real comment markers of the file type count (a `#` in a colour, URL or one-line string is not a comment).

With a motion, put the count AFTER `gcs` / `gcr` (`gcs3j`, `gcr200j`): a count before them always means rows, so `3gcsip` comments 3 rows and then `ip` runs as normal keys. If you pause after `gcs` longer than 'timeoutlen', a following `s` no longer makes `gcss` (gcs is then waiting for a motion and `s` cancels it); `gcsip` typed in one go is not affected.

`gcs` / `gcr` always change WHOLE rows (a charwise motion such as `gcse` changes every row it touches); the cursor stays on its text row; `.` repeats with the same motion, count or number of Visual rows. In a non-modifiable buffer they show one warning. Avoid `gcsgcs` (the second `gc` is the comment text object).

Fully supported languages: asm, bash, c, cmake, conf, cpp, cs, css, dockerfile, elixir, fish, gitconfig, go, haskell, hcl, html, i3config, java, javascript, julia, kitty, kotlin, lisp (Emacs Lisp / Common Lisp), lua, make, markdown, nix, perl, php, ps1 (PowerShell), python, r, ruby, rust, scala, sh, sql, swift, terraform, tex (LaTeX), tmux, toml, typescript, typst, vim, xml, yaml, zig, zsh. In gitconfig an UNQUOTED `#ff0000` is a comment for git itself, so `gcr` removes it: quote colours there.

Other filetypes use their 'commentstring': a marker counts only as the first non-blank character of the row.

---

17. Surrounding pairs (`vim-sandwich` + `nvim-autopairs`)

See Section 6: Working with parentheses, quotes, and brackets for the complete guide.

Quick reference:

Keymap	Description
`saiw"`	Add `"` around word
`sd"`	Delete surrounding `"`
`sr"'`	Replace `"` with `'`
`%`	Jump to matching bracket

Nested pairs, the `b` shortcut and wrapping a whole argument list: see "Nested pairs, `b` and exactness" and "Wrap a whole list or part of it" in section 6.

---

24. Yank history (`yanky.nvim`)

Keymap	Mode	Description
`p` / `P`	n, x	Paste after / before (with 300ms highlight)
`[y`	n	After pasting, cycle to previous yank entry
`]y`	n	After pasting, cycle to next yank entry

Command: `:YankyRingHistory` to browse all yank history. yanky.nvim loads right after the first screen (VeryLazy), so the yank history contains every yank of the session. In Visual mode `p` is yanky's: it overwrites the unnamed register with the replaced text (there is no separate keep-register map any more).

---

25. Undo history

Keymap	Description
`<Space>u`	Toggle Neovim's builtin undo tree (`nvim.undotree`) in a 30-column panel on the far left

Inside the panel there are no extra keys: moving the cursor onto an entry switches the buffer to that undo state (documented in `:h undotree.open()`). Close it with `<Space>u` again or `:q`.

---

29. Registers & macros

Registers

Keymap	Description
`"3y`	Yank to register 3
`"3p`	Paste from register 3
`"*y` / `"+y`	Yank to system clipboard
`:reg`	View all registers

Note: When a clipboard tool (wl-clipboard, xclip, ...) is installed, `clipboard` is `unnamedplus`, so `y`/`p` already use the system clipboard by default.

Example: two things at once (text `foo bar`):

1. `"ayiw` copies `foo` into register `a`.
2. `w"byiw` copies `bar` into register `b`.
3. `$"ap"bp` pastes both after the end of the line: `foo barfoobar`.

`:reg` shows `a` and `b`. Naming a register with `"a` does not touch the default yank register.

Macros

`Q` is an extra key for recording: `Qa` and `qa` both start recording into register `a` (plain `q` works too, `Q` is mapped to `q`).

Keymap	Description
`Qh`	Start recording macro to register `h`
`q`	Stop recording
`@h`	Play macro from register `h`
`5@h`	Play macro 5 times
`@@`	Replay the last played macro

---

60. Macros in depth

Macros record a sequence of keystrokes and replay them. They are one of the most powerful features in Vim for repetitive editing.

Key mapping: In this config `Q` is mapped to `q`, so `Q` and `q` both start recording (plain `q` works too). Stopping is always `q`.

Recording a macro

1. Press `Q` followed by a register letter (e.g., `Qa` to record into register `a`)
2. The command line (bottom line) shows `recording @a` (in a real terminal) – everything you do now is being recorded
3. Perform the editing actions you want to repeat
4. Press `q` to stop recording

Playing a macro

Keymap	What it does
`@a`	Play macro from register `a` once
`5@a`	Play macro 5 times
`@@`	Replay the last played macro
`100@a`	Play 100 times (stops early if it hits an error, e.g., end of file)

Scenario: add semicolons to the end of 20 lines

1. Place cursor on the first line
2. `Qa` – start recording to register `a`
3. `A;<Esc>` – go to end of line, add semicolon, back to normal mode
4. `j` – move down one line
5. `q` – stop recording
6. `19@a` – replay 19 more times (20 lines total)

Small version: lines `a`, `b`, `c`, `d`; do steps 2 to 5 on `a`, then `3@a` (4 lines: 1 recorded + 3 replays) gives `a;`, `b;`, `c;`, `d;`. With only `2@a` the last line `d` stays as it is.

Scenario: wrap each line in double quotes

Starting with:

```
apple
banana
cherry
```

1. Cursor on line 1
2. `Qa` – start recording
3. `I"<Esc>` – insert `"` at beginning
4. `A"<Esc>` – append `"` at end
5. `j` – move down
6. `q` – stop recording
7. `2@a` – replay for remaining lines

Result:

```
"apple"
"banana"
"cherry"
```

Scenario: append the same text to a block of lines

Starting with 4 consecutive lines that each end with `;`, add `// ok` to the end of every one.

1. Put the cursor on the first of the 4 lines, at any column.
2. `Qa` – start recording to register `a`
3. `A // ok<Esc>` – jump to the end of the line, type the text, back to Normal mode
4. `j` – move down one line, ready for the next run
5. `q` – stop recording. Only line 1 has changed so far (you did the edit while recording).
6. `3@a` – replay 3 more times for the other 3 lines (total lines - 1)

Result: all 4 lines end with `; // ok`.

- **Why it repeats well**: `A` goes to the end of the line wherever the cursor is, so the column does not matter, and the macro ends on `j`, so the next run starts on the next line.
- **Where the cursor ends**: one line **below** the last processed line (the final `j` of the last run). If that line is blank, the cursor is on the blank line. When the block is the last in the file the final `j` fails and the cursor stays on the last line.
- **Count**: use `number of lines - 1`, because the recording already did the first line. A count that goes past the last line stops early with an error, which is harmless.
- **Undo**: one `u` undoes the whole replay (lines 2-4); the line you edited by hand while recording needs a second `u`.

Scenario: turn `// name` lines into `names.add("name");` calls

Starting with 7 lines, indented 8 spaces, each `// apple`, `// banana`, …, `// orange`. Result: `names.add("apple");` and so on, same indent.

1. Put the cursor on the first line, at any column.
2. `Qa` – start recording to register `a` (`Q` is `q`)
3. `^` – go to the first non-blank character (the `//`)
4. `3x` – delete `//` (the two slashes and the space)
5. `i` + `names.add("` + `<Esc>` – insert the start of the call before the word
6. `A` + `");` + `<Esc>` – append the end of the call
7. `j` – move down one line
8. `q` – stop recording. Only line 1 has changed so far.
9. `6@a` – replay 6 more times (7 lines - 1)

- **Why it repeats well**: same reasoning as in the semicolon scenario above. The macro starts from a fixed place (`^`, the first non-blank, whatever the column was) and ends on `j`, so every replay starts on the next line at the right spot.
- **What is stored**: the register holds the raw keys; `:reg a` shows them.
- **Dead-key keyboard layouts (for example US International)**: the keys `^` and `"` need a following `<Space>` while you type them (and while recording); see Keyboard layouts with dead keys.

Scenario: convert a list of variables to assignments

Starting with:

```
name
age
email
```

Turn each into `self.name = name`:

1. Cursor on `name`
2. `Qa` – start recording
3. `0yiw` – yank the word
4. `Iself.<Esc>` – prepend `self.`
5. `A = <Esc>p` – append `=` and paste the word
6. `j` – next line
7. `q`, then `2@a` (gives `self.name = name`, `self.age = age`, `self.email = email`)

Scenario: turn CSV into SQL VALUES

Starting with `John,30,john@email.com`:

1. `Qa` – start recording
2. `I('<Esc>` – prepend `('`
3. `:s/,/','/g<CR>` – replace the commas on this line only (use `:s`, not `:%s`)
4. `A'),<Esc>` – append `'),`
5. `j` – next line
6. `q` then replay

Result on the first line: `John,30,john@email.com` becomes `('John','30','john@email.com'),`. With two lines (`John,30,john@email.com` and `Amy,25,amy@email.com`), recording on line 1 and then `@a` once gives:

```
('John','30','john@email.com'),
('Amy','25','amy@email.com'),
```

Tips for writing macros

- **Start from a consistent position**: Begin each macro from a fixed position in the line (`0` start of the line, `H` first non-blank) or from a search result. This makes the macro repeatable.
- **Use word motions, not character motions**: `w`, `e`, `b` work regardless of word length. `3l` only works for a specific column.
- **End on the next line**: If processing line-by-line, end the macro with `j` (move down) so replaying it processes subsequent lines.
- **Test with `@a` once**: Play the macro once to verify before running `100@a`.
- **Check existing registers**: `:reg` shows what's stored in each register. Macros and yanks share registers, so recording to `a` overwrites whatever was yanked to `a`.
- **Use a high count**: `999@a` will replay until it fails (e.g., end of file). Vim stops automatically on error.

Visual mode macros

You can apply a macro to every line in a visual selection:

1. Select lines with `V` + `j`/`k`
2. Type `:normal @a` and press Enter
3. The macro runs on each selected line

Example: lines `a`, `b`, `c` and a macro `a` that does `A;<Esc>` (without the `j`: the Ex command already moves from line to line). `ggVG`, then `:normal @a` (the command line shows `:'<,'>normal @a`) gives `a;`, `b;`, `c;`. Selecting only `b` and `c` (`jVj`) leaves `a` untouched. The macro-free form `:%norm A;` gives the same result.

---

61. The dot command (`.`) – repeating actions

The `.` key repeats the last change. This is arguably the most important efficiency tool in Vim.

What counts as a "change"

- Any editing in insert mode between `i`…`<Esc>` (typed text, deletions, etc.)
- Any operator command: `dd`, `dw`, `ciw`, `>>`, `gcc`, etc.
- Surroundings: `saiw"`, `sd"`, `sr"'`
- Operator-style plugin actions: surroundings (sandwich) and `gc` comments are repeated by Vim's own `.` machinery (they set an operator function); they do not call vim-repeat in the installed copies
- Other plugin mappings, only if the plugin registers with vim-repeat (see below)

vim-repeat is a small helper plugin with no keys or commands of its own. It tells `.` how to repeat the last action of a plugin mapping, which Vim alone would repeat only as its last built-in command. In the installed copies of this config's plugins, targets.vim, vim-matchup (its `ds%` / `cs%` surround actions) and gitsigns (its stage-hunk and reset-hunk actions) call it, so pressing `.` after them repeats the whole action. Whether `.` really repeats a given plugin key was not tried here; if a plugin does not register with vim-repeat, `.` repeats only the last built-in step. vim-repeat loads right after the first screen (VeryLazy).

Scenario: change a variable name one-by-one

1. Place cursor on the word `oldName`
2. `*` – search for it (highlights all occurrences)
3. `ciw` – change inner word, type `newName`, press `<Esc>`
4. `n` – jump to next occurrence
5. `.` – repeat the change (replaces `oldName` with `newName`)
6. `n` – next occurrence
7. `.` – repeat again
8. Skip an occurrence? Just press `n` without `.`

This gives you manual control over each replacement, unlike `:%s` which replaces all at once.

Example (here `*` only highlights the word and leaves the cursor in place): `foo a foo b foo` with the cursor on the first `foo`:

Keys	Result
`*ciwbar<Esc>n.`	`bar a bar b foo` (first and second changed)
`*ciwbar<Esc>nn.`	`bar a foo b bar` (the second is skipped with an extra `n`)

Scenario: add a prefix to multiple lines

1. On the first line: `I// <Esc>` (insert `// ` at start)
2. `j` – move down
3. `.` – repeat (adds `// ` to this line too)
4. `j.j.` – keep going

Example: lines `a`, `b`, `c` -> `I// <Esc>`, `j.`, `j.` -> `// a`, `// b`, `// c`.

Scenario: delete the first word on several lines

1. On the first line: `0dw` (go to start, delete word)
2. `j` – move down
3. `.` – repeat
4. Continue `j.` as needed

Example: lines `one x`, `two y` -> `0dw`, `j.` -> `x`, `y`.

Scenario: indent multiple blocks

1. On a line: `>>` (indent right)
2. `.` – indent again (double indent)
3. Move to another line, `.` – indent that line too

Combining `.` with counts

- `3.` repeats the last change 3 times (more exactly: the count replaces the one the change had). Example: `a b c d e f`, `ciw` + `X` + `<Esc>`, `w`, `3.` -> `X X d e f`: the repeat is `c3iw`, and `3iw` covers `b`, the space and `c`.
- `5>>` then `.` repeats the 5-line indent

---

62. Visual block editing (multi-cursor-like)

Visual block mode (`<Ctrl-v>`) lets you edit rectangular columns of text. This is the closest thing to multi-cursor editing.

Scenario: add a prefix to multiple lines at once

```
line one
line two
line three
```

1. Place cursor at the start of `line one`
2. `<Ctrl-v>` – enter block visual mode
3. `2j` – extend selection down 2 lines (column is now selected on 3 lines)
4. `I` – enter insert mode (capital I, for block insert)
5. Type `//` (or any prefix)
6. Press `<Esc>` – the prefix appears on ALL three lines

Result:

```
// line one
// line two
// line three
```

Scenario: append text to multiple lines

```
item1
item2
item3
```

1. `<Ctrl-v>` then `2j` – select the column
2. `$` – extend selection to end of each line
3. `A` – enter append mode (capital A)
4. Type `,` (or any suffix)

5. `<Esc>` – applied to all lines

Result:

```
item1,
item2,
item3,
```

Scenario: delete a column
If you have aligned text and want to remove a column:

1. `<Ctrl-v>` – block visual
2. Move to select the rectangular region (e.g., `3j10l`)
3. `d` – delete the block

Example: with the cursor on the `n` of `name`, `<Ctrl-v>jj4ld` removes the first 5 columns of all three lines:

Before	After
`name  age`	`age`
`bob   30`	`30`
`amy   25`	`25`

Scenario: replace a column
1. `<Ctrl-v>` – select the column
2. `c` – change (deletes the block and enters insert mode)
3. Type the replacement
4. `<Esc>` – applied to all lines

Example: lines `a1 b`, `a2 b`, `a3 b`, cursor on the first `a`: `<Ctrl-v>jjlcXY<Esc>` gives `XY b` on all three lines.

---

64. Everyday editing scenarios

Swap two lines
1. On the first line: `dd` (cut it)
2. Move to where you want it: `j` or `k`
3. `P` (paste above) or `p` (paste below)

Or use `<Alt-j>` / `<Alt-k>` to move lines up/down without cutting.

To move a line to a specific line number (`;3m10<CR>`) or by a relative distance (`;.m+2<CR>`), see Non-contiguous lines.

Example: lines `a`, `b`, `c`, cursor on `a`: `dd` then `p` gives `b`, `a`, `c`.

Swap two words (vim-swap)
Plugin: **vim-swap**. Put the cursor on an item of a comma-separated list (function arguments) and press `gs`: this starts "swap mode". There `h`/`l` move the current item left/right (swap with its neighbour), `j`/`k` choose another item, `1`-`9` choose the nth item, `s`/`S` sort ascending/descending, `r` reverses, `u`/`<Ctrl-r>` undo/redo, `<Esc>`

leaves swap mode. The digits only choose an item, they do not move it: to move an item several places press `h`/`l` once per step (see section 65).

For manual word swap:

1. On the first word: `diw` (delete inner word)
2. Move to the second word: `w` or `f`
3. `viwp` – select the second word and paste (swaps them)
4. `0P` – go to the start of the line and paste the word that `viwp` put in the register (before the first character)

On `one two` with the cursor on `one`: `diw` leaves ` two`; `w` jumps to `two`; `viwp` replaces it with `one` (the line is now ` one` and the register holds `two`); `0P` puts `two` in front: `two one`. Without step 4 the second word is lost. For function arguments or list items, `gs` above is simpler.

Duplicate a line

1. `yy` – yank the line
2. `p` – paste below

Or: `yyp` (same thing). Example: lines `a`, `b`, cursor on `a`: `yyp` gives `a`, `a`, `b`.

Duplicate a block of code

1. Select the block with `V` + `j`/`k`
2. `y` – yank
3. Navigate to destination
4. `p` – paste

Example: lines `a`, `b`, `c`, cursor on `a`: `Vjy` copies `a` and `b`, `j` `p` pastes them below `b`: `a`, `b`, `a`, `b`, `c`.

Fix indentation of entire file

1. `gg=G` – go to top, auto-indent everything to bottom

Example: in a Lua buffer the lines `if x then` / `print(1)` / `end` become `if x then` / `  print(1)` / `end` (2 spaces: shiftwidth is 2).

Remove all blank lines

1. `:%g/^$/d` – globally delete lines matching "empty"

Careful: `^$` only matches truly empty lines. On `a`, empty, `b`, a line with two spaces, `c` it leaves the whitespace-only line (`a`, `b`, ` `, `c`). To remove those too use `:g/^\s*$/d` (see section 71), which gives `a`, `b`, `c`.

Sort lines

1. Select lines with `V` + movement
2. `:sort` – sort alphabetically
3. `:sort!` – reverse sort
4. `:sort n` – numeric sort
5. `:sort u` – sort and remove duplicates

Before	Command	After
`pear` / `apple` / `fig`	`:sort`	`apple` / `fig` / `pear`
`pear` / `apple` / `fig`	`:sort!`	`pear` / `fig` / `apple`
`item 10` / `item 9` / `item 100`	`:sort n`	`item 9` / `item 10` / `item 100` (numbers by value)
`b` / `a` / `b`	`:sort u`	`a` / `b`
`c` / `b` / `a` / `z` / `y`, `Vjj` on the first three	`:sort`	`a` / `b` / `c` / `z` / `y` (only the selection is sorted)

Convert tabs to spaces (or vice versa)

1. `:set expandtab` (already set by default)
2. `:retab` – convert all tabs to spaces in the file
3. Or `:set noexpandtab` then `:retab!` for spaces-to-tabs

Example: lines `<Tab>a` and `<Tab><Tab>b` become `  a` and `    b` (tabstop is 2 here).

Wrap a selection in a tag/function

1. Select text with `v` or `V`
2. `sa` + the surrounding character (vim-sandwich)
3. For example: select `myVar`, then `sa"` wraps it as `"myVar"`
4. For function: type `sa`, then `f`, then the function name at the prompt that appears, then `<Enter>` – wraps as `funcName(myVar)`

On `myVar`: `saiwf`, type `fn`, `<Enter>` gives `fn(myVar)`.

---

65. Swapping function arguments (`vim-swap`)

Plugin: **vim-swap**. Swap delimited items (function arguments, list elements, etc.) without cutting and pasting.

Place your cursor on one of the arguments inside parentheses:

Keymap	What it does
`gs`	Start swap mode (then `h`/`l` swap with the neighbour, `j`/`k` choose an item, `1-9` pick an item, `<Esc>` exit)

The keys inside swap mode (`h` `l` `j` `k` `1-9` `s` `S` `r` `u` `<Ctrl-r>`, plus `g` / `G` to group / ungroup items) are vim-swap's own defaults per its help (`:help swap.txt`), not set in this config; only the `gs` start key is defined here (`lua/plugin_specs.lua`), and the plugin's other default keys (`g<`, `g>`) are switched off so the builtin `g<` stays.

Example: Given `func(a, b, c)` with the cursor on `b`, press `gs`, then `l`: `b` moves one place right, giving `func(a, c, b)`. Press `<Esc>` to leave swap mode.

Moving an item several places: there is no key that sends an item straight to position N. Inside swap mode the digits `1-9` only CHOOSE which item is current, they do not move it. To move an item two places, stay in one swap-mode session and press one key per step: `gs`, then `h` twice, then `<Esc>`. Example: given `f(a, c, b)` with the cursor on `b`, press `gs`, `h`, `h`, `<Esc>`: `b` moves two places left, giving `f(b, a, c)`.

- Inside swap mode `u` / `<Ctrl-r>` undo/redo a step, handy when you overshoot.

- Counts inside swap mode (`2h`) have not been tried here, so the guide makes no claim about them.
- Per `:help swap.txt` (not tried here): vim-swap's own `g<` / `g>` accept a count (`2g<` would move the item two places left in one command), but this config switches those mappings off on purpose, so they are not available.

Works with any comma-separated list: function arguments, array literals, dictionary entries, etc.

---

68. Useful Vim tricks

Run a normal-mode command on every line

`:g/pattern/normal @a` – run macro `a` on every line matching `pattern` `:g/pattern/normal dd` – delete every line matching `pattern` `:v/pattern/normal dd` – delete every line NOT matching `pattern` (inverse)

Examples:

Before	Command	After
`x` / `import a` / `y` / `import b`	`:g/^import/normal A;`	`x` / `import a;` / `y` / `import b;`
the same	`:g/^import/normal >>`	both `import` lines indented by 2 spaces
`TODO a` / `b` / `TODO c`	`:g/TODO/normal I[URGENT]`	`[URGENT] TODO a` / `b` / `[URGENT] TODO c`

Execute a command on a range

`:10,20normal A;` – append semicolons to lines 10-20 `:10,20normal I//` – comment out lines 10-20 `:'<,'>normal @a` – run macro `a` on visually selected lines

Example: lines `a`, `b`, `c`: `:2,3normal I//` gives `a`, `// b`, `// c`.

Increment/decrement numbers

Keymap	What it does	Example
`<Ctrl-a>`	Increment the number under cursor	`x = 7` with the cursor on the line: `<Ctrl-a>` -> `x = 8`
`<Ctrl-x>`	Decrement the number under cursor	`x = 7`: `<Ctrl-x>` -> `x = 6`
`10<Ctrl-a>`	Add 10 to the number	`x = 7`: `10<Ctrl-a>` -> `x = 17`
`g<Ctrl-a>` (visual block)	Sequential increment: line 1 gets +1, line 2 gets +2, line 3 gets +3...	three lines `0.`, select the column with `<Ctrl-v>2j`, `g<Ctrl-a>` -> `1.`, `2.`, `3.`
`g<Ctrl-x>` (visual block)	Sequential decrement (same idea, subtracting)	three lines `9`, select the column with `<Ctrl-v>2j`, `g<Ctrl-x>` -> `8`, `7`, `6`

Scenario: Generate a numbered list. Type `0.` on 5 lines, select them with `<Ctrl-v>`, then `g<Ctrl-a>` turns them into `1. 2. 3. 4. 5.`

Important: `g<Ctrl-a>` doesn't "know" what you want incremented – it purely acts on whatever number the highlighted column(s) overlap on each line, ignoring every

other number on the line. Wherever you place the block is what gets incremented. Only the digits inside the block count as the number.

Gotcha – starting value: because it *adds* `1×n` to each line, starting from `1` gives `1+1=2, 1+2=3, 1+3=4...` (starts at 2, not 1). If you want the sequence to start at 1, your placeholder number must start at `0`.

Example 1 – single number on the line (increments normally):

```
imgur_japan_0.jpg
imgur_japan_0.jpg
imgur_japan_0.jpg
```

Put the cursor on the `0`, `<Ctrl-v>`, `G` to extend the block down that same column to the last line, then `g<Ctrl-a>`:

```
imgur_japan_1.jpg
imgur_japan_2.jpg
imgur_japan_3.jpg
```

Example 2 – two numbers on the line, only one should change:

```
0001-photo-0.jpg
0001-photo-0.jpg
0001-photo-0.jpg
```

Here `0001` is a fixed ID that must stay identical on every line, and only the trailing `0` should become sequential. Place the cursor on the trailing `0` (not on `0001`), `<Ctrl-v>`, `G`, `g<Ctrl-a>`:

```
0001-photo-1.jpg
0001-photo-2.jpg
0001-photo-3.jpg
```

The `0001` is left untouched because the visual-block column never overlapped it – only the number your block touches gets incremented, no matter how many other numbers appear elsewhere on the line.

Common use case: renaming a batch of files. Open the filenames in a buffer (e.g. via a bulk-rename tool that spawns `$EDITOR` with one filename per line, in file order), reduce every line to the same placeholder with `:%s/.*/newname_0.jpg/`, then apply the block-select + `g<Ctrl-a>` trick above to turn the shared `0` into a sequence.

Example 3 – starting the sequence at 0 instead of 1:

`g<Ctrl-a>` always adds `1×n`, so it naturally starts at 1. To start at 0, do the sequential increment first, then shift the whole result down by one with a **plain** (non-`g`) `<Ctrl-x>` on the same block – plain block increment/decrement applies the *same* amount to every line instead of a growing amount:

```
newname_0.jpg          newname_1.jpg          newname_0.jpg
newname_0.jpg   g<C-a>  newname_2.jpg  <C-x>   newname_1.jpg
newname_0.jpg   ----->  newname_3.jpg  ----->  newname_2.jpg
newname_0.jpg           newname_4.jpg          newname_3.jpg
```

Reselect the exact same column block before pressing `<Ctrl-x>` (block selections don't persist across a `g<Ctrl-a>` – you need `<Ctrl-v>` + `G` again). This same "sequential increment, then uniform shift" combo also works for shifting a sequence to start at any number.

Example 4 – multi-digit, zero-padded numbers:

Vim preserves the digit width/leading zeros automatically as long as you don't overflow it:

```
photo_001.jpg
photo_001.jpg
photo_001.jpg
```

Put the cursor on the first `0` of `001`, widen the block over all three digits (`<Ctrl-v>ll`, then `G`), then `g<Ctrl-a>`:

```
photo_002.jpg
photo_003.jpg
photo_004.jpg
```

Warning: a block that covers only part of the digits increments only those digits (cursor on the first `0` with a 1-column block gives `photo_101.jpg`, `photo_201.jpg`, ...). Leading zeros are kept as long as the number does not outgrow its width.

If you want it to start at `001` instead of `002`, apply the same Example 3 shift: reselect the block, plain `<Ctrl-x>` once.

Letter sequences (a, b, c…) instead of numbers

`<Ctrl-a>`/`<Ctrl-x>` and their `g`-prefixed block variants also work on **letters**, but only once you opt in, since alphabetic increment isn't in Neovim's default `nrformats`:

```
:set nrformats+=alpha
```

(Add this to your Neovim config if you want it permanently; otherwise it only applies to the current session.)

With that set, bulk-renaming to `a, b, c, d...` works the same way as numbers:

```
a
a          g<C-a>     b
a   ----->  c
a           d
```

Note this starts at `b`, not `a` – same off-by-one as numbers, fixed the same way (Example 3): reselect the block and press plain `<Ctrl-x>` once to shift down:

```
b          a
c   <C-x>  b
d  ----->  c
e          d
```

`<Ctrl-a>`/`<Ctrl-x>` do NOT wrap at the alphabet boundary: `<Ctrl-a>` on `z` (or `Z`) leaves it unchanged. Case is kept (`a` -> `b`, `A` -> `B`).

Open the file under cursor

Keymap	What it does
`gf`	Open the file path under cursor (if it exists)
`<Ctrl-w>f`	Open file under cursor in a split
`gx`	Open the URL **or file** under the cursor (gx.nvim; also on a Visual selection)

For `gf`, put the cursor on a path such as `lua/mappings.lua` inside a string: if the file exists it opens in the current window. The path is looked up only from the current working directory ('path' is just that directory here, not the folder of the open file; in a Lua file `gf` on `lua.mappings` inside `require(...)` works too, because `.lua` is added and the dots become `/`) (started in the folder that contains `lua/` it opens; started in another folder, with the file open from the first, it finds nothing).

Change case

Keymap	What it does	Example
`~`	Toggle case of character(s). Since `tildeop` is set, use with a motion: `~w` toggles case of a word, `~e` to end of word.	`foo bar`: `~w` -> `FOO bar`; `Foo bar`: `~e` -> `fOO bar`
`gUiw`	Uppercase the entire word	`hello world` -> `gUiw` -> `HELLO world`
`guiw`	Lowercase the entire word	`HELLO world` -> `guiw` -> `hello world`
`gUU`	Uppercase the entire line	`foo bar` -> `gUU` -> `FOO BAR`
`guu`	Lowercase the entire line	`FOO BAR` -> `guu` -> `foo bar`
(in insert mode) `<Ctrl-u>`	Uppercase the current word (custom; the word touching the cursor, or the previous word if only spaces are before the cursor)	`foo` with Insert mode at its end: `<Ctrl-u>` -> `FOO`
(in insert mode) `<Ctrl-t>`	Toggle the case of the first letter of the current word (custom; `foo` <-> `Foo`)	`foo bar` with Insert mode inside `foo`: `<Ctrl-t>` -> `Foo bar`

Align text with tabular

Plugin: **tabular**. Aligns text around a character. `:Tabularize` works in every filetype from a fresh start (it loads on the first `:Tabularize`, and with the first Markdown file).

Command	What it does	Example
`:Tabularize /=`	Align all `=` signs in a selection or file	lines `a = 1` / `bb = 22` -> `a  = 1` / `bb = 22`
`:Tabularize /:`	Align colons (for JSON/YAML-like structures)	lines `a: 1` / `bbb: 22` -> `a   : 1` / `bbb : 22` (the colon column is aligned, with a space before each colon)

Full example with `VG:Tabularize /=` on three lines:

```
x = 1                 x         = 1
long_name = 22  -->   long_name = 22
yy = 333              yy        = 333
```

To align the pipes of a Markdown table, type `:Tabularize /|` (shown outside the table because a pipe breaks a table row).

Command abbreviations

This config sets some command abbreviations (type the short form as the first word of the command line, then press space or Enter):

Short	Expands to
`git`	`Git` (fugitive; only inside a git repository)
`man`	`Man` (manual pages)
`edit`	`Edit` (multi-file edit)
`pi`	`Lazy install`
`pud`	`Lazy update`
`pc`	`Lazy clean`
`ps`	`Lazy sync`
`z`	`Z` (zoxide jump; only when it is the whole command so far)
`norm`	`Norm` (`:norm` with a live preview while you type)

Two insert-mode abbreviations fix typos: `reqire` -> `require`, `serveral` -> `several`.

---

71. The global command (`:g`)

The global command runs an Ex command on every line matching a pattern. It's one of the most powerful built-in features.

Syntax: `:g/pattern/command`

The inverse (`:v`) runs on lines that do NOT match: `:v/pattern/command`

Common uses

Command	What it does	Result
`:g/TODO/d`	Delete every line containing `TODO`	lines `a TODO`, `b`, `c` -> `b`, `c`
`:v/TODO/d`	Delete every line that does NOT contain `TODO` (keep only TODO lines)	lines `a TODO`, `b`, `c TODO` -> `a TODO`, `c TODO`
`:g/^$/d`	Delete all blank lines	lines `a`, blank, `b` -> `a`, `b`
`:g/^\s*$/d`	Delete all blank lines (including whitespace-only)	lines `a`, a line with only spaces, `b` -> `a`, `b`
`:g/console\.log/d`	Delete all console.log lines	JavaScript lines `a()`, `console.log(1)`, `b()` -> `a()`, `b()`
`:g/pattern/normal @a`	Run macro `a` on every matching line	lines `a x`, `b`, `c x` with macro `a` = `A!<Esc>` (pattern `x`) -> `a x!`, `b`, `c x!`
`:g/pattern/normal A;`	Append semicolon to every matching line	lines `a x`, `b`, `c x` (pattern `x`) -> `a x;`, `b`, `c x;`
`:g/pattern/normal I//`	Comment out every matching line	lines `a x`, `b`, `c x` (pattern `x`) -> `// a x`, `b`, `// c x`
`:g/pattern/t $`	Copy every matching line to the end of file	lines `a x`, `b` (pattern `x`) -> `a x`, `b`, `a x`
`:g/pattern/m 0`	Move every matching line to the top of file	lines `a`, `b x`, `c` (pattern `x`) -> `b x`, `a`, `c`
`:g/^import/normal >>`	Indent all import lines	lines `import a`, `b`, `import c` -> `  import a`, `b`, `  import c`

Everyday scenarios

Sample for the commands below: lines `a`, `TODO b`, `c`, `TODO d`.

Command	Result
`:g/TODO/d`	`a` / `c`
`:v/TODO/d`	`TODO b` / `TODO d`
`:g/TODO/t $`	`a` / `TODO b` / `c` / `TODO d` / `TODO b` / `TODO d` (copies of the matches at the end)
`:g/TODO/m 0`	`TODO d` / `TODO b` / `a` / `c`

Delete all print/debug statements

```
:g/print(/d                    -- Python: delete all print() lines
:g/console\.log/d              -- JS: delete all console.log lines
:g/System\.out\.print/d        -- Java: delete all System.out.println lines
:g/fmt\.Print/d                -- Go: delete all fmt.Print lines
```

Keep only lines matching a pattern

```
:v/error/d                     -- keep only lines containing "error"
:v/\v(import|from)/d           -- keep only import statements
```

Extract all function signatures

```
:v/\vdef \w+\(/d                -- Python: keep only function definitions
:v/\v(public|private|protected)/d  -- Java: keep only method/field declarations
```

Add prefix/suffix to matching lines

```
:g/TODO/normal I[URGENT]         -- add "[URGENT] " before every TODO line
:g/^#/normal A <!---->           -- add comment marker after every markdown heading
```

Move lines matching a pattern to the top

```
:g/import/m 0                    -- move all import lines to the top of the file
```

This does not sort: each match is moved to line 0 in turn, so several matches end up in REVERSE order. Lines `a`, `TODO b`, `c`, `TODO d` with `:g/TODO/m 0` give `TODO d`, `TODO b`, `a`, `c`. To sort lines, use `:sort` (see "Sort lines" in section 64).

---

76. Common editing power combos

Quick-reference card of the most powerful editing combinations for daily use.

Changing text

Combo	What it does	Example
`ciw`	Change the word under cursor	`foo` -> type `bar` -> `bar`
`ci"`	Change text in double quotes	`"old"` -> type `new` -> `"new"`
`ci(`	Change text in parentheses	`func(old)` -> type `new` -> `func(new)`
`ci{`	Change text in braces	`{old}` -> type `new` -> `{new}`
`cit`	Change text in HTML tag	`<p>old</p>` -> type `new` -> `<p>new</p>`
`cc`	Change entire line	Clears the line and enters insert mode: `old line` -> type `new` -> `new`
`C`	Change from cursor to end of line	Deletes the rest of the line and enters insert mode: `foo bar` with the cursor on `b` -> type `baz` -> `foo baz`
`c$`	Same as `C`	`foo bar` with the cursor on `b` -> type `baz` -> `foo baz`
`ct)`	Change from cursor to before `)` (useful inside function arguments)	`f(old, x)` with the cursor on `o` -> type `new` -> `f(new)` (`old, x` is replaced)
`cf,` + char + label	Change from the cursor through a `,` picked with hop: `f` is hop's 2-character jump, so type `,` and the character after it, then the label (may be several lines away; see "Precision Selection")	`f` is hop.nvim here, not the built-in

Deleting text

Combo	What it does	Example
`diw`	Delete word under cursor	`foo bar` with the cursor in `foo` -> ` bar`
`daw`	Delete word + surrounding spaces	`foo bar baz` with the cursor on `bar` -> `foo baz`
`di"`	Empty out double-quoted string	`x "y" z` with the cursor on `y` -> `x "" z`
`da"`	Delete the entire quoted string including quotes	`x "y" z` with the cursor on `y` -> `x  z`
`di(`	Empty out parentheses	`f(old)` with the cursor in `old` -> `f()`
`da(`	Delete parentheses and their contents	`f(old) x` with the cursor in `old` -> `f x`
`dip`	Delete paragraph	lines `a`, `b`, (blank), `c` with the cursor on `a` -> (blank), `c`
`dd`	Delete line	lines `a`, `b` with the cursor on `a` -> `b`
`D`	Delete from cursor to end of line	`foo bar` with the cursor on `b` -> `foo`
`dt)`	Delete from cursor to before `)`	`f(old, x)` with the cursor on `o` -> `f()`

Copying text

Combo	What it does	Register `0` afterwards holds
`yiw`	Copy word under cursor	`foo` (for `foo bar`, cursor in `foo`)
`yi"`	Copy text inside double quotes	`foo` (for `x "foo" y`, cursor in `foo`)
`yi(`	Copy text inside parentheses	`a, b` (for `f(a, b)`, cursor inside)
`yap`	Copy paragraph	`a`, `b` and the blank line (for the lines `a`, `b`, (blank), `c`)
`yy`	Copy line	the whole line `a` (for the lines `a`, `b`)
`y$`	Copy from cursor to end of line	`bar` (for `foo bar`, cursor on `b`)

Selecting text

Combo	What it does	Selected text
`viw`	Select word	`foo` (for `foo bar`, cursor in `foo`)
`vi"`	Select inside quotes	`foo` (for `x "foo" y`, cursor in `foo`)
`vi(`	Select inside parentheses	`a, b` (for `f(a, b)`, cursor inside)
`vip`	Select paragraph	`a`, `b` (for the lines `a`, `b`, (blank), `c`; the blank line is not included)
`V5j`	Select 5 lines down	the current line and the 5 lines below it (6 lines)
`ggVG`	Select entire file	every line of the file

Quick transformations

Combo	What it does	Example
`gUiw`	Uppercase word	`hello world` -> `HELLO world`
`guiw`	Lowercase word	`HELLO world` -> `hello world`
`~w`	Toggle case of word	`foo bar` -> `FOO bar`
`>>`	Indent line	`foo` -> `  foo`
`<<`	Deindent line	`    foo` -> `  foo`
`==`	Auto-indent line	Lua `if a then` / `x` / `end` with the cursor on `x` -> `x` becomes `  x`
`gg=G`	Auto-indent entire file	the same three Lua lines -> `x` becomes `  x`
`gcc`	Comment/uncomment line	Lua `x = 1` -> `-- x = 1`
`gcip`	Comment/uncomment paragraph	Lua lines `a`, `b`, (blank), `c` with the cursor on `a` -> `-- a`, `-- b`, (blank), `c`
`saiw"`	Surround word with `"`	`hello` -> `"hello"`
`sd"`	Remove surrounding `"`	`"foo"` with the cursor on `foo` -> `foo`
`sr"'`	Replace `"` with `'` around current text	`"foo"` with the cursor on `foo` -> `'foo'`
`J`	Join current line with next	lines `foo`, `bar` -> `foo bar`

The most powerful patterns

Pattern	How it works	Example
`*` then `ciw` then `n.n.n.`	Find-and-replace one at a time with full control	`foo a foo b foo`: `*ciwbar<Esc>n.` -> `bar a bar b foo` (section 61)
`Qa` … `q` then `@a`	Record and replay any sequence of actions	`a`, `b`, `c`, `d`: record `A;<Esc>j` on `a`, then `3@a` -> `a;`, `b;`, `c;`, `d;` (section 60)
`V` select then `:norm @a`	Run a macro on selected lines	`a`, `b`, `c` with the macro `A;<Esc>`: `ggVG` `:normal @a` -> `a;`, `b;`, `c;` (section 60)
`:g/pattern/command`	Run a command on every matching line	`:g/TODO/d` on `a`, `TODO b`, `c` -> `a`, `c`
`:grep "text"` then `:cfdo ...`	Project-wide search and replace; `:cfdo` only walks a quickfix list you filled first (the other do-commands `:ldo`, `:bufdo`, `:argdo`, `:windo`, `:tabdo` are in section 67); (the substitute + `| update` recipe is in section 67)	see section 67
`<Space>rn`	Intelligent rename across project	N/A (an LSP rename: the symbol is renamed in every file that uses it)
`qf` list + `:cnext` / `:cprev`	Jump through search results or errors	N/A (moves the cursor to the next / previous entry of the quickfix list)
`.`	Repeat last change (combine with `n` for find-and-repeat)	`foo x foo`: `ciw` + `bar` + `<Esc>`, `ww`, `.` -> `bar x bar`

Chapter 5: Completion and snippets

14. Autocompletion (`nvim-cmp`)

Plugin: nvim-cmp. Sources: LSP, UltiSnips snippets, file paths, buffer words; in LaTeX files also omni (BibTeX/citations); in the `/` search line buffer words, in the `:` command line paths and command names. Each source is its own small plugin, listed in section 45 ("Completion sources and helpers").

Keymap	Description
`<Tab>`	If menu is open: select next item. Otherwise: normal tab.
`<CR>` (Enter)	Confirm an item you picked with `<Tab>` / `<Ctrl-n>`; if nothing is selected it is a plain newline
`<Ctrl-e>`	Dismiss / close the completion menu
`<Esc>`	Close the completion menu
`<Ctrl-d>`	Scroll documentation popup up
`<Ctrl-f>`	Scroll documentation popup down
`<Ctrl-n>` / `<Ctrl-p>`	Open the menu / select the next / previous item (inserts it)
`<Ctrl-y>`	Confirm the selected item
`<Down>` / `<Up>`	Select the next / previous item without inserting it
`<Tab>` / `<S-Tab>`	In the `:` and `/` command line: open / move through the completion menu

Completion labels are coloured like code (colorful-menu.nvim); match and kind colours follow the active colorscheme.

---

15. Snippets (`UltiSnips`)

Plugin: UltiSnips + vim-snippets. Custom snippets in `my_snippets/` directory.

Keymap	Description
`<Ctrl-j>`	Expand snippet / jump to next placeholder
`<Ctrl-k>`	Jump to previous placeholder
`<Space>fs`	Open the snippet gallery: fuzzy-search only your own snippets and insert one (normal mode); see section 52
`<Alt-s>`	Same gallery from insert mode; inserts at the cursor (see section 52)

Available snippet files: `all`, `cpp`, `java`, `markdown`, `nix`, `python`, `snippets`, `tex`, `vim`

Example: in a Java buffer type `for`, press `<Ctrl-j>`: it expands to a loop with the first placeholder (`i`) selected:

```
for (int i = start; i < end; i++) {
    // code
}
```

Type `index` (it replaces `i` everywhere it appears), then `<Ctrl-j>` moves to the next placeholder (`start`), the next `<Ctrl-j>` to `end`, then to `// code`. The full walk-through is in section 52 (How to use snippets).

Java snippets

Source: `my_snippets/java.snippets` (71 snippets). Each section below follows the same grouping as the snippet file.

The tables below are the quick reference. For each snippet's expanded code, the meaning of the placeholders and examples, see section 78 (Java chapter, `languages/java.md`, section 9).

Conventions (the same in every Java snippet):

- Triggers are camelCase and case-sensitive (`promptRead`, not `jpromptread`): pick them from the completion menu instead of typing them whole. Type `j` and read the list.
- Placeholders are generic English words naming the kind of thing to fill in (`type`, `name`, `condition`, `ExceptionType`), never hardcoded values.
- The Scanner variable is always named `input`. `fdiScanner` creates it; the other input snippets use an existing `input`.
- The one-line description is shown by nvim-cmp above the snippet body in the completion menu (scroll it with `<Ctrl-d>` / `<Ctrl-f>`); expand with `<Ctrl-j>`, jump between placeholders with `<Ctrl-j>` / `<Ctrl-k>`.
- `fdiScanner` uses the file name as the class name by default.
- When a snippet needs an import, its description names the exact import(s). The snippets do not add imports themselves, so check that the import line exists at the top of the file.
- Counterpart snippets name each other in their description (`interface` / `implements`, `equals` / `hashCode`, `comparable` / `comparator`, `twrScanner` / `twrPrintWriter`, `twrObjectOut` / `twrObjectIn`, `minArray` / `maxArray`).

Program skeleton and input

Trigger	Expands to / use it when
`fdiScanner`	Program skeleton with a Scanner named input (course package)
`promptRead`	Print a prompt and read a value (Type: Int, Double, Boolean, Line; import java.util.Scanner)
`readValidated`	Ask again until the input has the right type (Type: Int, Double, Boolean; import java.util.Scanner)
`readNumberThenLine`	Read a number and then a line, consuming the leftover newline (import java.util.Scanner)
`readUntilInt`	Loop reading numbers from input until a sentinel value (uses an existing Scanner named input; import java.util.Scanner)
`readUntilString`	Loop reading Strings from input until a sentinel text (uses an existing Scanner named input; import java.util.Scanner)
`randomInt`	Random int between a minimum and minimum + range - 1

Arrays, matrices and collections

Trigger	Expands to / use it when
`arrayNew`	Array created with a given size
`arrayLiteral`	Array created from literal values
`minArray`	Find the minimum of an array (maxArray for the maximum)
`maxArray`	Find the maximum of an array (minArray for the minimum)
`forMatrix`	Nested indexed for loops over a matrix
`foreachMatrix`	Nested foreach loops over a matrix
`arrayAdd`	Add an element to a fixed array with a counter (returns false when null or full)
`arrayRemove`	Remove an element from a fixed array with a counter (object arrays; returns false when not found)
`hashMap`	HashMap declaration (import java.util.HashMap)
`treeMap`	TreeMap (sorted by key) declaration (import java.util.TreeMap)
`arrayList`	ArrayList declaration (import java.util.List, java.util.ArrayList)
`hashSet`	HashSet declaration (import java.util.Set, java.util.HashSet)
`forMapEntry`	Iterate over the entries of a map (import java.util.Map)

Control flow

Trigger	Expands to / use it when
`for`	Indexed for loop
`foreach`	Enhanced for loop
`while`	While loop
`dowhile`	Do-while loop
`if`	If statement
`ifelse`	If-else statement
`ifelif`	If-else if-else statement
`tern`	Ternary conditional assigned to a variable
`switchtraditional`	Traditional switch statement with break
`switchmulti`	Traditional switch with two labels per case
`switcharrow`	Switch statement with arrow syntax
`switcharrowmulti`	Switch with arrow syntax and two labels per case
`switchyield`	Switch expression assigned to a variable
`switchyieldblock`	Switch expression with a block and yield

Methods

Trigger	Expands to / use it when
`staticMethod`	Private static method
`recursive`	Recursive method with a base case

Classes

Trigger	Expands to / use it when
`class`	Package-private class with 3 fields and a constructor (delete the extra fields)
`publicClass`	Public class with 3 private final fields and a constructor (delete the extra fields)
`subclass`	Subclass with a constructor calling super
`abstractClass`	Abstract class with one abstract method
`interface`	Interface with one method
`implements`	Class implementing an interface (the counterpart of interface)
`toString`	toString with two fields
`toStringSuper`	toString extending the superclass toString (for subclasses)
`equals`	equals using getClass (override hashCode too: hashCode)
`hashCode`	hashCode with Objects.hash (pair of equals; import java.util.Objects)
`defaultIfBlank`	Constructor field: use a default when a String is null or blank
`defaultIfBelowMin`	Constructor field: use a default when a number is below a minimum
`record`	Record with two components
`recordCompact`	Record with a compact constructor for validation
`enum`	Plain enum with three constants
`enumFields`	Enum whose constants carry a value
`instanceOf`	Type test with a binding variable (pattern matching)
`comparable`	compareTo method for natural ordering (add implements Comparable by hand)
`comparator`	Separate Comparator class (the counterpart of comparable; import java.util.Comparator)

Exceptions

Trigger	Expands to / use it when
`exception`	Custom checked exception with a fixed message
`exceptionData`	Custom checked exception carrying a value
`throwIf`	Guard that throws (ExceptionType: IllegalArgumentException, NullPointerException, ...)
`trycatch`	Try-catch block
`tryfinally`	Try-catch-finally block
`trywith`	Generic try-with-resources block

File I/O and serialization

Trigger	Expands to / use it when
`twrScanner`	Read a text file line by line with a Scanner (import java.util.Scanner, java.io.File, java.io.FileNotFoundException)
`twrPrintWriter`	Write to a text file with a PrintWriter (the counterpart of twrScanner; import java.io.PrintWriter, java.io.FileOutputStream, java.io.FileNotFoundException)
`twrBufferedReader`	Read a file with BufferedReader and readLine in a loop (import java.io.BufferedReader, java.io.FileReader, java.io.IOException)
`twrObjectOut`	Serialize an object to a file (import java.io.ObjectOutputStream, java.io.FileOutputStream, java.io.IOException)
`twrObjectIn`	Read a serialized object from a file (the counterpart of twrObjectOut; import java.io.ObjectInputStream, java.io.FileInputStream, java.io.IOException)
`serialUID`	serialVersionUID field for a Serializable class (class must implement java.io.Serializable)

JavaFX

Trigger	Expands to / use it when
`fxApp`	JavaFX application skeleton (import javafx.application.Application, javafx.stage.Stage, javafx.scene.Scene, javafx.scene.layout.BorderPane)
`fxProperty`	Model field with the JavaFX property trio (create the property in the constructor; import javafx.beans.property.StringProperty (or the property type used))
`fxAlert`	JavaFX alert dialog (AlertType: INFORMATION, WARNING, ERROR, CONFIRMATION; import javafx.scene.control.Alert, javafx.scene.control.Alert.AlertType)
`fxOnAction`	JavaFX button event handler

Other snippets

Every row gives the trigger and its description as the completion menu shows it. Explanations, expansions and placeholders are in the chapters named in the last column.

File	Trigger	Description	Chapter
all	`ltx` (regular expression `(?<!\w)ltx`)	Type ltx (not right after a word character) to insert the word LaTeX; works in every filetype	this section
all	`arw`	ASCII right arrow (–>) followed by text	this section
snippets	`snip`	Template for a new snippet definition (copied from vim-snippets; start of line)	this section
cpp	`bare`	Barebone C++ program: common includes, using std:: declarations and main	`languages/cpp.md`
cpp	`icd`	#include directive (start of line)	`languages/cpp.md`
cpp	`incset` `incmap` `incqueue` `incstr` `incvec` `incstack`	Include the header and add the using line, for example "Include and add using std::set (start of line)"	`languages/cpp.md`
cpp	`vec` `map` `umap` `set` `uset` `queue` `stack`	std container declaration, for example "std::vector declaration (needs using std::vector: bare or incvec)"; `map` also "overrides the vim-snippets map"	`languages/cpp.md`
cpp	`plist`	print a container as [a, b, c] (needs , ,)	`languages/cpp.md`
cpp	`pmat`	print a vector of vectors, one [a, b, c] row per line (needs , , ,)	`languages/cpp.md`
cpp	`pqueue`	print a priority_queue or stack by popping a copy (uses top(); std::queue has front() instead)	`languages/cpp.md`
cpp	`cout`	Print a labelled variable with std::cout (needs #include)	`languages/cpp.md`
cpp	`random`	Function returning a vector of random ints in [low, high] (needs #include and)	`languages/cpp.md`
cpp	`sol`	LeetCode style: create a Solution object (needs a	`languages/cpp.md`
[illegible]	`info` `warn` `error` `success` `hub` `systemd`	[illegible]	`languages/markdown.md`

onment.systemPackages;

`ltx` replaces the typed `ltx` with the word `LaTeX`. It does not fire when `ltx` follows a word character, so words that merely contain these letters are left alone. `arw` inserts `-->` (two hyphens and a greater-than sign, plain ASCII, not a Unicode arrow) and selects the placeholder `text`, so you can type the text right after it. `snip` (start of line) writes a template for a new snippet definition in a `.snippets` file: the keyword `snippet`, then the placeholders `Tab_trigger`, `Description` and the options `b`, then a body line and `endsnippet`.

Example: in a Python file `main` + `<Ctrl-j>` (at the start of a line) gives this block, with the `# code` placeholder on the indented line inside `def main():` selected:

```
def main():
    # code


if __name__ == "__main__":
    main()
```

---

45. Autocompletion in depth (`nvim-cmp`)

How it works

When you type in insert mode, **nvim-cmp** queries multiple sources and shows a popup menu with suggestions:

1. **LSP** (highest priority): Function names, variables, methods, types from the language server
2. **UltiSnips**: Snippet triggers (e.g., type `for` in a Java file)
3. **Path**: File paths when you start typing a path
4. **Buffer** (lowest priority, min 2 chars): Words already in the current buffer

For LaTeX files the sources are **omni** (BibTeX/citations), the texlab LSP (only when `texlab` is on PATH, LaTeX devShell), UltiSnips, buffer and path.

The smart tab behavior

`<Tab>` has three behaviors depending on context:

1. **Completion menu is visible**: Selects the next item in the menu
2. **Menu closed and the cursor is before a closing bracket or quote**: Jumps out past it (see Tabbing out of pairs)
3. **Otherwise**: Inserts a normal tab character

Completion keymaps

Keymap	In completion menu	Outside menu
`<Tab>`	Select next item	Jump out of a bracket or quote pair, else insert tab
`<CR>` (Enter)	Confirm the item you picked (with nothing picked: newline)	Insert newline
`<Ctrl-e>`	Close menu	Go to end of line
`<Esc>`	Close menu	Exit insert mode
`<Ctrl-d>`	Scroll docs up	Delete the character right of the cursor
`<Ctrl-f>`	Scroll docs down	(nothing)

Example (in a Python buffer with the pyright language server): line 1 is `pri_value = 1`, and on line 2 you type `prin`. The menu opens by itself, with the language server entries first and the snippets after them:

```
prin|
+--------------------------------------+
| print             Function           |    <- from the language server
| print~            Snippet            |    <- from UltiSnips
| ProfileFunction   Variable           |
+--------------------------------------+
```

1. `<Tab>` selects the first entry: the line now reads `print`.
2. `<CR>` confirms it, and no newline is inserted.
3. If you do NOT pick an entry (no `<Tab>`) and press `<CR>`, you get a normal newline and the menu closes (it may open again on the new line when something matches there), so `<CR>` never inserts a completion you did not choose (`prin` then `<CR>` left `prin` and started line 3).

Visual indicators

- Each completion item shows an icon indicating its kind (function, variable, method, keyword, etc.) via mini.icons
- Deprecated items appear with strikethrough
- The completion menu is semi-transparent (5% blend)

Completion sources and helpers

nvim-cmp itself only draws the menu. What it offers comes from small "source" plugins, each used under the `name` it has in `lua/config/nvim-cmp.lua`:

Plugin	Source name	What it offers	Where it is active
`cmp-nvim-lsp`	`nvim_lsp`	Names from the language server (functions, variables, types)	Normal Insert-mode completion; first in the list (in buffers without an attached server it offers nothing)
`cmp-nvim-ultisnips`	`ultisnips`	Snippet triggers from UltiSnips (see "Snippets for Developers" below)	Insert-mode completion, second in the list
`cmp-path`	`path`	File and folder paths	Insert-mode completion; also first in the `:` command line
`cmp-buffer`	`buffer`	Words already in the current buffer, from 2 typed characters	Insert-mode completion (last in the list); the only source in the `/` search line
`cmp-omni`	`omni`	Whatever the filetype's omni-completion function offers; in LaTeX files that is vimtex (commands, labels, citations)	Only in `tex` files, where it is listed first
`cmp-cmdline`	`cmdline`	Ex command names and their arguments	Only in the `:` command line

How the command line behaves: in `/` the menu offers buffer words. In `:` the `path` source is asked first and the `cmdline` source is used when it has nothing (nvim-cmp's group rule), so a path-like word completes as a path and an ordinary word as a command or argument. `<Tab>` / `<S-Tab>` open and move through the menu there (see the table in section 14).

Two helpers are not sources:

- `colorful-menu.nvim` colours each completion label the way code is coloured (name, argument list, type) using the language server's own label; the extra text (arguments, types) is drawn in the comment colour (settings in `lua/config/colorful_menu.lua`), and labels longer than 60 columns are cut. It works automatically, no keys.
- `vim-snippets` is a ready-made snippet collection loaded by UltiSnips next to the personal `my_snippets/` files (see "Ready-Made Snippets" in the snippets section below).

Libraries and dependencies of these plugins (for example `mini.icons`, which draws the kind icons) have no keys of their own.

---

52. Snippets for developers (`UltiSnips`)

What snippets are

Plugin: **UltiSnips** + **vim-snippets**. Snippets are templates that expand into boilerplate code when you type a trigger word.

Ready-made snippets (`vim-snippets`)

Besides the personal files in `my_snippets/`, UltiSnips also loads the `vim-snippets` collection: a library of common snippets for many languages that ships its own `UltiSnips/` folder. The plugin is installed as a dependency of UltiSnips and has no setup, keys or commands of its own, and the config lists the snippet folders it searches as `UltiSnips` and `my_snippets`. Its entries appear in the completion menu next to the personal ones (through the `ultisnips` source in section 45, "Completion sources and helpers"). To see what a language offers, open a file of that type, type the first letters of a common word such as `def` or `class` and look at the menu. The exact trigger list of the collection is not reproduced in this guide.

How to use snippets

1. In insert mode, type a trigger word (e.g., `for` in a Java file)
2. The trigger appears in the completion menu as a snippet
3. Press `<Ctrl-j>` to expand it
4. The snippet expands with **placeholders** (highlighted fields you need to fill in)
5. Press `<Ctrl-j>` to jump to the next placeholder
6. Press `<Ctrl-k>` to jump to the previous placeholder
7. Fill in each placeholder, and you're done

Example with `for` in a Java buffer:

```
for            <- you type the trigger
(Ctrl-j)        -> for (int i = start; i < end; i++) {
                       // code
                   }                          the first placeholder, i, is selected
index           -> for (int index = start; ... typing replaces the selected placeholder
everywhere it is used
(Ctrl-j)        -> the next placeholder, start, is selected; (Ctrl-j) again: end, then the
body
(Ctrl-k)        -> back to the previous placeholder
```

Filling the loop in (in a real Neovim, empty line inside a Java method): `for`, `<Ctrl-j>`, type `index` (every `i` is renamed), `<Ctrl-j>`, type `0` for `start`, `<Ctrl-j>`, type `10` for `end`, `<Ctrl-j>` jumps to the body.

Gotchas:

- `<Ctrl-j>` with no snippet trigger in front of the cursor does not expand anything: it just inserts a new line.
- An expanded snippet starts at the column where the cursor was. On a completely empty, unindented line it comes out unindented; format afterwards with `<Space>fm` (see the Java chapter).

The snippet gallery

The completion menu also lists the hundreds of built-in `vim-snippets` entries. The **snippet gallery** shows only your own snippets (the files in `my_snippets/`) for the filetype of the current buffer, plus the ones from `all.snippets`, without typing a trigger.

1. Press `<Space>fs` in normal mode, or `<Alt-s>` in insert mode
2. A floating fzf-lua picker opens: each line is `trigger  description  [file]`, sorted by trigger; the snippet text is shown in the preview on the right

3. Type to filter, move with `<Ctrl-n>` / `<Ctrl-p>` (see section 12, "Moving Inside Any Picker")
4. Press `<Enter>` to insert the snippet: from insert mode it lands exactly at the cursor, from normal mode right after the character under the cursor (like `a`); the usual placeholders work, jump with `<Ctrl-j>` / `<Ctrl-k>`
5. Press `<Esc>` to cancel: nothing is inserted (from insert mode you return to insert mode)

Before pressing `<Enter>`, check which line is HIGHLIGHTED, not just which lines the filter left. Example (in Normal mode in a Java buffer): typing `arrayL` leaves two entries and the highlighted one was `arrayLiteral`, not `arrayList`. Move with `<Ctrl-n>` / `<Ctrl-p>` to the one you want first. `<Enter>` on the highlighted `arrayLiteral` inserted `type[] name = {value1, value2, value3};`, and its fields worked with `<Ctrl-j>` / `<Ctrl-k>`.

Notes:

- Snippets with a regular-expression trigger, such as `ltx` (the trigger is `(?<!\w)ltx`), are listed and inserted like the others
- A few snippets cannot be inserted from the gallery because they need what only typing the trigger provides (a captured part of the trigger, or helper code of the file, for example the markdown headings `h1` … `h6`). Their line ends with `(type trigger)`; choosing one shows a warning and inserts nothing, so type the trigger instead
- If nothing is available for the filetype, a message says so
- `<Alt-s>` needs a terminal that sends Alt-s to Neovim (most do). If it does nothing, use `<Space>fs`. `<Ctrl-s>` is not used for the gallery: in insert mode it stays the LSP signature help

Custom snippets

Custom snippets live in the `my_snippets/` directory. Each file targets a specific language:

File	Language	Notable snippets
`all.snippets`	All filetypes	`ltx` (the word LaTeX), `arw` (ASCII `-->` then text); see section 15, Other snippets
`java.snippets`	Java	71 snippets: input, arrays and collections, control flow, methods, classes, exceptions, file I/O, JavaFX (full list in section 15, Java snippets)
`python.snippets`	Python	`print` (f-string), `impa`, `main`, `sol` (chapter `languages/python.md`, Snippets)
`cpp.snippets`	C++	`bare` program, includes, container declarations, print helpers, `random`, control flow (chapter `languages/cpp.md`, section 84)
`nix.snippets`	Nix	12 snippets: package lists, `delibmodule` / `delibalways`, `mkshell`, `mkderiv`, `flake`, `fetchgithub`, `systemd` (chapter `languages/nix.md`, section 86)
`tex.snippets`	LaTeX	`use` (`\usepackage`), `eqa` (equation with label), `mk` / `dm` (inline / display math) and 189 math-symbol snippets such as `leq`, `alpha`, `frac`, `matrix2` (chapter `languages/latex.md`, Snippets)
`markdown.snippets`	Markdown	208 snippets: keys, headings, links, images, front matter, message boxes, `mk` / `dm` (inline / display math) and 189 math-symbol snippets (chapter `languages/markdown.md`, Snippets)
`typst.snippets`	Typst	`mk` / `dm` (inline / display math) and 104 math-symbol snippets such as `leq`, `neq`, `integral`, `matrix2` (chapter `languages/typst.md`, Snippets)
`vim.snippets`	Vimscript	`fun` (function), `aug` (augroup) (chapter `languages/vim.md`, section 85)
`snippets.snippets`	Snippet files	`snip`: template for a new snippet definition

Creating your own snippets

Edit the appropriate file in `my_snippets/` (e.g., `my_snippets/python.snippets`):

```
snippet trigger "Description" b
def ${1:function_name}(${2:args}):
    ${3:pass}
endsnippet
```

- `trigger` is what you type
- `b` means it only triggers at the beginning of a line
- `${1}`, `${2}`, `${3}` are tab-stop placeholders (jump between them with `<Ctrl-j>`)

Result (with exactly this definition in a Python file): typing `trigger` at the start of a line and pressing `<Ctrl-j>` gives

```
def function_name(args):
    pass
```

with the cursor on the first placeholder, `function_name`: type the real name, `<Ctrl-j>` jumps to `args`, then to `pass` (`load` typed, `<Ctrl-j>`, `path` typed, `<Ctrl-j>`, `return 1` gives `def load(path):` / `    return 1`).

Chapter 6: Searching, replacing, files and the file tree

10. Searching, replacing, and refactoring text

This is one of the most important sections in the guide. It covers searching within a file, replacing text with various levels of control, and doing all of this across the entire project.

Searching in the current file

Keymap	Description
`/pattern`	Search **forward** for `pattern`. Press `<Enter>` to start the search.
`?pattern`	Search **backward** for `pattern`
`n`	Jump to the **next** match (with hlslens showing `[x/y]` count)
`N`	Jump to the **previous** match
`*`	Search **forward** for the exact word under cursor (whole word, literal; the cursor stays on the word, `3*` jumps to the 3rd match after the cursor)
`#`	Search **backward** for the exact word under cursor
`;noh<Enter>` (`;` is `:` here)	Clear the yellow search highlight. The search itself is kept, so `n` / `N` still work; the highlight comes back on the next search or `*`.

Search modifiers

Modifier	Where to put it	What it does	Example
`\c`	Anywhere in pattern	Force **case-insensitive**	`/\chello` finds `Hello`, `HELLO`, `hello`
`\C`	Anywhere in pattern	Force **case-sensitive**	`/\Chello` only finds `hello`
`\v`	At start of pattern	**Very magic**: regex works like Perl/Python (no need to escape `()`, `|`, `+`, etc.)	`/\vfunction\(.*\)`
`\<` and `\>`	Around pattern	**Whole word** match only	`/\<count\>` finds `count` but not `counter`

By default, search is case-insensitive but becomes case-sensitive if you type any uppercase letter (smart case). This is for `/` and `?` only: while you type a `:` command, smart case is off, so `:s` and `:g` ignore case completely (use `\C` or the `I` flag of `:s` for an exact match).

Search examples

Search	What it finds
`/hello`	`hello`, `Hello`, `HELLO` (smart case: all lowercase = case-insensitive)
`/Hello`	Only `Hello` (smart case: has uppercase = case-sensitive)
`/\vdef \w+\(`	All Python function definitions (very magic regex)
`/\v(TODO|FIXME|HACK)`	Any of these three words (very magic `|` for alternation)
`/\<user\>`	Only the word `user`, not `username` or `superuser`
`/error\c`	`error`, `Error`, `ERROR` (forced case-insensitive)

The same searches on a sample file (the match counts are the ones the search reports):

```
1  Hello world
2  hello again
3  HELLO
4  user, username, superuser
5  error Error ERROR
6  TODO: x
7  FIXME: y
8  def foo(a):
```

Search	Matches
`/hello`	3: lines 1, 2 and 3 (smart case: all lowercase ignores case)
`/Hello`	1: only line 1 (a capital makes it exact)
`/\<user\>`	1: only the first word on line 4; `username` and `superuser` are not matched
`/error\c`	3: the three words on line 5
`/\v(TODO|FIXME)`	2: lines 6 and 7
`/\vdef \w+\(`	1: `def foo(` on line 8

Substitution (find & replace in current file)

The substitute command has this structure: `:[range]s/old/new/[flags]`

Understanding the range (where to replace)

The range tells Vim which lines to search. If omitted, only the current line is affected.

Range	Meaning	Example
(none)	Current line only	`:s/old/new/`
`%`	**Entire file** (all lines)	`:%s/old/new/g`
`.`	Current line (same as no range)	`:.s/old/new/g`
`$`	Last line of file	`:$s/old/new/` (replaces on the last line only)
`.,$`	From current line to end of file	`:.,$s/old/new/g`
`1,.`	From first line to current line	`:1,.s/old/new/g`
`20,30`	From line 20 to line 30 (absolute)	`:20,30s/old/new/g`
`-3,+3`	From 3 lines above to 3 lines below cursor (relative)	`:-3,+3s/old/new/g`
`'<,'>`	Current visual selection (auto-filled when you press `:` in visual mode)	`:'<,'>s/old/new/g`

Understanding the flags (how to replace)

Flags go at the very end, after the last `/`.

Flag	What it does
(none)	Replace only the **first occurrence** on each line in range
`g`	**Global**: replace **all occurrences** on each line (not just the first)
`c`	**Confirm**: ask `y/n` for **each** replacement. You see the match highlighted and choose.
`i`	Case-**insensitive** matching
`I`	Case-**sensitive** matching. Needed here: `:s` ignores case completely, even with capitals in the pattern (`:%s/Old/X/g` on `old Old OLD` gave `X X X`, with `gI` only `Old` changed)
`n`	**Count only**: show how many matches there are without replacing anything
`e`	Suppress "pattern not found" error

Flag combinations

Command	What happens
`:%s/old/new/`	Replace the first `old` on each line in the file
`:%s/old/new/g`	Replace **every** `old` in the entire file
`:%s/old/new/gc`	Replace every `old`, but **ask confirmation** for each one
`:%s/old/new/gi`	Replace every `old` case-insensitively (`Old`, `OLD`, `old` all match)
`:%s/old/new/gn`	**Count** how many `old` exist in the file (no replacement)
`:%s/old/new/gce`	Confirm each, and don't error if not found

Results on the two lines `old old` / `old`:

Command	Result
`:%s/old/new/`	`new old` / `new`
`:%s/old/new/g`	`new new` / `new` (message `3 substitutions on 2 lines`)
`:%s/old/new/gn`	no change, message `3 matches on 2 lines`

And on the single line `old Old OLD` (`:s` ignores case here, see the `I` flag above):

Command	Result
`:%s/old/X/gi`	`X X X`
`:%s/Old/X/g`	`X X X` (the capital in the pattern does not matter)
`:%s/Old/X/gI`	`old X OLD` (only the exact `Old` changed)

The same holds with word boundaries: in a typed `:s` the case is always ignored unless you add `\C` (inside the pattern) or the `I` flag; a capital in the pattern does not help. (Reason: while you type a `:` command, smart case is switched off by an autocmd in `lua/custom-autocmd.lua`.) Run with typed keys in tmux 2026-10-05, real Neovim with this config, on the line `Result and result and RESULT`:

Command	Matches	Result
`:%s/\<result\>/X/ge`	all three words (case ignored)	`X and X and X`
`:%s/\<Result\>/X/ge`	all three words (a capital does NOT make it exact)	`X and X and X`
`:%s/\C\<result\>/X/ge`	only the exact lowercase `result` (`\C` forces exact case)	`Result and X and RESULT`
`:%s/\<result\>/X/gIe`	only the exact lowercase `result` (the `I` flag)	`Result and X and RESULT`
`:%s/\<Result\>/X/gIe`	only the exact `Result` (`I` plus the exact-case word)	`X and result and RESULT`

Take care with renames: because `:s` ignores case, renaming a variable `calc` also changes the class `Calc`, `Result`, `RESULT` and similar. Use `\C` (or the `I` flag) when only the exact case is meant. This applies equally to `:cfdo`, `:bufdo` and `:argdo` renames (see "Using regex with `:grep`" and "Example 6" below). `/` and `?` searches keep smart case.

Quick difference between the three common endings (same `:%s/old/new` start, only the end changes):

Ending	Replaces	Asks you?
`/` (no flag)	only the **first** `old` on each line	no
`/gc`	**every** `old` on each line	yes, `y/n` for each one
`/gcc`	**every** `old` on each line (the second `c` switches confirmation off again, so `/gcc` behaves like `/g`; `/gccc` asks again)	no

Do not confuse `/gcc` with the normal-mode `gcc` (toggle comment on the current line): inside `:s/…/…/` the letters are flags, outside it `gcc` is a command.

Confirmation mode (`c` flag) controls

When you use the `c` flag, Vim highlights each match and asks what to do:

Key	Action
`y`	**Yes**, replace this one and move to next
`n`	**No**, skip this one and move to next
`a`	**All**: replace this one and all remaining (stop asking)
`q`	**Quit**: stop replacing now
`l`	**Last**: replace this one and then stop
`<Ctrl-e>`	Scroll down to see more context
`<Ctrl-y>`	Scroll up to see more context

Example: on the lines `old old` / `old`, `:%s/old/new/gc` highlights the first `old` and the bottom line asks

```
replace with new? (y)es/(n)o/(a)ll/(q)uit/(l)ast/scroll up(^E)/down(^Y)
```

Press `y` (first `old` becomes `new`), `n` (the second stays `old`), `a` (the last one and everything after it is replaced without asking): the result is `new old` / `new`.

Changing the delimiter

If your search/replace text contains `/`, use a different delimiter to avoid confusion:

Command	What it does
`:%s#/usr/local/bin#/opt/bin#g`	Replace path using `#` as delimiter
`:%s|old|new|g`	Use `|` as delimiter

You can use almost any character as a delimiter. Just use the same character for all three separators.

Example: on the line `/usr/local/bin/x`, `:s#/usr/local/bin#/opt/bin#` gives `/opt/bin/x`.

Special replacement patterns

Pattern in replacement	What it means
`&`	The entire matched text
`\1`, `\2`, etc.	Capture group 1, 2, etc. from `\( \)` in the search
`\u`	Uppercase the next character
`\U`	Uppercase everything after this
`\l`	Lowercase the next character
`\L`	Lowercase everything after this
`\r`	Newline (line break)

Examples:

Command	Before	After
`:s/cat/[&]/`	`cat dog`	`[cat] dog`
`:s/\w\+/\U&/`	`foo bar`	`FOO bar`
`:s/\w\+/\L&/`	`FOO BAR`	`foo BAR`
`:s/\w\+/\l&/`	`FOO BAR`	`fOO BAR`
`:s/, /\r/g`	`a, b, c`	`a` / `b` / `c` (3 lines)
`:%s/\v(\w+), (\w+)/\2 \1/g`	`Smith, John` / `Doe, Jane`	`John Smith` / `Jane Doe`
`:%s/\v(\w+), (\w+)/\2, \1/g`	`Smith, John` / `Doe, Jane`	`John, Smith` / `Jane, Doe` (the comma stays)

Substitution examples

Command	What it does
`:%s/foo/bar/g`	Replace all `foo` with `bar`
`:%s/\<foo\>/bar/g`	Replace only whole-word `foo` (not `foobar`)
`:%s/foo/bar/gc`	Replace all, confirming each one
`:%s/foo//g`	Delete all occurrences of `foo`
`:%s/\v(\w+), (\w+)/\2, \1/g`	Swap two comma-separated words: `last, first` becomes `first, last`
`:%s/\<\(\w\)/\u\1/g`	Capitalize the first letter of every word
`:%s/$/;/`	Add a semicolon at the end of every line
`:%s/^\s*$\n//g`	Delete all blank lines
`:20,30s/TODO/DONE/g`	Replace only between lines 20-30
`:'<,'>s/old/new/g`	Replace only in the visual selection

Results:

Command	Before	After
`:%s/foo//g`	`foo foo x`	`  x` (two spaces are left)
`:%s/\<foo\>/bar/g`	`foo foobar`	`bar foobar`
`:%s/\<\(\w\)/\u\1/g`	`hello big world`	`Hello Big World`
`:%s/$/;/`	`a` / `b`	`a;` / `b;`
`:%s/^\s*$\n//g`	`a` / (empty) / `b` / (only spaces) / `c`	`a` / `b` / `c`
`:2,3s/x/y/`	`x` / `x` / `x` / `x`	`x` / `y` / `y` / `x`

Searching in the current buffer with `*` and `#`

These are the fastest ways to search for a word:

1. Place cursor on any word
2. Press `*` – all occurrences highlight, the hlslens overlay shows `[1/N]`
3. Press `n` to jump forward, `N` to jump backward
4. With no count the cursor stays on the word you pressed `*` on (custom behavior in this config: a whole-word, literal search; `3*` jumps to the 3rd match after the cursor instead)

This is often combined with `ciw` + `.` for selective replacement (see below).

Using `ciw` + `.` for selective single-file replacement

This is the **most practical replacement method** for everyday use. It gives you full control, replacing one occurrence at a time:

1. Place cursor on the word you want to replace (e.g., `oldName`)
2. `*` – search for it (all occurrences highlight)
3. `ciw` – delete the word and enter insert mode
4. Type the new word (e.g., `newName`), then press `<Esc>`
5. `n` – jump to the next occurrence
6. Decide: press `.` to replace this one too, or `n` to skip it
7. Repeat step 5-6 until done

Example: the line is `foo a foo b foo`, the cursor is on the first `foo`.

1. `*` (all three `foo` highlight, the cursor stays on the first)
2. `ciw`, type `bar`, `<Esc>`: `bar a foo b foo`
3. `n` jumps to the second `foo`: you decide to skip it, so `n` again jumps to the third `foo`
4. `.` repeats the change there: `bar a foo b bar`. The middle `foo` was left alone.

Why this is great: Unlike `:%s`, you see each occurrence in context and can decide whether to replace it. Unlike `:%s/old/new/gc`, you stay in normal mode between replacements and can scroll around.

---

67. Multi-file search and replace (complete guide)

This is the section you need when you want to find or replace text across your entire project – not just the current file.

Quick decision guide: which method to use

Scenario	Best method
Rename a function/variable/class (code-aware)	**LSP Rename** (`<Space>rn`)
Replace a plain string in many files	**`:grep` + `:cfdo`**
Replace only in certain file types (e.g., only `.py`)	**`:grep --type` or `:vimgrep`, then `:cfdo`**
Just find where something is used (no replace)	**`<Space>fg`** (live grep)
Replace with confirmation for each occurrence	**`:cfdo` with the `gc` flag**

Method 1: LSP rename (best for code symbols)

If you're renaming a function, variable, class, or any code symbol, this is the best method because it understands scope and language semantics.

1. Place cursor on the symbol you want to rename
2. Press `<Space>rn`
3. Type the new name
4. Press `<Enter>`

Example (with pyright): `a.py` contains `def load():` and `b.py` contains `from a import load` and `print(load())`. With the cursor on `load` in `a.py`, `<Space>rn` opens a small prompt

with `load` already in it; clear it (`<Ctrl-w>`), type `read` and press `<Enter>`. All three places are renamed (`def read():`, `from a import read`, `print(read())`), including the other file (it is changed in a buffer: save it with `:wa`). This works across files only in a project with a root marker such as `.git` or `pyproject.toml`; without one pyright only sees the open file (only `a.py` was renamed). A same-named symbol in another scope is left alone (in a project with a `pyproject.toml`: with `count` in two functions `f` and `g`, renaming `count` to `total` inside `f` changed only `f`).

What happens: The LSP server finds every reference to that symbol across the entire project and renames them all. It's smart: renaming `count` in one function won't affect `count` in another function.

Limitations: Only works for code symbols (not arbitrary text), and requires an LSP server that supports rename.

LSP rename versus `:grep` + `:cfdo`: can the fast text rename replace a slow `<Space>rn`?

Short answer: NOT a drop-in replacement. It is a fast alternative only when the word is unique enough, and only if you review what it matches first. `:cfdo` is a plain text substitution (like `:%s/old/new/g`), not a semantic rename. It does NOT run over the whole workspace and NOT over the open buffers: it runs over the FILES IN THE QUICKFIX LIST, that is exactly the files in which ripgrep (`:grep`) found a match (`:cfdo` opens each one, runs the command, and `| update` saves it). Open buffers are irrelevant (`:bufdo` is the one that walks open buffers).

	LSP rename (`<Space>rn`)	`:grep` + `:cfdo`
What it understands	Code: changes the symbol only (its declaration and real usages); a same-named method or variable in another class is NOT touched; strings and comments are not touched; knows overloads and scopes	Text: changes every occurrence of the characters or word: comments, strings, same-named but unrelated symbols, other file types (`pom.xml`, `.md`)
Speed	Java with jdtls: 10 to 17 s when healthy; can be slower or fail with a stale index (see When a rename does nothing)	About a second
Needs	An attached language server with a healthy index	Only ripgrep
Review before	No preview: the buffers change after the answer arrives	`:copen` to read every match first; the `c` flag confirms each change: `:cfdo %s/\<old\>/new/gc | update`
Undo	Several files change: see Undoing a multi-file replace	The same

Example (scratch copy of a small Java project, headless Neovim with this config): `:grep "add"` then `:cfdo %s/\<add\>/addition/ge | update`. The method `add` of class `Calc` exists in `Calc.java` and is called from three other files. The text rename ALSO changed

a comment that mentioned `add`, and a `names.add("apple")` line in another file, which is `java.util.List.add`, not `Calc.add`. In real code a `list.add(...)` call would be a compile error after the rename. `<Space>rn` on `Calc.add` changed only the method and its calls.

- The word-boundary form `\<word\>` avoids matching inside longer words.
- With a local variable `total`: `:grep "total"` then `:cfdo %s/\<total\>/result/g | update` also changed a string literal `"total is "` and a comment. Fine when you want that, wrong when the string is output text.
- In a folder WITHOUT a `.git` directory: a `.gitignore` entry (`target`) was NOT honored; `:grep` also listed a file in the ignored `target/` folder, so `:cfdo` would have changed it too. Ripgrep honors `.gitignore` only inside a git repository (inside one, an ignored `target/` folder is skipped). Check build output folders such as `target/` in the `:copen` list before running `:cfdo`.

Which to use (rule of thumb):

- A method, field, class or local variable whose name also occurs elsewhere (`add`, `get`, `name`, `size`, `value`): `<Space>rn`.
- A long, unique name (`calculateInvoiceTotal`), or a name in non-code text (config key, string, docs, comment wording): `:grep`, review with `:copen`, then `:cfdo`.
- `<Space>rn` fails or hangs: first the repair `<Space>jbc` (Java: When a rename does nothing); the text rename is a fallback only when the matches are reviewed.
- A Java package or folder cannot be renamed with `<Space>rn` ("not supported on this element"); rename the folder in the file tree with `r` instead: jdtls then rewrites the `package` and `import` lines by itself (File operations and language servers), with `:grep` + `:cfdo` as the fallback: Renaming a package or folder.
- Renaming a class by text does NOT rename its file.

Method 2: `:grep` + `:cfdo` (best for plain text)
This is the most versatile method. It uses ripgrep (very fast) to search the entire project, puts results in the quickfix list, then runs a command on each file of the list.

Requirement: fill the quickfix list FIRST. `:cfdo` (and `:cdo`) has no search of its own. It only runs a command on the files (or entries) that are ALREADY in the quickfix list, so the order is always:
1. fill the list (`:grep`, `:vimgrep`, `:make`, diagnostics: see

What populates the quickfix list),
2. check it with `:copen`, 3. run `:cfdo`. With an EMPTY list `:cfdo`

and `:cdo` do nothing and print no error (in headless Neovim: the commands returned without error; a visible message in a UI session was not seen). So "nothing happened" usually means the list was empty: look with `:copen`. The sibling commands for other file sets (location list, buffers, argument list, windows, tabs) are in Understanding `:cdo` vs `:cfdo` vs `:bufdo`.

Why `:cfdo` and not `:cdo`: `:grep` here creates ONE quickfix entry per match. `:cdo s/x/y/g` visits a line once per match; after the first visit replaced every `x` on the line, the next visit finds nothing and stops with `E486: Pattern not found`, so the rest is NOT replaced. `:cfdo %s/x/y/g` runs once per file and avoids this. If you prefer `:cdo`, add the `e` flag: `:cdo s/x/y/ge | update`.

Step-by-step: replace all without confirmation

```
:grep "oldFunction"                       -- search entire project
:cfdo %s/oldFunction/newFunction/g | update   -- replace in every matching file and save it
```

Step-by-step: replace with confirmation for each occurrence

```
:grep "oldFunction"                       -- search entire project
:cfdo %s/oldFunction/newFunction/gc | update  -- 'c' flag asks y/n for EACH occurrence
```

When the `c` flag is active, for each match you see it highlighted and can press:

- `y` to replace this one
- `n` to skip this one
- `a` to replace all remaining in this file (then moves to the next file, where you are prompted again)
- `q` to stop entirely

Step-by-step: review results before replacing

```
:grep "oldFunction"                       -- search entire project
:copen                                    -- open the quickfix window to review all results
```

Now you can see every file and line that matches. Use `:cnext`/`:cprev` (or `j`/`k` in the quickfix window then `<Enter>`) to jump through them. Once satisfied:

```
:cfdo %s/oldFunction/newFunction/g | update   -- replace and save
```

Using regex with `:grep`

`:grep` passes the pattern directly to ripgrep, so you can use ripgrep regex. It is smart-case: an all-lowercase pattern ignores case, a pattern with a capital is exact (add `-s` to force exact case). For an exact-case project-wide rename BOTH sides must be exact: `:grep -s "word"` to collect the files and `\C` (or the `I` flag) in the substitution, for example `:cfdo %s/\Cword/new/g | update`. Otherwise `:s` still changes `Word` and `WORD` in those files (in a single file, see "Substitution" above):

Command	What it finds
`:grep "TODO"`	All lines containing `TODO`
`:grep "TODO|FIXME"`	Lines with `TODO` or `FIXME` (type `|` with the backslash; a rendered table may hide it)
`:grep "\buser\b"`	Only the whole word `user`
`:grep "def \w+\("`	Python function definitions
`:grep "console\.log"`	All `console.log` calls

What `:grep "TODO"` puts in the quickfix list (the list has one row per match: file, line number and the text of the line):

```
src/c.py │ 1│# TODO: fix
src/c.py │ 2│print(1)  # TODO later
src/d.py │ 1│# TODO: three
```

`:grep "TODO\|FIXME"` (on a small project with `TODO` and `FIXME` in three files) fills the list with every line containing either word. The order of the files can vary, because ripgrep searches in parallel. A line with two matches (such as `x and x` for `:grep "x"`) appears twice in the list: this is why `:cfdo` is used below.

Limiting to specific file types
Ripgrep supports file type filters:

Command	What it searches
`:grep "pattern" --type py`	Only Python files
`:grep "pattern" --type js`	Only JavaScript files
`:grep "pattern" --type java`	Only Java files
`:grep "pattern" src/`	Only files in the `src/` directory
`:grep "pattern" --glob "*.tsx"`	Only `.tsx` files

Method 3: `:vimgrep` + `:cfdo` (built-in, slower but portable)
`:vimgrep` is Vim's built-in search (doesn't require ripgrep). It's slower but lets you use Vim regex and file glob patterns:

Command	What it does
`:vimgrep /pattern/ **/*`	Search all files recursively
`:vimgrep /pattern/ **/*.py`	Search only Python files
`:vimgrep /pattern/ **/*.{js,ts}`	Search JS and TS files
`:vimgrep /pattern/ src/**/*`	Search only in `src/` directory

Then use `:cfdo` as before:

```
:vimgrep /oldName/ **/*.java
:cfdo %s/oldName/newName/g | update
```

Method 4: `<Space>fg` for finding (no replace)
`<Space>fg` (live grep via fzf-lua) is the fastest way to **find** where something is used, but it doesn't directly support replace. Use it for:

- Exploring: "Where is this function called?"
- Investigating: "Which files reference this config key?"
- Planning: "How many places use this pattern?" before deciding on a replace strategy

After reviewing results in fzf, you can then use `:grep` + `:cfdo` for the actual replacement.

Complete examples

Example 1: rename an API endpoint across the project
You renamed `/api/users` to `/api/v2/users`:

```
:grep "/api/users"                          -- find all references
:copen                                      -- review: make sure you're not catching wrong
things
:cfdo %s#/api/users#/api/v2/users#g | update   -- replace (using # as delimiter since / is
in the text) and save
```

Example 2: replace a deprecated function name (with confirmation)

```
:grep "getUser"
:cfdo %s/getUser/fetchUser/gc | update   -- confirm each one ('y' to replace, 'n' to skip)
```

Example 3: delete all `console.log` statements in JavaScript files

```
:grep "console\.log" --type js            -- find them
:cfdo g/console\.log/d                    -- delete every line containing the match
:cfdo update                              -- save the files
```

Example 4: add a comment before every TODO

```
:grep "TODO"
:cfdo %s/TODO/NOTE: was TODO/g | update
```

Example 5: replace only in Python files in the src/ directory

```
:grep "old_function" --type py src/
:cfdo %s/old_function/new_function/g | update
```

Example 6: case-insensitive project-wide replace

```
:grep -i "oldname"                        -- ripgrep's -i flag for case-insensitive
:cfdo %s/oldname/newname/g | update       -- :s ignores case in this config anyway; for
exact case use :grep -s and \C
```

Understanding `:cdo` vs `:cfdo` vs `:bufdo`

These are the "do-command family": each runs `{cmd}` over a different list. None of them searches by itself; they only walk a list that already exists (the quickfix and location commands need it filled first, see the requirement in Method 2).

Command	Which list it walks	Filled by / source
`:cdo {cmd}`	Every **entry (line)** of the quickfix list; may visit the same file several times	`:grep`, `:vimgrep`, `:make`, diagnostics keys `<Space>qw` / `<Space>qb` (see What populates the quickfix list)
`:cfdo {cmd}`	Once per **file** of the quickfix list	The same sources
`:ldo {cmd}` / `:lfdo {cmd}`	Like `:cdo` / `:cfdo`, but over the **location list** (one per window); open it with `:lopen`	`:lgrep`, `:lvimgrep`, `:lmake`
`:bufdo {cmd}`	Every buffer in the **buffer list** (see `:ls`), shown in a window or not. Project files that are not open are NOT visited (documented `:bufdo` behaviour)	Files you opened
`:argdo {cmd}`	Every file of the **argument list**: the files given on the command line (`nvim a b c`) or set with `:args <glob>`, e.g. `:args src/**/*.java`	`:args`
`:windo {cmd}`	Every **window of the current tab page**	Your splits
`:tabdo {cmd}`	Every **tab page** (runs in the current window of each tab)	Your tabs

Which one when:

I want to change…	Use
Project text found by a search	`:grep "old"` then `:cfdo ...`
Per-window results (several searches at once, each in its own window)	`:lgrep "old" .` then `:lfdo ...`
The files I already have open	`:bufdo ...`
An explicit set of files	`:args <files or glob>` then `:argdo ...`
The windows I see now	`:windo ...`
Every tab	`:tabdo ...`

Scratch Java project, headless Neovim:

- `:lgrep "total" .` then `:lfdo %s/\<total\>/loc/ge | update` changed both files that had matches (13 entries, 6 + 7 substitutions).
- With two files loaded and a third never opened, `:bufdo %s/\<total\>/buf/ge | update` changed the two loaded ones. The third had no match anyway, so this alone does not prove it would be skipped; that a file outside the buffer list is not visited is the documented behaviour of `:bufdo`.
- `nvim a b`, then `:argdo %s/\<total\>/arg/ge | update` changed both files.
- With two split windows `:windo ...` and with two tabs `:tabdo ...` (same substitution with `| update`) changed both files.

- With an EMPTY quickfix or location list, `:cdo`, `:cfdo` and `:lfdo` do nothing and print no error (headless; a visible message in a UI session was not seen). "Nothing happened" usually means an empty list: check with `:copen` / `:lopen`.

Gotcha: always add the `e` flag for `:bufdo`, `:argdo`, `:windo` and `:tabdo` substitutions: `%s/old/new/ge`. Without it, `:bufdo %s/\<total\>/buf/g | update` STOPPED at the first buffer without a match with `E486: Pattern not found` and the following buffers were not processed (the same stop rule as the `:cdo` stop below). `:cfdo` after `:grep` is safe without `e` only because every file in the list has a match.

The `|` chain: `{cmd} | update` runs the command and then saves the buffer. Without `update` (or `set hidden` / autowrite) a modified buffer may refuse to be left (`E37: No write since last change`). The auto-save plugin of this config saves when you leave a buffer, but do not rely on it for these commands.

`:argdo` as a project-wide file set without grep: `:args **/*.java`, then `:argdo %s/old/new/ge | update`.

For search-and-replace use `:cfdo %s/old/new/g` (see Method 2 for why a plain `:cdo s/old/new/g` can stop early). `:cdo` is fine for commands that act on the entry's line once, or with the `e` flag.

Example: the file `x.txt` has the single line `x and x`.

Steps	What happens
`:grep "x" x.txt`	The quickfix list has TWO entries, both for line 1 (one per `x`)
`:cdo s/x/y/g`	Entry 1 (`(1 of 2)`): the line becomes `y and y`. Entry 2 (`(2 of 2)`): nothing is left to replace, so it stops with `E486: Pattern not found: x`
`:cdo s/x/y/ge`	The same, but the `e` flag hides the error: no message
`:cfdo %s/x/y/g`	One visit per file: `y and y`

Undoing a multi-file replace
If the replace went wrong, each file has its own undo history:

1. `:cfdo undo` – undo the last change in every affected file
2. `:cfdo update` – save the reverted files

Or use `:cfdo earlier 1f` to go back one save-state in each file (both files were restored exactly with both recipes).

---

12. Fuzzy finding & project-wide search (`fzf-lua`)

Plugin: **fzf-lua**. A powerful popup interface that connects to FZF (a command-line fuzzy finder). It lets you search file names, search text inside files, browse buffers, and more. The popup opens centered on screen at 70% height.

Moving inside any picker (lists with a search bar)
Several things look the same: a search bar on top and a filtered list below it. They are the fzf-lua pickers (`<Space>ff`, `<Space>fg`, `<Space>fb`, `<Space>fr`, `<Space>gbl` …), the snacks pickers (the branch menu you get by clicking the branch in the statusline, the code-action menu `<Space>ca`, every other `vim.ui.select` list) and Telescope (`:Telescope`,

`<Space>db`). The `:DevDocs` lists are `vim.ui.select` lists, so they are snacks pickers too. The search bar takes your typing, so `j` and `k` type the letters `j` and `k` there. Move through the list with these keys (all in a real terminal):

Keys	What it does
`<Ctrl-n>` / `<Ctrl-p>`, or `<Down>` / `<Up>`	Next / previous item. Works in all three kinds of picker.
`<Ctrl-j>` / `<Ctrl-k>`	The same, but only in fzf-lua and the snacks pickers (NOT in Telescope)
`<Tab>`	Move down. In fzf-lua and Telescope it also marks the item, so you can choose several at once
`<Enter>`	Choose the item (or the marked ones)
`<Esc>`	fzf-lua: close the picker. snacks and Telescope: the first `<Esc>` leaves the search bar for Normal mode, where `j` / `k` move the list; a second `<Esc>` closes it

The same `<Ctrl-n>` / `<Ctrl-p>` also move through the completion menu (section 14); on the `:` command line use `<Tab>` / `<S-Tab>`.

Keymaps

Keymap	Description
`<Space>ff`	**Find files**: search file names in the project
`<Space>fg`	**Live grep**: search text content across all files in the project
`<Space>fh`	Search Neovim help tags
`<Space>ft`	Search tags (functions, classes) in the current buffer (needs `ctags`; the Nix nvim wrapper provides universal-ctags)
`<Space>fb`	Search currently open buffers
`<Space>fr`	Search recently opened files
`<Space>fs`	Search only your own snippets (`my_snippets/`) and insert one; also `<Alt-s>` in insert mode (section 52, The snippet gallery)
`<Space>gbl`	Fuzzy-search git branches (`<Enter>` checks the branch out)
`<Space>gB`	Branch menu (the same menu as clicking the branch name in the statusline): choose a branch and Neovim switches to it. Move with `<Ctrl-n>` / `<Ctrl-p>`, see "Moving Inside Any Picker"

`<Space>ff` has no preview window and shows git status icons next to modified/ untracked files; `.gitignore` is respected.

Inside the fzf-lua popup

Key	What it does
Type text	Filters results in real-time
`<Enter>`	Open the selected result
`<Esc>`	Cancel and close the popup
`<Ctrl-j>` / `<Ctrl-k>`	Move down / up in the results list
`<Ctrl-n>` / `<Ctrl-p>`	Move down / up (alternative keys)

`<Space>fg` – Live grep (project-wide text search) in depth

This is one of the most important keymaps for developers. It searches inside every file in your project directory using **ripgrep** (`rg`) under the hood.

What it does

1. Press `<Space>fg`
2. A popup appears with a search prompt
3. As you type, ripgrep searches **all files** in the project folder and shows matching lines in real-time
4. Results show: file path, line number, and the matching line
5. Press `<Enter>` to jump directly to that file and line

What the popup looks like after typing `TODO` (the list is on top, the preview of the highlighted match below; move with `<Ctrl-n>` / `<Ctrl-p>` or `<Down>` / `<Up>`, open with `<Enter>`):

```
+---------------------------- Grep -----------------------------+
| > TODO                                                3/3 (0) |
|  :: <ctrl-g> to Fuzzy Search                                  |
| > src/c.py:1:3:# TODO: fix                                    |
|   src/c.py:2:13:print(1)  # TODO later                        |
|   src/d.py:1:3:# TODO: three                                  |
+---------------------- buf 1: c.py ----------------------------+
|  1 # TODO: fix                                                |
|  2 print(1)  # TODO later                                     |
+---------------------------------------------------------------+
```

Each result is `file:line:column:text`.

Plain text search

Just type normal text. For example, typing `getUserById` finds every file and line containing that string.

Regex search

Live grep supports **full regex** (ripgrep regex syntax). You don't need to learn all of regex, but here are the most useful patterns:

Pattern you type	What it finds	Example matches
`TODO`	Literal text `TODO`	`// TODO: fix this`
`TODO|FIXME`	`TODO` OR `FIXME`	Both `// TODO` and `// FIXME`
`def \w+\(`	Python function definitions	`def process_data(`, `def main(`
`class \w+`	Class declarations	`class UserService`, `class App`
`import.*from`	ES6-style imports	`import { foo } from 'bar'`
`console\.log`	`console.log` (dot is escaped)	`console.log("debug")`
`function\s+\w+`	JavaScript function declarations	`function handleClick`
`\berror\b`	Whole word `error` only	`error` but not `errors` or `errorHandler`
`https?://`	URLs (http or https)	`https://example.com`
`v[0-9]+\.[0-9]+`	Version strings	`v1.0`, `v2.13`

Use cases for live grep

Scenario	What to search
Find where a function is called	Type the function name
Find all TODOs	Type `TODO`
Find a specific error message	Type part of the error string
Find all API endpoints	Type `@GetMapping` (Java) or `app.get(` (Express) or `@app.route` (Flask)
Find all imports of a module	Type `import.*moduleName` (regex)
Find environment variable usage	Type `process.env` or `os.environ`
Find hardcoded strings	Type the string in quotes

`<Space>ff` – Find files (file name search)

Searches **file names** (not content). Useful when you know the file you want but not the exact path.

- Type `userserv` to find `UserService.java` (fuzzy matching)
- Type `config.py` to find configuration files
- Type `.env` to find environment files (only if they are not git-ignored: `.gitignore` is respected; a `.ignore` file containing `!.env` makes them visible)
- Type `test` to see all test files

The difference between search methods

Method	Keymap	What it searches	Best for
Live grep	`<Space>fg`	Text **inside** files across the entire project	Finding where code/text is used
Find files	`<Space>ff`	**File names** in the project	Opening a file by name
Buffer search	`<Space>fb`	Names of **currently open** files	Switching between open files
Recent files	`<Space>fr`	Files you **recently edited**	Returning to a file you had open earlier
Buffer tags	`<Space>ft`	Functions/classes in **current file** (needs `ctags`)	Jumping to a function in the current file
In-file search	`/pattern`	Text in **current file only**	Finding something in the file you're editing
Word under cursor	`*`	Current word in **current file**	Quick highlight and jump to next occurrence

---

26. Quickfix & location list

Commands

Command	Description
`:copen` / `:cclose`	Open / close quickfix window
`:cnext` / `:cprev`	Next / previous item
`:cfirst` / `:clast`	First / last item
`:cc [nr]`	Jump to specific entry
`:cdo {cmd}`	Run command for each entry
`:colder` / `:cnewer`	Navigate quickfix history
`:lopen` / `:lclose`	Location list (per-window)
`\x`	Close quickfix and location list windows

Inside the quickfix window (nvim-bqf, quicker.nvim)

quicker.nvim formats the list (grouped by file, file-name column at most 40 characters or half the screen); the list cannot be edited as a buffer. nvim-bqf adds these keys inside the quickfix window (the preview does not start by itself):

Key	Description
`p` / `P`	Toggle the preview of the item / toggle auto-preview while moving
`zp`	Toggle the preview between normal and maximum size
`<Ctrl-f>` / `<Ctrl-b>`	Scroll the preview down / up
`<Tab>` / `<S-Tab>`	Mark the item and move down / up; `z<Tab>` clears the marks
`zn` / `zN`	New list from the marked / unmarked items
`zf`	Fuzzy filter the list (fzf)
`<` / `>`	Go to the older / newer quickfix list
`o`	Open the item and close the quickfix window
`O`	Open the item in a window that already shows the buffer (otherwise open it) and close the quickfix window, like `o` (the quickfix window was closed afterwards; `<Enter>` is the key that keeps the list open)
`t` / `T`	Open in a new tab (`T` stays in the quickfix window)
`<Ctrl-x>` / `<Ctrl-v>` / `<Ctrl-t>`	Open in a horizontal split / vertical split / new tab

Example: `:grep "TODO"` gives 3 hits and `:copen` opens the list. Press `<Tab>` twice: the first two hits are marked (each `<Tab>` marks and moves down). Press `zn`: a NEW list is made with just those 2 hits (the old list is kept). `<` goes back to the older list with all 3 hits, `>` forward to the new one. `o` on a hit opens it and closes the quickfix window, `<Enter>` opens it and keeps the list open.

---

51. Quickfix workflows for developers

The quickfix list is a central tool for developers. It's a list of locations (file + line number) that you can jump through. Many features populate it.

What populates the quickfix list

Source	How to populate	Description
Project-wide search	`:vimgrep /pattern/ **/*`	Search results across all files
LSP diagnostics	`<Space>qw`	Diagnostics of all open buffers
Buffer diagnostics	`<Space>qb`	Errors/warnings in current file only
Build errors	`:make`	Compiler output
Grep	`:grep pattern`	Uses ripgrep (configured in this setup)

Navigating the quickfix list

Command / Keymap	What it does
`:copen`	Open the quickfix window at the bottom
`:cclose` or `\x`	Close the quickfix window (`\x` also closes all location lists)
`:cnext`	Jump to the next item
`:cprev`	Jump to the previous item
`:cfirst` / `:clast`	Jump to the first / last item
`:cc 5`	Jump to item number 5
`:colder` / `:cnewer`	Go to the previous / next quickfix list (history)

Batch operations on quickfix items

Command	What it does
`:cfdo %s/old/new/g | update`	Run a substitution once in every file of the quickfix list, then save it
`:cdo s/old/new/ge | update`	The same per quickfix entry (the `e` flag is needed, see the Multi-File Search and Replace section)

Example workflow: Rename a string across the project:

1. `:grep "oldName"` to populate quickfix with all occurrences
2. `:cfdo %s/oldName/newName/g | update` to replace in all files and save them

trouble.nvim: better quickfix UI

Plugin: **trouble.nvim**. A nicer interface for browsing diagnostics and quickfix items.

Keymap / Command	What it does
`<Space>dw`	Toggle Trouble with the diagnostics of every loaded buffer, grouped by file (`:Trouble diagnostics toggle`; see "Diagnostics in depth" in the code chapter)
`:Trouble`	Open Trouble window

Trouble shows diagnostics grouped by file with icons and colors, making it easier to triage errors.

---

11. File explorer (`nvim-tree`)

Plugin: nvim-tree.lua. A sidebar file tree. It loads on the first `<Space>s` or the first `:NvimTreeToggle`, `:NvimTreeOpen`, `:NvimTreeFocus`, `:NvimTreeFindFile` or `:NvimTreeFindFileToggle`; `nvim <dir>` and the dashboard entry open it too. Which folder it shows: `nvim` or `nvim .` shows the folder you started in; `nvim <folder>` shows that folder; `nvim <folder>/file` shows the folder you started in (not the file's folder). From then on it keeps that folder until the working folder changes: after `:cd`, `:tcd`, `:Z` or `:z` it shows the new folder (also when it is open, or was closed and is opened again).

Keymap	Context	Description
`<Space>s`	global	Toggle the file explorer on/off
`<Enter>`	in tree	Open file (cursor moves to file) / expand directory. With 2 or more editor windows open and the file not already shown in one of them, it asks "Pick window:": a letter (A, B, C, …) is drawn in the middle of each window's status line, press the letter of the window you want; any other key cancels. A file that is already open in a window is just focused. The dimming plugin vimade is switched off only while the letters are shown (otherwise they were nearly invisible) and switched on again right after. The tree itself always opens at the far left of the screen.
`<Tab>`	in tree	Open file but **keep cursor in the tree** (great for opening multiple files)
`<BS>`	in tree	Close (collapse) the directory under the cursor; on a file it closes its parent directory
`a`	in tree	Create a new file. Type the name and press Enter. Add `/` at the end for a directory.
`d`	in tree	Delete file/directory (asks for confirmation)
`r`	in tree	Rename file/directory. A language server that knows file operations (jdtls for Java) is told and can fix the references: File operations and language servers
`c`	in tree	Copy file to clipboard
`x`	in tree	Cut file to clipboard
`p`	in tree	Paste from clipboard
`q`	in tree	Close the file explorer
`D`	in tree	Move file/directory to the trash (asks for confirmation; needs `trash`)
`g?`	in tree	Show the help with every key
`<Ctrl-v>` / `<Ctrl-x>` / `<Ctrl-t>`	in tree	Open in a vertical split / horizontal split / new tab
`-` / `<Ctrl-]>`	in tree	Make the parent / the folder under the cursor the root
`H` / `I`	in tree	Toggle dotfiles / git-ignored files
`f` / `F`	in tree	Live filter: start / clear
`R`	in tree	Refresh
`y` / `Y` / `gy`	in tree	Copy the name / relative path / absolute path
`]c` / `[c`, `]e` / `[e`	in tree	Next / previous git item, diagnostic item
`E` / `W`	in tree	Expand all / collapse all

`nvim <dir>` opens the tree on that directory and makes it the working directory.

Moving between tree and code: Use `<Ctrl-w>h` / `<Ctrl-w>l` or `<Left>` / `<Right>` arrow keys.

File operations and language servers (nvim-lsp-file-operations)
Why: renaming or moving a file or folder changes things the language server tracks, above all in Java the `package` line and the `import` lines of other files. nvim-lsp-file-operations tells the server about every rename, move, create and delete done in the file tree (the LSP requests `workspace/willRenameFiles`, `didRenameFiles` and the create/delete equivalents), and the server answers with the text edits that keep the project consistent.

How it is wired:

- It adds NO keys. You use the tree's own keys (`r` rename, `x` + `p` cut and paste, `a` create, `d` delete, see the table above). The one command is `:LFO` (`create`, `delete`, `rename`), for scripts.
- It loads right after nvim-tree's setup, so it costs nothing at startup. The matching client capabilities (`workspace.fileOperations`) are advertised to every language server from `lua/lsp_utils.lua`; a server only offers file operations to a client that advertises them.
- Servers that react: jdtls (it offers `willRename` for `**/*.java` files and for folders; this is the case documented here) and tinymist (it offers `willRename` for `**/*.typ` files). For servers that do not handle file operations, nothing changes.
- Only the file tree is hooked. The vim-eunuch commands `:Rename` and `:Move` and a plain `:w` do NOT tell the server.

Example (Java, Maven project): the folder `src/main/java/com/acme/util` holds `Helper.java` with `package com.acme.util;`, and `App.java` has `import com.acme.util.Helper;`.

1. `<Space>s` opens the tree, put the cursor on the folder `util`.
2. `r`, type `helpers`, `<Enter>`.
3. The folder is renamed on disk. In the moved `Helper.java` the line becomes `package com.acme.helpers;` and the file is saved. A file that imports it (here `App.java`) gets `import com.acme.helpers.Helper;` in its BUFFER: the buffer is marked modified but NOT saved.
4. `:wa` saves all changed buffers.

If jdtls is not attached (a project without a Maven or Gradle marker, or the server still starting), the rename is only a rename on disk and the `package` / `import` lines stay as they were: use the manual steps in Renaming a package or folder.

---

30. Working with directories

Keymap / Command	Description
`<Space>cd`	Change working directory to current file's directory (window-local)
`:cd <path>`	Change directory globally
`:lcd <path>`	Change directory for current window only
`:tcd <path>`	Change directory for current tab
`:pwd`	Print current working directory
`:Z {keywords}` (or `:z {keywords}`)	`cd` to the best zoxide match (e.g. `:z nix nixos`) and print `cd <dir>`; without arguments it opens the fuzzy zoxide directory picker; without zoxide installed one warning

Path modifiers (for use in commands)

Modifier	Meaning	Example
`%`	Current file path	`/home/user/project/src/main.lua`
`%:h`	Directory of current file	`/home/user/project/src`
`%:t`	Filename only	`main.lua`
`%:p`	Full absolute path	`/home/user/project/src/main.lua`

---

57. File management for developers

File operations

Plugin / Feature	What it does
nvim-tree (`<Space>s`)	Visual file browser. Create (`a`), delete (`d`), rename (`r`), copy (`c`), cut (`x`), paste (`p`).
vim-eunuch	Unix file commands (they do NOT notify language servers: to have jdtls fix `package` and `import` lines after a rename or move, use `r` / `x` + `p` in the file tree, see File operations and language servers), all available from a fresh start (lazy `cmd` list): `:Rename <newname>`, `:Move <path>` (move the file, creating directories), `:Duplicate <name>`, `:Copy <path>`, `:Delete` / `:Remove` / `:Unlink` (delete the file; `:Delete` also the buffer, and it needs `!` for a file that has content), `:Mkdir <dir>` (always creates the missing parent folders, like `mkdir -p`; do NOT type `-p`, it would become part of the folder name; a `!` makes no difference; with no argument it creates the folder of the current file; an existing folder only gives the message "Directory already exists"), `:Chmod <mode>`, `:Cfind` / `:Lfind` / `:Clocate` / `:Llocate` (find / locate into the quickfix / location list), `:SudoEdit`, `:SudoWrite`, `:Wall` (write all) and `:W` (= `:Wall`).
gx.nvim (`gx`)	Open the URL or file path under cursor in a browser.
`:CopyPath absolute`	Copy the full file path to clipboard.
`:CopyPath relative`	Copy path relative to project root.
`:CopyPath nameonly`	Copy just the filename.

Examples of the vim-eunuch commands (all in a scratch project, checking the files on disk and the buffer afterwards; the cursor is in the buffer `lua/old.lua` and Neovim was started in the project root):

Command	Result on disk
`:Rename new.lua`	`lua/old.lua` becomes `lua/new.lua` (a bare name is relative to the file's folder); the buffer follows the file
`:Duplicate copy.lua`	`lua/copy.lua` is written next to it, and the buffer now shows `lua/copy.lua` (the original stays)
`:Mkdir a/b/c`	the folders `a/b/c` are created, relative to the folder Neovim was started in (here the project root), NOT to the file's folder
`:Delete`	refuses with `File not empty (add ! to override)` as long as the file has content
`:Delete!`	the file is deleted from disk and its buffer is closed (Neovim shows the previous buffer)
`:Move lua/sub/m2.lua`	the file is moved there (the missing folder `lua/sub` is created) and the buffer is renamed to `lua/sub/m2.lua`, with its text intact
`:Remove`	the file is deleted from disk; the buffer keeps its name but is reloaded EMPTY and unmodified (`u` brings the TEXT back into the buffer, marked modified, but the file stays deleted until you save it with `:w`)

Project structure navigation

Keymap	What it does
`<Space>s`	Toggle file tree sidebar
`<Space>ff`	Fuzzy-find any file in the project
`<Space>fg`	Search for text across all project files
`<Space>cd`	Change the working directory of THIS window only (`:lcd`) to the current file's folder and print it
`<Space>t`	Toggle the symbol outline (aerial) for the current file

---

63. Working with multiple files

Opening several files

Method	How
From command line	`nvim file1.py file2.py file3.py` (opens all as buffers)
From inside Neovim	`<Space>ff` to find and open files one at a time
Split open	`:vs file2.py` opens file2 in a vertical split next to current file
Tab open	`:tabe file2.py` opens in a new tab
From file tree	`<Space>s`, navigate to file, press `<Tab>` to open without leaving the tree

Comparing two files side by side

1. Open the first file

2. `:vs second_file.py` – open the second file in a vertical split
3. Now both files are visible side-by-side
4. Use `<Ctrl-w>h` / `<Ctrl-w>l` to switch between them
5. Use `:diffthis` in each window to enable diff mode (highlights differences)
6. Leave diff mode: `:diffoff` turns it off only in the window you are in, so the other window still shows the diff until you run it there too. `:diffoff!` turns it off in all windows of the tab at once. To get rid of the second file as well, close its window with `:q` (or `<Ctrl-w>c`); with one window left diff mode ends by itself. `:only` / `<Ctrl-w>o` keeps just the current window.

What it looks like for two files that differ in two lines (the changed lines are highlighted in colour, `*` marks them here):

```
  d1.txt                    |  d2.txt
    1  one                  |    1  one
*   2  two                  | *  2  TWO
    3  three                |    3  three
    ...                     |    ...
* 15  fifteen               | * 15  15
   16  sixteen              |   16  sixteen
```

In the real window, a line that exists only in one file is highlighted in green there (an added line), and the other window shows diagonal hatching (`/////`) at that position: these are filler lines that keep both windows aligned (seen on screen: the left file had one line, the right file three, so the left window showed hatching under its line and the right window showed line 3 in green). The bar at the top of the screen (the buffer tabs) lists buffers in the order they were opened, not the positions of the windows: after `nvim d1.txt` then `:vs d2.txt`, `d2.txt` is the second tab even though its window is on the left or right of the split.

Or use the diffview plugin: `:DiffviewOpen` for git diffs.

Copying between files

1. In file A: select text with `V` or `v`, then `y` to yank
2. Switch to file B: `<Ctrl-w>l` or `gb` or `<Space>bp`
3. Navigate to where you want the text
4. `p` to paste

Since clipboard is `unnamedplus`, yanked text is shared across all buffers and even with external applications.

Running the same edit across multiple files

Use the quickfix list:

1. `:grep "TODO"` – find all files with "TODO"
2. `:cfdo %s/TODO/DONE/g | update` – replace in every file of the list and save it

Closing files you're done with

Keymap	What it does
`\d`	Close current buffer, keep window
`\D`	Close all other buffers EXCEPT those with unsaved changes or a still-running terminal (one message says how many were kept)
`<Space>q`	Save the buffer if modified, then close the window (last window: quits nvim)

Chapter 7: Windows, buffers, terminal, sessions and the interface

7. Windows, splits, and buffers

This section explains how to open, navigate, resize, and close split windows entirely with the keyboard.

Key concepts

- **Buffer**: A file loaded into memory. You can have many buffers open but only see some of them.
- **Window**: A visible area showing a buffer. You can split your screen into multiple windows.
- **Tab**: A collection of windows. Think of it as a different workspace layout.

Panel layout: short single-task panels open on the LEFT (undo tree `<Space>u`, `<Space>rr` output, `:help` on a screen of at least 200 columns, which then opens as a full-height split on the far left), persistent panels on the RIGHT (Claude Code). Inactive windows are dimmed (vimade); the current window is always full colour.

Creating splits

Keymap / Command	Description
`<Ctrl-w>s` or `:sp`	Split the current window **horizontally** (new window appears below)
`<Ctrl-w>v` or `:vs`	Split the current window **vertically** (new window appears to the right)
`<Space>-`	Split the current window **horizontally** (new window below, same buffer)
`<Space>|`	Split the current window **vertically** (new window to the right, same buffer)
`:sp <file>`	Open `<file>` in a new horizontal split
`:vs <file>`	Open `<file>` in a new vertical split

Config note: `splitbelow` and `splitright` are set, so new splits always open below/right.

```
<Ctrl-w>s  (or <Space>-)       <Ctrl-w>v  (or <Space>|)

+------------------+           +---------+---------+
| old (above)      |           | old     | new     |
+------------------+           |         | (focus) |
| new (focus)      |           |         |         |
+------------------+           +---------+---------+
```

Put two chosen buffers side by side or one above the other

The rows above split with the SAME buffer (`<Space>|`, `<Space>-`, `<Ctrl-w>v`) or open a file PATH (`:vs <file>`). To show two buffers that are already open, chosen by name or number, use `:sbuffer`. The two buffers do not have to be neighbours in the buffer list: you address them by name or number, never by "next/previous". Typical use:

a window was closed by accident, or you want two of several open files next to each other.

Side by side (vertical split):

1. `:ls` Lists the buffers with their NUMBERS and names. The number is stable and is not the position in the list.
2. `:b <name>` or `:b <number>` Shows the wanted LEFT buffer in the current window. `<Tab>` completes the name, and a partial unique name is enough (e.g. `:b a`).
3. `:vert sbuffer <name or number>` Splits vertically and shows the second buffer in the NEW window on the RIGHT (`splitright` is set). The cursor is in the new window.
4. `<Ctrl-w>h` / `<Ctrl-w>l` Move between the two windows.

One above the other: the same, but `:sbuffer <name or number>` without `vert`. The new window opens BELOW (`splitbelow` is set) and gets the cursor. Move with `<Ctrl-w>k` / `<Ctrl-w>j`.

Command	Effect
`:b <name|number>`	Show that buffer in the current window
`:sbuffer <name|number>`	New horizontal split below showing that buffer
`:vert sbuffer <name|number>`	New vertical split to the right showing that buffer
`:vs <file>` / `:sp <file>`	Split and open a FILE PATH (loaded if needed); not a buffer name or number
`<Ctrl-^>` or `:b#`	Flip the current window between the current and the previously used buffer (see "Buffer management")
`:ls` or `:buffers`	List the buffers with their numbers

`:bn` / `:bp` only move to the neighbours in the list, so they do not help when the two wanted buffers are far apart.

Example with four open buffers (`:ls` shows `1 a.txt`, `2 b.txt`, `3 c.txt`, `4 d.txt`): to see `a.txt` and `d.txt` side by side, run `:b 1` then `:vert sbuffer 4`, or by name `:b a` then `:vert sbuffer d`. No need to go through 2 and 3.

With 4 buffers open: `:vert sbuffer <name>` with another buffer in the current window gave two windows side by side (current buffer left, requested buffer right, cursor in the new right window); `:sbuffer <name>` gave a stacked layout (new window below, cursor in it); `:vs <file>` gave side by side; `:sbuffer 99` (no such buffer) gives `E86: Buffer 99 does not exist`.

Gotchas:

- If the current window already shows the buffer you name, you get the same buffer twice (for the horizontal case).
- Closing one of the windows (`:q`, or `<Space>q`, which also saves) does not close the buffer: it stays in `:ls`. Only `\d` closes the buffer (see "Buffer management").

Navigating between windows

Keymap	Description
`<Ctrl-w>h` or `<Left>`	Move to the window on the **left**
`<Ctrl-w>j` or `<Down>`	Move to the window **below**
`<Ctrl-w>k` or `<Up>`	Move to the window **above**
`<Ctrl-w>l` or `<Right>`	Move to the window on the **right**
`<Ctrl-w>w`	Cycle to the **next** window
`<Ctrl-w>W`	Cycle to the **previous** window
`<Ctrl-w>p`	Jump to the **previously active** window

```
+-----+-----+    from A: <Ctrl-w>l -> B,  <Ctrl-w>j -> C
|  A  |  B  |    from B: <Ctrl-w>h -> A,  <Ctrl-w>j -> C
+-----+-----+    from C: <Ctrl-w>k -> A or B (see below)
|     C     |    <Ctrl-w>p from C -> back to the window you came from
+-----------+
```

C spans the full width, so `<Ctrl-w>k` from C goes to the window that lies above the cursor's screen column: A while the cursor is in the left half, B in the right half (with the cursor at columns 0 to 39 and 45 to 70 of an 80 column screen). `<Ctrl-w>j` from A or B always lands in C.

Resizing windows

Keymap	Description
`<Ctrl-w>=`	Make all windows **equal size**
`<Ctrl-w>+`	Increase current window height by 1 line
`<Ctrl-w>-`	Decrease current window height by 1 line
`<Ctrl-w>>`	Increase current window width by 1 column
`<Ctrl-w><`	Decrease current window width by 1 column
`10<Ctrl-w>+`	Increase height by 10 lines
`10<Ctrl-w>>`	Increase width by 10 columns
`<Ctrl-w>_`	Maximize current window height (make it as tall as possible)
`<Ctrl-w>|`	Maximize current window width (make it as wide as possible)
`:resize 20`	Set window height to 20 lines
`:vertical resize 80`	Set window width to 80 columns

Example (two windows, one above the other, the cursor in the upper one): `<Ctrl-w>_` makes the upper window as tall as possible and squeezes the lower one down to a single line; `<Ctrl-w>=` gives both the same height again.

Auto-resize: When you resize your terminal, all windows resize equally, except the Claude Code panel, which goes back to 30% of the screen width.

Moving windows around

Keymap	Description
`<Ctrl-w>H`	Move current window to the **far left** (becomes full height)
`<Ctrl-w>J`	Move current window to the **very bottom** (becomes full width)
`<Ctrl-w>K`	Move current window to the **very top** (becomes full width)
`<Ctrl-w>L`	Move current window to the **far right** (becomes full height)
`<Ctrl-w>r`	**Rotate** windows in the current row/column
`<Ctrl-w>R`	Rotate windows in reverse
`<Ctrl-w>x`	**Swap** current window with the next one
`<Ctrl-w>T`	Move current window to a **new tab**

Example, starting with A and B side by side and C below them, the cursor in A:

```
start                <Ctrl-w>L on A       <Ctrl-w>J on A       <Ctrl-w>x on A
+-----+-----+        +-----+-----+        +-----------+        +-----+-----+
|  A  |  B  |        |  B  |     |        |     B     |        |  B  |  A  |
+-----+-----+        +-----+  A  |        +-----------+        +-----+-----+
|     C     |        |  C  |     |        |     C     |        |     C     |
+-----------+        +-----+-----+        +-----------+        +-----------+
                                          |     A     |
                                          +-----------+
```

`<Ctrl-w>x` swaps A with the next window in the same row (B). `<Ctrl-w>H` is like `<Ctrl-w>L` but puts A on the far left, `<Ctrl-w>K` is like `<Ctrl-w>J` but puts A at the very top.

Buffer tabs (the top line, bufferline.nvim)

The top line shows one tab per open buffer (bufferline). Click a tab to switch to it. The `x` at the right of a tab closes it; while that buffer has unsaved changes the `x` is replaced by a `●`, and clicking the `●` closes it too. For a changed file Vim then asks `Save changes to "name"? [Y]es, (N)o, (C)ancel`: Yes saves and closes, No discards and closes, Cancel keeps the buffer and shows the warning "unsaved changes, buffer kept". A terminal whose program is still running is kept ("running terminal, buffer kept"). A right-click opens Neovim's own menu (see the Mouse table in the cheat sheet).

Key	Effect
`<Space>bp`	Pick a buffer (`:BufferLinePick`): per the plugin's docs every visible tab shows a character, and typing that character jumps to that buffer

Example (README.md is the current buffer and has unsaved changes; the second line is what you see after `<Space>bp`, then pressing `b` jumps to notes.txt):

```
 a.lua    | notes.txt   ▌README.md ●
 a a.lua  | b notes.txt ▌R README.md ●
```

Quickfix, fugitive and git buffers never get a tab. The tabs show no file icons and no diagnostics, and they are ordered by buffer number.

Changing the order of the tabs

The tabs are sorted by buffer number, which is the order in which the files were first opened, so a new file always appears on the right. There is no key for reordering; use these commands (type them after `:`):

Command	Effect
`:BufferLineMovePrev`	Move the current tab one place to the left
`:BufferLineMoveNext`	Move the current tab one place to the right

Example: you opened `hello.java` and then `calculator.java`, so the tabs read `hello.java | calculator.java` and `calculator.java` is the current buffer. `:BufferLineMovePrev` makes them `calculator.java | hello.java`; `:BufferLineMoveNext` puts them back. The new order stays while you switch between the buffers (within one Neovim session).

This only changes the order of the tabs. If the two files are shown side by side in two split windows and you want to swap which one is on the left, that is a window operation: `<Ctrl-w>x` (see "Moving windows around" above); with `hello.java` on the left and `calculator.java` on the right, `<Ctrl-w>x` gives `calculator.java | hello.java`.

Closing windows

Keymap / Command	Description
`<Space>q`	Close the current window (saves if modified, `:x`). It fires after a short pause because `<Space>qb` / `<Space>qw` also exist
`<Space>Q`	Force quit Neovim, **discarding unsaved changes**, after a Yes/No confirmation (default No)
`:q`	Close current window
`:q!`	Close current window discarding unsaved changes
`:only` or `<Ctrl-w>o`	Close ALL other windows, keep only the current one
`\x`	Close the quickfix window and all location lists of the tab (the cursor stays in the current window)

Buffer management

Keymap / Command	Description
`gb`	Go to the **next** buffer; `{N}gb` (e.g. `3gb`) goes to buffer number N (an invalid number warns "Invalid bufnr")
`gB`	Go to the **previous** buffer. Do not give it a count: `{N}gB` does nothing (an invalid number warns "Invalid bufnr"); use `{N}gb` to jump to buffer N
`<Space>bp`	**Pick** a buffer: each open buffer shows a letter, press it to switch
`\d`	Close/delete the current buffer (window stays open, shows previous buffer). On the last buffer an empty buffer is left. A named file with changes is saved first by auto-save (BufLeave); a buffer auto-save does not save (unnamed, read-only, Typst/LaTeX) is not deleted: you land in the previous buffer and the unsaved one stays loaded. On the only, unnamed buffer with typed text, Vim's confirm dialog "Save changes?" appears (the unsaved buffer is shown for a moment while it asks); your answer decides whether it is closed (in a real terminal).
`\D`	Close all other buffers, but **keep** buffers with unsaved changes and terminals that are still running (one message "kept N buffer(s) (unsaved or running terminal)")
`<Ctrl-^>`	Switch to the **alternate buffer**: the buffer you were in before this one. Press it again to come back, so you can flip between two files. It is the same as `:b#` or `:e #`. On a US keyboard `^` is Shift-6, so the keys are Ctrl-Shift-6; many terminals cannot send that, which is why Neovide gets the plain `<Ctrl-6>` as well (see section 77)
`:ls` or `:buffers`	List all open buffers; the flag columns are explained below ("Reading the `:ls` flags")
`:b <name>`	Switch to a buffer by (partial) name
`:b 3`	Switch to buffer number 3

Reading the `:ls` flags

Each `:ls` line shows the buffer number, then up to four flag columns, then the name. Example: `1 #a + "a.txt"`, `2  h   "b.txt"`, `3 %aF "term://..."`.

Flag	Meaning
`%`	The buffer in the current window (line 3 above)
`#`	The alternate buffer, the one `<Ctrl-^>` flips to (line 1)
`a`	Active: loaded and visible in a window
`h`	Hidden: loaded but shown in no window (line 2)
`+`	Modified, unsaved changes (line 1)
`F`	Terminal buffer whose job has finished (a terminal showing `Process exited`)
`u`	Unlisted buffer; only shown with `:ls!`
`R`	Terminal buffer with a running job
`?`	Terminal buffer without a job (`:terminal NONE`)
`-`	Buffer with `modifiable` off
`=`	Read-only buffer
`x`	Buffer with read errors

Use these when choosing buffers for Put two chosen buffers side by side or one above the other: `h` buffers are loaded but not on screen. A closed test terminal that shows `hF` is the case described in The test terminal window does not come back after you close it. `:help :ls` also lists `:ls` filter flags (for example `:ls h` for hidden buffers only).

Tabs

Command	Description
`:tabnew`	Open a new empty tab
`:tabe <file>`	Open `<file>` in a new tab
`gt`	Go to the next tab
`gT`	Go to the previous tab
`:tabclose` or `\t`	Close the current tab
`:tabonly` or `\T`	Close all other tabs

With exactly two tabs `gt` toggles between them: `:tabnew` (you are now in tab 2), `gt` goes back to tab 1, `gt` again to tab 2.

Closing floating windows

Some plugins open floating windows (diagnostics, hover docs, etc.):

Keymap	Description
`<Esc>`	Close any floating window (custom mapping)

---

8. Terminal integration

Opening a terminal

Command / Keymap	Description
`:term` or `:terminal`	Open terminal in the current window
`:sp | term`	Open terminal in a horizontal split below
`:vs | term`	Open terminal in a vertical split to the right
`<Space>rr`	Run code (opens a terminal in a vertical split on the **left** of the code window)

The terminal automatically starts in insert mode (you can type immediately) and hides line numbers. (in a real terminal: both start in insert mode).

Navigating in and out of terminal

Keymap	Context	Description
`<Esc>`	In terminal	**Exit terminal mode** and enter Normal mode. Now you can navigate away from the terminal window using `<Ctrl-w>h/j/k/l` or arrow keys. Exception: in a Claude terminal `<Esc>` goes to Claude; there `<Ctrl-w>h/j/k/l` work directly from terminal mode, and `<Ctrl-q>` (or `<Ctrl-\><Ctrl-n>`) leaves terminal mode (see section 9).
`i` or `a`	In terminal (Normal mode)	Re-enter terminal mode (start typing commands again)
`<Ctrl-w>h/j/k/l`	In terminal (Normal mode)	Move to another window
`<Left>/<Right>/<Up>/<Down>`	In terminal (Normal mode)	Move to another window (arrow key shortcuts)

Workflow example: You run code with `<Space>rr`. A terminal opens showing output. To go back to your code: press `<Esc>` to exit terminal mode, then `<Ctrl-w>l` (or `<Right>`) to move to the code window (the output is on the left). To close the terminal: `<Space>q` while in the terminal window.

Closing a terminal

Method	Description
`<Space>q`	While the terminal window is focused (press `<Esc>` first), close the window. A program that is still running keeps running in a hidden buffer (`\D` keeps such buffers)
Type `exit`	In an interactive shell terminal (`:term`), `exit` ends the shell and the window closes. A `<Space>rr` run does NOT close its window when the program ends: the output stays (it ends with the line `[Process exited 0]`) until you close it with `<Space>q`
`\d`	Delete the terminal buffer If the program is still running, Vim asks `Close "term://..."? [Y]es, (N)o, (C)ancel` first (in a real terminal).

---

58. Session and productivity

Auto-save (auto-save.nvim)

Plugin: **auto-save.nvim**. Files are automatically saved when you:

- Switch to another application (`FocusLost`)
- Leave the current buffer (`BufLeave`)

It never saves unnamed, read-only or special buffers (terminals, help, ...) and never saves Typst and LaTeX files (their watchers would recompile on every save). After each save a small popup (nvim-notify, title "AutoSave") shows "AutoSave: saved at HH:MM:SS"; if the format-check hint ("file is not formatted") appears too, the popups stack one under the other without overlapping. Auto-saves never format the file (see Formatting (conform.nvim)): format on save runs only on explicit saves (`:w`, `:x`, `ZZ`). For Typst and LaTeX, which are never auto-saved, use `<Space>fm` or `:w`.

Session management (persistence.nvim, vim-obsession)

Plugin: **persistence.nvim** saves a session for the current folder (and git branch) automatically when you quit, once a real file was opened. It is never restored by itself: restore it from the dashboard (`r` this folder, `L` last session). Windows of Claude Code, terminals, nvim-tree, the outline, help and quickfix are left out of the saved session. `<Space>sv` (restart) brings windows, tabs and files back on its own.

Plugin: **vim-obsession** (manual alternative). Save and restore your entire Neovim session (open files, window layout, etc.). It loads right after the first screen, so a session started with `nvim -S Session.vim` keeps being updated while you work (no need to type `:Obsession` again).

Command	What it does
`:Obsession`	Start recording the session (saves to `Session.vim`)
`:Obsession!`	Stop recording and delete the session file
`nvim -S Session.vim`	Restore the session from the command line

Collaborative editing (instant.nvim)

Plugin: **instant.nvim**. Real-time collaborative editing.

- The plugin loads on its first `:Instant...` command. Host and port are arguments of `:InstantStartServer` / `:InstantStartSession` / `:InstantJoinSession` (e.g. `:InstantStartSession 127.0.0.1 8081`); the built-in server defaults to port 8080.
- Uses your system username automatically

Example (with two Neovim instances on one machine): the host runs `:InstantStartServer 127.0.0.1 8081`, then `:InstantStartSession 127.0.0.1 8081`; the guest runs `:InstantJoinSession 127.0.0.1 8081` and sees "Connected!" (the host sees "Peer connected! 2 connected."). The host's open buffers appear in the guest's buffer list (`:ls`; the buffer numbers are the guest's own). What either side types in a shared buffer shows up in the other one at once (in both directions), together with the other person's name at their cursor line.

---

32. Statusline (`lualine.nvim`)

Section	Position	Contents
A	Leftmost	Filename + a lock icon (Nerd Font) when the file is read-only
B	Left	Git branch (click: pick a branch and check it out), `↑[n]` / `↓[n]` commits ahead / behind the upstream (the upstream is refreshed with a silent `git fetch origin` at most once a minute; long branch names are cut at 20 characters), diff stats (+~-), diagnostic counts (same icons as the sign column), Python virtualenv (Python buffers only)
C	Center-left	Pending command (e.g. `2d`), `recording @a` while a macro is being recorded (the command line is hidden, so Neovim's own message is not visible), spell indicator (`[SPELL]`)
X	Center-right	Active LSP (gear icon; `(+N)` = other attached clients; click: popup with the attached clients), `[N]>L` (long-line warning, see "Long-line warning" below), `[N]trailing` (first line with trailing whitespace), `MI:N` (mixed tabs and spaces)
Y	Right	Encoding (only when not UTF-8) and file format (only when not unix), both in red; `[CN]` input-method badge on macOS
Z	Rightmost	Progress through the file (%). The line:column position is shown only in inactive windows

Example (in a git repo, the cursor in the first line of a Lua file with one added, one changed and one removed line; the Nerd Font icons are left out, the real line has a branch icon before `main`, a diagnostic icon before the count `5` and a gear before `lua_ls`):

```
a.lua   main \ +1 ~1 -1 \ <icon> 5                    lua_ls (+1)  Top
A       B                                            X            Z
```

`Top` is the progress through the file (`Top`, a percentage or `Bot`).

Long-line warning

The statusline warns when a line is as long as the coloured vertical line (`colorcolumn`) of the current file, or longer. The warning sits on the right, left of `[N]trailing`.

Reading it, for example `[4]>88 (+1)`:

Part	Meaning
`[4]`	Line number of the first too-long line
`>88`	The limit used: the column of the red vertical line in this file
`(+1)`	How many other lines are also too long (left out when only one line is too long)

It is only a count, not a list of lines: jump to the first one with `:<number><CR>` (for `[4]`: `:4<CR>`), fix it, and the next one is reported.

- A line of exactly the limit already counts. Tabs count by their display width.
- The limit is the `colorcolumn` of the current buffer, which depends on the file type (default 100; see "Filetype-specific settings"). Files with no `colorcolumn` (plain text) never show the warning.
- Not shown in Insert mode (it appears after `<Esc>`), and not in special buffers (help, terminal, pickers, anything that is not a normal file).
- `:set filetype=lua` in a buffer changes the limit for that buffer only until it is closed; reopening the file detects the file type again.

Switch it off or on with the command `:LongLineWarn`:

Command	Effect
`:LongLineWarn<CR>` or `:LongLineWarn toggle<CR>`	Switch the warning on if it is off, off if it is on; prints `Long-line warning ON` or `Long-line warning OFF`
`:LongLineWarn on<CR>`	Switch it on
`:LongLineWarn off<CR>`	Switch it off

The setting is global (all buffers and windows) and is on at every start. There is no key for it.

---

33. UI features

Feature	Description
which-key.nvim	Press `<Space>` and wait: a popup shows all available leader keybindings
Dashboard	Start screen for a bare `nvim` (no file, directory or stdin); menu and keys in "Dashboard (Start Screen)" below
nvim-notify	Animated notification popups (fade + slide, 1500ms)
Colorschemes	On Nix systems the base16 theme named by `NVIM_BASE16_THEME` (fallback Catppuccin Mocha); on other systems one of 19 themes chosen at random at each start. UI colours (yank flash, hop keys, notifications, float borders) follow the active theme. Details: "Colorschemes" below
dropbar.nvim	Breadcrumb bar at top showing file > class > function
nvim-colorizer	Color codes (hex, rgb) are highlighted with their actual color; plain color words such as `red` are not
mini.indentscope	Visual `|` guide for current indent scope (loads right after the first screen; `ii`/`ai` exist from then on)
fidget.nvim	LSP progress messages in bottom-right corner
nvim-lightbulb	Lightbulb icon when code actions are available
vim-illuminate	Highlights the other uses of the word under the cursor (`<Alt-n>` / `<Alt-p>` jump between them)
vimade	Dims inactive windows
Borders	Floating windows and the completion menu have a single-line border (exception: `:Lazy` uses rounded corners)

Centred command line (tiny-cmdline.nvim)

Pressing `:` opens the command line as a small bordered window in the middle of the screen instead of at the bottom. The bottom line of the editor is permanently hidden (`cmdheight=0`) and Neovim's built-in message window (`ui2`) draws messages.

- `/` and `?` (search) keep the normal place at the bottom.
- The completion menu (`<Tab>` in the `:` line) opens just under the centred window.
- Pending keys such as `2d` show in the statusline, and `recording @a` shows there while you record a macro.
- Colours: window, border and title are derived from the active colorscheme (on Nix systems the base16 theme named by `NVIM_BASE16_THEME`, otherwise the Catppuccin Mocha fallback or the random theme), and follow `:colorscheme`. A theme without the needed colours keeps the plugin's default links.
- The plugin has no keys or commands of its own, and this config keeps its default settings (60 % wide, between 40 and 80 columns).

Breadcrumb bar (dropbar.nvim)

Plugin: **dropbar.nvim**. A line at the top of a window shows the path to the code under the cursor: the file path, then the class and function you are in. It follows the cursor by itself and appears for files that have a Treesitter parser or a language server with symbols (and for markdown and terminal buffers). Files over 1 MB, help

buffers, floating windows and windows that already have their own winbar do not get it. This config loads the plugin with its default settings and maps no key for it.

Action	Effect
Click a part of the bar with the mouse	Opens a menu of the entries at that level (for example the other functions of the class); `<CR>` or a click on an entry jumps there (plugin default)
In that menu: `q` or `<Esc>`	Close the menu (plugin default)

For a keyboard-driven view of the same information use the symbol outline (`<Space>t`, "Symbol Outline" in the navigation chapter).

LSP progress messages (fidget.nvim)

Plugin: **fidget.nvim**. While a language server is busy (starting up, indexing a project), a small progress message appears in the bottom-right corner and disappears when the server is done. It is only informational: there is nothing to press, and the config calls `setup {}` with the default settings. If the message stays for a while after opening a file, the server is still working; wait until it vanishes before expecting completion or go-to-definition.

Colorschemes

Twenty-two plugins in this config are colour themes or their helper library: 19 themes of the random list, plus catppuccin, nvim-base16 and lush.nvim. None of them is loaded at startup (all are lazy): lazy.nvim loads a theme plugin only when its colorscheme is selected.

Which theme is active at startup

System	What happens
Nix / NixOS (the folder `/etc/nixos` or `/etc/nix` exists)	The base16 theme named by the environment variable `NVIM_BASE16_THEME` is applied through nvim-base16 as `base16-<name>`. If the variable is missing or empty, or that theme fails to load, `base16-catppuccin-mocha` is used
Any other system	One theme out of the 19 below is picked at random at every start

If even the Nix fallback fails, a warning ("Failed to load base16 colorscheme …") is shown and Neovim's built-in `default` colorscheme is used. If a random theme fails to load, a warning ("Failed to load colorscheme …") is shown and `default` is used.

The Catppuccin Mocha look is the convention of this repository. In Neovim it comes only from the Nix fallback `base16-catppuccin-mocha` (nvim-base16). The catppuccin plugin is installed too, but it is not in the random list and has no setup of its own: use it by hand with `:colorscheme catppuccin-mocha`.

Switching and listing themes

There is no key or custom command for it; use the built-in commands:

Command	Effect
`:colorscheme <Tab>`	Lists and completes the names of the colorschemes Neovim knows (plugin colorschemes that are not loaded yet are expected to be included, because lazy.nvim registers them; not tried)
`:colorscheme <name>`	Switches to that theme; lazy.nvim loads the plugin first when needed
`:colorscheme`	Shows the name of the active theme

A manual switch lasts until you quit Neovim; the next start picks the theme again as described above. The per-theme settings in the table below (style, italics, background) are set only by the startup loader. A manual `:colorscheme <name>` does not run that code, so the theme starts from the plugin's own defaults (read from the code, not tried).

The UI colours of this config (yank flash, cursor and float borders, hop hint keys, notification background, completion menu, git signs) are derived from the active theme again after every `:colorscheme`, so they follow a manual switch.

The 19 themes of the random list

Plugin	Colorscheme loaded at startup	Other names the plugin provides
onedark.nvim	`onedark` (style "darker")	N/A (the plugin has only this one colorscheme file)
edge	`edge` (default style, italics on)	N/A (the plugin has only this one colorscheme file)
sonokai	`sonokai` (italics on)	N/A (the plugin has only this one colorscheme file)
gruvbox-material	`gruvbox-material` (hard background, original foreground, italics on)	N/A (the plugin has only this one colorscheme file)
everforest	`everforest` (hard background, italics on)	N/A (the plugin has only this one colorscheme file)
nightfox.nvim	`carbonfox`	`nightfox`, `dayfox`, `dawnfox`, `duskfox`, `nordfox`, `terafox`
onedarkpro.nvim	`onedark_dark`	`onedark`, `onedark_vivid`, `onelight`, `vaporwave`
material.nvim	`material` (style "darker")	`material-darker`, `material-lighter`, `material-oceanic`, `material-palenight`, `material-deep-ocean`
arctic	`arctic` (needs lush.nvim)	N/A (the plugin has only this one colorscheme file)
kanagawa.nvim	`kanagawa-dragon`	`kanagawa`, `kanagawa-wave`, `kanagawa-lotus`
modus-themes.nvim	`modus`	`modus_operandi`, `modus_vivendi`
jellybeans.nvim	`jellybeans`	`jellybeans-default`, `-hc`, `-mono`, `-muted`, `-warm` and light variants
github-theme	`github_dark_default`	`github_dark`, `github_dark_dimmed`, `github_dark_high_contrast`, `github_dark_colorblind`, `github_dark_tritanopia`, `github_light*`
ashen.nvim	`ashen`	N/A (the plugin has only this one colorscheme file)
melange-nvim	`melange`	N/A (the plugin has only this one colorscheme file)
makurai-nvim	`makurai_dark`	`makurai_autumn`, `makurai_light`
vague.nvim	`vague`	N/A (the plugin has only this one colorscheme file)
kanso.nvim	`kanso`	`kanso-ink`, `kanso-mist`, `kanso-pearl`, `kanso-zen`
citruszest.nvim	`citruszest`	N/A (the plugin has only this one colorscheme file)

The "other names" are the colorscheme files found in the installed plugin folders, so they depend on the plugin version.

Not in the random list: catppuccin (`catppuccin`, `catppuccin-mocha`, `-macchiato`, `-frappe`, `-latte`) and nvim-base16 (many `base16-<name>` themes; the one used on Nix). lush.nvim is only a library that arctic needs; it provides no colorscheme (see "Libraries and dependencies" in "
39. Other plugins

" in `10-various.md`).

Line number column (`statuscol.nvim`)

The column at the left of every window is drawn by statuscol.nvim as one column with three parts, from left to right: the sign column, the line number and the fold column. Each part has a click handler of the plugin (its default handlers; not described further here).

Part	What it shows
Signs	The sign column is one cell wide (`signcolumn=yes:1`). The config defines signs for git changes (gitsigns) and for diagnostics (error, warning, info, hint glyphs)
Number	The line number; `number` and `relativenumber` are both on (see "Automatic behaviors" for which window gets relative numbers). `relculright` is off, so the number of the cursor line is not right-aligned
Folds	The fold markers; fold levels deeper than 3 show a blank instead

Example (in a git repository, the cursor in line 1 of a file with no diagnostics; compared with the last commit, line 3 was changed, one line below it was deleted and two lines were added at the end; the cursor line shows its absolute number, the other lines their distance from the cursor):

```
sign number text
       1    return {          <- cursor line: absolute number
       1      a = 1,
~      2      b = 20,         <- git sign: changed line
_      3      c = 3,          <- git sign: lines deleted below
       4      e = 5,
+      5      f = 6,          <- git sign: added line
+      6      g = 7,          <- git sign: added line
       7    }
```

The sign column is one cell wide: when a line also has a diagnostic (for example a lua_ls hint such as "unused local"), the diagnostic sign is drawn instead of the git sign.

The fold keys are in "Code folding (`nvim-ufo`)".

Icons and UI libraries

These plugins have no keys and no commands of their own; other plugins use them. Icons need a Nerd Font in the terminal.

Plugin	Role
mini.icons	Supplies file-type and LSP-kind icons (the completion menu kind icons come from it). The config calls its `mock_nvim_web_devicons()`, so plugins that only know nvim-web-devicons get mini.icons answers; it is lazy and loads when a plugin asks for it
nvim-web-devicons	The older icon plugin. It has no spec of its own: lazy.nvim installs it because trouble.nvim lists it as a dependency (the nvim-tree config also has a `web_devicons` setting). Which of the two answers when both are present was not checked
nui.nvim	Popup, menu and layout building blocks. Needed by nvim-java, nvim-dbee and ascii.nvim
ascii.nvim	A collection of ASCII-art pictures, only needed by the dashboard, which takes a random one for its header at each start

Input popups and big files (`snacks.nvim`)

Plugin: **snacks.nvim**. Loaded at startup; three of its parts are enabled, with no keys or commands of their own.

Part	What it does here
input	Replaces `vim.ui.input`: questions that ask for text appear as a small popup at the cursor with a darkened backdrop
picker	Replaces `vim.ui.select` (the snacks picker option `ui_select` is on by default and the config does not turn it off), so selection lists such as the branch menu or code actions are filterable pickers. The keys inside them are in "Moving Inside Any Picker" (`05-search-and-files.md`)
bigfile	A file over 1.5 MB, or one whose lines average more than 5000 characters, gets the filetype `bigfile`: no Treesitter and no filetype keys. The language server of the real filetype starts a little later, without semantic tokens or completion. `:lsp stop` drops it; `:set ft=<language>` (for example `:set ft=json`) returns to full mode

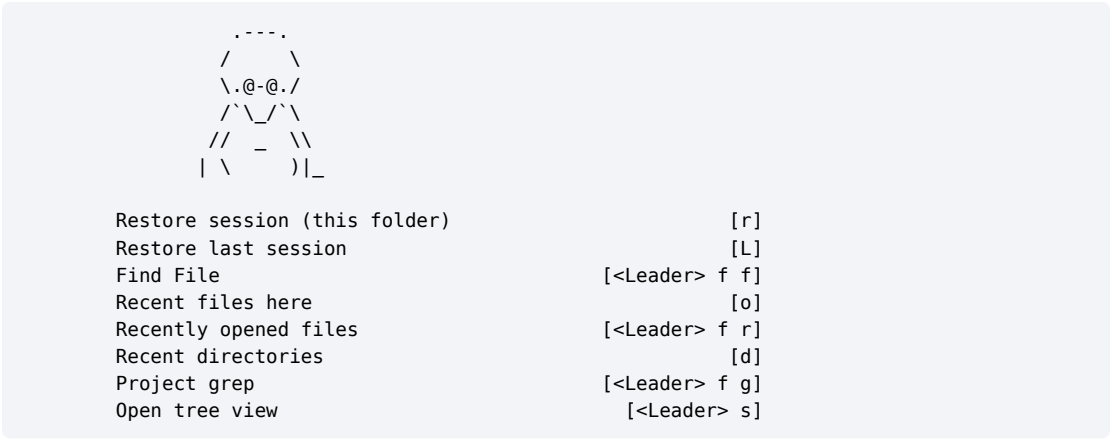
Dashboard (start screen, dashboard-nvim)

The dashboard opens for a bare `nvim` (no file, no directory, no stdin) or with `:Dashboard`. `<Enter>` runs the item under the cursor. These single-letter keys work only inside the dashboard (the items that show `[<Leader> ...]` on the right are the global keys, they work everywhere, see "Help keys" below):

Key	Item
`r`	Restore session (this folder)
`L`	Restore last session
`o`	Recent files here (only files under the current directory)
`d`	Recent directories (zoxide picker; the item is shown only when `zoxide` is installed)
`m`	Search keymaps
`e`	New file
`q`	Quit Neovim

The other items show their normal key: Find File `<Space>ff`, Recently opened files `<Space>fr`, Project grep `<Space>fg`, Open tree view `<Space>s`, Search help `<Space>fh`, Claude Code `<Space>cc`, Open Nvim config `<Space>ev`, Open WhatsApp `\w` (the second-to-last item, just above Quit Nvim; it runs `:WhatsApp`, see "WhatsApp in Neovim"). With nothing saved, `r` / `L` show one warning ("no saved session for this folder" / "no saved session"). Sessions are never restored automatically.

What a bare `nvim` shows (in a terminal; the header picture is random, here a penguin, and the Nerd Font icons before each item are left out; the real list continues below "Open tree view"):



Keymap	Description
`\h`	Open the dashboard in the current window (the previous buffer stays open in the background; inside the dashboard it only says "already in the dashboard")
`\w` / `\W`	Open WhatsApp (open or jump to its tab) / close it; the same as `:WhatsApp` and `:WhatsApp close`, and the dashboard item `Open WhatsApp` uses `\w` (works in any normal-mode buffer)
`\H`	Close the dashboard and return to the previous buffer (the dashboard buffer is deleted so it does not pile up); outside the dashboard it only warns "not in the dashboard", with no previous buffer "no previous buffer to resume"

Help keys: the user guide and Claude (any buffer, and the dashboard)

The same two keys work in every normal buffer and as dashboard items (shown as `[<Leader> ?]` and `[<Leader> a]`), so you learn them once.

Keys	What it does
`<Space>?`	Open this user guide as a PDF (`user-guide/neovim-user-guide.pdf`, built from the markdown files) in its own zathura window next to Neovim, always in light colors (Catppuccin Latte) even when your zathura and desktop are dark (`lua/zathura_light.lua`, shared with the LaTeX and Typst viewers, writes a small zathurarc in Neovim's cache folder that includes yours and overrides only the colors), always at page 2, the table of contents (zathura's remembered last page is ignored); close it with `q` in zathura. Pressing the key again while it is open only shows a notice. Without zathura the system viewer opens the PDF
`<Space>a`	Ask Claude how to do something in Neovim: a fresh `claude` session, always on the Sonnet model (`--model sonnet`), with the `answering-neovim-usage-questions` skill loaded, started in the nvim config folder with permissions bypassed (`--dangerously-skip-permissions`: it reads the guide and config, and can run commands or edit files, without asking). In a normal buffer it opens in a vertical split on the right (40% of the screen); on the dashboard it opens in its own tab. Type your question; `/exit` ends the session and closes the split or tab

In the Claude split, `<Esc>` goes to Claude (it interrupts a running answer). `<Ctrl-w>h` (or `j`/`k`/`l`) moves to another window directly, even from terminal mode; `<Ctrl-q>` (or `<Ctrl-\><Ctrl-n>`) leaves terminal mode. To edit the guide itself open the `.md` files directly (`<A-m>` previews one in the browser).

`:Dashboard` does the same as `\h`. To close the current buffer and get the dashboard instead: `:Dashboard | bdelete #` (a buffer with unsaved changes refuses with E89). `\d` deletes the buffer but shows the previous one, not the dashboard.

Chapter 8: Code: LSP, running, debugging, folding, treesitter

13. LSP: language server protocol

Plugin: nvim-lspconfig (default server definitions; Neovim's builtin `vim.lsp` does the rest). Provides IDE features. The keys below work per buffer according to what the attached servers support: if no attached server supports it, `K` and `gd` fall back to Vim's builtin versions and `<Space>rn` / `<Space>ca` show one warning.

Configured language servers

Server	Language	Program(s) that must be on PATH
`pyright` + `ruff`	Python	`pyright-langserver`, `ruff`
`lua_ls`	Lua	`lua-language-server`
`bashls`	Bash	`bash-language-server`
`yamlls`	YAML	`yaml-language-server`
`marksman`	Markdown	`marksman`
`nixd`	Nix	`nixd`
`jdtls`	Java (via nvim-java)	`java` (Java devShell)
`clangd`	C/C++	`clangd` (c-cpp devShell)
`ltex_plus`	Grammar and spelling (LanguageTool) for markdown, tex, plain TeX, typst, gitcommit, text	`ltex-ls-plus`
`typos_lsp`	Typos in identifiers and comments, every real file	`typos-lsp`
`tinymist`	Typst	`tinymist`
`texlab`	LaTeX	`texlab` (LaTeX devShell)
`rust_analyzer`	Rust	`rust-analyzer` AND `cargo`
`gopls`	Go	`gopls` AND `go`
`hls`	Haskell	`haskell-language-server-wrapper`
`sourcekit`	Swift, Objective-C	`sourcekit-lsp`
`ts_ls`	JavaScript / TypeScript	`typescript-language-server` (the project's `node_modules/.bin` copy is preferred)
`phpactor`	PHP	`phpactor` (php devShell)
`r_language_server`	R, Rmd, quarto	`R` plus the R package `languageserver`: the first R file runs one silent background check; restart nvim after installing the package

A server is enabled only when ALL its programs are on PATH; otherwise it is skipped silently (no warning when you open a file of that language outside its devShell).

The programs come from the Nix system or the language's devShell; nothing is downloaded by Neovim. `:LspStart <name>` tells you which program is missing.

LSP keymaps

Keymap	Description
`gd`	**Go to definition**: jump to where the symbol is defined (several different places open the location list)
`K`	**Hover**: show documentation in a floating window
`<Space>rn`	**Rename**: rename the symbol everywhere it's used
`<Space>ca`	**Code action**: show available fixes/refactors
`<Space>fm`	**Format** the file (or only the selection in Visual mode) on demand with the formatter of the file type, async; see Formatting (conform.nvim). Saving with `:w` formats too
`<Space>fo`	**Toggle format on save** for all buffers; prints `Format on save: on` or `off`

Example (with pyright; the same file is used for the next examples):

```
def load(path):          # line 1
    ...
x = load("a.txt")        # line 10, the cursor is on "load"
```

Keys	What happens
`gd`	The cursor jumps to line 1, `def load(path):`; `<Ctrl-o>` brings it back to line 10
`K`	A small bordered float shows the signature: `(function) def load(path: Unknown) -> str` (the type text comes from pyright); it closes when you move
`<Space>rn`	A "New Name" popup opens with `load` in it; erase it with `<BS>`, type `read`, `<Enter>`: line 1 becomes `def read(path):` and line 10 `x = read("a.txt")`. To cancel: `<Esc>` only leaves Insert mode (the popup stays open, in Normal mode); then `:q<Enter>` closes it with nothing renamed and your file still open

Why press `K` twice (going into the hover float). The first `K` shows the documentation in a small float and leaves your cursor where it was; the float disappears as soon as you move. A second `K` moves the cursor INTO the float, where it stays until you close it. The float is then a read-only Markdown window (in a Lua file with `lua_ls`), so you can use normal Neovim keys on the documentation:

Keys in the float	What happens
`j` `k`, `gg`, `G`	Move and scroll through a long documentation text that does not fit in the small float
`V` then `j`, then `y`	Copy lines of the documentation into a register, to paste them into your code or notes (it is read-only, so nothing in the float can be changed)
`/word<Enter>`	Search inside the documentation (`/Returns` jumped to the line with that word)
`q` or `<Esc>`	Close the float; the cursor is back in your file on the same spot

So going inside is not only "more readable": it is how you scroll long docs, search them and copy from them. For a short signature the first `K` is enough.

Built-in Neovim LSP and diagnostic keys

Neovim's own LSP keys also work next to the custom ones (in every buffer with a server that supports them):

Keymap	Mode	Description
`grn`	n	Rename (same as `<Space>rn`)
`gra`	n, x	Code action (same as `<Space>ca`)
`grr`	n	References (quickfix list)
`gri`	n	Implementation
`grt`	n	Type definition
`grx`	n	Run the code lens of the line
`gO`	n	Document symbols (outline in the location list)
`<Ctrl-s>`	i, s	Signature help
`]d` / `[d`	n	Next / previous diagnostic of any severity (`<Space>de` / `<Space>dE` jump to errors only)
`]D` / `[D`	n	Last / first diagnostic in the buffer
`<Ctrl-w>d`	n	Show the diagnostics under the cursor

LSP commands

Command	What it does
`:LspInfo`	LSP status (same as `:checkhealth vim.lsp`)
`:LspAttached`	Small popup with the servers attached to this buffer (`q` / `<Esc>` closes; also opens when you click the LSP name in the statusline; with no server attached it only shows a notification)
`:LspLog`	Open the LSP log file
`:LspRestart [name...]`	Restart the servers of this buffer, or the named ones
`:LspStop [name...]`	Stop them
`:LspStart [name...]`	Start the enabled servers of this buffer that are not running, or the named ones
`:LspInlayHints enable` / `disable`	Switch inlay hints on / off globally (off by default)

glance.nvim: peek without jumping

Plugin: glance.nvim. Preview definitions/references in a popup, without leaving your current file.

Keymap	Description
`<Space>gd`	Peek at definitions
`<Space>gr`	Peek at all references
`<Space>gi`	Peek at implementations

Example (`<Space>gr` with the cursor on `load` in `x = load("a.txt")`): a panel opens across the window with two panes. The left pane previews the code (the file with its line numbers), the right pane is titled `References (2)` and lists `def load(path):` and `x = load("a.txt")`. `<Esc>` closes the panel and the cursor is back on the line where it was.

```
+----------------------------------+-------------------------+
| preview of the selected entry    | References (2)          |
| (the whole file, left)           |   def load(path):       |
|                                  |   x = load("a.txt")     |
+----------------------------------+-------------------------+
```

Diagnostics (errors, warnings)

Nerd Font signs in the gutter: ⊗(error), ⚠(warning), ⓘ(info), 💡 (hint). No underline and no inline text: the message appears in a floating window when the cursor rests on the line (it closes when you move, leave the window or enter Insert mode).

Keymap	Description
`<Space>db`	Telescope picker with the diagnostics of the current file
`<Space>dw`	Toggle the Trouble diagnostics list (the diagnostics of every buffer Neovim has loaded, grouped by file; see "Diagnostics in depth")
`<Space>de`	Jump to next error
`<Space>dE`	Jump to previous error
`<Space>dd`	Show diagnostic detail in floating window
`<Space>dt`	Toggle diagnostics on/off
`<Space>qw`	Send the diagnostics of all open buffers to the quickfix list
`<Space>qb`	Send buffer diagnostics to quickfix list

Example (with pyright on a file whose last line is `print(undefined_name)`): the line gets a ⊗sign in the gutter, and after a moment a float appears under it:

```
 11   print(undefined_name)
      +-------------------------------------------------------------------+
      | Diagnostics:                                                      |
      |  Pyright: "undefined_name" is not defined [reportUndefinedVariable]|
      +-------------------------------------------------------------------+
```

`<Space>de` from the top of the file jumps to that line; `<Space>qw` opens the quickfix list with one entry for it (`hello.py |11| "undefined_name" is not defined`).

---

44. Language server protocol (LSP) in depth

What LSP is

LSP is a protocol that lets Neovim communicate with language-specific servers (programs that understand your code). The server analyzes your code and provides:

- **Diagnostics**: Errors and warnings shown in the gutter and floating windows
- **Go to definition**: Jump to where a function/class/variable is defined
- **Hover**: Show documentation for the symbol under cursor

- **Rename**: Rename a symbol across the entire project
- **Code actions**: Quick fixes, auto-imports, refactorings
- **Formatting**: Auto-format your code according to language standards (in this config the file formatters of Formatting (conform.nvim) do the work; the language server formats only where no formatter is installed)
- **Completion**: Suggestions as you type

How LSP is managed (nvim-lspconfig)

Each server is configured in `lua/config/lsp.lua` (plus `after/lsp/<name>.lua`) with Neovim's builtin `vim.lsp.config` / `vim.lsp.enable`. **nvim-lspconfig** only supplies the default server definitions. A server is enabled only when its program is on PATH: the programs come from the Nix system (`neovim.nix`) or from the language's devShell, nothing is downloaded by Neovim. Java (jdtls) is managed by nvim-java.

Configured servers and what they provide

Server	Language	What it provides
pyright	Python	Type checking, import resolution, diagnostics. Disables import sorting (ruff handles that).
ruff	Python	Fast linting, quick fixes and import sorting. Complementary to pyright. Formatting is `ruff format` through conform.nvim.
lua_ls	Lua	Full Lua analysis with `vim` global recognized. Its formatting is switched off: Lua is formatted by stylua (on save and with `<Space>fm`, see Formatting (conform.nvim)).
bashls	Bash/Shell	Shell script analysis and diagnostics.
yamlls	YAML	Schema validation and formatting for YAML files.
marksman	Markdown	Link validation, heading completion. Formatting is done with Prettier on save and with `<Space>fm` (not by marksman).
nixd	Nix	Nix language analysis. Formatter: nixpkgs-fmt.
jdtls	Java	Full Java IDE features via nvim-java (see Java section). Auto-configured.
clangd	C/C++	Compilation, diagnostics, code completion for C/C++.
ltex_plus	Prose (markdown, tex, typst, gitcommit, text)	Grammar and spell checking with LanguageTool; problems are diagnostics.
typos_lsp	Every real file	Finds typos in identifiers and comments; offers fixes as code actions.
tinymist	Typst	Diagnostics, completion (PDF export is left to `:TypstWatch`).
texlab	LaTeX	Diagnostics, hover, symbols, rename.
rust_analyzer	Rust	Full Rust analysis (needs `cargo` too).
gopls	Go	Full Go analysis (needs `go` too).
hls	Haskell	Haskell language server.
sourcekit	Swift, Objective-C	Swift analysis (C/C++ stay with clangd).
ts_ls	JavaScript / TypeScript	TypeScript, JSX, refactoring.
languageserver	R	Hover, signature help, completion. Needs the R package `languageserver` installed.

Each server starts only when its program is installed (see the table in section 13); `:LspAttached` (or a click on the LSP name in the statusline) shows what is attached.

How language servers are registered (lua/config/lsp.lua)
One file registers every server, so there is no separate chapter for languages such as Rust, Go, TypeScript, Haskell, Swift, PHP, R or Nix: their whole setup is the entry quoted below. There is no Mason in this config: no tool installer manages servers. The programs come from the Nix system or from a devShell (see section 43), and a server is started only if its program is found on PATH.

Shared capabilities
Verbatim (`lua/config/lsp.lua`, top of the file):

```
-- Capabilities shared by every server
vim.lsp.config("*", {
  capabilities = require("lsp_utils").get_default_capabilities(),
})
```

`vim.lsp.config("*", ...)` is the wildcard: every server gets these client capabilities in addition to its own settings. `lsp_utils.get_default_capabilities()` currently returns Neovim's own `vim.lsp.protocol.make_client_capabilities()`, whose 0.12 defaults already include what nvim-ufo needs for folding.

The server table
Abridged: the Python, Lua, Nix, LaTeX, Rust, Go, Haskell, Swift, TypeScript and PHP, R entries are quoted verbatim; the entries for bashls, clangd, yamlls, marksman, ltex_plus, typos_lsp and tinymist are left out (tinymist is quoted in the Typst chapter). Lines starting with `-- ...` mark the cuts.

```
-- Servers: configured here (plus after/lsp/<name>.lua), enabled only when the binary exists
---@type table<string, vim.lsp.Config>
local servers = {
  pyright = { cmd = { "pyright-langserver", "--stdio" } },
  ruff = { cmd = { "ruff", "server" } },
  -- ...

  -- Lua setup
  lua_ls = {
    cmd = { "lua-language-server" },
    -- lua_ls still advertises formatting with format.enable = false: hide it, so nothing
(LSP
    -- format, 'formatexpr' for gq) uses lua_ls's formatter next to stylua
    on_init = function(client)
      client.server_capabilities.documentFormattingProvider = false
      client.server_capabilities.documentRangeFormattingProvider = false
    end,
    settings = {
      Lua = {
        -- one Lua formatter: stylua (through conform.nvim)
        format = { enable = false },
        diagnostics = {
          disable = { "duplicate-set-field" },
          globals = { "vim" },
        },
        workspace = {
          checkThirdParty = false,
```

```
        },
      },
    },
  },

  -- ...

  -- Nix setup
  nixd = {
    -- --log=error: the default level writes every request to stderr, i.e. into lsp.log
    cmd = { "nixd", "--log=error" },
    settings = {
      nixd = {
        formatting = { command = { "nixpkgs-fmt" } },
      },
    },
  },

  -- ...

  -- LaTeX (texlab comes from the LaTeX devShell; enabled only when executable)
  texlab = { cmd = { "texlab" } },

  -- Rust (rust-analyzer + cargo: rust devShell); binaries checked in `needs` below
  rust_analyzer = { cmd = { "rust-analyzer" } },

  -- Go (gopls + go: go devShell); binaries checked in `needs` below
  gopls = {
    cmd = { "gopls" },
    -- nvim-lspconfig's list also has gotmpl: no filetype detection sets it (:checkhealth
vim.lsp
    -- warns "Unknown filetype 'gotmpl'")
    filetypes = { "go", "gomod", "gowork" },
  },

  -- Haskell (haskell-language-server: haskell devShell; nixpkgs ships only the -wrapper
binary
  -- (+ haskell-language-server-<ghc version>), no plain haskell-language-server)
  hls = { cmd = { "haskell-language-server-wrapper", "--lsp" } },

  -- Swift (sourcekit-lsp: swift devShell, or Xcode on macOS). nvim-lspconfig also lists c/
cpp:
  -- dropped, clangd serves those (no second client next to clangd)
  sourcekit = { cmd = { "sourcekit-lsp" }, filetypes = { "swift", "objc", "objcpp" } },

  -- JavaScript/TypeScript (typescript-language-server: node devShell). No cmd here: nvim-
lspconfig's
  -- cmd is a function that prefers the project's node_modules/.bin copy; binary checked in
`needs`
  ts_ls = {},

  -- PHP (phpactor: php devShell). nvim-lspconfig's filetypes/root_markers are kept; the
config only
  -- pins the command, so the binary check below is exactly executable("phpactor")
  phpactor = { cmd = { "phpactor", "language-server" }, filetypes = { "php" } },

  -- R (languageserver R package: R devShell). Enabled lazily by the one-time probe below,
never
  -- by the loop that enables the other servers
  r_language_server = { cmd = { "R", "--no-echo", "-e", "languageserver::run()" } },
}
```

In plain words:

- Each entry is `name = { cmd = {...}, ... }`. Only what differs from nvim-lspconfig's default definition is written; anything left out (filetypes, root markers) is the default. More settings of a server live in `after/lsp/<name>.lua`.
- **Python:** `pyright` and `ruff`, see the Python chapter.
- **Nix (`nixd`):** `--log=error` because the default level writes every request into the LSP log file; the formatter is `nixpkgs-fmt`.
- **LaTeX (`texlab`):** only the command; texlab comes from the LaTeX devShell.
- **Rust (`rust_analyzer`) and Go (`gopls`):** the comment says which devShell supplies them. `gopls` trims nvim-lspconfig's filetype list to `go`, `gomod`, `gowork` (the default also lists `gotmpl`, which no filetype detection sets, and `:checkhealth vim.lsp` would warn about it).
- **Haskell (`hls`):** the command is `haskell-language-server-wrapper --lsp`, because nixpkgs ships only the `-wrapper` binary (and a per-GHC-version one), no plain `haskell-language-server`.
- **Swift (`sourcekit`):** filetypes are `swift`, `objc`, `objcpp`. The default list also has `c` and `cpp`; they are dropped so clangd serves those and no second client attaches next to it.
- **TypeScript / JavaScript (`ts_ls`):** the entry is empty (`{}`) on purpose. nvim-lspconfig's `cmd` is a function that prefers the project's `node_modules/.bin` copy of the server; setting `cmd` here would replace it. The binary check for it is in `needs` (next section).
- **PHP (`phpactor`):** only the command is pinned; the default filetypes and root markers are kept.
- **R (`r_language_server`):** starts R and runs the `languageserver` package. It is never enabled by the plain loop, only by the probe described below.
- **Lua (`lua_ls`):** see "Lua: lua_ls and stylua" below.

When is a server enabled

Verbatim:

```
-- servers that are NOT enabled at startup (something else enables them later)
local deferred = { r_language_server = true }

-- binaries a server needs, when that is not just cmd[1]
local needs = {
  -- nvim-lspconfig's rust_analyzer root_dir runs `cargo metadata` (it warns "cargo not
found")
  rust_analyzer = { "rust-analyzer", "cargo" },
  -- nvim-lspconfig's gopls root_dir runs `go env` (it would fail without `go`)
  gopls = { "gopls", "go" },
  ts_ls = { "typescript-language-server" },
}

--- first binary the server needs that is not on PATH (nil: all present, or nothing known to
check)
---@param name string
---@return string?
local function missing_binary(name)
  local config = vim.lsp.config[name]
  local cmd = config and config.cmd
  local bins = needs[name] or (type(cmd) == "table" and { cmd[1] }) or {}
  return vim.iter(bins):find(function(bin)
    return not utils.executable(bin)
```

```
  end)
end

for name, config in pairs(servers) do
  vim.lsp.config(name, config)
  if not deferred[name] and not missing_binary(name) then
    vim.lsp.enable(name)
  end
end
```

In plain words:

- `deferred`: servers that the loop must not enable. Only `r_language_server`.
- `needs`: servers that need more than their own `cmd[1]`. `rust_analyzer` also needs `cargo` (its root detection runs `cargo metadata`), `gopls` also needs `go` (it runs `go env`), and `ts_ls` has no `cmd` here, so its binary `typescript-language-server` is named explicitly.
- `missing_binary(name)` returns the first required program that is not on PATH, or `nil`.
- The loop at the end registers every server (`vim.lsp.config`) and calls `vim.lsp.enable` only if nothing is missing. So outside a devShell nothing starts and nothing warns; `:LspStart <name>` tells you which program is missing.

R: the one-time probe

`R` on PATH is not enough, the R package `languageserver` must be installed too, and starting R to find out is slow. So the config asks once, at the first R-like file. Abridged: the comment, the state variables and the autocommand are quoted verbatim; the probe function (the `vim.system` call that runs R and, on success, enables the server and re-triggers `FileType` for R buffers already open) is left out and described below.

```
-- r_language_server: `R` alone is not enough, the `languageserver` R package must be
installed, and
-- starting R to find out is slow. So: ONE probe, at the first R-ish file, async, cached for
the
-- session. Silent when R or the package is missing, or when the probe fails or times out.
When it
-- succeeds the server is enabled and started for the R buffers already open (vim.lsp.enable
only
-- hooks FileType events that come later; the buffer that triggered the probe is already
past it).
local r_probe = nil ---@type "running"|"ok"|"no"|nil
local R_PROBE_TIMEOUT_MS = 10000

-- ... local function probe_r_language_server() ... end

vim.api.nvim_create_autocmd("FileType", {
  group = lsp_keys_group,
  pattern = { "r", "rmd", "quarto" }, -- the filetypes of r_language_server
  desc = "One-time probe for the R languageserver package",
  callback = probe_r_language_server,
})
```

In plain words:

- The probe runs R in the background (`quit(status = !requireNamespace("languageserver", quietly=TRUE))`) with a 10 second time-out. Exit code 0 means the package is there.

- The answer is cached for the session (`r_probe` is `"running"`, `"ok"` or `"no"`). Missing R, a missing package, an error and a timeout are all silent.
- On success the server is enabled and started for the R buffers that are already open (`vim.lsp.enable` alone only hooks later `FileType` events, and the buffer that triggered the probe is already past that point). After you install the package, restart nvim, because a `"no"` stays for the session.

Lua: lua_ls and stylua

Two things stop `lua_ls` from formatting, so that stylua (run by conform.nvim, see Formatting (conform.nvim)) is the only Lua formatter: `format = { enable = false }` in the settings, and `on_init` removes the formatting capabilities (lua_ls still advertises them with the setting off). Both are in the `lua_ls` entry above. The rest of the Lua server settings are in `after/lsp/lua_ls.lua` (verbatim):

```
-- settings for lua-language-server can be found on https://luals.github.io/wiki/settings/
---@type vim.lsp.Config
return {
  ---@type lspconfig.settings.lua_ls
  settings = {
    Lua = {
      runtime = {
        -- Tell the language server which version of Lua you're using (most likely LuaJIT in
the case of Neovim)
        version = "LuaJIT",
      },
      hint = {
        enable = true,
      },
    },
  },
}
```

In plain words: `runtime.version = "LuaJIT"` (Neovim runs LuaJIT), `hint.enable = true` turns on inlay hints (shown with `:LspInlayHints enable`), and the `lua_ls` entry adds `globals = { "vim" }` and ignores `duplicate-set-field`. lazydev.nvim (below) supplies the Neovim API types.

The run key lives in `after/ftplugin/lua.lua`. Abridged:

```
-- Disable inserting comment leader after hitting o/O/<Enter>
vim.opt_local.formatoptions:remove { "o", "r" }

for _, lhs in ipairs({ "<F9>", "<leader>rf" }) do -- <leader>rf: the same without function
keys
  vim.keymap.set("n", lhs, "<cmd>luafile %<CR>", { buffer = true, silent = true, desc = "run
lua file" })
end
```

In plain words:

- `<F9>` and `<Space>rf` run the file with `:luafile %`.
- Formatting has no Lua-specific key: `<Space>fm` and format on save run stylua through conform.nvim (the global key, see Formatting (conform.nvim)). A project `stylua.toml` is honoured, and a missing stylua is skipped silently.

typos_lsp: where it attaches

typos_lsp is registered like the others (`cmd = { "typos-lsp" }` in the server table). Its extra rules are in `after/lsp/typos_lsp.lua` (verbatim):

```
local root_markers = { "typos.toml", "_typos.toml", ".typos.toml", "pyproject.toml",
"Cargo.toml", ".gitignore" }

-- filetypes typos_lsp must never attach to
local skip_ft = {
  -- Snacks.bigfile buffers: would receive the whole file (see lua/config/bigfile.lua)
  bigfile = true,
  -- start screens
  dashboard = true,
  alpha = true,
  snacks_dashboard = true,
}

---@type vim.lsp.Config
return {
  root_markers = root_markers,
  -- typos_lsp has no filetype list, so it would start on every buffer that gets a filetype:
  -- skip special buffers (buftype ~= "": help, terminal, quickfix, plugin panels...) and
skip_ft
  root_dir = function(bufnr, on_dir)
    if vim.bo[bufnr].buftype ~= "" or skip_ft[vim.bo[bufnr].filetype] then
      return
    end
    on_dir(vim.fs.root(bufnr, root_markers))
  end,
}
```

In plain words: typos_lsp has no filetype list, so left alone it would start on every buffer with a filetype. The `root_dir` function refuses special buffers (`buftype ~= ""`: help, terminal, quickfix, plugin panels) and the filetypes in `skip_ft` (the big-file buffers and start screens). For real files the root is the nearest folder with one of the `root_markers`, where a `typos.toml` can hold your own accepted words.

LSP keymaps (all languages)

These keys work per buffer according to what the attached servers support: `K` and `gd` fall back to Vim's builtin versions when no attached server supports hover / definition; `<Space>rn` and `<Space>ca` show one warning instead ("rename: no attached language server supports it", or "… no language server attached to this buffer").

Keymap	What it does	When to use
`gd`	**Go to definition**. If there's only one definition, jumps directly. If multiple, opens a location list so you can pick. Deduplicates results.	When you want to see where a function/class/variable is defined.
`K`	**Hover documentation**. Shows docs in a floating window with a border (at most 100 x 40).	When you need to check what a function does, its parameters, return type, etc.
`<Space>rn`	**Rename symbol**. Renames the symbol under cursor everywhere it appears in the project.	When refactoring: changing a function name, variable name, etc.
`<Space>ca`	**Code action**. Shows a menu of available fixes and refactorings.	When the lightbulb icon appears, or when you want to auto-import, extract a variable, fix a lint warning, etc.
`<Space>fm`	**Format file** (Visual mode: only the selection). Runs the formatter of the file type asynchronously (stylua, prettier, ruff format, ...); the LSP formatter only when none is installed.	Whenever you want clean formatting, also when format on save is off.
`<Space>fo`	**Toggle format on save** for all buffers.	To save once without formatting; press again to turn it back on.

A worked example of `gd`, `K` and `<Space>rn` on a small Python file is in section 13 ("LSP keymaps").

Formatting (conform.nvim)

conform.nvim (stevearc/conform.nvim) is the one formatter front end for every file type, also for files without a language server. It runs an external formatter program on the buffer text and applies only what changed, so marks and the cursor stay. Config: `lua/config/conform.lua`.

Where the programs come from: most formatters are installed together with Neovim (the nix wrapper); the others must be on PATH. A formatter that a project devShell provides wins over the global one when you start Neovim inside that devShell. A formatter that is not installed is skipped silently; when no formatter of the file type is available the language server formats instead (if one is attached). `:ConformInfo` shows what will run for the current buffer.

File type	Formatter
Lua	stylua
Markdown, YAML, JSON, JSONC, CSS, SCSS, HTML, JavaScript, TypeScript, JSX, TSX	prettier
sh, bash	shfmt
fish	fish_indent
Nix	nixfmt
TOML	taplo
Typst	typstyle
TeX, plain TeX, BibTeX	latexindent
C, C++, Objective-C, Objective-C++	clang-format
Java	google-java-format
Rust	rustfmt
Go	goimports, then gofumpt
Python	ruff format
XML	xmllint
SQL	sql-formatter

Format on save

Every explicit save formats the file first: `:w`, `:update`, `:x`, `ZZ`, `:wq` and `<Space>w`. The formatter gets 1000 ms. This applies to real file buffers only (not terminals, help or the quickfix window).

Auto-saves never format: the file that auto-save.nvim writes when you leave a buffer or Neovim loses focus is written as it is, so a file you are in the middle of editing is not reshuffled (see Auto-save). Typst and LaTeX files are never auto-saved at all, so format them with `<Space>fm` or by saving with `:w`.

Keys and commands

Key / command	Mode	What it does
`<Space>fm`	Normal, Visual	Format the whole file; in Visual mode (`v`, `V` or Ctrl-v) only the selection. Asynchronous; works also when format on save is off. The buffer changes as one undo step (`u` undoes it) and is not saved
`<Space>fo`	Normal	Toggle format on save for ALL buffers. Prints `Format on save: off` or `Format on save: on`
`:FormatDisable` + Enter	Command-line	Turn format on save off for all buffers
`:FormatDisable!` + Enter	Command-line	Turn it off for the current buffer only (type `!` with Shift+1 right after `FormatDisable`, then Enter)
`:FormatEnable` + Enter	Command-line	Turn it on again (undoes both of the above)
`:ConformInfo` + Enter	Command-line	Show the formatters configured for this file type and which one will run

Example (any Lua file with messy spacing like `local x=1`): press `<Space>fm`: the line becomes `local x = 1` and the buffer shows as modified; `u` brings the old text back. Press `<Space>fo` once, save with `:w` and the file is written unformatted; press `<Space>fo` again to turn format on save back on.

After a save that did not format (an auto-save, or format on save switched off) Python and Lua files can show the hint `<file>: file is not formatted (ruff)` or `(stylua)`. It is only a hint; press `<Space>fm` to format.

Peeking without jumping (glance.nvim)
Plugin: **glance.nvim**. Instead of jumping away to a definition (which changes your context), you can peek at it in an inline popup:

Keymap	What it does
`<Space>gd`	Peek at definitions in a popup. You see the code without leaving your current file. Press `<Esc>` to close.
`<Space>gr`	Peek at all references. See every place in the project that uses this symbol.
`<Space>gi`	Peek at implementations. See how interfaces/abstract methods are implemented.

Neovim's builtin `grn`, `gra`, `grr`, `gri`, `grt` and `gO` also work (see section 13).

When to use Glance vs `gd`: Use Glance when you want to quickly check something and come back. Use `gd` when you want to actually navigate to the definition and work there.

Diagnostics in depth
Diagnostics are the errors, warnings, and hints that the LSP server reports about your code.

How they appear:

- Nerd Font signs in the gutter: ⊗(error), ⚠(warning), ⓘ(info), 💡 (hint)
- A floating window automatically appears after ~500ms when your cursor rests on a line with diagnostics; it closes when you move the cursor or enter insert mode
- The statusline (left side) shows the diagnostic counts with the same icons, e.g. `1 3`

Navigation:

Keymap	What it does
`<Space>de`	Jump to the next **error** (skips warnings/hints)
`<Space>dE`	Jump to the previous **error**
`<Space>dd`	Manually open the diagnostic float for the current line
`<Space>db`	Open a Telescope picker showing all diagnostics in the current file
`<Space>dw`	Toggle Trouble (`:Trouble diagnostics toggle`): the diagnostics of every buffer Neovim has loaded, grouped by file, not only the current one. A file that was never opened is listed only when its language server sends project-wide diagnostics. The key's `desc` says "Workspace Diagnostics"
`<Space>dt`	Toggle diagnostics on/off globally (a message says which; the automatic float stays off while disabled)

Sending diagnostics to quickfix:

Keymap	What it does
`<Space>qw`	Put the diagnostics of all open buffers into the quickfix list
`<Space>qb`	Put current buffer diagnostics into the quickfix list (with none you only get "No diagnostics in this buffer")

Then use `:cnext` / `:cprev` to jump through them one by one.

The lightbulb (nvim-lightbulb)

Plugin: **nvim-lightbulb**. A lightbulb icon appears in the sign column whenever the LSP has code actions available for the current line. This is your cue to press `<Space>ca`.

The lightbulb filters out noisy ruff actions (`source.fixAll.ruff`, `source.organizeImports.ruff`) to avoid false positives.

Editing the Neovim config in Lua (lazydev.nvim)

Plugin: **lazydev.nvim**. When you edit a Lua file (for example this config), the Lua language server (`lua_ls`) must know the Neovim API. lazydev adds those definitions to the server's workspace, so `vim.*` gets completion, hover (`K`) and signature help instead of "undefined global" warnings. It also adds the modules you `require(...)` in the open file as you go, so only what is used is loaded.

Item	Detail
Loads	Only in Lua files (lazy `ft = "lua"`); no cost elsewhere
Extra libraries (`lua/plugin_specs.lua`)	The luv types when the file mentions `vim.uv`; the nvim-lspconfig types when it mentions `lspconfig` (type help for `after/lsp/*.lua` files)
Keys	None
Needs	`lua_ls` attached to the buffer (see the server table above)
Not set up	lazydev's optional nvim-cmp source for `require("...")` module names is not configured here, so module-name completion inside `require(...)` only lists modules that are already loaded in the workspace

How the results reach the menu: the names come from `lua_ls` through the normal LSP source (see section 45, "Completion sources and helpers").

---

18. Code folding (`nvim-ufo`)

Plugin: nvim-ufo. Folds code blocks using the LSP folding ranges, falling back to indentation.

Keymap	Description
`za`	Toggle fold at cursor
`zA`	Toggle all folds under cursor recursively
`zc` / `zo`	Close / open fold at cursor
`zC` / `zO`	Close / open all folds recursively
`zR`	Open **all** folds in the file
`zM`	Close **all** folds in the file
`zr`	Open one more fold level (`2zr` = two levels; counted from the folds you see, so it works right after `zM`; in a buffer without ufo folds: one warning)
`zm`	Close one more fold level (accepts a count)
`<Space>K`	Preview folded lines in a popup
`zi`	Toggle folding feature on/off

Example (a Python file; `zM` closes every fold). Before:

```
1  def load(path):
2      with open(path) as f:
3          data = f.read()
4      parse(data)
5      return data
```

After `zM` the whole function is one line: the first line of the fold, then `⋯ 4` at the right edge, the number of hidden lines:

```
1  def load(path):                                    ⋯ 4
```

`za` on that line opens it again, `zR` opens every fold, and `<Space>K` shows the hidden lines in a popup without opening the fold.

Java example (in a real Neovim on a class, without jdtls running: ufo then uses its other fold providers). A class `public class X {` holding several methods:

- `zc` on any line inside a method closes that method; the line shows `⋯ N` (N = hidden lines). `zo` opens it, `za` toggles it.
- `zM` closes everything: only the class line `public class X {` is left, with the count of all hidden lines.
- After `zM`, `za` on the class line opens ONLY the class: the methods inside stay folded (one level). `zR` opens everything.

---

47. Code folding in depth (`nvim-ufo`)

What it is

Plugin: **nvim-ufo** + **promise-async**. Code folding collapses blocks of code (functions, classes, if-blocks, etc.) into a single line to help you see the big picture.

How it works

nvim-ufo uses the LSP server's folding ranges to determine what can be folded, and falls back to indentation when no server provides folds.

Folded lines show a preview: the first line of the fold + a count like `⋯ 42` showing how many lines are hidden.

Folding keymaps

Keymap	What it does	When to use
`za`	Toggle the fold under cursor	Quick open/close of a single fold
`zR`	Open ALL folds in the file	When you want to see everything
`zM`	Close ALL folds in the file	When you want the bird's-eye view
`zr`	Open one more fold level (`{N}zr` = N levels; in a buffer without ufo folds: one warning)	Gradually reveal more detail
`zm`	Close one more fold level (`{N}zm` = N levels)	Step back to less detail
`zo` / `zc`	Open / close fold at cursor	Precise control
`zO` / `zC`	Open / close all nested folds at cursor	Deep open/close
`<Space>K`	Preview folded lines in popup	See what's inside without unfolding
`zi`	Toggle folding on/off globally	Temporarily disable all folding

See "Code folding" (section 18) for a before/after picture.

Workflow tip: Press `zM` to close all folds when you open a large file. This gives you an outline view. Then use `za` to open only the sections you care about. Use `<Space>K` to peek inside folds without opening them.

21. Treesitter & text objects

nvim-treesitter (plugin)

Provides tree-sitter syntax highlighting (started automatically per filetype when a parser exists). On Nix systems the parsers come from the nix store and nothing is installed by Neovim; on other systems these parsers are installed automatically: cpp, diff, dockerfile, git_config, git_rebase, gitcommit, html, json, lua, python, toml, vim.

The real setup (`lua/config/treesitter.lua`, verbatim):

```
-- nvim-treesitter `main` branch API (the old `nvim-treesitter.configs` module no longer
exists).
local ok, ts = pcall(require, "nvim-treesitter")
if not ok then
  return
end

ts.setup({})

-- On Nix systems grammars come from the nix store (neovim.nix, nvim-treesitter.withPlugins)
-- and the filesystem is read-only, so only install parsers on non-Nix systems.
local is_nix = vim.uv.fs_stat("/etc/nixos") or vim.uv.fs_stat("/etc/nix")
if not is_nix then
  ts.install({ "cpp", "diff", "dockerfile", "git_config", "git_rebase", "gitcommit", "html",
"json", "lua", "python", "toml", "vim" })
end

-- Highlighting is no longer a module option: start it per buffer when a parser exists.
vim.api.nvim_create_autocmd("FileType", {
  group = vim.api.nvim_create_augroup("treesitter_highlight", { clear = true }),
  callback = function(args)
    if vim.bo[args.buf].filetype == "help" then
      return
    end
    pcall(vim.treesitter.start, args.buf)
  end,
})
```

In plain words:

- This uses the `main` branch API of nvim-treesitter; the old `nvim-treesitter.configs` module no longer exists. If the plugin cannot be loaded, the file stops silently.
- On Nix systems (`/etc/nixos` or `/etc/nix` exists) nothing is installed: the grammars come from the nix store (`neovim.nix`, `withPlugins`) and the file system is read-only. On other systems the twelve listed parsers are installed.
- Highlighting is started per buffer by a `FileType` autocommand: `pcall(vim.treesitter.start, buf)`. The `pcall` makes a filetype without a parser a silent no-op, and the `help` filetype is skipped.

targets.vim (plugin)

Adds many additional text objects for quotes, brackets, arguments, separators. Works automatically with `d`, `c`, `y`, `v`. If the cursor is not inside the pair, `i(`, `i"` and friends look forward on the line.

vim-matchup (plugin)

Enhanced `%` matching for language keywords (`if`/`else`/`end`, `do`/`while`, etc.). Shows offscreen match in popup. Also: `g%` (backwards `%`), `[%` / `]%` (start / end of the enclosing pair), `z%` (into the next pair), text objects `i%` / `a%`. With `g%` and `[%` / `]%`, `g%` from `if` goes backwards to `end`. `z%` in a real terminal: from `if` it moves to the closing `)` of the next pair inside the block.

Example (in a real `.lua` file):

```
if x then      -- line 1
  a()          -- line 2
else           -- line 3
  b()          -- line 4
end            -- line 5
```

With the cursor on `if`, `%` goes to `else` (line 3), the next `%` to `end` (line 5), the next one back to `if`. `g%` from `if` goes backwards, to `end`.

---

46. Treesitter in depth (`nvim-treesitter`)

What treesitter is

Plugin: **nvim-treesitter**. It parses your code into a syntax tree (like an AST) and uses that for:

- **Syntax highlighting**: More accurate than regex-based highlighting. Understands the actual structure of the code.
- **Symbols**: the aerial outline (`<Space>t`) can read the symbols from the tree.

Installed parsers

On Nix-managed systems (a folder `/etc/nixos` or `/etc/nix` exists) the parsers come from the nix store (home-manager); Neovim installs nothing. On other systems Neovim installs this fixed set at startup: cpp, diff, dockerfile, git_config, git_rebase, gitcommit, html, json, lua, python, toml, vim. Neovim itself bundles the parsers for c, lua, vim, vimdoc, query and markdown, so those highlight everywhere. Other languages get no tree-sitter highlighting there until you run `:TSInstall <lang>`. On a simulated non-nix system: each of the 12 grammars is downloaded, but the install needs the `tree-sitter` command and a C compiler (`gcc`/`cc`); without the `tree-sitter` command every grammar fails with `Error during "tree-sitter build": ... ENOENT ... 'tree-sitter'` and nothing is installed (the error lines appear again at every start). So on a non-nix machine install `tree-sitter` (the CLI) and a C compiler first.

---

19. Code running

Custom function in `lua/mappings.lua`. Opens the output in a vertical split terminal on the left. If the file has no name (the buffer was never saved), the filetype has no runner, or the needed program is not on PATH, you get one warning (naming the devShell to start nvim in) instead of a terminal. A named buffer with unsaved changes runs the version on disk, so save first (`:w`).

Keymap	Description
`<Space>rr`	Run current file (auto-detects language)

Supported: Python, Java, C, C++, C#, JavaScript, TypeScript, Go, Rust, Bash, Lua, Ruby, PHP. Special cases: Java with jdtls attached runs `:JavaRunnerRunMain` (nvim-java's own runner split at the bottom, not the `<Space>rr` terminal on the left); Rust inside a cargo project runs `cargo run`; C# with a `.csproj` runs `dotnet run --project`; Go runs `go run .` for the whole package.

Example: in a saved `hi.py` containing `print("hi")`, `<Space>rr` opens the output on the left and keeps your code on the right. The output stays after the program ends, with a line `[Process exited 0]` below it, until you close it with `<Space>q`:

```
+--------------------------+----------------------+
| hi                       | print("hi")          |
|                          |                      |
| [Process exited 0]       |                      |
+--------------------------+----------------------+
   output (left)              your code (right)
```

After running, the terminal output appears in a split. See Terminal integration for how to navigate to/from it and close it.

Filetype-specific

Keymap	Filetype	Description
`<Space>rf` / `<F9>`	Python	Run with `python -u` via AsyncRun (`uv run python -u` inside a uv project)
`<Space>rf` / `<F9>`	C++	Compile (clang++, else g++, C++20) and run in a split below; only mapped when a compiler is on PATH
`<Space>rf` / `<F9>`	LaTeX	Compile with vimtex
`<Space>rf` / `<F9>`	Lua	Run the file inside Neovim (`:luafile %`)
`<Space>rf` / `<F9>`	Vim script	Source the file (`:source %`)

---

55. Code running in depth

The universal runner

The `<Space>rr` keymap detects the current filetype and runs the appropriate command in a **vertical split terminal** on the left. If the file is unsaved, the filetype has no runner, or the compiler/interpreter is not on PATH, you get ONE warning and no terminal opens (the message names the devShell to start nvim in).

How it works:

1. Detects the filetype of the current buffer
2. Builds the correct command (e.g., `python3 file.py`, `go run .`)
3. Opens a vertical split
4. Starts a terminal in that split running the command
5. Output appears in real-time

After running:

- The terminal window opens on the LEFT of your code
- Press `<Esc>` in the terminal to enter Normal mode
- Navigate back to your code with `<Ctrl-w>l` or `<Right>`
- Close the terminal window with `<Space>q`, or delete its buffer with `\d`
- Run again with `<Space>rr` (it opens a new terminal each time)

Language-specific details

Language	Command used	Notes
Python	`python3 <file>`	N/A (plain run; needs `python3` on PATH)
Java	`:JavaRunnerRunMain` when jdtls (nvim-java) is attached; otherwise `java <file>`	With jdtls nvim-java opens its own runner split at the bottom (not the `<Space>rr` terminal on the left)
C	`gcc -Wall -Wextra -std=c11 <file> -o <binary> && <binary>`	Compiles and runs; the binary sits next to the source; needs gcc (c-cpp devShell)
C++	`g++ -Wall -Wextra -std=c++20 <file> -o <binary> && <binary>`	Compiles and runs; needs g++ (c-cpp devShell)
C#	`dotnet run --project <nearest .csproj>`	Without a project file: `dotnet run <file>`
JavaScript	`node <file>`	N/A (plain run; needs `node` on PATH)
TypeScript	`node <file>`	Node runs `.ts` directly
Go	`go run .` in the file's directory	Runs the whole package
Rust	`cargo run` when a `Cargo.toml` is above the file	Otherwise `rustc <file>` and runs the binary
Bash	`bash <file>`	N/A (plain run; needs `bash` on PATH)
Lua	`nvim -l <file>`	Neovim's own LuaJIT
Ruby	`ruby <file>`	N/A (plain run; needs `ruby` on PATH)
PHP	`php <file>`	N/A (plain run; needs `php` on PATH)

Filetype-specific runners

Some filetypes have an additional `<Space>rf` runner (also `<F9>`):

Filetype	What `<Space>rf` / `<F9>` does
Python	Runs with `python -u <file>` via AsyncRun (unbuffered output; `uv run python -u` in a uv project)
C++	Compiles with `clang++` (else `g++`) `-Wall -Wextra -std=c++20 -O2` and runs it in a horizontal split; only mapped when a compiler is on PATH
LaTeX	vimtex compile; only when `latex` is on PATH

---

36. Debugging

Plugin	Keymap / Command	Description
nvim-dap	(lazy-loaded)	Debug Adapter Protocol client. Java debugging auto-configured via nvim-java.
nvim-gdb	`<Space>dp`	Python buffers only: start pdb on the current file (Linux/Windows only; elsewhere one warning); during the session `<Space>dc` `dn` `ds` `df` `dB` `du` `dv` step and inspect (table in section 2). `:GdbStart` (gdb) works inside the c-cpp / rust devShells

---

56. Debugging in depth

Debug adapter protocol (DAP, nvim-dap)

Plugin: **nvim-dap**. DAP is a standardized protocol (created by Microsoft) for communication between an editor and a debugger. It's the same protocol used by VS Code.

Java: Debugging is auto-configured via nvim-java (only inside the Java devShell, where `java` is on PATH). Open a Java file, set a breakpoint with `<Space>jp`, and use `<Space>jtC` (debug the current test class) or `<Space>jtM` (debug the current test method). For a `main` class use `:DapContinue`; stepping commands and the other debug keys: Java section 7.

Python: In Python buffers `<Space>dp` starts `python -m pdb` on the current file through nvim-gdb. Only available on Linux/Windows. During the session use `<Space>dc` (continue), `dn` (next), `ds` (step), `df` (finish), `dB` (breakpoint), `du` (until) and `dv` (evaluate); the full table is in section 2 ("Python debugger keys").

GDB integration (nvim-gdb)

Plugin: **nvim-gdb**. A visual front end for GDB, LLDB, pdb and a few other debuggers: it starts the debugger in a terminal pane under your code and marks the current line in the source window. Available on Linux and Windows only (disabled on macOS). It loads the first time one of the start commands below runs.

Command	Effect
`:GdbStart gdb -q ./a.out`	Start GDB on a program (you give the whole gdb command line)
`:GdbStartLLDB lldb ./a.out`	The same with LLDB
`:GdbStartPDB python -m pdb file.py`	Python's pdb (in Python buffers `<Space>dp` runs this for the current file)
`:GdbStartBashDB bashdb script.sh`	bashdb for shell scripts
`:GdbStartRR`	Replay a recording made with `rr`

The command forms are the plugin's documented usage (its README); the pdb one is what `<Space>dp` runs, the others were not run for this guide.

Example (with gdb 17 on a small C file): compile with `gcc -g a.c -o a.out`, open `a.c` in Neovim and run `:GdbStart gdb -q ./a.out`. A gdb terminal opens below the source. Move to the source window (`<Ctrl-\><Ctrl-n>`, then `<Ctrl-w>k`), put the cursor on

`int y = x * 3;` and press `<F8>`: a `●` appears in the sign column. `<F5>` (continue) only works while the program runs: before that it prints "The program is not being run." in the gdb pane; start it with `:GdbRun`. It stops at the breakpoint and the line is marked with `▶`. `<F10>` steps over to the next line (the `▶` moves down one line).

```
  ●  5   int y = x * 3;          before :GdbRun: breakpoint
  ▶  5   int y = x * 3;          stopped at the breakpoint
  ▶  6   printf("%d\n", y);      after <F10>
```

The plugin's own start keys (`<Space>dd`, `dl`, `dp`, `db`, `dr`) are switched off in `lua/plugin_specs.lua` so they do not replace the diagnostic keys; `<Space>dp` is a Python-buffer key defined in `after/ftplugin/python.lua`. During a session the plugin sets these keys in the source window (buffer-local, removed again when the session ends):

Key	Command	Effect
`<F8>`	`:GdbBreakpointToggle`	Toggle a breakpoint on the cursor line
`<F5>`	`:GdbContinue`	Continue
`<F10>`	`:GdbNext`	Step over
`<F11>`	`:GdbStep`	Step into
`<F12>`	`:GdbFinish`	Step out of the current frame
`<F4>`	`:GdbUntil`	Run until the cursor line
`<Ctrl-p>` / `<Ctrl-n>`	`:GdbFrameUp` / `:GdbFrameDown`	Move one stack frame up / down (plugin default)
`<Space>dv`	(evaluate)	Evaluate the word under the cursor (Normal) or the selection (Visual); moved here from the plugin's `<F9>`

Two commands without a key: `:GdbCreateWatch <command>` (for example `info locals` in GDB) opens a watch window that re-runs the command at every step, and `:GdbLopenBacktrace` / `:GdbLopenBreakpoints` put the backtrace / breakpoints into the location list. The Python-only Space keys for the same actions (`<Space>dc`, `dn`, `ds`, `df`, `dB`, `du`) are in the Python guide ("Debugging with pdb"). The F-keys, `<Ctrl-p>` / `<Ctrl-n>` and the commands are the plugin's defaults from its README and help (`:help nvimgdb`).

---

43. How the development toolchain fits together

When you open a code file in Neovim, several systems activate automatically behind the scenes:

```
You open a file
  |
  v
Treesitter parses the syntax tree --> accurate highlighting (plus symbols for the aerial
outline)
  |
  v
LSP server starts (e.g., pyright for Python) --> diagnostics, go-to-definition, hover,
rename, code actions
```

```
  |
  v
Completion engine (nvim-cmp) connects to LSP --> autocomplete suggestions as you type
  |
  v
Gitsigns reads git status --> change markers in gutter
  |
  v
Lightbulb watches LSP --> shows icon when code actions are available
  |
  v
Diagnostics config --> errors/warnings appear as Nerd Font signs, a float opens on
CursorHold
```

You don't need to start any of this manually. It all happens on file open.

Enter a language devShell from a running Neovim (`:DevEnv`)

Language servers and tools come from per-language Nix devShells (see "Configured servers and what they provide" in section 44). Normally direnv loads the devShell when you `cd` into a project and you start `nvim` there. If Neovim is already running without the devShell (for example a `.java` file opened from outside its project, so `java` is not on PATH), you do not have to quit:

1. `:DevEnv <lang>` Type the command and a devShell name, for example `:DevEnv java`, then `<CR>`. Press `<Tab>` after `:DevEnv ` to list the available names.
2. Wait. The first call for a flake takes about 10 seconds and shows a progress notification (`DevEnv: evaluating the java devShell (about 10 s) ...`). Neovim stays usable meanwhile.
3. Read the result (a check, not an action): one INFO notification, `DevEnv: <lang> ready: ...`, lists what became available (language servers, `nvim-java`, `vimtex`, ...). If something fails you get a WARN notification instead.

What it does, in order:

- Runs `nix print-dev-env --json` on the flake folder, asynchronously.
- Takes only an allowlist of variables from the result: `PATH` (prepended to the current one), `JAVA_HOME`, `CLASSPATH`, `NODE_PATH`, `XDG_DATA_DIRS`, `NIX_CFLAGS_COMPILE`, `NIX_LDFLAGS` and `JAVA_TOOL_OPTIONS` (the Lombok agent). Nothing else (not `HOME`, `SHELL`, ...) is changed.
- Enables every language server whose programs are now on PATH (section 44) and replays the file type for open buffers, so a server attaches to the file you already have open.
- Starts the language-specific parts: nvim-java, jdtls and spring-boot for `java`; vimtex for `latex`; typst.vim for `typst`.
- Caches the result per flake (keyed on `flake.lock` and `flake.nix`), so the next call is instant. A cache whose Nix store paths were garbage-collected is ignored and evaluated again.

Things to know:

- It is a command only: there is no keybinding.
- It needs `nix` with the `nix-command` and `flakes` features on the machine.
- The flake's shell hook does not run, so the extension links the Java devShell makes (java-debug, java-test) are not created; see section 78 for what that means.

- The environment applies to this Neovim session only. Quit and restart, and you have to run it again (or start Neovim from the devShell).
- Opening a `.java` or `.tex` file without its tool shows a one-time hint: `java not found on PATH: run :DevEnv java to enter its devShell` (same for `latex`).
- The folder holding the flakes is the default below. Change it with `vim.g.devenv_base` (Lua) or the `$NVIM_DEVENV_BASE` environment variable. Only sub-folders that contain a `flake.nix` are listed as devShells.

Every devShell it supports (path: `~/nix/templates/krit/dev-environments/language-specific/<name>`):

`:DevEnv` name	Path	What it provides (from its `flake.nix`)
`c-cpp`	`~/nix/templates/krit/dev-environments/language-specific/c-cpp`	clang-tools (clangd), cmake, conan, cppcheck, doxygen, gtest, lcov, vcpkg, codespell, CLion
`go`	`~/nix/templates/krit/dev-environments/language-specific/go`	go, gopls, gotools, golangci-lint
`haskell`	`~/nix/templates/krit/dev-environments/language-specific/haskell`	ghc, cabal-install, haskell-language-server, ormolu
`java`	`~/nix/templates/krit/dev-environments/language-specific/java`	JDK (`JAVA_HOME`), Maven, Gradle, Lombok agent, jdtls wrapper, java-debug and java-test extensions (see section 78)
`jupyter`	`~/nix/templates/krit/dev-environments/language-specific/jupyter`	python313, poetry, ruff, ipykernel, pip, a `.venv` virtual-environment hook, and for notebooks in Neovim chafa, ffmpeg, imagemagick, librsvg and (Linux) `NVJUP_CHROMIUM` (see Jupyter notebooks)
`latex`	`~/nix/templates/krit/dev-environments/language-specific/latex`	texlive (scheme-full: `latex`, `latexmk`), texlab, tectonic, pandoc, zathura, latex2html, latex2mathml
`nix`	`~/nix/templates/krit/dev-environments/language-specific/nix`	nixd, nixfmt, statix, nh, niv, cachix, lorri, vulnix, dhall-nix
`node`	`~/nix/templates/krit/dev-environments/language-specific/node`	nodejs, typescript, typescript-language-server, eslint, prettier, pnpm, yarn
`php`	`~/nix/templates/krit/dev-environments/language-specific/php`	php, composer, phpactor
`python`	`~/nix/templates/krit/dev-environments/language-specific/python`	Python with black, flake8, isort, pylint, numpy, matplotlib, pip and a virtual-environment hook; pyright, ruff
`r`	`~/nix/templates/krit/dev-environments/language-specific/r`	R with the `languageserver` and `knitr` packages
`rust`	`~/nix/templates/krit/dev-environments/language-specific/rust`	Rust toolchain, rust-analyzer, cargo-deny, cargo-edit, cargo-watch, openssl, pkg-config
`shell`	`~/nix/templates/krit/dev-environments/language-specific/shell`	shellcheck, shfmt, bash-language-server
`swift`	`~/nix/templates/krit/dev-environments/language-specific/swift`	swift, sourcekit-lsp
`typst`	`~/nix/templates/krit/dev-environments/language-specific/typst`	typst, tinymist, typstyle, prettypst, typstwriter, utpm, zathura

Language chapters: Java section 78, LaTeX section 80, Typst section 82.

53. Documentation lookup

devdocs.nvim (plugin)

Plugin: **devdocs.nvim**. Browse programming documentation (the DevDocs collection) without leaving Neovim. It needs `jq`, `curl` and `pandoc` on the PATH.

Command	What it does
`:DevDocs fetch`	Download the list of available documentation sets (the metadata)
`:DevDocs install`	Pick a set from a list (each entry shows its download size) and install it
`:DevDocs install <doc>`	Install the set named `<doc>` directly (e.g. `lua~5.1`; some sets need a version after `~`)
`:DevDocs get`	Pick one of the installed sets, then pick a page; the page opens read-only in a horizontal split
`:DevDocs get <doc>`	Skip the first list and pick a page of the installed set `<doc>`
`:DevDocs delete` / `:DevDocs delete <doc>`	Remove an installed set (from a list, or by name)

Press `<Tab>` after `:DevDocs ` to complete the sub-commands, and after `install`, `get` or `delete` to complete set names (available sets for `install`, installed ones for `get` and `delete`). The plugin ships no completion; it is added in `lua/config/devdocs.lua`, and the first `<Tab>` on a set name loads the plugin. Without a sub-command, `:DevDocs` only prints the available ones. The lists are `vim.ui.select` lists, so they look like the snacks pickers (see section 12). The plugin loads only when you run `:DevDocs`; on its first load it fetches the metadata in the background. Downloads and conversion run in the background and need the network; reading an installed set is local. No keys are mapped and no plugin options are set (the install list is not preset, so nothing downloads on its own). Documentation is stored in the `devdocs` folder of Neovim's data directory.

Example (with the Lua 5.1 set; tested headless: it installs and `:DevDocs get lua~5.1` opens its page list):

1. `:DevDocs fetch` downloads the list of available sets (needed only if `:DevDocs install` shows an empty list).
2. `:DevDocs install lua~5.1` downloads and converts the set; wait until the messages stop.
3. `:DevDocs get` lists `lua~5.1`; press `<Enter>`, then pick a page from the second list (the Lua set has a single page, `Index`) and press `<Enter>` again: it opens in a read-only split.

devdocs.nvim: a set is missing from `:DevDocs get`

`:DevDocs get` lists only sets whose entry in `~/.local/share/nvim/devdocs/state.json` has `"extracted": true`. A set stuck at `{"downloaded": true}` without `extracted` (the extraction was interrupted or failed) is on disk but invisible. Look at the file and search for entries that are downloaded but not extracted.

Set names (slugs) use `~`, not `-`: `openjdk~25`, `python~3.15`. An install downloads `https://documents.devdocs.io/<slug>/db.json`, turns it into `docs/<slug>.json` with `jq`, then converts every entry to markdown with `pandoc` into `docs/<slug>/`. If a slug is rejected, refresh the list with `nvim --headless -c 'DevDocs fetch' -c 'qa!'`.

The work is asynchronous, so a headless run must wait for `extracted` to become true, otherwise it quits too early. Install a set (large sets such as `openjdk~25`, about 115 MB of JSON, take a while):

```
nvim --headless -c 'DevDocs install <slug>' \
  -c 'lua vim.wait(900000, function() local f=io.open(vim.fn.stdpath("data").."/devdocs/
state.json"); local s=vim.json.decode(f:read("*a")); f:close(); return (s["<slug>"] or
{}).extracted == true end, 1000)' \
  -c 'qa!'
```

Repair a downloaded but unextracted set without downloading again: delete the partial output, then extract only:

```
rm -rf ~/.local/share/nvim/devdocs/docs/<slug>
nvim --headless -c 'lua require("devdocs.docs").ExtractDocs("<slug>")' \
  -c 'lua vim.wait(900000, function() local f=io.open(vim.fn.stdpath("data").."/devdocs/
state.json"); local s=vim.json.decode(f:read("*a")); f:close(); return (s["<slug>"] or
{}).extracted == true end, 1000)' \
  -c 'qa!'
```

Check the result (this is the list `:DevDocs get` shows):

```
nvim --headless -c 'lua
print(vim.inspect(vim.tbl_keys(require("devdocs").GetInstalledDocs())))' -c 'qa!'
```

`:DevDocs delete <slug>` only sets `downloaded` and `extracted` to false and leaves the key in `state.json`. To remove a set completely, delete `docs/<slug>/` and `docs/<slug>.json`, and drop the key (for example with `jq 'del(.["<slug>"])'`). Setting `ensure_installed = { ... }` in the devdocs.nvim options would install sets on startup.

Hover documentation (LSP)

Press `K` on any symbol to see its documentation in a floating window. This pulls from:

- Function signatures and return types
- Docstrings / JSDoc / Javadoc
- Type information

Example (Python buffer): `K` on `open` in `with open(path) as f:` shows a bordered float with the signature, one parameter per line (`(function) def open(` / `file: FileDescriptorOrPath,` / `mode: OpenTextMode = "r",` / … / `) -> TextIOWrapper[_WrappedBuffer]`). On a keyword such as `with` there is nothing to show and the message "No information available" appears instead.

---

59. Useful developer commands

Command	What it does
`:LspInfo`	LSP status (runs `:checkhealth vim.lsp`)
`:Lazy`	Open plugin manager
`:Lazy update`	Update all plugins
`:JSONFormat`	Pretty-print JSON through Python (whole file or visual range; no key; `<Space>fm` formats JSON with prettier)
`:ToPDF`	Convert markdown to PDF via pandoc
`:Redir <cmd>`	Capture any Neovim command output (e.g., `:Redir messages`)
`:Telescope keymaps`	Browse all defined keymaps
`:checkhealth`	Diagnose Neovim installation issues
`:Flog`	Git log graph
`:DiffviewOpen`	Side-by-side diff view
`:Neogit`	Full git UI
`:DevDocs get`	Browse programming documentation
`:YankyRingHistory`	Browse yank history
`:LspAttached`	Popup with the LSP servers attached to this buffer
`:LspLog`	Open the LSP log
`<Space>u`	Toggle the undo tree (the `:Undotree` command exists only after the first `<Space>u`)
`:AerialToggle!`	Toggle the symbol outline (`<Space>t`)
`:Dashboard`	Open the start screen
`:CodeDiff`	VSCode-style side-by-side diff
`:DBUI` / `:Dbee`	SQL clients

Part III: everyday scenarios & recipes

Practical, step-by-step walkthroughs for common tasks.

---

75. Real-world developer workflows

Step-by-step walkthroughs of common developer tasks entirely within Neovim.

Workflow: investigating a bug

1. `<Space>fg` – search for the error message text across the project
2. `<Enter>` on the relevant result – jumps to the file and line
3. `gd` – go to the definition of the function that causes the error
4. `K` – read the function's documentation
5. `<Space>gr` – see everywhere this function is called (glance references)
6. `<Ctrl-o>` – jump back to where you were
7. `<Space>hb` – check git blame: who changed this and when
8. `]c` / `[c` – navigate to nearby git changes (hunks)

9. `<Space>hp` – preview what the hunk changed
10. Fix the issue, `<Space>w` to save, `<Space>rr` to run and test

Workflow: code review (reviewing your own changes)

1. `<Space>gs` – open git status
2. Navigate to a changed file, press `<Enter>` to open it
3. `]c` – jump to the first changed hunk
4. `<Space>hp` – preview the change
5. `]c` – next change, repeat
6. `:DiffviewOpen` – for a full side-by-side diff of all changes
7. When satisfied: `<Space>gw` to stage, `<Space>gc` to commit

Workflow: refactoring a function name across the project
If it's a code symbol (function, class, variable):

1. Place cursor on the name
2. `<Space>rn` – LSP rename, type new name, Enter
3. Done. All references updated intelligently.

If it's arbitrary text (e.g., a string, API path, config key):

1. `<Space>fg` – search for it first, verify all the places it appears
2. `:grep "oldText"` – populate the quickfix list
3. `:copen` – review the matches
4. `:cfdo %s/oldText/newText/gc | update` – replace with confirmation (`y` / `n` each) and save all files

Workflow: adding a feature in a new branch

1. `<Space>gbn` – create a new branch (type name, Enter)
2. `<Space>s` – open file tree, navigate to where you'll add files
3. `a` in the tree – create a new file
4. Write code; saving with `:w` formats it (`<Space>fm` formats on demand); `<Space>rr` to run/test
5. `<Space>de` – jump through any errors
6. `<Space>ca` – apply code action fixes
7. `<Space>gw` – stage the file
8. `<Space>gc` – commit
9. `<Space>gP` – push

Workflow: quickly editing a config file

1. `<Space>ff` – fuzzy find the config file by name
2. Make your changes
3. `<Space>w` – save
4. If it's the Neovim config: `<Space>sv` to restart Neovim (writes all buffers, restores windows/tabs/files; terminals such as Claude Code are not restarted)

Workflow: working with JSON

1. Open the JSON file
2. If it's messy: `<Space>fm` formats it with prettier (`:JSONFormat` also pretty-prints it)
3. `<Space>fg` in another terminal to find references to JSON keys
4. `za` to fold/unfold sections for readability

5. `ci"` to change a value inside quotes
6. `<Space>w` to save

Result of step 2: the one-line file `{"a":1,"b":[2,3]}` becomes seven lines with two-space indentation:

```
{
  "a": 1,
  "b": [
    2,
    3
  ]
}
```

Workflow: writing documentation (Markdown)

1. Open the `.md` file
2. `<Alt-m>` – live preview in browser (needs internet: Preview)
3. Write content; wrapping is auto-enabled for markdown
4. `^^` – add a footnote
5. `+` (operator) – convert lines to a bulleted list
6. `:Tabularize /|` – align a markdown table
7. `:AddRef label url` – add a reference link
8. `<Space>cz` – toggle spell check
9. `]s` / `[s` – navigate misspelled words
10. `z=` – fix spelling

Workflow: pair programming with split views

1. `:vs <file>` – open another file side-by-side
2. `<Ctrl-w>l` / `<Ctrl-w>h` – switch between the two files
3. `van` in file A – select the surrounding syntax node; repeat `an` to grow the selection (for example to the whole function), `in` shrinks it again; then `y` to copy
4. `<Ctrl-w>l` – switch to file B
5. `p` – paste the function
6. `<Ctrl-w>=` – equalize window sizes if they got uneven
7. `<Ctrl-w>o` – when done, close all splits except current

---

35. Java development (`nvim-java`)

The Java keys work only in a Java buffer with the Java language server (jdtls) attached: open nvim inside the Java devShell (`java` on PATH) or run `:DevEnv java` (section 43). Everywhere else the same keys (except the four global debug keys `<Space>jp`, `<Space>jP`, `<Space>jh`, `<Space>jx`) show one warning "Java: jdtls not attached (open nvim inside the Java devShell, or run :DevEnv java)". which-key groups: `<Space>j` Java, `jb` build, `jr` runner, `jt` test, `je` extract.

Build & run

Keymap	Description
`<Space>jbb`	Build workspace
`<Space>jbc`	Clean workspace
`<Space>jrr`	Run main class
`<Space>jrs`	Stop running main
`<Space>jrc`	Run compact source file (Java 25, `java <file>`)
`<Space>jrl`	Toggle runner log window
`<Space>jrp`	Profiles UI

Testing

Keymap	Description
`<Space>jtc`	Run all tests in current class
`<Space>jtC`	Debug all tests in current class
`<Space>jtm`	Run test method under cursor
`<Space>jtM`	Debug test method under cursor
`<Space>jtr`	View last test report

Refactoring

Keymap	Description
`<Space>jev`	Extract variable
`<Space>jeo`	Extract variable (all occurrences)
`<Space>jec`	Extract constant
`<Space>jem`	Extract method
`<Space>jef`	Extract field
`<Space>jj`	Change JDK runtime
`<Space>jd`	Configure debugger (DAP)
`<Space>jp`	Toggle breakpoint (global map, works in any buffer)
`<Space>jh`	Show value under cursor while the debugger is paused
`<Space>jP`	Clear ALL breakpoints in all files (no undo; capital P)
`<Space>jx`	Terminate the debug session

Worked examples of these keys are in section 78 (`languages/java.md`).

---

54. Java development in depth

Plugin: **nvim-java**.

This is the most feature-rich language setup in this config. It provides a full Java IDE experience. nvim-java loads when you open the first Java file of the session (not at startup), so opening a Java file takes a moment longer the first time; non-Java sessions do not pay for it.

How it works

nvim-java wraps the Eclipse JDT Language Server (jdtls) and adds:

- Build system integration
- Test runner and debugger
- Spring Boot tools
- Refactoring commands
- DAP (Debug Adapter Protocol) for step-through debugging

All of this only starts inside the Java devShell (`java` on PATH). Elsewhere `.java` files open without Java tooling, and the `<Space>j` keys show one warning "Java: jdtls not attached (open nvim inside the Java devShell, or run :DevEnv java)".

On Nix systems the JDK comes from the Java devShell (`JAVA_HOME`) and nvim-java never downloads one. On other systems nvim-java auto-installs a JDK.

Build & run

Keymap	Command	What it does
`<Space>jbb`	`:JavaBuildBuildWorkspace`	Compile the entire workspace
`<Space>jbc`	`:JavaBuildCleanWorkspace`	Clear the jdtls workspace cache (after a Yes/No question; jdtls is restarted automatically)
`<Space>jrr`	`:JavaRunnerRunMain`	Run the main class
`<Space>jrs`	`:JavaRunnerStopMain`	Stop the running program
`<Space>jrc`	(Lua function, Java buffers only)	Run a Java 25 compact source file with `java <file>` in a bottom terminal split; see Compact source files
`<Space>jrl`	`:JavaRunnerToggleLogs`	Show/hide the runner log window (it opens as a full-width, 15-line split at the bottom)
`<Space>jrp`	`:JavaProfile`	Profiles UI

Testing

Keymap	Command	What it does
`<Space>jtc`	`:JavaTestRunCurrentClass`	Run all `@Test` methods in the current class
`<Space>jtC`	`:JavaTestDebugCurrentClass`	Debug all tests (with breakpoints)
`<Space>jtm`	`:JavaTestRunCurrentMethod`	Run only the test method under cursor
`<Space>jtM`	`:JavaTestDebugCurrentMethod`	Debug only the test under cursor
`<Space>jtr`	`:JavaTestViewLastReport`	Show pass/fail results from the last test run

Debugging

Keymap	Command	What it does
`<Space>jd`	`:JavaDapConfig`	Configure the debug adapter (auto-runs on Java file open, but can be re-triggered)

DAP is configured automatically when jdtls starts. Four global keys:

Keymap	Same as	What it does
`<Space>jp`	`:DapToggleBreakpoint`	Toggle a breakpoint on the cursor line
`<Space>jP`	`:DapClearBreakpoints`	Remove ALL breakpoints in all files at once; no undo (capital P on purpose)
`<Space>jh`	`:lua require("dap.ui.widgets").hover()`	While paused: value of the variable under the cursor in a float
`<Space>jx`	`:DapTerminate`	End the debug session

Stepping and continuing deliberately have no keys: type `:DapContinue`, `:DapStepOver`, `:DapStepInto`, `:DapStepOut`. Workflow: Java section 7.

Refactoring

Keymap	Command	What it does
`<Space>jev`	`:JavaRefactorExtractVariable`	Extract the expression under the cursor (or selection) to a local variable
`<Space>jeo`	`:JavaRefactorExtractVariableAllOccurrence`	Extract and replace ALL occurrences of the expression
`<Space>jec`	`:JavaRefactorExtractConstant`	Extract a constant
`<Space>jem`	`:JavaRefactorExtractMethod`	Extract a method
`<Space>jef`	`:JavaRefactorExtractField`	Extract a field
`<Space>jj`	`:JavaSettingsChangeRuntime`	Switch the JDK version

Only the `:JavaBuild*` and `:JavaRefactor*` commands exist after jdtls has attached; the other `:Java*` commands exist as soon as nvim-java is loaded but do nothing useful without jdtls.

Worked examples of these keys are in section 78 (`languages/java.md`).

Chapter 9: Git

20. Git integration

vim-fugitive (plugin)

The fugitive keys (and the gitlinker keys below) exist only inside a git repository: nvim started in one, or a file of one opened. Outside a repository these keys are not mapped: `<Space>` just moves the cursor one column right and the next keys run as their normal Vim/plugin meaning (`<Space>gs` becomes `l` plus vim-swap's `gs`). `<Space>gbl` works everywhere (fzf-lua).

`<Space>gn` (Neogit) and `<Space>gD` (Diffview) belong to the same group: both are defined in `lua/config/fugitive.lua`, which loads together with fugitive when git is detected, so they exist only in a git repository too (once defined they stay for the rest of the session, even after `:cd` out of the repository). The plugins behind them are not repo-limited: `:Neogit` and `:DiffviewOpen` load on demand from any directory (Neogit itself still needs a repository to show anything).

Keymap	Mode	Description
`<Space>gs`	n	Git status window
`<Space>gw`	n	Git add current file
`<Space>ga`	n	Git add all changes (whole repository: new, modified and deleted files, `git add -A`)
`<Space>gu`	n	Unstage the current file (`git restore --staged`)
`<Space>gx`	n	Discard the changes in the current file (`git restore`). Asks Yes/No first, the default is No
`<Space>gv`	n	Vertical diff of the current file against the index (`:Gvdiffsplit`)
`<Space>gL`	n	Git log of the current file (`:Git log --oneline -- %`)
`<Space>gc`	n	Git commit
`<Space>gA`	n	Amend the last commit (`:Git commit --amend`)
`<Space>gz` / `<Space>gZ`	n	Stash the changes / pop the stash (`:Git stash`, `:Git stash pop`)
`<Space>gm` / `<Space>gR`	n	Puts `:Git merge ` / `:Git rebase ` on the command line: type the rest and press Enter
`<Space>gn`	n	Open Neogit (`:Neogit`)
`<Space>gD`	n	Open Diffview (`:DiffviewOpen`)
`<Space>gp`	n	Git pull
`<Space>gP`	n	Git push (opens terminal split)
`<Space>gb`	x	Git blame selected lines
`<Space>gbn`	n	Create new branch (prompts for name)
`<Space>gbd`	n	Puts `:Git branch -D ` on the command line: type the branch name and press Enter (force delete)
`<Space>gf`	n	Puts `:Git fetch ` on the command line (add arguments, then Enter)
`<Space>gbl`	n	Fuzzy-search git branches and check one out (fzf-lua)

Example: in a repository with one changed tracked file and one new file, `<Space>gs` opens the status window:

```
Head: main
Push: origin/main
Help: g?

Untracked (1)
? notes.txt

Unstaged (1)
M lua/mappings.lua
```

Close the window again with `gq`. `<Space>gw` in the `mappings.lua` buffer moves it from "Unstaged" to "Staged" (the header becomes `Staged (1)`), `<Space>gu` moves it back.

`<Space>gd`, `<Space>gr` and `<Space>gi` are not git keys: they are Glance (peek at definitions, references, implementations, see "Peeking Without Jumping (Glance)" in the code chapter). For a git diff use `<Space>gv`, for restoring use `<Space>gx`.

Clicking the branch name in the statusline also opens a branch picker (`git checkout` of the chosen local or remote branch).

gitsigns.nvim (plugin)
Shows `+` `~` `_` signs in the gutter for added/changed/deleted lines.

The gitsigns keys exist only in buffers of git-tracked files. In any other buffer they are not mapped.

A **hunk** is a contiguous block of changed lines. With the hunk keys you can stage (or discard) just one part of a file instead of the whole file.

Keymap	Mode	Description
`]c`	n	Jump to next git change (hunk)
`[c`	n	Jump to previous git change
`<Space>hp`	n	Preview the hunk in a floating window
`<Space>hb`	n	Show git blame for current line
`<Space>hs`	n, x	Stage the hunk; in visual mode stages the selected lines
`<Space>hr`	n, x	Reset the hunk (discards the change). Asks Yes/No first, the default is No; in visual mode resets the selected lines
`<Space>hu`	n	Unstage the last staged hunk
`<Space>hS`	n	Stage the whole buffer
`<Space>hR`	n	Reset the whole buffer (discards all changes in the file). Asks Yes/No first
`<Space>hd`	n	Diff the file against the index
`<Space>ht`	n	Toggle showing deleted lines

Example: compared with the last commit, `b = 2,` was changed to `b = 20,`, the line `d = 4,` was deleted and two lines were added at the end. The gutter shows:

```
  1  return {
  2    a = 1,
~ 3    b = 20,      <- changed line
_ 4    c = 3,       <- one or more lines were deleted below this line
  5    e = 5,
+ 6    f = 6,       <- added line
+ 7    g = 7,       <- added line
  8  }
```

The cursor on the `b = 20,` line and `<Space>hs` stages only that hunk: `git diff --staged` then contains just the `b` change, while the deleted and added lines are still unstaged, and the sign of that hunk changes to `┃`. `<Space>hu` unstages it again. `<Space>hr` asks `Reset this hunk (discard the change)? (Y)es, [N]o:`; with `y` the line is `b = 2,` again.

gitlinker.nvim (plugin)

Keymap	Mode	Description
`<Space>gl`	n, x	Copy permalink for current line(s)
`<Space>gbr`	n	Open repository in browser

Example (with the placeholder remote `https://github.com/user/repo.git`): select lines 3 to 5 of `lua/mappings.lua` (`Vjj`) and press `<Space>gl`; the link is `https://github.com/user/repo/blob/<full commit hash>/lua/mappings.lua#L3-L5`. The link needs the current commit to exist on the remote branch (the local `origin/<branch>` has to contain it): without that nothing is copied and no message appears.

Neogit (plugin)

Neogit is a full-screen git interface in the style of Emacs Magit: one status buffer shows the branch, the unstaged and staged changes and recent commits, and short keys act on the item under the cursor. It uses diffview.nvim for diff views and fzf-lua for pickers; both load with it. It loads on its first command and needs a git repository to be useful.

Key / command	Effect
`<Space>gn`	Open Neogit (`:Neogit`); the key exists only in a git repository, see the note at the top of section 20
`:Neogit`	Open the status buffer (a new tab) from any directory inside a repository
`:NeogitCommit [<sha>]`	Open the commit view of a commit (`HEAD` when no argument)
`:NeogitLogCurrent [<file>]`	Log of a file (the current file when no argument; with a visual range, the history of those lines)
`:NeogitResetState`	Reset the flags Neogit remembers for its popups

Inside the status buffer the keys are Neogit's own defaults (this config does not change them; see `:help neogit`): `?` opens a popup that lists the available keys, `s` stages and `u` unstages the item under the cursor, `x` discards it (with a confirmation), `<Tab>` folds or unfolds it, `<CR>` opens the file, `c` opens the commit popup, `q` closes the window. These keys were read from the plugin's help, not tried here.

vim-flog (plugin)

vim-flog shows the history of the repository as a graph: one line per commit, with lines joining the branches and merges. It is a viewer; commits themselves are made with fugitive.

Command / key	Effect
`:Flog`	Open the commit graph of the current repository in a new tab (the first run asks git to write a commit-graph file)
`g?` (inside Flog)	Show Flog's own list of keys
`<CR>` (inside Flog)	Open the commit under the cursor in a side window
`dd` (inside Flog)	Diff the commit under the cursor against the current `HEAD`
`u` (inside Flog)	Reload the graph
`gq` or `ZZ` (inside Flog)	Quit Flog

Example (a merge of two branches, hashes shortened):

```
  • [0ed345a] (HEAD -> main) Merge branch 'feature'
1 ├┐
```

```
2 │ • [4bf2d52] (feature) feature b
3 • │ [3c36aff] main b
4 ├─┘
5 • [fe7ed82] (origin/main) w2
6 • [f3c1cbb] wip
```

Each row also shows the date and the author; the lines on the left are the branches and the merge.

There is no key for `:Flog` in this config. Flog requires vim-fugitive (it is a documented prerequisite, and fugitive is loaded in a git repository), so open it inside a repository. Inside the Flog window any fugitive command can be used, and Flog's `gs` / `gu` / `gU` (show staged / untracked / unstaged changes) apply there instead of vim-swap's `gs`. The in-window keys are the plugin's defaults from its help (`:help flog-mappings`), not tried here.

diffs.nvim (plugin)
diffs.nvim makes diffs easier to read. It works on its own, without keys.

What	Effect
Automatic highlighting	Code inside the diffs shown by fugitive, neogit and gitsigns gets syntax highlighting, with intra-line changes marked; the integrations are switched on in `lua/plugin_specs.lua`
Conflict markers	Inline merge conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`) in working files are detected and highlighted; its resolve keymaps are off by default and this config does not turn them on (use the diffview keys above); diagnostics are switched off in a buffer while it has conflicts
`:Diff`	Diff of the current file against git. Without arguments it compares the index with the working file, so it shows unstaged changes only; `:Diff <revision>` (for example `HEAD~3` or a branch) compares the current file at that revision with the working file; `:Diff review` opens a review of the whole repository; `:vertical Diff` splits vertically

This plugin is not fetched from GitHub but from forge.barrettruth.com. The argument forms of `:Diff` come from the plugin's help (`:help diffs.nvim-commands`) and were not tried here.

codediff.nvim (plugin)
codediff.nvim shows two versions of a file side by side with VSCode-like highlighting of the changed words inside each line.

Command	Effect
`:CodeDiff`	Open an explorer of the changed files of the repository; pick a file to see its side-by-side diff
`:CodeDiff <revision>`	Compare a branch, commit or `HEAD~N` with the working tree
`:CodeDiff <revision> <revision>`	Compare two revisions
`:CodeDiff --staged`	Only the changes in the git index

There is no key for it in this config. The first use downloads a small native diff library (network needed once; `:CodeDiff install` does it explicitly). More forms exist (pull requests, history, other repositories): see the plugin's README. The forms above come from that README and were not tried here.

Other Git tools

Plugin	Command / Trigger	Description
neogit	`:Neogit`	Full git UI (magit-like; loads on the first `:Neogit*` command, in any directory; also `:NeogitCommit`, `:NeogitLogCurrent`, `:NeogitResetState`)
diffview.nvim	`:DiffviewOpen`, `:DiffviewFileHistory` (`:DiffviewClose` once a view was opened)	Side-by-side diff viewer and 3-way merge tool; file history panel
vim-flog	`:Flog`	Visual git log graph
diffs.nvim	`:Diff` (and automatic)	Unified diff of the current file against the index (unstaged changes; details in its section above); also colours the diffs shown by fugitive, neogit and gitsigns, and conflict markers
codediff.nvim	`:CodeDiff`	VSCode-style side-by-side diff (downloads a small native library on first use)

Resolving merge conflicts (diffview.nvim)

During a merge, `:DiffviewOpen` opens the 3-way merge tool. Inside a diffview view:

Keymap	Description
`<Space>gCo` / `<Space>gCt`	Choose OURS / THEIRS for the conflict
`<Space>gCb` / `<Space>gCa`	Choose BASE / BOTH
`]C` / `[C`	Next / previous conflict (capital C; `]c` / `[c` stay the gitsigns hunk keys)

Example (`main` and `feature` both changed the line `b = 2,`): `:DiffviewOpen` shows OURS and THEIRS side by side at the top and the working file (LOCAL) below, with the conflict markers in it. In the LOCAL window `]C` jumps to the conflict, then:

```
<<<<<<< HEAD                <Space>gCo (ours)    <Space>gCt (theirs)    <Space>gCa (both)
  b = "ours",                 b = "ours",          b = "theirs",          b = "ours",
=======                                                                   b = "theirs",
  b = "theirs",
>>>>>>> feature
```

`<Space>gCb` (BASE) gives the part between the `|||||||` and `=======` markers; with git's default conflict style that part does not exist, so the conflicting lines simply disappear. `u` undoes a choice.

`<Space>cb` and `<Space>ca` keep their normal meaning inside diffview. fugitive's `:Gvdiffsplit!` is the other way to resolve a conflict.

---

48. Git workflow in depth

The Git plugin ecosystem

This config includes several git-related plugins that each handle a different aspect:

Plugin	What it does	How to use
vim-fugitive	Run git commands from inside Neovim. The core git plugin.	`<Space>gs` for status, `<Space>gc` for commit, etc.
gitsigns.nvim	Shows which lines changed in the gutter. Navigate between changes.	`]c` / `[c` to jump between hunks, `<Space>hp` to preview.
gitlinker.nvim	Generate shareable URLs to specific lines of code.	`<Space>gl` to copy a permalink.
neogit	A full git UI inside Neovim (like Magit for Emacs).	`:Neogit` to open (loads on that command; also `:NeogitCommit`, `:NeogitLogCurrent`, `:NeogitResetState`).
diffview.nvim	Side-by-side diff viewer for comparing branches, commits, etc.	`:DiffviewOpen` to open.
vim-flog	Visual git log/graph showing branch history.	`:Flog` to open.
diffs.nvim	Syntax highlighting inside the diffs of fugitive, neogit and gitsigns; conflict markers. `:Diff` shows the file against the index.	Automatic, `:Diff`.
codediff.nvim	VSCode-style side-by-side diff.	`:CodeDiff`.

Merge conflicts: `:DiffviewOpen` is the merge tool (keys in section 20, "Resolving Merge Conflicts").

Daily Git workflow

A typical workflow entirely from within Neovim:

1. **Check status**: `<Space>gs` opens the fugitive status window
2. **Stage**: `<Space>gw` stages the current file, `<Space>ga` stages everything in the repository, `<Space>hs` stages only the hunk under the cursor (or use `s` in the status window); `<Space>gu` unstages the current file
3. **Review changes**: `<Space>hp` to preview hunks, or `]c`/`[c` to navigate between them
4. **Commit**: `<Space>gc` opens a commit message buffer. Write message, then `:wq`
5. **Push**: `<Space>gP` pushes (opens a terminal split showing progress)
6. **Pull**: `<Space>gp` pulls latest changes (`<Space>gz` stashes your changes first, `<Space>gZ` pops the stash)
7. **Blame**: Select lines in visual mode, then `<Space>gb` to see who wrote them
8. **Create branch**: `<Space>gbn` prompts for a branch name
9. **Share code**: `<Space>gl` copies a permalink to the current line

Understanding gitsigns.nvim

The gutter signs mean:

- `+` : This line was **added** (new code)
- `~` : This line was **modified** (changed from last commit)
- `_` : A line was **deleted below** this line
- `‾` : A line was **deleted above** this line
- `|` : This line has both additions and deletions (change-delete)

Hunk navigation: `]c` jumps to the next changed block (hunk), `[c` jumps to the previous. This is very useful during code review.

Chapter 10: Claude, Markdown, LaTeX/Typst, spelling, URLs

9. AI assistant window (Claude Code, `claude-code.nvim`)

Claude Code keys (claude-code.nvim)

Keymap	Mode	Description
`<Space>cc`	n	**Toggle** Claude Code terminal (opens/closes it)
`<Space>a`	n	Fresh Claude session for a quick "how do I do X in Neovim" question: vertical split on the right, started in the nvim config folder with the `answering-neovim-usage-questions` skill loaded, always on the Sonnet model (`--model sonnet`) and permissions bypassed (no prompts); `/exit` closes it (on the dashboard it opens in its own tab). See section 33 "Help keys"
`<Space>cR`	n	Resume/continue the last Claude conversation (`claude --continue`)
`<Space>cV`	n	Start Claude in verbose mode
`:ClaudeCodeResume`	cmd	Start `claude --resume` (pick an older conversation; command only, no key)
`<Ctrl-h/j/k/l>`	t, n (Claude panel)	Move to the window left / below / above / right
`<Ctrl-w>h/j/k/l`	t (any Claude terminal, including the `<Space>a` split)	Move to the window left / below / above / right. Works even after you typed and pressed `<Esc>` (which goes to Claude). Press `<Ctrl-w>` (hold Ctrl, press `w`, release both), then `h`, `j`, `k` or `l`
`<Ctrl-q>`	t (any Claude terminal, including the `<Space>a` split)	**Leave terminal mode** (Normal mode in the same window), the one-chord form of `<Ctrl-\><Ctrl-n>`. Needed because `<Esc>` goes to Claude. Afterwards `:q`, `<Space>q`, `<Space>Q` and `<Space>cc` work; `i` goes back into Claude. Only Claude terminals have it, other terminals leave with `<Esc>`
`<Ctrl-f>` / `<Ctrl-b>`	t (Claude panel)	Scroll a page down / up

Claude Code opens as a **vertical split on the right**, 30% of the screen width. It is a terminal buffer. There is no toggle key in terminal mode. To navigate:

1. **Move to the Claude window**: `<Ctrl-w>l` or `<Right>` (it opens on the right). Focusing it puts you in terminal (insert) mode by itself.
2. **Move back to code**: `<Ctrl-h>` or `<Ctrl-w>h` (both work directly in the Claude terminal; no need for `<Ctrl-\><Ctrl-n>` first). `<Ctrl-w>j/k/l` move to the other neighbours the same way. Afterwards you are in Normal mode in the target window. `<Esc>` is sent to Claude (for example to interrupt it) and does **not** leave terminal mode.
3. **Close Claude**: from the code window `<Space>cc` toggles it closed. Inside the Claude terminal press `<Ctrl-q>` first (or `<Ctrl-\><Ctrl-n>`), then `<Space>cc` or `<Space>q`. If Claude is the only window (for example after `:only`) you are in terminal mode, so `:q`, `<Space>q` and `<Space>Q` are typed into Claude until you press `<Ctrl-q>`.
4. **Type in Claude**: If in Normal mode inside the Claude terminal, press `i` to re-enter terminal mode

Example for `<Space>a`: press it, then type a plain question such as `how do I delete everything inside quotes` and press `<Enter>`. The skill it loads answers as numbered steps that name the exact keys. `/exit` ends the session and closes the split.

Claude runs in the git root of the current file. Files Claude changes are reloaded in their buffers (checked every second while the panel is open). The panel is left out of saved sessions, and `\D` never deletes it while it runs.

---

49. AI-assisted development in depth

Claude Code in depth (claude-code.nvim)

Plugin: **claude-code.nvim**

Claude Code is an AI coding assistant that runs in a terminal inside Neovim. The plugin loads right after the first screen is drawn (VeryLazy), so its keys and `:ClaudeCode*` commands exist a fraction of a second after Neovim starts.

Keymap	Mode	What it does
`<Space>cc`	n	Toggle the Claude Code terminal. Opens as a vertical split on the right (30% of the width).
`<Space>cR`	n	Resume/continue the last Claude conversation (`claude --continue`)
`<Space>cV`	n	Start Claude in verbose mode

Using Claude Code:

1. Press `<Space>cc` to open
2. It opens as a terminal buffer in a vertical split on the right
3. Type your request and press Enter
4. Claude can read and edit your files directly
5. To go back to your code press `<Ctrl-w>h` (or `<Ctrl-h>`); a plain `<Esc>` goes to Claude (e.g. to interrupt it) and does not leave terminal mode. To close the panel, press `<Ctrl-q>` (leaves terminal mode), then `<Space>cc`

Claude Code uses your project's git root as the working directory. `:ClaudeCodeResume` starts `claude --resume`. Inside the panel `<Ctrl-h/j/k/l>` and `<Ctrl-w>h/j/k/l` move to the neighbour windows (the `<Ctrl-w>` forms also work in the `<Space>a` split and after typing and pressing `<Esc>`) and `<Ctrl-f>` / `<Ctrl-b>` scroll.

---

27. Markdown support

Preview (mdkite.nvim)

Keymap	Description
`<Alt-m>`	Toggle markdown preview in browser (`:MdKite toggle`; markdown buffers only). Needs internet; with a preview running, `<Alt-m>` in a second Markdown buffer stops it (`:MdKite start` there moves it). Details: Preview (mdkite.nvim)

Truth tables and Boolean algebra (truth-table.nvim)

For digital-logic notes: `:TruthTable A B` builds a table, `:TruthTableExpand A xor B` adds columns, `:TruthTableKarnaugh` draws a Karnaugh map, `:TruthTableDeMorgan` / `:TruthTableSimplify` preview rewrites that `:TruthTableApplyStep` writes as `≡` steps, `:TruthTableVerify!` proves them, `:TruthTableTutor` is a built-in course. Keys `<Space>T...` (tables) and `<Space>l...` (rewrites); `and@` in insert mode types `∧`. Full guide with a worked lesson: Truth tables and Boolean algebra.

Footnotes (vim-markdownfootnotes)

Keymap	Mode	Description
`<Space>mf`	n	Add footnote (markdown buffers only; elsewhere one warning)
`<Space>mr`	n	Return from footnote (markdown buffers only)
`^^`	n, i	Insert footnote number (markdown files only). Because of this map, a single `^` or `@` typed in insert mode appears after a short pause (500 ms), since Neovim waits for a possible second key
`@@`	n, i	Return from footnote (markdown files only; it shadows the macro replay `@@` in Markdown buffers)

Text objects & operators (Markdown only)

Keymap	Description
`vic` / `vac`	Select inside / around code block
`+`	Convert lines to unordered list (operator: `+ip` for paragraph)
`<Space>mb{motion}` (or `<Space>mb` on a Visual selection)	Add a hard line break (a trailing `\`) to each line, e.g. `<Space>mbip`; blank lines and lines already ending in `\` are skipped
`:AddRef <label> <url>`	Add reference link at end of buffer

Examples (all in a Markdown buffer):

Keys	Before	After
`+ip`	`plain one` / `plain two`	`+ plain one` / `+ plain two`
`<Space>mbip`	`plain one` / `plain two`	`plain one\` / `plain two\`
`<Space>mf` (cursor at the end of the line; the mark goes in at the cursor)	`Some text here.`	`Some text here.[^1]` and a new last line `[^1]:` with the cursor behind it, ready for the note
`:AddRef s https://example.com`	`See [site][s].`	the same line, then a blank line, `<!-- Reference links -->` and `[s]: https://example.com` at the end of the buffer
`vic` (cursor inside a fenced block)	```` ```lua ```` / `a = 1` / `b = 2` / ```` ``` ````	the two lines `a = 1` and `b = 2` are selected, without the fences

Other Markdown plugins

- **render-markdown.nvim**: In-editor rendering (pauses in insert mode). Max file: 1.5MB.
- **tabular**: Table alignment. Command: `:Tabularize`
- **ltex-ls-plus (LSP)**: grammar and spell checking (LanguageTool) for markdown, tex, typst, gitcommit and text files, when `ltex-ls-plus` is installed. Problems are diagnostics: `]d` / `[d`, `<Space>dd`; fixes via `<Space>ca`.
- **marksman (LSP)**: link and heading navigation, completion, symbols.

---

28. LaTeX and Typst support

LaTeX (`vimtex`)

Only available if `latex` is installed (vimtex also needs `latexmk` to compile).

Keymap	Description
`<Space>rf` / `<F9>`	Start / stop continuous compilation
`\ll`	Start / stop continuous compilation (same as `<Space>rf` / `<F9>`)
`\lv`	View PDF

The texlab language server adds diagnostics, hover, symbols and rename when `texlab` is on PATH (LaTeX devShell). Auto-save never saves LaTeX files.

Typst (`typst.vim`, tinymist)

Keymap	Description
`<Space>tw`	Typst buffers: `:TypstWatch`, recompile on save and open the PDF

Needs the `typst` program (typst devShell): without it `<Space>tw` shows one warning and the Typst plugin does not load. Errors go to the quickfix list. The PDF viewer is `$TYPST_PDF_VIEWER` if set, else `zathura` if installed. LSP: `tinymist`. Typst buffers use `textwidth=100` and wrap. Auto-save never saves Typst files.

---

31. Spell checking

Languages: English, Italian, German, French.

Keymap	Description
`<Space>cz`	Toggle spell checking on/off
`]s` / `[s`	Next / previous misspelled word
`z=`	Show spelling suggestions (the popup lists up to 19: keys `1-9`, `0`, `a-j`; press the key, no `<Enter>`)
`zg`	Add the word to the English word list (`spell/en.utf-8.add`)
`2zg` / `3zg` / `4zg`	Add the word to the Italian / German / French list
`zw`	Mark word as wrong (same counts)
`zug`	Undo the last `zg`
`:e ~/.config/nvim/spell/en.utf-8.add`	Show the allowed words of a language (`it`, `de`, `fr` in the other files); delete a line to remove a word

`spellfile` has one word list per language, in the order of `spelllang` (en, it, de, fr): `zg` adds to the first, a count picks another. The lists are `spell/*.utf-8.add` inside the config, which is tracked in a PUBLIC repository: `spell/README.md` says these words are public, so review new words before committing. At startup Neovim silently recompiles any list whose compiled `.add.spl` file is missing or older, so the words of the tracked lists are accepted on a fresh checkout.

Example: in a text file with the line `The recieve button`, `<Space>cz` switches spell checking on, `recieve` is marked, and with the cursor on it `z=` opens a popup titled "Spelling suggestions" that lists the suggestions, each with a key (`1 → receive`, `2 → relieve`, …; this config's popup lists the keys `1-9`, `0` and `a-j`). Press `1` (no `<Enter>`): the line is now `The receive button`. `zg` on `recieve` instead adds the word to `spell/en.utf-8.add`, so it is no longer marked.

Besides the builtin spell checker two language servers report problems as diagnostics: `ltex_plus` (grammar and spelling in prose files; the language is detected per file, so Italian, German and French text is checked in its own language; force one with a magic comment, see section 80) and `typos_lsp` (typos in identifiers and comments of source files). The statusline shows `[SPELL]` while spell checking is on.

---

38. URL & Unicode (`gx.nvim`, `vim-highlighturl`, `unicode.vim`)

Keymap	Mode	Description
`gx`	n, x	Open the URL or file under the cursor (gx.nvim)
`ga`	n	Show Unicode info for character under cursor
`<Ctrl-x><Ctrl-z>`	i	Complete a Unicode character by name or `U+code`
`<Ctrl-x><Ctrl-g>`	i	Complete a digraph
`<Space>cu`	n	Swap `<Ctrl-x><Ctrl-z>` between completing the character and completing its name
`<F4>` + motion (needs a function key; alternatives below)	n, x	Turn 2-character digraph pairs in the text into their characters

Commands: `:UnicodeSearch {name or U+hex}`, `:UnicodeName`, `:UnicodeTable`. The plugin loads on the first `ga`, `<Space>cu` or `:UnicodeSearch`; the insert keys, `<F4>` and `:UnicodeName` / `:UnicodeTable` exist only after that. Without function keys use `<Ctrl-k>` + two letters in Insert mode (built-in Vim digraph input, outside a snippet; `<Ctrl-k>` `a` `:` gives `ä`) or `<Ctrl-x><Ctrl-g>`. Example: `<F4>$` on `a:e:o:u:` gives the umlauts (per the plugin doc).

Examples: in Insert mode `<Ctrl-k>` `a` `:` types `ä`. `ga` with the cursor on that `ä` prints `'ä' U+00E4 Dec:228 LATIN SMALL LETTER A WITH DIAERESIS (a: a") ä /\%ue4 "\u00e4"`: the character, its code point and decimal value, its name, its digraph and its HTML entity.

URLs in buffers are automatically highlighted (vim-highlighturl plugin).

Chapter 11: Various: custom commands, other plug-ins, configuration, Neovide

34. Custom commands

Command	Description
`:CopyPath nameonly`	Copy filename to clipboard
`:CopyPath relative`	Copy `<project-root>/path/to/file` (root = nearest `.git` or `pyproject.toml`; a warning when there is none)
`:CopyPath absolute`	Copy absolute path
`:JSONFormat`	Pretty-print JSON through Python (whole file or visual range; no key; `<Space>fm` formats JSON with prettier)
`:Redir <cmd>`	Run the command and show its output in a new tab (scratch buffer, wiped when closed)
`:Edit <pattern>...`	Open every file matching the glob patterns (`:Edit src/*.lua`); `:edit` typed as the first word expands to `:Edit`
`:Datetime [format]`	Show date and time (optional format argument)
`:ToPDF`	Convert markdown to PDF (requires pandoc)
`:Z {keywords}`	zoxide jump (see Working with directories)
`:TermHL`	Show the current buffer (e.g. a log with ANSI colour codes) rendered with its colours in a read-only terminal buffer
`:LogAutocmds`	Toggle logging of all autocommand events to `~/.local/state/nvim/log-autocmds.log` (the file is emptied each time logging starts)
`:StripTrailingWhitespace`	Remove trailing whitespace (same as `<Space><Space>`)
`:Notifications`	Show the notification history (nvim-notify)
`:Inspect` / `:InspectTree`	Show the highlight groups / the Treesitter tree at the cursor

Examples (in the file `lua/mappings.lua` of a git repository at `/tmp/demo`):

Command	Result
`:CopyPath nameonly`	The clipboard (register `+`) holds `mappings.lua`; Neovim prints "Filepath copied to clipboard!"
`:CopyPath relative`	`<project-root>/lua/mappings.lua`
`:CopyPath absolute`	`/tmp/demo/lua/mappings.lua`
`:Datetime`	Prints the date and time, for example `2026-10-05 10:57:32+0200`
`:Edit lua/*.lua`	With `lua/mappings.lua`, `lua/x.lua` and `lua/y.lua` on disk, all three are in the buffer list (the last one, `lua/y.lua`, is shown)
`:Redir echo 1`	A new tab opens with a scratch buffer that contains the output (`1`); the buffer is wiped when the tab is closed

Plugin manager shortcuts (lazy.nvim)

Type these in command mode, then press space (or Enter) to expand:

Shortcut	Expands to
`pi`	`:Lazy install`
`pud`	`:Lazy update`
`pc`	`:Lazy clean`
`ps`	`:Lazy sync`

---

39. Other plugins

Plugin	Trigger	Description
`auto-save.nvim`	Automatic (active from right after the first screen)	Saves on `FocusLost` / `BufLeave` (a small nvim-notify popup "AutoSave: saved at HH:MM:SS"); never formats; never saves unnamed, read-only or special buffers, nor Typst and LaTeX files
`better-escape.vim`	`jk` (insert)	Fast escape from insert mode (200ms window)
`vim-repeat`	`.`	Makes plugin actions repeatable with `.`
`vim-swap`	`gs` (n, x)	Interactively swap function arguments / list items
`vim-eunuch`	`:Rename`, `:Delete`	Unix file operations (language servers are not told; for that rename in the file tree, see section 11)
`vim-obsession`	`:Obsession`	Session save/restore
`instant.nvim`	`:InstantStartServer`, `:InstantStartSession {port}`, `:InstantJoinSession {host} {port}`	Collaborative editing (loads on its first `:Instant...` command)
`firenvim`	Browser	Neovim in browser text areas
`aerial.nvim`	`<Space>t`	Symbol outline (section 37)
`treesj`	`gS`	Split / join code blocks
`vim-illuminate`	`<Alt-n>`, `<Alt-p>`, `<Alt-i>`	Word references: `<Alt-n>` / `<Alt-p>` jump to the next / previous one, `<Alt-i>` selects the reference (Visual and operator-pending mode)
`vimade`	Automatic	Dims inactive windows
`persistence.nvim`	Dashboard `r` / `L`	Saves a session per folder when you quit (see Session Management)
`snacks.nvim`	Automatic	Nicer input / select popups; light mode for big files
`colorful-menu.nvim`	Automatic	Completion labels coloured like code
`live-command.nvim`	`:norm`	Live preview of `:norm` while you type
`lazydev.nvim`	Lua files	Neovim API completion when editing the config
`vim-oscyank`	`:OSCYank {text}`, `:OSCYankVisual`, `:OSCYankRegister`	Copy to the system clipboard through the terminal (works over SSH; Linux). `:OSCYank` needs the text as an argument; in Visual mode use `:OSCYankVisual`, for a register `:OSCYankRegister {reg}`
`vim-be-good`, `vim-dadbod-ui`	`:VimBeGood`, `\w`, `\W`, `:Scriptnames`, `:Verbose {command}`	(see below)

telescope.nvim (second picker)

Plugins: **telescope.nvim** and **telescope-symbols.nvim**. Telescope is a second popup picker next to fzf-lua (the file, grep and buffer pickers are fzf-lua, see `05-search-and-files.md`). It is lazy: it loads only on its first `:Telescope` command, so nothing starts at launch.

Command / key	Effect
`<Space>db`	Telescope list of the diagnostics of the current buffer (the config calls `telescope.builtin.diagnostics` for this buffer only)
`:Telescope keymaps`	Searchable list of all keymaps
`:Telescope symbols`	Pick a symbol from a list and insert it at the cursor (emoji, kaomoji, gitmoji, math, latex, julia, nerd font glyphs)

telescope-symbols.nvim has no command, key or setup of its own; it only ships the symbol lists (JSON files) that the built-in `:Telescope symbols` picker reads from the runtime path. Lazy.nvim loads it together with Telescope. The keys inside the picker (and the ways Telescope differs from fzf-lua) are in "Moving Inside Any Picker" in `05-search-and-files.md`. `:Telescope symbols` was read from the plugin code, not tried.

Copying over SSH (vim-oscyank)

Plugin: **vim-oscyank** (enabled on Linux only; loaded on its first command). It wraps text in an OSC 52 escape sequence and writes it to the terminal, which then puts it into the system clipboard of the machine you sit at, also when Neovim runs on another machine over SSH. Your terminal (and tmux, if used) must allow OSC 52 clipboard writes. This configuration maps no key for it.

Command	Effect
`:OSCYank {text}`	Copy the given text (the argument is required)
`:OSCYankVisual`	Copy the Visual selection (run it from Visual mode)
`:OSCYankRegister {reg}`	Copy the content of a register

Example (inside tmux with `set-clipboard on` and `allow-passthrough on`): `:OSCYank hello` shows `[oscyank] 5 characters copied` and the text `hello` arrives in the terminal's clipboard (in tmux it appeared in tmux's own paste buffer). Pasting on the local machine then gives `hello`.

Keyboard layout switching (vim-xkbswitch)

Plugin: **vim-xkbswitch** (`lyokha/vim-xkbswitch`). It switches the keyboard layout automatically when you enter and leave Insert mode, so that Normal-mode keys keep working while you type text in another layout (for example a non-Latin one).

- **Only on macOS**, and only when the `xkbswitch` command is found on `PATH`. On Linux and Windows it is declared but disabled: it is not installed and does nothing.
- It loads on the first `InsertEnter` and the config only turns it on (`XkbSwitchEnabled = 1`); there are no keys and no commands of our own.

Neovim in the browser (firenvim)

Plugin: **firenvim**. With the Firenvim browser extension installed, a browser text area can be edited in a Neovim window. The config applies the following only when Neovim was started by Firenvim:

Setting	Value
Which fields	`textarea` elements only
Takeover	`never`: Firenvim does not open by itself; you start it from the extension
Command line	Neovim's own (`cmdline: neovim`)
Filetype	Buffers from `github.com` and `stackoverflow.com` are Markdown, `sqlzoo*` buffers are SQL
Look	No sign column, ruler, command display, statusline or tabline in the browser window; lualine and bufferline are not loaded there

The plugin build (`:Lazy build firenvim`) installs the native part for the browser, using the `PATH` of the running Neovim. The browser-side button or shortcut for a manual takeover belongs to the extension, not to this config.

Live preview of :norm (live-command.nvim)

Plugin: **live-command.nvim**. When `norm` is typed as an Ex command (at the start of the command line, after a range, after `|`, after `:g/pattern/` or `:v/pattern/`, or after a command modifier such as `:silent!`), a command-line abbreviation turns it into `:Norm`. `:Norm` behaves like `:norm` but shows the effect in the buffer, with the changed text highlighted inline, while you are still typing. The word `norm` inside a pattern, a string or other text is left alone.

You type	Effect
`:%norm Atext`	Preview of appending `text` to every line; `<Enter>` runs it, `<Esc>` leaves the buffer unchanged
`:g/<pattern>/norm dd`	The preview also works after `:g/.../` and `:v/.../`
`:silent! norm ...`	The preview also works after command modifiers

Example (in a terminal): with the three lines `a` / `b` / `c`, typing `:%norm Atext` (without `<Enter>`) turns the buffer into `atext` / `btext` / `ctext` while you type, and the command line shows `:%Norm Atext`. `<Enter>` keeps the result, `<Esc>` restores `a` / `b` / `c`. The other rows of the table are standard `:norm` uses.

Vim-script debugging (vim-scriptease)

Plugin: **vim-scriptease**. Loaded on the first use of one of its three commands (the lazy list names `:Scriptnames`, `:Messages`, `:Verbose`). The plugin has more commands; only these three are loaded on demand here.

Command	Effect (from the plugin's help)
`:Messages`	Load the output of `:messages` into the quickfix list; `:Messages clear` clears the messages. Not the same as the built-in lower-case `:messages`
`:Scriptnames`	Load the list of `:scriptnames` into the quickfix list and open it
`:Verbose {command}`	Like `:verbose {command}`, but the output goes to a file and is shown in the preview window (handy for noisy commands; a count in front raises the verbosity)

WhatsApp in Neovim (whatsapp.nvim)

Plugin: **whatsapp.nvim**. An unofficial WhatsApp client: the chat list is on the left, the conversation on the right and a compose buffer underneath. It runs a small Go program (`bin/wa-bridge`, built on the whatsmeow library) as a child process and shows up on your phone as a linked device. It may violate the WhatsApp terms of service, and attachments can be malicious, so every open asks for confirmation. This config maps two keys in Normal mode, `\w` (open or jump to the WhatsApp tab, same as `:WhatsApp`) and `\W` (close it, same as `:WhatsApp close`), and the dashboard has an `Open WhatsApp` item (second to last, above `Quit Nvim`). The plugin has its own keys inside its window, and `g?` lists all of them.

The plugin loads on the first `:WhatsApp`. Its repository keeps the Lua part in the `dist/` subfolder, so the config adds that folder to the runtime path when the plugin loads.

Building the bridge. The bridge needs `make`, `go` and a C compiler (SQLite uses cgo). lazy.nvim builds it on install and update only when all three are on Neovim's PATH (they do not have to be on your shell's PATH); otherwise the build is skipped without a message. If you run `:WhatsApp` without a built bridge you get a warning (nothing at startup) that names the fix: once the three tools are on Neovim's PATH, run `:Lazy build whatsapp.nvim` and restart Neovim. The build downloads the Go modules, so it needs the network.

Command	Effect
`:WhatsApp pair`	Show a QR code; scan it on the phone under WhatsApp > Settings > Linked Devices > Link a Device (needed once)
`:WhatsApp`	Open (or jump to) the WhatsApp tab; `nvim +WhatsApp` starts Neovim straight into it
`:WhatsApp {chat name}`	Open the chat with that name
`:WhatsApp toggle`, `close`, `status`, `stop`, `names`	Toggle or close the tab, show the connection status, stop the bridge, refresh contact names
`:WhatsApp logout`	Unlink the device and forget the local session

Keys inside the WhatsApp window (from the plugin's README):

Key	Effect
`<CR>` / `i`	Open the chat under the cursor / open it and start composing
`u`	Jump to the next unread chat
`gU` / `gu` / `~`	Mark unread / read / toggle (mirrored to the phone)
`<Tab>` / `<S-Tab>`	Cycle between the list, the conversation and the compose pane
`R` / `r`	Reply to / react to the message under the cursor
`K` / `L`	Load older messages / toggle bubbles and plain lines
`:w` in the compose pane	Send the message (several lines are fine)
`gx`, `[m`, `]m`	Download if needed and open an attachment / move between attachments
`g?` / `q`	Show every key and command / go back

Each chat keeps its own unsent draft until Neovim exits. New messages in a chat you are not viewing raise a `vim.notify` notification. Pictures are drawn inside the bubble in terminals with the Kitty graphics protocol (kitty, Ghostty); elsewhere you get a tag line and `gx`.

Practice game (vim-be-good)

Plugin: **vim-be-good**. A small game collection to practice basic Vim movements. Open an **empty** buffer first (in a non-empty file the plugin raises an error), then run `:VimBeGood`. The menu lists the games: `words`, `ci{`, `relative`, `hjkl`, `whackamole`, `snake` and `random`. Delete a line of the list to pick it: deleting a difficulty selects that difficulty, deleting a game starts it. The instructions are at the top of each game. For example, in `relative` you delete the line that says `DELETE ME` using relative jumps, and in `whackamole` you move to the character that has the caret under it and flip its case.

The plugin loads on the first `:VimBeGood`; no keys are mapped. `let g:vim_be_good_delete_me_offset = 35` would fix the offset of the `relative` game instead of a random one (not set in this config). Please note that the difficulty only works on a few games.

Libraries and dependencies

These plugins have no commands or keys. Other plugins need them, and lazy.nvim loads them when one of those plugins starts or asks for them.

Plugin	What it is	Needed by (in this configuration)
plenary.nvim	Lua helper library (async, paths, jobs)	claude-code.nvim, neogit and Telescope
promise-async	Promise and async library	nvim-ufo (code folding)
lush.nvim	Library for writing colour themes in Lua	the arctic colorscheme (see "Colorschemes" in `06-windows-terminal-sessions.md`)
nui.nvim	Popup, menu and layout building blocks	nvim-java, nvim-dbee, ascii.nvim (see "Icons and UI libraries" in `06-windows-terminal-sessions.md`)
nvim-web-devicons, mini.icons	File and kind icons	see "Icons and UI libraries" in `06-windows-terminal-sessions.md`

SQL databases (nvim-dbee, vim-dadbod-ui)

Two independent SQL clients. Neither has connections by default: this repo is public, so connections are never stored in the config.

Keymap	Mode	Description
`<Space>Dt`	n	Dbee: toggle the UI
`<Space>Do`	n	Dbee: open the UI
`<Space>Dc`	n	Dbee: close the UI
`<Space>Du`	n	Dadbod: toggle the UI (connections, saved queries, results)
`<Space>Da`	n	Dadbod: add a connection
`<Space>Df`	n	Dadbod: find the DB buffer

Commands: `:Dbee`, `:DB <url> <query>` (`:%DB <url>` runs the whole buffer as a query), `:DBUI`, `:DBUIToggle`, `:DBUIAddConnection`, `:DBUIFindBuffer`.

Connections come from the environment (set them in an untracked shell file, direnv or sops, never in the repo):

- nvim-dbee: `$DBEE_CONNECTIONS`, a JSON array such as `[{"name":"local","type":"postgres","url":"postgres://user:pw@localhost:5432/db"}]`. Connections added inside the UI are not saved.
- vim-dadbod-ui: `$DADBOD_CONNECTIONS`, a JSON object `{"name":"url"}`. Saved queries go to `~/.local/share/nvim/db_ui`.

`:checkhealth` shows a known, accepted `vim.validate{}` deprecation warning from nvim-dbee. Connections come from the environment, never from the config: `$DBEE_CONNECTIONS` (JSON list, dbee) and `$DADBOD_CONNECTIONS` (JSON object name -> URL, dadbod), for example `export DBEE_CONNECTIONS='[{"name":"test","type":"sqlite","url":"/path/test.sqlite"}]'`. Keys:

Where	Keys	What it does
dbee	`<Space>Do` open, `<Space>Dt` toggle, `<Space>Dc` close	`<Space>Do` opens four windows: drawer, SQL editor, result, call log
dbee drawer	`<CR>`, `o`, `r`, `cw`, `dd`	select / expand, toggle, refresh, rename, delete; the drawer's own help is listed inside it
dbee editor	`BB`, `<CR>`	`BB` runs the whole file (Visual: the selection), `<CR>` runs the statement under the cursor; the rows appear in the result window
dbee result	`L` / `H`, `E` / `F`, `yaj` / `yac`, `<C-c>`	next / previous page, last / first page, yank the row as JSON / CSV, cancel
dadbod (`<Space>Du`)	`o`, `S`, `R`, `A`, `H`, `d`, `r`, `q`, `?`	in the drawer: open/toggle, open in a vertical split, redraw, add a connection, toggle details, delete, rename, close, help; the drawer tree shows connection, New query, Saved queries, Tables
dadbod SQL buffer	`<Space>S`, `<Space>W`, `<Space>E`	run, save, edit bind parameters; `:DB sqlite:/path select ...` runs one query and shows the rows

In an automated test the dadbod result window opened after `<Space>S` but stayed empty, so check that step in a real terminal.

---

40. Configuration management

Keymap / Command	Description
`<Space>ev`	Open `init.lua` in a new tab
`<Space>sv`	Write all buffers and restart Neovim (windows, tabs and files are restored; terminals such as Claude Code are not restarted). Builtin `ZR` restarts without writing
`:Lazy`	Open plugin manager UI

New or changed config needs a restart. Mappings, options and plugin specs in `lua/*.lua` are read at startup: a Neovim that was already running when keys were added or changed (for example by a tool or another editor) does not have them. The config's own comment says re-sourcing `$MYVIMRC` is not supported with lazy.nvim, so do not try `:source`. Restart with `<Space>sv` (see the row above); after `<Space>sv` Neovim has reloaded and the new keys are available. To check that a key exists afterwards, press `<Space>` and read the which-key popup, or run `:verbose nmap <Space>jp` (replace the key; `:verbose map` is default Neovim). | `:Lazy update` | Update all plugins |

---

41. Filetype-specific settings

Filetype	Settings
Python	4-space indent, `<Space>rf` (or `<F9>`) to run, `<Space>fm` (and every `:w`) formats with `ruff format`
Lua	`<Space>rf` (or `<F9>`) to execute (`:luafile %`), `<Space>fm` (and every `:w`) formats with stylua
C++	`<Space>rf` (or `<F9>`) to compile and run (only when a C++ compiler is on PATH, e.g. the c-cpp devShell)
Markdown	Word wrap enabled, syntax highlighting continues up to column 3000 on long lines; `<Space>fm` (and every `:w`) formats with prettier; `<Space><Space>` does not strip trailing spaces
JSON	`<Space>fm` (and every `:w`) formats with prettier (in Visual mode the selection); `:JSONFormat` pretty-prints through Python
Typst	`<Space>tw` TypstWatch, `textwidth=100`, wrap
Vim script	`<Space>rf` (or `<F9>`) sources the file
Line-length marker	The coloured column marker (`colorcolumn`) sits at 100 by default and, per language, exactly at that language's convention: 80 for C, C++, shell, YAML, Vim script, Haskell, R, JavaScript and TypeScript (also jsx/tsx); 88 for Python (ruff format); 100 for Java, Rust, Swift, Nix, Typst; 120 for Lua, PHP, TeX; plain `.txt` files show none. A line touching the marker is over that language's limit. Nothing wraps or reflows.

Filetype syntax plugins (vim-tmux, vim-toml)

These two plugins add syntax highlighting and filetype settings. They have no keys or commands in this configuration and load only when a file of that type is opened.

Plugin	Files	Notes
vim-tmux	tmux configuration files (filetype `tmux`)	Installed only when the `tmux` program is found on the machine. Per its README it also sets the comment string, makes `K` jump to the matching place in `man tmux` and adds `g!` (run lines as tmux commands); those extras were not tried here
vim-toml	`.toml` files (filetype `toml`)	Follows the plugin's `main` branch

---

42. Automatic behaviors

These happen without any keypress:

Behavior	Description
Auto-create directories	Missing parent directories are created on save
Auto-resize windows	Windows resize equally when terminal is resized (the Claude Code panel goes back to 30% width)
Relative line numbers	Relative in the focused window in normal mode; absolute in insert mode and in other windows; none in terminals
Non-UTF-8 warning	Warns if file encoding is not UTF-8
Yank highlight	Yanked text highlighted for 300ms
Cursor restore	Cursor returns to original position after yank
Auto-quit	When only utility windows remain in a tab (quickfix, aerial outline, nvim-tree), the tab closes; on the last tab Neovim quits
Diagnostic float	Diagnostics auto-show when cursor rests on a line
Smart case	Case-insensitive search unless uppercase is used (only `/` and `?`; `:s` and `:g` always ignore case)
Colorscheme	Fixed base16 theme on Nix systems (`NVIM_BASE16_THEME`, fallback Catppuccin Mocha); random on other systems
Auto-save	Files save automatically on focus lost / buffer leave (not unnamed, read-only or special buffers, not Typst/LaTeX); these saves never format
Format on save	Every explicit save (`:w`, `:x`, `ZZ`) formats the file with the formatter of its file type (conform.nvim); `<Space>fo` toggles it for all buffers, `:FormatDisable` / `:FormatDisable!` / `:FormatEnable` also work, `<Space>fm` formats on demand. See Formatting (conform.nvim)
File changed on disk	Checked when Neovim gets focus and when idle. An unmodified buffer is reloaded ("File changed on disk. Buffer reloaded!"); if the buffer was changed too it is kept ("File changed on disk and in the buffer (buffer kept)"); a deleted file keeps its buffer (one warning)
Format check after save	After saving a Python or Lua file, `ruff format --check` / `stylua --check` run in the background; when the save did not format (an auto-save, or format on save switched off) and the file is still unformatted it gives the hint `<file>: file is not formatted (ruff)` (`(stylua)` for Lua); a file the tool cannot check (syntax error) gives `<file>: <tool> could not check the file (syntax error?)` plus the first error line. Nothing is changed
`nvim <directory>`	The directory becomes the working directory and the file tree opens there
Git plugins	fugitive and gitlinker load when the working directory or an opened file is inside a git repository; neogit loads on its first `:Neogit*` command (with diffview and fzf-lua)
Big files	Files over about 1.5 MB (or with very long lines) open in a light mode: no Treesitter, no completion, the language server starts a little later. `:set ft=<language>` gives the full mode back
Help window	On a screen of at least 200 columns `:help` opens as a full-height split on the far left

---

Part II: developer guide

Everything below is aimed at developers. It explains the plugins and tools in this config that make Neovim a full development environment, what they do under the hood, why they matter, and how to use them effectively.

---

66. Shell commands from inside Neovim

Running a shell command

Command	What it does
`:!ls`	Run `ls` and show the output (press Enter to return)
`:!python %`	Run the current file with python (`%` is the current filename)
`:!git diff`	Run git diff without leaving Neovim
`:!mkdir -p src/utils`	Create directories

Inserting command output into the buffer

Command	What it does
`:read !date`	Insert the output of `date` below the cursor
`:read !ls`	Insert directory listing into the buffer
`:read !curl -s <url>`	Insert the contents of a URL
`:%!sort`	Replace the entire buffer with its sorted version
`:%!python -m json.tool`	Format the entire buffer as JSON with 4-space indent (`:JSONFormat` does the same with 2 spaces and leaves invalid JSON untouched)

Examples:

Command	Before	After
`:read !echo hi` (cursor on line 1)	`a` / `b`	`a` / `hi` / `b`
`:%!sort`	`b` / `c` / `a`	`a` / `b` / `c`
`:%!python3 -m json.tool`	`{"a":1,"b":[2,3]}`	7 lines: `{`, `  "a": 1,`, `  "b": [`, `    2,`, `    3`, `  ]`, `}`

Filtering a selection through a command

1. Select lines with `V`
2. Type `:!sort` – the selected lines are replaced with the sorted result
3. Or `:!awk '{print $2}'` – replace with second column only

Examples: the four lines `z` / `c` / `b` / `a` with `Vj` on lines 2 and 3, then `:!sort` (the command line shows `:'<,'>!sort`): the result is `z` / `b` / `c` / `a`, only the selected lines are sorted. The two lines `1 x` / `2 y` selected with `Vj` and `:!awk '{print $2}'` become `x` / `y`.

The asyncrun.vim plugin

Plugin: **asyncrun.vim**. Runs commands asynchronously (non-blocking) and sends output to the quickfix list.

Command	What it does
`:AsyncRun make`	Run make in the background, results go to quickfix
`:AsyncRun python %`	Run current file, output in quickfix

The quickfix window auto-opens (6 lines tall) when AsyncRun starts.

Example: `:AsyncRun echo hi` opens the quickfix window with three lines: `[echo hi]`, `hi` and `[Finished in 0 seconds]`.

---

77. Neovide (graphical Neovim)

Neovide is a graphical program that runs Neovim in its own desktop window, instead of inside a terminal such as kitty and tmux. It is the same Neovim and the same configuration (`~/.config/nvim`); only the window around it changes. Nothing in this guide depends on it: everything works in the terminal.

Starting it

Command	What it does
`neovide`	Open an empty Neovim window
`neovide file.txt`	Open a file in a Neovide window

You start it from a terminal (or your application launcher). A Neovide window has no tmux and no terminal around it, so tmux keys do nothing there; the editor itself, `:terminal` and the Claude panel work as usual.

What this configuration sets for Neovide

The settings live in the Neovide block of `ginit.vim` in the nvim config (they apply only when Neovide runs):

Setting	Value	Effect
`guifont`	JetBrainsMono Nerd Font, size 10	The font of the window (the same font as the kitty terminal; change the name or `:h10` in the Neovide block of `ginit.vim`)
`neovide_transparency`	1.0	Opaque window (no see-through background)
`neovide_cursor_animation_length`	0.1	The cursor glides to its new place in 0.1 seconds
`neovide_cursor_trail_size`	0.3	A short trail behind the moving cursor
`neovide_cursor_vfx_mode`	empty	No particle effects around the cursor

Change a value by editing that block; the `neovide_...` names are the standard Neovide options.

Keys that only exist in a GUI

A terminal swallows some key combinations, so `ginit.vim` adds them for GUIs (`:imap`, `:cmap` and `:nmap` show them with a description ending in "(GUI)"):

Keys	Mode	What it does
`<Shift-Insert>`	Insert, command line	Paste the system clipboard
`<Ctrl-6>`	Normal	Jump to the alternate (previously open) buffer, like `<Ctrl-^>`

`ginit.vim` also has small blocks for other graphical frontends (nvim-qt and fvim); they do nothing in Neovide.

Chapter 12: Plugin catalog

83. Plugin catalog

This chapter lists every plugin of the config in one place, grouped by what it is for. Each entry says in one or two sentences what the plugin is and what it is used for here, and links to the in-depth section of the guide that explains it (keys, commands, examples). Plugins that only load under a condition (a language, an operating system, a tool on PATH) say so.

The catalog is kept in sync by `build-pdf.py --check`: it fails when a plugin of the config has no entry here, when an entry has no working link to a section that really names the plugin, or when an entry names a plugin that is no longer installed. Whoever adds, removes or renames a plugin updates this chapter and the in-depth section in the same change.

Loaded at startup. An entry that begins with this label is loaded in every session without any trigger from you: either while Neovim starts or right after the first screen appears (lazy.nvim's `VeryLazy`), or on a bare `nvim` start for the dashboard. Entries without the label are loaded only when needed (a key, a command, a file type, a git repository, Insert mode and so on), so their startup cost is zero until then. The colorscheme of the current session is also loaded at startup, which theme that is depends on the machine (see Colorschemes).

LSP and code intelligence

Language servers, diagnostics and navigation

- `nvim-lspconfig`: **Loaded at startup.** Ships the ready-made definitions of language servers (command, filetypes, root markers). This config enables the servers itself in lua/config/lsp.lua; the plugin only provides the defaults. Keys such as `K`, `gd`, `<Space>rn`, `<Space>ca` are this config's own. In depth: How LSP Is Managed
- `lazydev.nvim`: Teaches the Lua language server the Neovim API, so editing this config gives completion, hover and no false "undefined global vim" warnings. Loads only in Lua files. In depth: Editing the Neovim config in Lua (lazydev.nvim)
- `conform.nvim`: The one formatter front end for every file type, also files without a language server: runs stylua, prettier, ruff format, shfmt and others on every explicit save (`:w`), and on demand with `<Space>fm`; never on auto-saves. `<Space>fo` toggles format on save. In depth: Formatting (conform.nvim)
- `glance.nvim`: **Loaded at startup.** Shows definitions, references and implementations in a popup so you can peek without leaving your file. Keys: `<Space>gd`, `<Space>gr`, `<Space>gi`. In depth: Peeking Without Jumping (Glance)
- `nvim-lightbulb`: Shows a lightbulb in the sign column when the LSP has a code action for the current line; filters out ruff's two always-on actions. Press `<Space>ca` to use it. In depth: The Lightbulb
- `trouble.nvim`: A tidy list window for diagnostics grouped by file. In this config it is opened by `<Space>dw` (`:Trouble diagnostics toggle`) to look at errors and warnings across the project. In depth: Trouble (Better Quickfix UI)

- `fidget.nvim`: **Loaded at startup.** Shows small progress messages in the bottom-right corner while a language server is working (indexing, starting up). No keys; defaults only. In depth: LSP progress messages (fidget.nvim)
- `aerial.nvim`: A sidebar outline of the file's functions, classes and methods, built from Treesitter or the LSP. Toggle it with `<Space>t`; `[t` / `]t` jump between symbols. In depth:
  37. Symbol outline (`aerial.nvim`)
- `dropbar.nvim`: **Loaded at startup.** A bar at the top of the window (winbar) showing where you are as a path, like file > class > function. Used here only with its defaults; no keys are configured. In depth: Breadcrumb bar (dropbar.nvim)
- `devdocs.nvim`: Reads DevDocs documentation (language and library references) inside Neovim, in a read-only split. Needs `jq`, `curl` and `pandoc`. Loads only on `:DevDocs`; no keys are mapped. In depth: devdocs.nvim (plugin)
- `nvim-treesitter`: **Loaded at startup.** Parses code into a syntax tree for accurate highlighting. Started per buffer when a parser exists; also feeds aerial, treesj and the smart comment code. Parsers come from the Nix store on Nix systems. In depth:
  46. Treesitter In Depth

Java

- `nvim-java`: Wires the Java tooling together: starts the jdtls language server, test and debug extensions and adds the `:Java*` commands. Used here for building, running, testing, debugging and refactoring Java projects with the `<Space>j` key family. Only for Java files (it loads when you open one). In depth:
  78. Java (nvim-java, jdtls, tests, debugging)
- `spring-boot.nvim`: A second language server for Spring Boot projects that attaches next to jdtls. Brought in automatically by nvim-java; there is nothing to configure or press for it. In depth: spring-boot (Spring Boot tools)
- `nvim-dap`: The general debugger client (breakpoints, stepping) implementing the Debug Adapter Protocol. Here it is installed only for Java debugging through nvim-java; there is no panel; four keys (`<Space>jp` breakpoint, `<Space>jP` clear all breakpoints, `<Space>jh` value under the cursor, `<Space>jx` stop) and the typed `:Dap*` commands (`:DapContinue`, `:DapStepOver`, ...) do the rest. In depth: Debug Adapter Protocol (DAP)

Debugging and running code

- `nvim-gdb`: Visual front end for GDB, LLDB and pdb: starts the debugger in a terminal pane and marks the current line in your code. Used mainly to step through Python with pdb (`<Space>dp`) and for C/C++ with `:GdbStart` in the c-cpp devShell. In depth: GDB Integration and Debugging with pdb
- `nvjup`: **Loaded at startup.** Edits and runs Jupyter notebooks: opening an `.ipynb` file shows its cells, outputs and a kernel session (run with `<Space>nr`, `<C-CR>`). Needs `uv` for a one-time build (`:Lazy build nvjup`) and `ipykernel` (jupyter devShell). In depth: Jupyter notebooks (nvjup)
- `asyncrun.vim`: Runs a shell command as a background job and streams its output into the quickfix window (opened 6 lines tall). Here `<Space>rf` / `<F9>` for Python goes through it. In depth: The AsyncRun Plugin

Completion and snippets

Completion

- `nvim-cmp`: **Loaded at startup.** The completion engine: shows a popup menu with suggestions from the language server, snippets, paths and buffer words while you type, and in the `:` and `/` command lines. Keys are tuned so `<Tab>` picks and `<CR>` only confirms a picked item. In depth:
  14. Autocompletion (`nvim-cmp`)
- `cmp-nvim-lsp`: **Loaded at startup.** Completion source that feeds the language server's suggestions (functions, variables, types) into the nvim-cmp menu. In depth: Completion sources and helpers
- `cmp-path`: **Loaded at startup.** Completion source for file and folder paths, in insert mode and in the `:` command line. In depth: Completion sources and helpers
- `cmp-buffer`: **Loaded at startup.** Completion source that offers words already present in the current buffer; also completes words in the `/` search line. In depth: Completion sources and helpers
- `cmp-omni`: **Loaded at startup.** Completion source that passes through a filetype's omni-completion function; used here only in LaTeX files (BibTeX and citation completion). In depth: Completion sources and helpers
- `cmp-cmdline`: **Loaded at startup.** Completion source for the command line: offers Ex command names and arguments (and paths) after `:`, and buffer words in the `/` search. In depth: Completion sources and helpers
- `cmp-nvim-ultisnips`: **Loaded at startup.** Bridge that lists UltiSnips snippet triggers in the completion menu, so a snippet can be picked from the menu like any other item. In depth: Completion sources and helpers
- `colorful-menu.nvim`: **Loaded at startup.** Colours the labels in the completion menu like code (name, arguments, type) instead of plain text; works automatically, no keys. In depth: Completion sources and helpers

Snippets and pairs

- `ultisnips`: **Loaded at startup.** The snippet engine: type a trigger, expand it into a template and jump through its placeholders. Personal snippets live in `my_snippets/`. In depth:
  15. Snippets (`UltiSnips`)
- `vim-snippets`: **Loaded at startup.** A ready-made collection of snippets for many languages that UltiSnips loads alongside the personal ones; no config. In depth: Ready-made snippets (`vim-snippets`)
- `nvim-autopairs`: Inserts the closing bracket or quote automatically when you type the opening one. Default behaviour, no keys of its own. In depth: Auto-pairing (nvim-autopairs plugin)

Editing and motions

Text objects and surroundings

- `vim-sandwich`: **Loaded at startup.** Adds, deletes and replaces surrounding characters (quotes, brackets) with `sa`, `sd`, `sr`. Here `s` alone is disabled (use `cl`), and its "query" text objects live on `iS` / `aS`. In depth: Adding surrounding pairs (vim-sandwich plugin)

- `targets.vim`: **Loaded at startup.** Extra text objects: any-bracket `ib`/`ab`, arguments `ia`/`aa`, quotes `iq`/`aq`, separators `i,`/`a,`, and it looks forward on the line when the cursor is not inside the pair. In depth: targets.vim (plugin)
- `vim-matchup`: Better `%` that jumps between `if`/`else`/`end` style keywords as well as brackets, highlights the pair and shows an off-screen partner in a popup. Also gives `i%`/`a%`, `g%`, `[%`, `]%`, `z%`. In depth: vim-matchup (plugin)
- `vim-swap`: **Loaded at startup.** Swaps items of a comma-separated list or argument list interactively with `gs`. Only that one key is kept (plugin defaults off, so `g<` and `g>` stay builtin). In depth:
  65. Swapping function arguments (`vim-swap`)
- `treesj`: Splits a one-line list, argument list, table or block into one item per line, and joins it back, using the Treesitter syntax tree. One key: `gS`. In depth: Split and join code (treesj)

Moving around and searching in the buffer

- `hop.nvim`: `f` + two characters labels every match on screen; press the label to jump. It replaces the builtin `f`. Hint colours follow the active colorscheme's Search highlight. In depth:
  22. Jump navigation (`hop.nvim`)
- `nvim-hlslens`: Shows a `[x/y]` match counter next to the current search match. `n`, `N`, `*`, `#` are remapped in config/hlslens.lua; `*` and `#` search the whole word literally and keep the cursor in place (with a count they jump). In depth:
  23. Search lens (`nvim-hlslens`)
- `vim-illuminate`: **Loaded at startup.** Highlights other uses of the word under the cursor (LSP, else Treesitter; plain text matching in nix files) when there are at least 2. `<Alt-n>` / `<Alt-p>` jump between them, `<Alt-i>` selects one. In depth: Word references (vim-illuminate)

Editing helpers

- `vim-commentary`: **Loaded at startup.** Toggles comments with `gc` + motion, `gcc`, and `:[range]Commentary`. A custom smart-comment module sits next to it. In depth: vim-commentary (plugin)
- `tabular`: `:Tabularize /char` aligns text around a character (equals signs, colons, table pipes). Loads on first `:Tabularize` or with the first Markdown file. In depth: Align Text
- `vim-repeat`: **Loaded at startup.** Makes the `.` key repeat plugin actions (surround, comment, swap and so on) and not only builtin edits. It has no keys or commands of its own. In depth:
  61. The dot command (`.`) – repeating actions
- `better-escape.vim`: Typing `jk` quickly in Insert mode returns to Normal mode, with a 200 ms window between the two keys. In depth: Leaving insert mode
- `yanky.nvim`: **Loaded at startup.** Keeps a history of yanks, highlights pasted text for 300 ms, and lets `[y` / `]y` swap a fresh paste for an earlier or later yank. `p`/`P` are yanky's paste in Normal and Visual mode. In depth:
  24. Yank history (`yanky.nvim`)
- `nvim-ufo`: **Loaded at startup.** Code folding that uses the language server's folding ranges (indentation as fallback) and shows a preview of folded lines. Keys are the standard `z` fold keys plus `<Space>K` to preview a fold. In depth:
  18. Code folding (`nvim-ufo`)

- `promise-async`: **Loaded at startup.** A small helper library that nvim-ufo (code folding) needs for asynchronous work. Nothing to use directly. In depth: 47. Code Folding In Depth
- `unicode.vim`: Unicode info (`ga`), completion of characters by name, digraph helpers. In depth: 38. URL & Unicode
- `gx.nvim`: `gx` opens the URL or file under the cursor (also on a Visual selection) in the browser or default program. In depth: Open the file under cursor
- `whitespace.nvim`: **Loaded at startup.** Highlights trailing whitespace and provides `:StripTrailingWhitespace`, which `<Space><Space>` calls. Not active in Markdown, where two trailing spaces are a line break. In depth: Miscellaneous editing

Search and files

- `fzf-lua`: **Loaded at startup.** Popup fuzzy finder on top of the fzf program. Used for file search, live grep, help tags, buffer tags, buffers, recent files, git branches, keymaps (dashboard `m`) and the zoxide picker. In depth: 12. Fuzzy finding & project-wide search (`fzf-lua`)
- `telescope.nvim`: Second picker framework, loaded only on `:Telescope`. In this config it is used for the `<Space>db` buffer diagnostics list and for `:Telescope keymaps`; telescope-symbols also uses it. In depth: Telescope (second picker)
- `telescope-symbols.nvim`: Data package for telescope: adds symbol/emoji sources so `:Telescope symbols` can insert them. No key of its own in this config. In depth: Telescope (second picker)
- `nvim-tree.lua`: Sidebar file tree toggled with `<Space>s`; `<Tab>` opens a file but keeps the cursor in the tree. Has its own window picker when several windows are open. In depth: 11. File explorer (`nvim-tree`)
- `nvim-bqf`: Better quickfix window: item preview, marking items, fuzzy filter. The preview does not start by itself here. Only inside the quickfix window. In depth: Inside the quickfix window (nvim-bqf, quicker.nvim)
- `quicker.nvim`: Nicer quickfix and location list display (grouped by file). Editing the list as a buffer is switched off. In depth: Inside the quickfix window (nvim-bqf, quicker.nvim)
- `nvim-lsp-file-operations`: Tells language servers about renames, moves, creates and deletes done in the nvim-tree file tree, so jdtls rewrites the `package` and `import` lines when a Java file or package folder is renamed there. No keys, only the command `:LFO`; it loads after nvim-tree. In depth: File operations and language servers (nvim-lsp-file-operations)
- `vim-eunuch`: Unix file commands inside Neovim: rename, move, delete, mkdir, chmod, sudo write. They do not notify language servers. In depth: File operations

Sessions and saving

- `persistence.nvim`: Saves one session per folder (and git branch) when you quit. Never restored automatically; restore it from the dashboard items. In depth: Session Management
- `vim-obsession`: **Loaded at startup.** Manual session recording with `:Obsession` into `Session.vim`; keeps updating a session started with `nvim -S Session.vim`. In depth: Session Management

- `auto-save.nvim`: **Loaded at startup.** Saves a changed file when you leave the buffer or Neovim loses focus; shows a small popup "AutoSave: saved at HH:MM:SS"; these saves never format. In depth: Auto-Save

Git

Everyday Git in the buffer

- `gitsigns.nvim`: **Loaded at startup.** Shows `+ ~ _ ‾ |` signs in the gutter for added, changed and deleted lines and gives hunk keys (jump, preview, stage, reset, blame line). In this config it is the everyday tool for staging or discarding one block of a file. In depth: gitsigns.nvim (plugin)
- `vim-fugitive`: Runs git from inside Neovim (`:Git`, `:Gwrite`, `:Gvdiffsplit`). Here it carries most of the `<Space>g...` keys: status, add, unstage, commit, amend, stash, pull, branch, fetch; the cmdline abbreviation `git` becomes `Git`. Only inside a git repository (nvim started in one, or a file of one opened); outside one its keys do not exist. In depth: vim-fugitive (plugin)
- `gitlinker.nvim`: Builds a web URL (permalink) to the current line or visual selection on the host (GitHub, Azure DevOps via a custom callback) and can open the repository in the browser. Only inside a git repository (nvim started in one, or a file of one opened); outside one its keys do not exist. In depth: gitlinker.nvim (plugin)

Git interfaces and diffs

- `neogit`: A Magit-style full-screen git interface (status, stage, commit, log). Here it is opened with `<Space>gn` or `:Neogit` and uses diffview and fzf-lua as optional integrations. The key exists only inside a git repository; `:Neogit` works anywhere. In depth: Neogit (plugin)
- `diffview.nvim`: Side-by-side diff viewer for the working tree, branches and commits, plus a file-history panel and a 3-way merge tool. Here `<Space>gD` opens it and the conflict keys `<Space>gCo/gCt/gCb/gCa`, `]C`, `[C` work inside it. The key `<Space>gD` exists only inside a git repository; `:DiffviewOpen` works anywhere. In depth: Resolving merge conflicts (diffview.nvim)
- `vim-flog`: Draws the git commit graph (branches and merges) in a buffer. Here it is opened with `:Flog`; there is no key for it. Needs vim-fugitive, so open it inside a git repository. In depth: vim-flog (plugin)
- `diffs.nvim`: **Loaded at startup.** Adds syntax/treesitter highlighting inside the diff buffers of fugitive, neogit and gitsigns, highlights conflict markers, and provides `:Diff`. In depth: diffs.nvim (plugin)
- `codediff.nvim`: VSCode-style side-by-side diff opened with `:CodeDiff`. No key is mapped. In depth: codediff.nvim (plugin)

Databases

- `vim-dadbod`: Engine behind `:DB`: runs a query against a database URL (`:DB sqlite:file select ...`, `:%DB url` for the whole buffer). It has no screen of its own; vim-dadbod-ui adds one. In depth: SQL databases (nvim-dbee, vim-dadbod-ui)
- `vim-dadbod-ui`: Browsable connection tree, saved queries and result pane for vim-dadbod. Keys `<Space>Du` (toggle), `<Space>Da` (add connection), `<Space>Df` (find buffer). In depth: SQL databases (nvim-dbee, vim-dadbod-ui)

- `nvim-dbee`: SQL client with drawer, editor, result and call-log windows and a Go backend. Opened with `<Space>Do` / `<Space>Dt`, closed with `<Space>Dc`. In depth: SQL databases (nvim-dbee, vim-dadbod-ui)

User interface

Bars, messages and start screen

- `which-key.nvim`: **Loaded at startup.** Shows a popup of the keys that can follow what you have typed (press `<Space>` and wait). The only groups it names are the Java ones (`<Space>j...`). In depth: Which-Key: See Available Keybindings
- `lualine.nvim`: **Loaded at startup.** The statusline at the bottom: file name, git branch and diff, diagnostics, active LSP, trailing-whitespace and mixed-indent warnings, progress. Clicking the branch or the LSP name opens a picker or popup. In depth:
  32. Statusline (`lualine.nvim`)
- `bufferline.nvim`: **Loaded at startup.** The top line showing one tab per open buffer (not Vim tabpages). Click to switch, click the x or the modified dot to close; qf, fugitive and git buffers are hidden from it. In depth: Buffer Tabs (the Top Line)
- `dashboard-nvim`: **Loaded at startup.** The start screen shown for a bare `nvim`, with a menu (sessions, files, keymap search, user guide, Claude help) and random ASCII art. `\h` reopens it, `\H` closes it again. The menu also has an Open WhatsApp item (`\w`). In depth: Dashboard (Start Screen)
- `statuscol.nvim`: **Loaded at startup.** Builds the column left of the text: signs (git, diagnostics), line numbers and the fold column (fold levels deeper than 3 are not shown). Clicks on the three parts have their own handlers. In depth: Line number column (`statuscol.nvim`)
- `tiny-cmdline.nvim`: **Loaded at startup.** Shows the `:` command line as a centred floating window (search stays at the bottom); needs the hidden command line, so pending keys and macro recording show in the statusline. Colours follow the active colorscheme. In depth: Centred command line (tiny-cmdline.nvim)
- `nvim-notify`: **Loaded at startup.** Animated popup notifications (fade in, slide out, 1.5 s) that replace the plain message line for everything that calls vim.notify. The popup background follows the active colorscheme; `:Notifications` shows the history. In depth:
  33. UI features
- `snacks.nvim`: **Loaded at startup.** A toolbox of which three parts are used: nicer `vim.ui.input` popup at the cursor, the picker (used for `vim.ui.select` and some lists), and the light mode for very big files (over 1.5 MB or lines averaging over 5000 characters). In depth: Input popups and big files (`snacks.nvim`)

Visual helpers

- `vimade`: **Loaded at startup.** Dims the windows that do not have focus, to half strength without animation, so the active window stands out. In depth:
  33. UI features
- `nvim-colorizer.lua`: **Loaded at startup.** Paints color codes (hex, rgb and similar) in the text with the color they stand for. Plain color words are deliberately left alone. In depth:
  33. UI features

- `mini.indentscope`: **Loaded at startup.** Draws a thin vertical line for the indent block the cursor is in, and adds the `ii` / `ai` text objects plus `[i` / `]i` to jump to the top or bottom of the scope. In depth: More text objects
- `vim-highlighturl`: Highlights URLs in any buffer so they stand out. In depth: 38. URL & Unicode

Icons and UI libraries

- `mini.icons`: **Loaded at startup.** The icon provider. It also pretends to be nvim-web-devicons for plugins that only know that one, and gives the completion menu its kind icons. In depth: Icons and UI libraries
- `nvim-web-devicons`: Library only: file icons for plugins that ask for it (trouble.nvim, nvim-tree). mini.icons stands in for it. In depth: Icons and UI libraries
- `nui.nvim`: Library only: popup, menu and layout building blocks used by nvim-java, nvim-dbee and ascii.nvim. Nothing to use directly. In depth: Icons and UI libraries
- `ascii.nvim`: A library of ASCII-art pictures; the dashboard picks a random one that fits the window height. In depth: Icons and UI libraries

Colorschemes

- `catppuccin`: The Catppuccin theme family (mocha, macchiato, frappe, latte), the palette this repo uses everywhere. Installed but not in the random list and not configured; select it with `:colorscheme catppuccin-mocha`. On Nix the Mocha look comes from nvim-base16 instead. In depth: Colorschemes
- `nvim-base16`: Collection of base16 themes. This is what makes the Nix startup theme: `base16-<NVIM_BASE16_THEME>`, fallback `base16-catppuccin-mocha`. Not in the random list. In depth: Colorschemes
- `lush.nvim`: Library for writing themes in Lua; only installed because arctic needs it. No colorscheme, no commands used here. In depth: Colorschemes
- `arctic`: Dark VS Code-like theme; one of the random startup themes on non-Nix systems. Provides `arctic`. In depth: Colorschemes
- `ashen.nvim`: Dark, muted ash-toned theme; random-list member. Provides `ashen`. In depth: Colorschemes
- `citruszest.nvim`: Vivid dark theme with citrus accents; random-list member. Provides `citruszest`. In depth: Colorschemes
- `edge`: Clean, modern dark theme; random-list member (default style, italics on). Provides `edge`. In depth: Colorschemes
- `everforest`: Green, low-contrast forest theme; random-list member (hard background, italics on). Provides `everforest`. In depth: Colorschemes
- `github-theme`: GitHub's light and dark palettes; random list loads `github_dark_default`. Also dimmed, high-contrast, colorblind and light variants. In depth: Colorschemes
- `gruvbox-material`: Softer gruvbox variant; random-list member (hard background, original foreground, italics on). Provides `gruvbox-material`. In depth: Colorschemes
- `jellybeans.nvim`: Port of the jellybeans theme; random-list member, with several variants (hc, mono, muted, warm, light). In depth: Colorschemes
- `kanagawa.nvim`: Theme inspired by Hokusai's wave painting; random list loads `kanagawa-dragon`; also wave and lotus. In depth: Colorschemes
- `kanso.nvim`: Calm, kanagawa-inspired theme; random list loads `kanso`; also ink, mist, pearl, zen. In depth: Colorschemes

- `makurai-nvim`: Dark theme with autumn and light variants; random list loads `makurai_dark`. In depth: Colorschemes
- `material.nvim`: Material Design theme; random list loads `material` with style "darker"; also oceanic, palenight, deep-ocean, lighter. In depth: Colorschemes
- `melange-nvim`: Warm, earthy theme; random-list member. Provides `melange`. In depth: Colorschemes
- `modus-themes.nvim`: Accessible high-contrast Modus themes; random list loads `modus`; also `modus_operandi` (light) and `modus_vivendi` (dark). In depth: Colorschemes
- `nightfox.nvim`: Fox theme family; random list loads `carbonfox`; also nightfox, dayfox, dawnfox, duskfox, nordfox, terafox. In depth: Colorschemes
- `onedark.nvim`: Atom One Dark port; random list loads it via its Lua setup with style "darker". In depth: Colorschemes
- `onedarkpro.nvim`: Another One Dark port; random list loads `onedark_dark` (the config comment says `onedark_vivid` lacks contrast). In depth: Colorschemes
- `sonokai`: Monokai-pro-inspired theme; random-list member (italics on). Provides `sonokai`. In depth: Colorschemes
- `vague.nvim`: Minimal, low-saturation dark theme; random-list member. Provides `vague`. In depth: Colorschemes

Writing: Markdown, LaTeX, Typst

- `mdkite.nvim`: Live browser preview of a Markdown file with Mermaid diagrams, KaTeX math and scroll sync; `<Alt-m>` toggles it in Markdown buffers. A pure-Lua server, no npm build; the preview page needs internet. Only in Markdown files. In depth: Preview (mdkite.nvim)
- `kitehost.nvim`: The small HTTP server library that mdkite.nvim uses to serve the preview. Loaded together with mdkite.nvim, nothing to use directly. In depth: Preview (mdkite.nvim)
- `truth-table.nvim`: **Loaded at startup.** Builds truth tables, Karnaugh maps and step-by-step Boolean-algebra rewrites inside Markdown notes (`:TruthTable`, `:TruthTableKarnaugh`, `:TruthTableTutor`; keys `<Space>T...` and `<Space>l...`); `and@` types `∧` in note buffers. In depth: Truth tables and Boolean algebra (truth-table.nvim)
- `cds-lua-fp`: **Loaded at startup.** Small Lua helper library that truth-table.nvim needs; nothing to use directly. In depth: Truth tables and Boolean algebra (truth-table.nvim)
- `render-markdown.nvim`: Draws headings, lists, code blocks, tables and checkboxes nicely inside the buffer; pauses in Insert mode. Only in Markdown files. In depth: Rendering inside the buffer (render-markdown.nvim)
- `vim-markdownfootnotes`: Fast footnote insertion and jump back in Markdown files. Only in Markdown files. In depth: Footnotes
- `vimtex`: **Loaded at startup.** Full LaTeX support (compile with `latexmk`, forward and inverse search with the PDF viewer, table of contents, motions and text objects). Only loaded when `latex` is on PATH, in practice inside the LaTeX devShell. In depth: LaTeX (vimtex, texlab, ltex, PDF viewer)
- `typst.vim`: Typst syntax colours, indentation, `:TypstWatch`, `:make` and the `:Toc` outline commands for `.typ` files. Only loaded when `typst` is on PATH. In depth: Typst (typst.vim, tinymist, watch and preview)

AI and collaboration

- `claude-code.nvim`: **Loaded at startup.** Runs the Claude Code CLI in a side panel inside Neovim; toggle with `<Space>cc`, resume or continue older conversations. In depth: Claude Code
- `instant.nvim`: Real-time collaborative editing: start a server and a session, others join. In depth: Collaborative Editing

Plugin manager, libraries and other tools

Plugin manager and libraries

- `lazy.nvim`: **Loaded at startup.** The plugin manager that installs, loads (lazily) and updates every other plugin. `:Lazy` opens its window; `pi`, `pud`, `pc`, `ps` are typed shortcuts. In depth: Plugin Manager Shortcuts
- `plenary.nvim`: **Loaded at startup.** Lua helper library (async, paths, jobs) required by other plugins. Nothing to use directly. In depth: Libraries and dependencies

Terminal, browser and file types

- `vim-oscyank`: Copies text to the system clipboard through the terminal (OSC 52 escape sequence), so it works over SSH. No key is mapped in this config; it is used through commands. In depth: Copying over SSH (vim-oscyank)
- `firenvim`: **Loaded at startup.** Lets Neovim edit browser text areas (needs the Firenvim browser extension). In this config takeover is manual only. In depth: Neovim in the browser (firenvim)
- `live-command.nvim`: **Loaded at startup.** Live preview of `:norm` while you type it; typing `:norm` is turned into `:Norm`. In depth: Live preview of :norm (live-command.nvim)
- `vim-scriptease`: Helpers for debugging Vim script and the config: `:Messages`, `:Scriptnames`, `:Verbose`. In depth: Vim-script debugging (vim-scriptease)
- `vim-tmux`: Syntax highlighting, commenting and `K` lookup for `tmux.conf` files; loads only for the `tmux` filetype and only when tmux is installed. In depth: Filetype syntax plugins (vim-tmux, vim-toml)
- `vim-toml`: Syntax highlighting for `.toml` files (for example `pyproject.toml`); follows the plugin's `main` branch. Only in `.toml` files. In depth: Filetype syntax plugins (vim-tmux, vim-toml)
- `vim-xkbswitch`: Switches the keyboard layout automatically when you enter and leave Insert mode. macOS only (needs the `xkbswitch` command); declared but disabled elsewhere, so it does nothing on Linux. In depth: Keyboard layout switching (vim-xkbswitch)
- `whatsapp.nvim`: WhatsApp chats inside Neovim through an unofficial client with a Go bridge; used through `:WhatsApp`, `\w` (open), `\W` (close) and a dashboard item. The bridge is built only when make, go and a C compiler are on PATH. In depth: WhatsApp in Neovim (whatsapp.nvim)
- `vim-be-good`: A small game collection to practice basic Vim movements, started with `:VimBeGood` in an empty buffer. In depth: Practice game (vim-be-good)

Chapter 13: Java

78. Java (nvim-java, jdtls, tests, debugging)

This section is one complete walk through everything that is specific to Java in this config: the tools, why each one is there, how it works, how to use it, and what to do when it does not work. It covers only Java-only things (nvim-java and its parts, the language servers, the test runner, the debugger, the Java keys, commands and snippets). Global tools with global keys (`gd`, `K`, `<Space>rn`, git, fzf, …) get only a pointer to their own section. Sections 35 and 54 hold the short key lists and section 56 the debugging basics; this section puts everything in one order.

Unless marked otherwise, everything below was tried with real keys in a Maven test project (JDK 25 devShell, `Main`, `Calc`, `CalcTest`).

1. What you get and why

Part	Why it is in the setup	What it does	Where it comes from
nvim-java	One plugin that wires all the Java pieces together	Starts jdtls, finds the debugger and test extensions, adds the `:Java*` commands	lazy plugin `nvim-java/nvim-java`, loaded on `ft = "java"`
jdtls	Java has no useful editing without a real compiler front end	Completion, diagnostics (compile errors while you type), go to definition, rename, code actions, formatting, refactoring	nvim-java's own jdtls 1.54.0, in `~/.local/share/nvim/nvim-java/packages/jdtls`
spring-boot	Understands Spring annotations and `application.properties`	A second language server that attaches next to jdtls	nvim-java, only started when `java` is on PATH
java-debug (DAP adapter)	Without it the debugger cannot talk to the JVM	Lets nvim-dap debug Java	VS Code extension `vscjava.vscode-java-debug` linked by the devShell
java-test	Runs JUnit tests and reports per-test results	`<Space>jt...` keys	VS Code extension `vscjava.vscode-java-test` linked by the devShell
nvim-dap	The general debugger client (breakpoints, stepping)	`:Dap*` commands	plugin `mfussenegger/nvim-dap`, a dependency of nvim-java
Lombok	Many Java projects use it; jdtls must load it as an agent	Annotations like `@Getter` compile	`lombok` package of nvim-java, and `JAVA_TOOL_OPTIONS` of the devShell
typos_lsp	Spell checker for code (global tool, not Java-only)	Underlines misspelled words in identifiers and comments, also in Java	`typos-lsp` from the Nix system
Snippets	Typing the same loops and switches over and over	See part 9 below	`my_snippets/java.snippets`

Tools you get only through the devShell: the JDK (25), `mvn`, `gradle`, `lombok`, `jdtls` wrapper, and the two VS Code extensions. The Neovim config itself never installs them.

How nvim-java is loaded

The real spec (`lua/plugin_specs.lua`):

```
{
  "nvim-java/nvim-java",
  ft = "java",
  dependencies = { "MunifTanjim/nui.nvim", "mfussenegger/nvim-dap" },
  config = function()
    local is_nix_managed = vim.uv.fs_stat("/etc/nixos") or vim.uv.fs_stat("/etc/nix")
    local has_java = vim.fn.executable("java") == 1
    require("java").setup({
      jdk = { auto_install = not is_nix_managed },
      java_test = { enable = true },
      java_debug_adapter = { enable = true },
      spring_boot_tools = { enable = has_java },
    })
    vim.lsp.config("jdtls", { settings = { java = { home = vim.env.JAVA_HOME } } })
    if has_java then vim.lsp.enable("jdtls") end
  end,
}
```

In plain words:

- `ft = "java"`: nvim-java loads when you open the first Java file of the session, not at startup. Non-Java sessions pay nothing. The very first Java file of a session opens a little slower.
- On a Nix system `jdk.auto_install` is false: nvim-java never downloads a JDK. The JDK comes from the devShell (`JAVA_HOME`).
- jdtls and spring-boot start only if `java` is on PATH. That is how the config stays silent outside the devShell.
- `java.home = $JAVA_HOME` tells jdtls which JDK to compile with.
- jdtls itself is NOT taken from the devShell (the devShell `jdtls` is only a wrapper script); nvim-java uses its own jdtls 1.54.0.

spring-boot (Spring Boot tools)

Plugin: **spring-boot.nvim**. It has no spec of its own in the plugin file: nvim-java declares it as a dependency, and nvim-java's setup uses it to start the Spring Boot language server. It has no keys and no commands in this config.

Item	What it is
What it starts	A second language server (the Spring Boot tools of VS Code, installed by nvim-java as its `spring-boot-tools` package, default version 1.55.1) that attaches next to jdtls. nvim-java passes the server path to the plugin's setup and registers the client commands the server needs
When it starts	Only when `java` is on PATH: the config sets `spring_boot_tools = { enable = has_java }`, so in practice only inside the Java devShell
What it offers	Per the plugin's README (not tried here): finding Spring beans and web endpoints through workspace symbols, completion and navigation in `application.properties` and `application.yml`, snippet completion and code actions
Where you see it	The statusline shows `jdtls (+2)` and the attached clients are `jdtls,spring-boot,typos_lsp` (see " 2. Quick Start ")
Caveat	It attaches in every Java buffer, even in a project without Spring; that is harmless (see section 5)

The Java devShell (`use_dev_env java`)
A Java project has a `.envrc` with one line:

```
use_dev_env java
```

direnv loads the devShell when you `cd` into the folder. The devShell (template `dev-environments/language-specific/java/flake.nix`) gives you `JAVA_HOME` (JDK 25), `mvn`, `gradle`, Lombok (`JAVA_TOOL_OPTIONS` has a `-javaagent:` for it) and, in its shell hook, links the extensions for Neovim:

```
~/.local/share/nvim/nvim-java/packages/java-debug-adapter/extension -> vscode-java-debug
~/.local/share/nvim/nvim-java/packages/java-test/extension          -> vscode-java-test
./.direnv/tools/jdtls                                               -> jdt-language-server
```

`.direnv/` is ignored by git, so nothing shows up in `git status`. First time in a new folder: `direnv allow`.

Entering the devShell from a running Neovim (`:DevEnv java`). If Neovim is already open without the devShell (for example a `.java` file opened from outside the project), run `:DevEnv java` instead of quitting:

1. `:DevEnv java` then `<CR>`. The first call takes about 10 seconds and shows `DevEnv: evaluating the java devShell (about 10 s) ...`; later calls are instant (cached per flake).
2. A notification `DevEnv: java ready: ...` lists what started (`java`, `nvim-java (jdtls, spring-boot-tools)`, language servers). jdtls attaches to the `.java` buffers that are already open.

It applies only `PATH`, `JAVA_HOME`, `CLASSPATH`, `JAVA_TOOL_OPTIONS` (Lombok) and a few other allowlisted variables, then starts nvim-java. The devShell's shell hook does not

run, so the extension links above are not created: jdtls works anyway (nvim-java uses its own copies), but you get a warning when the java-debug or java-test extension is missing from `~/.local/share/nvim/nvim-java/packages`; debugging and tests may then be unavailable until nvim-java installs them or direnv enters the devShell once. Opening a `.java` file without `java` also shows a one-time hint, `java not found on PATH: run :DevEnv java to enter its devShell`. All devShell names and the full description: section 43.

The other option is unchanged: quit, `cd` into the project and start `nvim` from there with direnv.

Outside the devShell

`java` is not on PATH, so jdtls and spring-boot do not start. `.java` files still open with syntax highlighting, snippets and typos_lsp, but without Java language features. Every `<Space>j...` key shows ONE warning, `Java: jdtls not attached (open nvim inside the Java devShell, or run :DevEnv java)`. These global keys are fallbacks: without them `<Space>jrr` would fall through to plain Vim keys (`<Space>`, `j`, `rr`). The universal run key `<Space>rr` (section 19) then runs plain `java <file>` if `java` exists, otherwise gives one warning.

2. Quick start

1. In a terminal: `cd` into the project folder (the one with `pom.xml`). direnv loads the devShell and prints `Java Environment Active (JDK 25)`. First time: `direnv allow`.
2. `nvim src/main/java/demo/Main.java`
3. Wait for `jdtls (+2)` in the statusline (jdtls, spring-boot, typos_lsp). All three attach: `jdtls,spring-boot,typos_lsp`. Start times of a fresh project are about 20 to 50 seconds (a cold first start); in an everyday session jdtls starts fast, and the slow part is a project-wide rename (`<Space>rn`, see When a rename does nothing). A big project's first start is slower. Before that, keys such as `<Space>jtm` or `:DapContinue` show a warning or do nothing.
4. Run: `<Space>jrr`. A full-width, 15-line terminal split opens at the bottom with the program output. If the project has more than one class with a `main` method, a picker opens first (see The runner window).
5. Test: open `CalcTest.java` and press `<Space>jtc`, then `<Space>jtr` for the result tree.
6. Debug: `<Space>jp` on a line with code, then `<Space>jtC`. The window stops at the line with a `→` sign. Step by step: section 7.

3. Project layout rules

The root folder

jdtls needs one root folder per project. nvim-java decides it with these markers (`lua/java-core/ls/servers/jdtls/root.lua`), two groups, the first group that matches wins:

Group	Markers	Meaning
1	`mvnw`, `gradlew`, `settings.gradle`, `settings.gradle.kts`, `.git`	multi-module projects; the git root is the last resort on purpose
2	`build.xml`, `pom.xml`, `build.gradle`, `build.gradle.kts`	single-module projects

Consequences:

- The git root wins over a `pom.xml` below it, by design: in a multi-module Maven project you cannot tell the parent from a submodule, and jdtls does not break when the root is higher than the real parent `pom.xml`.
- A Maven project nested in a bigger git repository (`repo/java/pom.xml`) still works: the root is the git root and jdtls finds the `pom.xml` below it. `<Space>jrr` works in this layout (in the user's real layout).
- No git folder: the folder with `pom.xml` is the root. A loose `.java` file with neither: jdtls has no project (create a `pom.xml` or run `git init`).
- Maven and Gradle projects are both imported by jdtls.
- Expected layout: code in `src/main/java/...`, tests in `src/test/java/...`.

First start and where files appear

- The first start of a project is slower: jdtls imports the project, runs Maven or Gradle and builds an index. The statusline and notifications show progress.
- Compiled classes go to `target/` (Maven) next to `pom.xml`. jdtls also writes Eclipse files into the project (`.project`, `.classpath`, `.settings/`). In the test project `.gitignore` contains only `target/` and `.direnv/`, so `.project`, `.classpath` and `.settings/` show up in `git status`; add them to the project's `.gitignore` if you do not want them in git.
- jdtls workspace data goes to `~/.cache/nvim/jdtls/` (`config_<hash>` and `workspace/proj_<hash>`), never into the project. The workspace folder name is a hash of the folder where Neovim was STARTED (`vim.fn.getcwd()` at startup), not of the jdtls root. Always start Neovim from the same project folder, otherwise jdtls builds a second workspace.
- Saving: the auto-save plugin saves a buffer when you leave it or when Neovim loses focus. Refactor results and edits are therefore written to disk without `:w` once you switch windows (a refactor was saved this way). Auto-saves do not format; saving with `:w` does (see `<Space>fm` below).

4. Keys and commands

All `<Space>j` keys are normal mode only. The real maps are created per buffer when jdtls attaches and removed when it detaches (`LspAttach` / `LspDetach` in `lua/mappings.lua`); before that, the global fallback shows the warning. Exception: the four debug keys `<Space>jh`, `<Space>jp`, `<Space>jP` and `<Space>jx` (see below) are plain global maps without that warning. which-key shows the groups `j` Java, `jb` build, `jr` runner, `jt` test, `je` extract.

Build (`<Space>jb`)

Key	Command	What it does	Mode
`<Space>jbb`	`:JavaBuildBuildWorkspace`	Compile the whole workspace; a "Background task" notification shows while it runs	n
`<Space>jbc`	`:JavaBuildCleanWorkspace`	Delete the jdtls workspace cache of the project after a Yes/No question, then restart jdtls (see below)	n

Runner (`<Space>jr`)

Key	Command	What it does	Mode
`<Space>jrr`	`:JavaRunnerRunMain`	Run the main class	n
`<Space>jrs`	`:JavaRunnerStopMain`	Stop the running program (the java process is gone and the runner shows `Process finished with exit code::129`); with nothing running it does nothing visible	n
`<Space>jrc`	(Lua function, Java buffers only)	Run a Java 25 compact source file (no package line, no top-level type) with `java <file>` in a bottom terminal split, see Compact source files	n
`<Space>jrl`	`:JavaRunnerToggleLogs`	Show or hide the runner log window (toggles between 1 and 2 windows)	n
`<Space>jrp`	`:JavaProfile`	Run profiles window (main class and arguments), see below	n

Test (`<Space>jt`)

Key	Command	What it does	Mode
`<Space>jtc`	`:JavaTestRunCurrentClass`	Run all tests of the class in the buffer	n
`<Space>jtm`	`:JavaTestRunCurrentMethod`	Run only the test method under the cursor	n
`<Space>jtr`	`:JavaTestViewLastReport`	Show the result tree of the last run	n
`<Space>jtC`	`:JavaTestDebugCurrentClass`	Debug the class (breakpoints stop)	n
`<Space>jtM`	`:JavaTestDebugCurrentMethod`	Debug the test method under the cursor	n

Refactor / extract (`<Space>je`)

Key	Command	What it does	Mode
`<Space>jev`	`:JavaRefactorExtractVariable`	Extract the expression to a local variable	n
`<Space>jeo`	`:JavaRefactorExtractVariableAllOccurrence`	Same, and replace ALL occurrences	n
`<Space>jec`	`:JavaRefactorExtractConstant`	Extract a constant	n
`<Space>jem`	`:JavaRefactorExtractMethod`	Extract a method (a whole statement needs a selection FIRST, see section 8)	n
`<Space>jef`	`:JavaRefactorExtractField`	Extract a field	n

`<Space>jev`, `<Space>jeo` and `<Space>jec` need no selection (cursor on the expression is enough). To extract a whole statement or several lines with `<Space>jem`, select the code first, press `<Esc>`, then `<Space>jem` (details in section 8).

Debug (`<Space>jh`, `<Space>jp`, `<Space>jP`, `<Space>jx`)

Global maps (not buffer-local), so they work in any buffer, also when jdtls is not attached (where the other `<Space>j` keys only show the warning); they load nvim-dap on demand. All four keys `<Space>jh`, `<Space>jp`, `<Space>jP` and `<Space>jx` work: breakpoint sign, debugger paused, value float, session ended. Stepping and continuing have no keys on purpose: use the typed commands in section 7.

Key	Same as	What it does	Mode
`<Space>jp`	`:DapToggleBreakpoint`	Breakpoint on or off on the cursor line (`B` sign). which-key text: `Java debug: toggle breakpoint`	n
`<Space>jP`	`:DapClearBreakpoints`	Remove ALL breakpoints in ALL files at once; no undo (set them again with `<Space>jp`). Capital P on purpose: the mirror of `<Space>jp` (one breakpoint), kept apart because it is destructive. which-key text: `Java debug: clear all breakpoints`	n
`<Space>jh`	`:lua require("dap.ui.widgets").hover()`	While the debugger is paused: show the value of the variable under the cursor in a float. which-key text: `Java debug: show value under cursor (paused)`	n
`<Space>jx`	`:DapTerminate`	End the debug session: the program is stopped. The breakpoints stay where they are. which-key text: `Java debug: terminate session`	n

Note: a Neovim that was already running when these keys were added (`<Space>jP` included) does not have them, because mappings are read at startup. Restart it with `<Space>sv` (writes all buffers and runs `:restart`; terminal buffers such as the Claude panel are not relaunched, see the `<Space>sv` entry in section 40).

Settings and debugger setup

Key	Command	What it does	Mode
`<Space>jd`	`:JavaDapConfig`	Configure the debug adapter again (it is done automatically when jdtls starts)	n
`<Space>jj`	`:JavaSettingsChangeRuntime`	Pick another JDK runtime for jdtls; only works if runtimes are configured, see below	n

The `:Java*` commands

With `:command Java` after jdtls attached:

- Existing at that point: `JavaBuildBuildWorkspace`, `JavaBuildCleanWorkspace`, `JavaRefactorExtract...` (five), `JavaRunnerRunMain`, `JavaRunnerStopMain`, `JavaRunnerToggleLogs`, `JavaRunnerSwitchLogs`, `JavaTestRunCurrentClass`, `JavaTestRunCurrentMethod`, `JavaTestRunAllTests`, `JavaTestDebugCurrentClass`, `JavaTestDebugCurrentMethod`,

`JavaTestDebugAllTests`, `JavaTestViewLastReport`, `JavaDapConfig`, `JavaProfile`, `JavaSettingsChangeRuntime`.

- Three have no key: `:JavaRunnerSwitchLogs`, `:JavaTestRunAllTests` (all tests of the project) and `:JavaTestDebugAllTests`.
- Only `:JavaBuild*` and `:JavaRefactor*` appear after jdtls attached; the other `:Java*` commands exist as soon as nvim-java is loaded but do nothing useful without jdtls.
- `:JavaRefactor...` commands accept a range, so `:'<,'>JavaRefactorExtractMethod` is valid.

The runner window

`<Space>jrr` opens the runner as a full-width terminal split at the bottom (14 text rows plus the status line, 200 columns wide at 200 columns). It prints the real command, then the program output, then the exit code. Output for `Main`:

```
/nix/store/...-openjdk-25.0.2+10/lib/openjdk/bin/java -cp .../target/classes demo.Main
Picked up JAVA_TOOL_OPTIONS: -javaagent:.../lombok.jar
result: 20
Process finished with exit code::0
```

(The `Picked up JAVA_TOOL_OPTIONS` line comes from the devShell's Lombok setting; it is harmless.) `<Space>jrl` hides and shows this window again. To go back to the code: `<Esc>`, then `<Ctrl-w>k` or `<Up>`.

Several classes with a `main` method: `<Space>jrr` first opens a picker titled `Select the main class (module -> mainClass)`, with one line per class, for example `1. <project> -> pkg.Main` and `2. <project> -> pkg.Other`. `<C-n>` / `<C-p>` move, `<Enter>` on the wanted line runs that class. The output then appears in the bottom terminal split and ends with `Process finished with exit code::0`. Seen in a practice project with two main classes.

Known quirk (an nvim-java bug): once the program has finished, typing a key in this terminal buffer gives `E900: Invalid channel id`. Press `<Esc>` (or `<Ctrl-\><Ctrl-n>`) first to leave terminal mode, then close the window with `:q` or `<Ctrl-w>c`.

Profiles window (`<Space>jrp`)

A profile stores VM arguments and program arguments for the runner of one main class. Flow:

1. `<Space>jrp` opens a window `Profiles` with the entry `New Profile` and the hint `[a]ctivate [d]elete [b]ack [q]uit`.
2. `<Enter>` on `New Profile` opens a form with three boxes: `Name`, `VM arguments`, `Program arguments`, and the hint `[s]ave [b]ack [q]uit`.
3. In the form (normal mode): `i` types in the box, `<Esc>` leaves it, `<Tab>` or `j` goes to the next box, `k` to the previous one, `s` saves.
4. Saved example: name `prof1`, program arguments `hello`. The profile is stored in `~/.local/share/nvim/nvim-java-profiles.json`, keyed by project folder and main class, and marked active:

```
{"/path/to/project": {"nvim-java-test -> demo.Main": [{"name": "prof1", "is_active": true,
"prog_args": "hello", "vm_args": ""}]}}
```

5. The next `<Space>jrr` uses it. The runner printed `... demo.Main hello`.

6. To switch or remove profiles: `<Space>jrp`, choose the profile, `a` activates it, `d` deletes it, `q` closes.

Clean workspace (`<Space>jbc`)

1. `<Space>jbc` opens a Yes/No list with the question `Do you want to delete ".../.cache/nvim/jdtls/workspace/proj_<hash>"`.
2. `<Enter>` on `1. Yes` deletes that folder and restarts jdtls (the folder was recreated fresh a moment later with a new timestamp, jdtls came back). `No` does nothing.
3. Your source files and `target/` are not touched.
4. The folder named in the question is computed from jdtls's root folder, while jdtls really uses a folder named from the folder where Neovim was started. They are the same when you start Neovim in the project root (the case of a project that is its own git root). If your git root is above the folder where you start Neovim, the question may name a folder that does not exist; then delete the real one by hand: `rm -r ~/.cache/nvim/jdtls/workspace`.

Change runtime (`<Space>jj`)

In the test project it only shows a notification: `No configured runtimes available` plus a link to the nvim-java README. The list of JDKs comes from jdtls setting `java.configuration.runtimes`, which this config does not set (only `java.home = $JAVA_HOME`). So `<Space>jj` is a no-op here; the JDK in use is the devShell JDK.

Compact source files (Java 25, `<Space>jrc`)

A compact source file (JEP 512) has no `package` line and no top-level `class`, `interface`, `record` or `enum`; it holds only methods (and fields), for example a bare `void main() { ... }`. jdtls compiles a stub class for such a file and reports `Unresolved compilation problem: declared package "" does not match the expected package`, and `<Space>jrr` still fails on it. Use `<Space>jrc` instead:

1. `<Space>jrc` (Normal mode, in the compact file's buffer). Saves the buffer first if it was modified, then opens a bottom terminal split and runs the JDK source launcher `java <file>`.
2. The terminal inherits `JAVA_HOME` and `JAVA_TOOL_OPTIONS` from Neovim, so it needs `java` on PATH: open Neovim inside the Java devShell, or run `:DevEnv java`. Without `java`, the key shows a warning.
3. On a file that has a `package` line or a top-level type, the key only shows the warning `Java: not a compact source file ...`. Use `<Space>jrr` for normal classes.

Limits: `<Space>jrc` runs a single file with no project classpath, so it cannot use other classes of the project. The key exists only in Java buffers.

Diagnostics: for compact files the jdtls diagnostic `declared package "" does not match the expected package` (code 536871240) is dropped, so the file shows no false error. A real package mismatch in a normal class (the `package` line disagrees with the folder) is still shown.

5. Language server features in a Java buffer

All need jdtls attached. The general keys are described elsewhere (section 13 for LSP); here is what they do in Java.

Key / command	What it does in Java
`gd`	Go to definition; several results open the location list. Opens the target file as a NEW buffer (the previous file stays open in the tab line); `<C-o>` jumps back to the call site, `<C-i>` forward again (default Neovim)
`K`	Hover: type and Javadoc. A hover float shows a "java" progress bar for a few seconds while loading; `<Esc>` closes it
`<Space>rn`	Rename a class, method or variable in all files (NOT a package or folder: on the name in a `package a.b.c;` line Neovim answers "not supported on this element"; rename the folder in the file tree with `r` instead, see Renaming a package or folder). The `New Name` box opens already filled with the old name (delete it with `<BS>` and type the new one; the text behind the box does NOT change while you type); after `<Enter>` the rename appears only after a delay: the answer takes about 10 to 17 s (small project; can be longer), so wait before pressing anything else. If nothing has changed after about a minute, see When a rename does nothing. It applies to ALL files that use the name, the other files are changed only in memory (hidden buffers, `:ls` shows them with `+`) until `:wa` (press `<Enter>` first if a "Press ENTER" prompt is showing); text in comments is not changed. A second `<Space>rn` while the first one is still pending starts from the NEW name (a mistaken second run gave `additaddition`)
`<Space>ca`	Code actions: quick fix, organize imports, generate getters, constructors, `toString`; also the way to extract with a selection (see Refactoring). Which generate entries work and how: see Generate getters, setters and constructors below
`<Space>fm`	Format the file (Visual mode: the selection) with google-java-format, run by conform.nvim (see Formatting (conform.nvim)); the jdtls formatter (Eclipse style) is used only when google-java-format is not installed. Example: `calc.add(2,3)*4` becomes `calc.add(2, 3) * 4`. Saving with `:w` formats too; auto-saves do not, and `<Space>fo` switches format on save off and on. It runs asynchronously: wait a moment. It can re-indent the whole file (the template's 4 spaces became 2) and wrap long lines (`int  x=1+2 ;` became `int x = 1 + 2;`)
`:LspInlayHints enable` / `disable`	Inlay hints (parameter names, types); off by default (enable turns hints on)
`<Space>t`	Symbol outline (aerial); in Java it shows `Main` and `main`
diagnostics	Compile errors and warnings appear while you type, without running anything (in compact source files the false `declared package ""` error is hidden, see Compact source files)

Other notes:

- **illuminate**: other uses of the word under the cursor are highlighted (Java is in its filetype list).
- **typos_lsp**: attaches to Java buffers too (root markers include `.gitignore`), also outside the devShell. If `<Space>rn` or `<Space>ca` only warn "no attached language server supports it", only typos_lsp is attached and jdtls is missing.
- **Completion** and **snippets** work as in other languages (sections 14 and 15).
- The coloured line marker (`colorcolumn`) is at 100 for Java.
- spring-boot attaches next to jdtls in every Java buffer, even in a project without Spring; it is harmless.

Go to definition, back, hover, references

Example: the cursor is on `add` in `calc.add(10, 3)`.

1. `gd` Opens the file with the definition (`public int add(...)`) as a new buffer; the previous file stays open in the tab line at the top.
2. `<C-o>` Jumps back to the call site (`<C-i>` goes forward again; default Neovim).
3. `K` Hover popup with the signature, for example `int pkg.Calc.add(int a, int b)`. It may show a `java` loading bar for a few seconds first. `<Esc>` closes it.
4. `<Space>gr` Opens Glance (Peeking without jumping) with a panel `References (N)` listing every use, including the definition itself (in the practice project: 4 files); the preview is on the left. `<C-n>` / `<C-p>` move, `<Esc>` closes.

When a rename does nothing

Applies to `<Space>rn` and, in the same way, to other project-wide jdtls actions (find references, extract, ...). Symptom: the `New Name` box works, you press `<Enter>`, and nothing changes and no message appears (jdtls fails on the server side and Neovim does not show the error). This is the most likely cause found so far, NOT the only one: other causes exist, and case-specific debugging may still be needed.

1. Wait first. Even when everything works the request takes about 10 to 17 s, and the text does not change while you type; the buffers change only after the answer arrives. Up to about 20 s of nothing can be normal.
2. `:LspAttached` Check that `jdtls` is in the list; `:LspInfo` (`:checkhealth vim.lsp`) shows the clients and the root folder. Without jdtls, see "jdtls never attaches" in Troubleshooting.
3. After about a minute with no change, look at the jdtls log of the project workspace (an Eclipse-style log, one per workspace). In a shell: `ls -t ~/.cache/nvim/jdtls/workspace/*/.metadata/.log | head -3`. The newest one is normally the project you work in; the confirmation box of `<Space>jbc` also names the workspace folder.
4. Search it, with `<log>` replaced by that path: `grep -n "Problem with rename" <log>`, and `grep -n "^!ENTRY .* 4 0 " <log> | tail` (an entry whose number after the plugin name is 4 is an error). Read the lines after a hit with `sed -n '<line>,+12p' <log>`.
5. What the log showed in that case: `Problem with rename for file:///.../Main.java` with `AssertionFailedException: assertion failed: Search for method declaration did not find, original element: int add(int, int)` preceded by `Error filtering index locations based on qualifier.` and

`java.io.FileNotFoundException: .../.metadata/.plugins/org.eclipse.jdt.core/<number>.index (No such file or directory)`

Meaning: the search index files of the workspace were missing or stale, so jdtls could not find where the method is used and the rename failed. Reproduced by deleting two `<number>.index` files of a scratch workspace: the rename request then answered with 0 changed files after about 4 s and wrote the same error.

6. `<Space>jbc` Fix (Clean workspace): choose `1. Yes` with `<Enter>`. jdtls restarts and rebuilds the workspace (progress in the lower right; wait about 1 minute; the `.index` files reappear). Then the same rename returned edits for 4 files. It is a cache clean: save unsaved work first, and the first start afterwards is slower. It is NOT guaranteed to fix every failed rename; if the log shows a different error, that error is the lead.
7. After a successful rename the open buffer has changed and the other files are changed in memory only (hidden buffers, `:ls` shows `+`): save them all with `:wa`.

Rename: `:grep` + `:cfdo` instead of `<Space>rn`?

Not a drop-in replacement. It takes about a second but is plain text: it also changes comments, strings and unrelated same-named methods (`Calc.add` and `List.add`). Use it only for a unique name and only after reviewing the matches with `:copen`; for common names (`add`, `get`, `size`) stay with `<Space>rn`. Details, table and examples: LSP rename versus `:grep` + `:cfdo`.

Renaming a package or folder. `<Space>rn` with the cursor on a name in the `package a.b.c;` line does NOT work: Neovim answers "not supported on this element". jdtls renames only classes, methods and variables there. A package is its folder, so you rename the FOLDER in the file tree and jdtls fixes the `package` and `import` lines. Example: folder `objects/<old>` becomes `objects/<new>`.

1. In the file tree (`<Space>s`, then `r` on the folder): type the new folder name and press `<Enter>`.
2. jdtls is told about the rename (nvim-lsp-file-operations, see File operations and language servers) and rewrites the lines: in the moved files `package a.<old>;` becomes `package a.<new>;` and is saved; in the files that import them the `import a.<old>.X;` lines change in their buffers only.
3. `:wa` saves the buffers that changed.

Only the file tree does this. A folder moved with `:Move`, `:Rename` or in a shell is a plain move on disk. The same happens when jdtls is not attached yet. The manual fallback then, or when you want to review every change first:

1. Rename the folder (tree `r`). This renames the folder on disk only; the `package` and `import` lines still contain the old name.
2. `:grep -rn "<old>" .` Collects every line with the old name. `:copen` opens the list: read the matches and make sure none is something else (a comment, a string, an unrelated name).
3. `:cfdo %s/<old>/<new>/ge | update` Fixes every `package` and `import` line in all files of the list and saves them. The `e` flag stops an error when a file has no match.
4. `:LspRestart` Restarts the language server so it reads the new folder layout. If errors remain, run `<Space>jbc` (clean workspace) and choose `1. Yes`.

Add a missing import (auto-import)

Example: a file contains `List<String> names = new ArrayList<>();` and no imports.

1. Put the cursor ON the unknown word (`List`). If the cursor is elsewhere, the menu does not offer imports at all.
2. `<Space>ca` Opens the `Code actions` list (fzf-lua) with entries such as `Import 'List' (com.sun.tools.javac.util)`, `Import 'List' (java.awt)`, `Import 'List' (java.util)`, `Add all missing imports`, `Create class 'List<T>'`, `Change to ...`, `Organize imports`.
3. `Import java.util` Type this to filter the list, then CHECK that the highlighted line is `Import 'List' (java.util)`.
4. `<Enter>` `import java.util.List;` is added at the top (imports end up sorted, `ArrayList` before `List`).
5. Repeat for `ArrayList`: cursor on it, `<Space>ca`, filter `Import java.util`, `<Enter>`.

Traps:

- The filter order is not the numbering. After typing only `Import`, the first line was the `java.awt` entry: always read the highlighted line.
- Filtering only `java.util` for `ArrayList` put `Change to 'List' (java.util)` on top, and `<Enter>` REPLACED `ArrayList` with `List`. Undo with `u`. Keep the word `Import` in the filter.
- `Add all missing imports` failed in the test with the notification `Failed to choose imports ... java-refactor/action.lua:288: bad argument #1 to 'ipairs' (table expected, got nil)` (a file where `List` had several candidate packages and another name did not exist at all). Do not rely on it; use the single `Import '<Name>' (<package>)` entries.
- `Organize imports` is in the list; not tried.

Generate getters, setters and constructors

Example class `Person` with the fields `name` and `age`.

1. Put the cursor on a field line inside the class. The cursor must be inside the class, on a field; elsewhere the menu offers fewer generate actions.
2. `<Space>ca` opens the code-action list (an fzf-lua picker): type to filter, `<C-n>` / `<C-p>` or the arrows move, `Enter` runs the entry, `Esc` cancels.

The list shows every generate action twice, and the numbers are only the order. One group has NO trailing dots (`Extract interface...`, `Generate Getters and Setters`, `Generate Getters`, `Generate Setters`, `Generate Constructors...`, `Generate hashCode() and equals()...`, `Generate toString()...`, `Override/Implement Methods...`, `Organize imports`), the other group has dots on the accessor entries too (`Generate Getters and Setters...`, `Generate Getters...`, `Generate Setters...`, `Generate Constructors...`, ...).

Entry	Result
`Generate Getters and Setters`, `Generate Getters`, `Generate Setters` (NO dots)	Writes the methods for ALL fields at once, no questions asked (`getName`, `setName`, `getAge`, `setAge`). To get fewer, delete the extra methods afterwards
The same three entries WITH dots	Do NOT work. They show the notification `"java.action.generateAccessorsPrompt" is not supported yet!` and the file stays unchanged (nvim-java does not implement that prompt; see `lua/java-refactor/client-command-handlers.lua` in the nvim-java plugin). Use the entries without dots
`Generate Constructors...`	Works, opens a `Select Fields` list (see below)
`Generate toString()...`, `Generate hashCode() and equals()...`, `Override/Implement Methods...`	Use the same multi-select as the constructor (per the nvim-java source, `lua/java-refactor/action.lua`). Not tried one by one

The `Select Fields` list (nvim-java's own multi-select) behaves differently from other pickers:

- `Enter` does NOT finish: it TOGGLES the highlighted field (a `*` appears in front of it) and the list reopens.
- `<C-n>` / `<C-p>` move; `Tab` does nothing here.
- `<Esc>` finishes the selection and writes the constructor. The first `<Esc>` sometimes had to be pressed twice.
- Result with both fields marked: `public Person(String name, int age) { this.name = name; this.age = age; }`; with only `name` marked: `public Person(String name) { this.name = name; }`. The constructor is inserted before the fields.

Indentation: the inserted code uses TAB indentation (jdtls does not format what it inserts), so it looks off next to 4-space code. `<Space>fm` formats the file and re-indents it; saving with `:w` does the same.

6. Tests (JUnit)

Why and how

java-test knows JUnit 4 and 5 and TestNG. nvim-java asks jdtls for the test classes and methods, then starts the tests through the debug adapter machinery. That is why the test terminal is called `[dap-terminal]` and why debugging a test works with the same keys.

Using it

Example test (`src/test/java/demo/CalcTest.java`):

```
class CalcTest {
    @Test
    void addWorks() {
```

```
        assertEquals(5, new Calc().add(2, 3));
    }

    @Test
    void multiplyWorks() {
        assertEquals(6, new Calc().multiply(2, 3));
    }
}
```

1. `<Space>jtc` anywhere in the class runs both tests; with the cursor inside one method, `<Space>jtm` runs only that one. In tmux, real nvim: `<Space>jtm` with the cursor on the assert line of the second method, then `<Space>jtr`, listed only that method (`multiplyWorks`), not `addWorks`; the whole-class run `<Space>jtc` lists all test methods. The cursor must be inside a method that has a test annotation, see `cursor is not on a test method`.
2. A terminal window named `[dap-terminal] Launch All Java Tests` opens at the bottom (the tab line also shows it). Content: only the start lines (`Picked up JAVA_TOOL_OPTIONS...`) and `[Process exited 0]`. The terminal does NOT print pass or fail text. For `<Space>jtm` as well: the terminal shows only the `Picked up JAVA_TOOL_OPTIONS: -javaagent:.../lombok.jar` line and `[Process exited 0]`.
3. The results are in the report: to see it press `<Space>jtr`; it opens a floating window with a tree. (In a tmux test of ours, after `<Space>jtc` only the terminal was visible and no report popup appeared by itself. The user saw the report with a check mark after a `<Space>jtc` class run.) All passing:

```
demo.CalcTest
  addWorks(demo.CalcTest)
  multiplyWorks(demo.CalcTest)
```

Each line has a status icon in front (a different icon for pass and fail). With a wrong expected value (`assertEquals(7, ...)`): the failing method gets the failure message and stack below it:

```
  multiplyWorks(demo.CalcTest)
    org.opentest4j.AssertionFailedError: expected: <7> but was: <6>
    at org.junit.jupiter.api.Assertions.assertEquals(Assertions.java:531)
    at demo.CalcTest.multiplyWorks(CalcTest.java:15)
```

4. Close the report with `<Esc>` or `q`. `<Space>jtr` again after a new run shows the new result. The exit code in the terminal is 0 even when tests fail; trust the report.

To debug a test (stop at a breakpoint instead of just running it) see section 7, "Learning exercise: debug a test".

`cursor is not on a test method`

`<Space>jtm` can warn `cursor is not on a test method`. The message comes from nvim-java (`java-test/api.lua`): `execute_current_test_method` warns when no test method is found at the cursor; the methods come from jdtls/java-test for the current file. Causes, most likely first:

1. The method has no `@Test` above it (the neighbouring method had one). Add `@Test` (or `@ParameterizedTest` etc.) above the method; after that `<Space>jtm` ran it.

2. The cursor is outside a method, for example on the class line or between methods. Put the cursor on a line inside the method.

The test terminal window does not come back after you close it

The first `<Space>jtm` or `<Space>jtc` opens the window `[dap-terminal] Launch All Java Tests` at the bottom; for a tiny test the line `[Process exited 0]` appeared within about 2 seconds. After you close that window (`:q`, `<Space>q`, or any accidental unsplit), further runs do NOT open a new window. Only the notice `INFO::"run current test method"` (or the class variant) appears. `:ls` then still lists the buffer `[dap-terminal] Launch All Java Tests` with the flags `hF` (`h` = hidden). The notice only means the run was STARTED, not that it finished.

Why: nvim-java does not create the test terminal. Its runner (`lua/java-dap/runner.lua`, `Runner:run_by_config`) only calls `require('dap').run(config, ...)`, and nvim-dap (`lua/dap/session.lua`, `terminals.acquire`) creates the terminal. nvim-dap first reuses a terminal buffer from a pool of buffers of finished sessions (filled by `terminals.release`) and in that case opens NO window. Only when the pool is empty does it create a new buffer AND a window, with the command from its setting `terminal_win_cmd` (default `belowright new`). So the first run opens the window, later runs write into the same buffer, and nothing reopens the window you closed. This is the default behaviour of nvim-dap, not a choice of this config (the config sets neither `terminal_win_cmd` nor `dap.defaults`).

Observed: the program runner window (`<Space>jrr`) DOES come back after `:q` the next time it runs, while the test terminal does not. In the source, nvim-java's own runner (`lua/java-runner/run.lua`) creates a NEW terminal buffer for every run (`nvim_create_buf` plus `nvim_open_term`) and shows it with its own window logic (`runner.lua`, `set_buffer` / `create`), so a window is opened each run. It does not use nvim-dap's terminal pool.

The nvim-dap setting `dap.defaults.fallback.terminal_win_cmd` is not configured here; the way to work with the closed window is the steps below.

To see the test terminal again:

1. `:ls` Look for the line `[dap-terminal] Launch All Java Tests`; `h` means hidden.
2. `:sbuffer dap-terminal` Shows it in a new horizontal split below. Or `:vert sbuffer dap-terminal` for side by side. The buffer is listed and shows `[Process exited 0]`. Not tellable by us: whether the content shown came from the LATEST run or from an earlier one. Details of `:sbuffer`: Put two chosen buffers side by side or one above the other.
3. `<Ctrl-w>k` / `<Ctrl-w>j` Back to the code window. You can keep the terminal in a split permanently to watch the exit line of every run.

Reliable signals that a test run has finished:

- The exit line (`[Process exited 0]`) in the terminal buffer.
- `<Space>jtr` (report). It shows the LAST finished run only (after `<Space>jtm` on one method it lists only that method, after `<Space>jtc` all methods).

The debug runs `<Space>jtC` and `<Space>jtM` also go through `dap.run` (`run_by_config` in `api.lua` is used for both; the debug configuration differs only in `debug = true`).

Needs

- A JUnit dependency in `pom.xml` (the test project: `junit-jupiter` 5.10.2 and `maven-surefire-plugin` 3.2.5, release 21).
- jdtls attached and the `java-test` extension linked by the devShell (see section 1).
- Network the first time Maven has to download the JUnit jars into `~/.m2`.

7. Debugging

Why and how

nvim-dap is the debugger client; java-debug is the adapter that talks to the JVM; nvim-java registers the Java configurations for you. This config has four debug keys (`<Space>jp` breakpoint, `<Space>jP` clear all breakpoints, `<Space>jh` value under the cursor, `<Space>jx` stop; see section 4) and NO dap-ui panel (no variables or stack window). Everything else is a typed `:Dap*` command: stepping and continuing deliberately have no keys (`:DapContinue`, `:DapStepOver`, `:DapStepInto`, `:DapStepOut`). You work with the sign column and small floating windows. The four keys:

Step	How	Result
Breakpoint on / off	`<Space>jp` (same as `:DapToggleBreakpoint`) on the line (put it on a line with code, e.g. the `assertEquals` line, not on the `void addWorks() {` line)	A `B` sign in the sign column
Debug a test class	`<Space>jp`, then `<Space>jtC`	No picker. A notice `debug current test class` appears; after about 15 seconds the window shows a `→` sign on the breakpoint line; the program is paused
Debug one test method	`<Space>jtM` with the cursor in the method	same, only that method (same start as `<Space>jtC`)
Debug the program (`main`)	`<Space>jp` in `Main.java`, then `:DapContinue`	A picker "Configuration" opens (5 identical entries `nvim-java-test -> demo.Main`; why there are several: see The Configuration picker); press `<Enter>` on the first: the program stops at the breakpoint with `→`
Step over	`:DapStepOver`	`→` moves to the next line; the line you were on has run
Step into	`:DapStepInto`	On a call to your own method it opens that file (`calc.add(2, 3)` opens `Calc.java` at `int sum = a + b;`; the file of your own method opens at its first line inside). On a line without a call, or into library code, it opens an empty `unknown` buffer and a DAP warning "Adapter reported frame ... Invalid cursor line"
Step out	`:DapStepOut`	Back to the caller file
Look at a value	put the cursor on the variable, `<Space>jh` (same as `:lua require("dap.ui.widgets").hover()`), only while paused	A small float with the value (shows with the command; the key: a variable showed `13`; `<Space>jh` on a variable after the breakpoint line showed its value). `<Esc>` closes it
Continue	`:DapContinue` while paused	Runs to the next breakpoint or to the end (the test terminal shows `[Process exited 0]`; for the program, the terminal shows the rest of the output and `[Process exited 0]`)
Stop	`:DapTerminate`	The session ends (in the test it closes; afterwards a new session starts cleanly after it)
Debug console	`<Space>jx` (same as `:DapRepl`)	Opens a REPL window; you can type expressions and see their values

Debug workflow

1. `<Space>jp` on a line with code (not a comment and not the method signature line).
2. `:DapContinue`, then in the picker choose an entry that ends with your main class (for example `... -> tutorial.Main`). For tests `<Space>jtC` / `<Space>jtM` start directly, with no picker.
3. While paused (the `→` sign): `:DapStepOver`, `:DapStepInto` or `:DapStepOut` to move; `<Space>jh` with the cursor on a variable shows its value.
4. `:DapContinue` to run on, to the end or to the next breakpoint.
5. `<Space>jx` to stop the session.
6. Optional: `<Space>jP` removes all breakpoints in all files when you are done.

What the stepping commands do (for beginners):

Command	Meaning
`:DapStepOver`	Run THIS line and stop on the next one. Method calls on the line run completely; you do not go inside them.
`:DapStepInto`	Go INSIDE the method that is called on this line. Works for your own code that has source; into library code you get an empty `unknown` buffer and a DAP warning (see the step table).
`:DapStepOut`	Run the rest of the current method and stop back in the caller.
`:DapContinue`	Run until the next breakpoint, or to the end of the program.

Program output appears only when its line has run: the `→` line has NOT run yet, so a `System.out.println(...)` on it prints nothing until you step over it.

Learning exercise (try it)

Use a small `main` method that calls `compute(a, b)` on line A, followed by a few lines that print and use the result.

1. `<Space>jp` on line A, then `:DapContinue` and choose the first entry in the picker: the program stops with `→` on line A.
2. `:DapStepOver`: `→` moves to the next line; line A has run, but a print on the next line has not printed yet.
3. `:DapStepOver` again: the output of the print line appears in the debug terminal (only now, when its line has run) and `→` moves on. Put the cursor on a variable assigned earlier and press `<Space>jh`: a float shows its value; `<Esc>` closes it.
4. `:DapContinue`: the program runs to the end; the terminal shows the rest of the output and `[Process exited 0]`.
5. Start again (`<Space>jx` first if a session is still open, then `<Space>jp` on line A and `:DapContinue` plus picker). On line A, `:DapStepInto` opens the file of `compute` at its first line inside; `:DapStepOver` once there; `:DapStepOut` goes back to the caller file.

Learning exercise: debug a test

Use the test class from section 6: a test `multiplyWorks` whose line is `assertEquals(6, new Calc().multiply(2, 3));`.

1. `<Space>jp` on the `assertEquals(...)` line of `multiplyWorks`: a `B` sign appears.

2. `<Space>jtC` (debug the whole test class; `<Space>jtM` with the cursor in the method debugs only that method): there is NO picker. A notice `debug current test class` appears.
3. Wait about 15 seconds: `→` is on the breakpoint line. The debug terminal shows only the `Picked up JAVA_TOOL_OPTIONS` line until the run ends.
4. `:DapStepOver`: the assert line runs and `→` moves on.
5. `:DapContinue`: the tests run to the end; the terminal shows `[Process exited 0]`.
6. `<Space>jtr`: the report shows a check mark per test.

Without a breakpoint `<Space>jtC` just runs the tests to the end.

Difference to debugging a program:

	`:DapContinue`	`<Space>jtC` / `<Space>jtM`
For	a class with a `main` method	JUnit tests (whole class / the method under the cursor)
Picker	yes, the `Configuration` picker; choose the entry that ends with your main class	no, it starts directly
Start from	any file	the test file

Stopping inside the code that the test calls):

1. Open the file of the called method (for example `Calc.java`) and put the breakpoint inside it with `<Space>jp`.
2. Go back to the test file and press `<Space>jtC`.
3. The debugger pauses inside the called method when the test reaches it. `<Space>jh` on a parameter shows its value.

Practical rule: to stop inside the code that the test calls, set the breakpoint in that file, then start the test debug from the test file.

The Configuration picker

`:DapContinue` without a running session opens a picker titled `Configuration`. It lists `<project name> -> <main class>` for every class with a `main` method (for example one line for `Main` and one for `Snippets`). The number at the start is only the position in the list. Any entry with the main class you want is identical to the others: pick the first. Many identical entries are normal (5 identical entries in one session; 40 entries in a long session, 4 in a fresh one).

Why the list grows (see the nvim-java source, `lua/java-dap/init.lua`: `vim.list_extend(nvim_dap.configurations.java, dap_config)`): every time the debug configuration is set up, nvim-java APPENDS the detected main classes and never clears the old ones. A set-up happens each time jdtls attaches and on `<Space>jd` / `:JavaDapConfig`, so the list grows during a long session. The 40 entries were 20 pairs. A restart of Neovim resets the list.

Notes:

- Without a breakpoint, `<Space>jtC` simply runs the tests to the end.
- The debugged program's output goes to the `[dap-terminal]` window.

- Breakpoints only work when the program is started through the debugger. `<Space>jrr` runs without a debugger: it starts a plain `java` process (read from the nvim-java source, `lua/java-runner/run.lua`: the command is started with `jobstart`, no dap involved), so breakpoints are IGNORED and the whole program runs and prints all its output. The `B` sign stays on the line either way. To stop at breakpoints start with `:DapContinue` (and the picker) as above, or `<Space>jtC` / `<Space>jtM` for tests.
- More than one breakpoint (breakpoints set with `<Space>jp` on two lines of a small `main` method): `:DapContinue` plus the picker stops at the FIRST breakpoint the program reaches (`→` on that line; the terminal shows only the `Picked up JAVA_TOOL_OPTIONS` line). `:DapContinue` while paused runs on and stops at the SECOND one (the terminal now also shows the output printed in between). A third `:DapContinue`, with nothing left to stop at, runs to the end: the terminal shows the rest of the output and `[Process exited 0]`. The order of stops follows the order in which the program reaches the lines, not the order in which you set the breakpoints. `<Space>jx` stops early at any point.
- Remove breakpoints: `<Space>jp` (or `:DapToggleBreakpoint`) on the line that has the `B` removes it. It TOGGLES: with the cursor on a different line it ADDS a breakpoint there (read from nvim-dap `toggle_breakpoint`). "I have tons of breakpoints, how do I remove them all?": `<Space>jP` (same as `:DapClearBreakpoints`) clears the breakpoints of all files at once, `<Space>jp` on the line removes only that one. There is no undo; set them again with `<Space>jp`.
- If a debug window stays open after `<Space>jx` / `:DapTerminate`, close it with `<Space>q` in that window.
- See section 36 and section 56 for the general DAP notes.

8. Refactoring (extract)

How it works: each `<Space>je...` key runs `vim.lsp.buf.code_action` filtered to one kind (`refactor.extract.variable`, `.constant`, `.function`, `.field`). So `<Space>je...` and `<Space>ca` use the same jdtls actions; the keys just pick the right one for you.

Rule: select first, then act. To extract a whole statement or several lines into a method, the code MUST BE SELECTED BEFORE you press `<Space>jem`. Without a selection only the small expression under the cursor is extracted. Variable and constant extraction (`<Space>jev`, `<Space>jec`, `<Space>jeo`) are the opposite: they need NO selection, the cursor on the expression is enough.

The cursor or selection decides what is extracted. After the action a small `New Name` window opens with a proposed name (`i`, `_4`, `extracted`). The change is ALREADY applied; the window renames it:

- `<Enter>` accepts the name in the box (to rename, delete the text with `<BS>` first, then type the new name; it must be a valid Java identifier or you get "is not a valid Java identifier").
- `<Esc>` closes the window and keeps the proposed name.

Undo needs `u` twice (once for the rename step, once for the extraction).

Try-it code (`Main.java`), results with the cursor on the shown part:

```
Calc calc = new Calc();
int total = calc.add(2, 3) * 4;
String label = "result";
System.out.println(label + ": " + total);
```

Key	Cursor	Result
`<Space>jev`	on `calc.add(2, 3)`	`int i = calc.add(2, 3);` and `int total = i * 4;` (the `New Name` box proposes `i`)
`<Space>jeo`	on an expression that appears several times	same, but every identical occurrence uses the new variable
`<Space>jec`	on the `4`	`private static final int _4 = 4;` above the method and `... * _4;` (rename to `FACTOR` gives `private static final int FACTOR = 4;`)
`<Space>jem`	on the identifier `calc`, no selection	extracts only that identifier: `extracted(calc).add(2, 3)` and `private static Calc extracted(Calc calc) { return calc; }`
`<Space>jem`	selection made first, see below	extracts the selected statement(s) into a method
`<Space>jef`	on an expression	extracts to a field (same flow)

Extracting a whole statement or block into a method needs a selection, and the key is normal mode only. Select first, then leave Visual mode, then press the key. Exact flow for the `System.out.println(label + ": " + total);` line:

1. `^` Put the cursor on the first character of the statement (`^` jumps to the first non-blank character).
2. `v` Start a characterwise selection. The bottom line must say `-- VISUAL --`, NOT `-- VISUAL LINE --`.
3. `$` Extend the selection to the end of the line, so the highlight covers the whole statement including the `;` (in this config `$` in Visual mode goes to the last visible character).
4. `<Esc>` Leave Visual mode. The highlight disappears; that is normal, Neovim remembers the selection.
5. `<Space>jem` The `New Name` box appears and the line is already replaced by the call.

Why this order: `<Space>jem` (like all `<Space>j...` keys) is a Normal-mode mapping only (`keymap.set("n", ...)` in `lua/mappings.lua`), so it cannot be pressed while Visual mode is active; there `<Space>` and `j` are Visual-mode keys that move the selection. The command reads the range of the LAST Visual selection (the marks `'<` and `'>`), which Neovim keeps after `<Esc>`. `gv` re-selects the last selection (default Neovim behaviour, not tried here).

What goes wrong:

- Capital `V` (linewise) selection: only a part was extracted. On a longer `println("sum=" + sum + ...)` line only the text `"sum="` was extracted:

`System.out.println(extracted() + sum + ...` plus a method returning `"sum="`. On the try-it line only `System.out` was extracted.
- Starting `v` in the middle of the line: partial selection, odd result.
- `<Space>jem` before jdtls is ready: only a warning, nothing is extracted. Wait for jdtls (see section 2).

Result:

```
        extracted(total, label);
    }

    private static void extracted(int total, String label) {
        System.out.println(label + ": " + total);
    }
```

Another statement: `System.out.println("sum=" + sum + " limited=" + limited + " product=" + product);` became `extracted(sum, limited, product);` plus a new `private static void extracted(int sum, int limited, int product)` method.

Pressing `<Space>jem` while the selection is still active does NOT work (visual mode has no such key; `<Space>ca` there would even run `c`, change, on the selection).

9. Snippets

Source: `my_snippets/java.snippets` (71 snippets, grouped below as in the file). Type the trigger in insert mode in a Java buffer and expand it with `<Ctrl-j>` (section 15); `<Ctrl-j>` / `<Ctrl-k>` jump to the next / previous placeholder. A compact trigger + description table of all of them is in `04-completion-snippets.md` ("Java snippets"); this section explains each snippet and shows what it expands to.

Conventions (the same in every snippet):

- Triggers are camelCase and case-sensitive (`promptRead`, not `jpromptread`): pick them from the completion menu instead of typing them whole. Type `j` and read the list.
- Placeholders are generic English words naming the kind of thing to fill in (`type`, `name`, `condition`, `ExceptionType`), never hardcoded values.
- The Scanner variable is always named `input`. `fdiScanner` creates it; the other input snippets use an existing `input`.
- The one-line description is shown by nvim-cmp above the snippet body in the completion menu (scroll it with `<Ctrl-d>` / `<Ctrl-f>`).
- `fdiScanner` uses the file name as the class name by default.
- When a snippet needs an import, its description names the exact import(s). The snippets do not add imports themselves, so check that the import line exists at the top of the file.
- Counterpart snippets name each other in their description (`interface` / `implements`, `equals` / `hashCode`, `comparable` / `comparator`, `twrScanner` / `twrPrintWriter`, `twrObjectOut` / `twrObjectIn`, `minArray` / `maxArray`).

How to read the code blocks below: they show the expansion with the placeholder names, in tab-stop order. When a name appears several times in the real expansion (for example the loop variable `i`), you type it once and every copy follows. Delete the placeholder text you do not need, or overwrite it.

Group: Program skeleton and input

The Scanner `Type` placeholder. The input snippets end in `input.next` plus a placeholder `Type`. Type the name of what you want to read, with a capital first letter, and it becomes the method:

You type for `Type`	Method	Reads	Variable type
`Int`	`nextInt()`	an integer	`int`
`Double`	`nextDouble()`	a decimal number	`double`
`Boolean`	`nextBoolean()`	`true` / `false`	`boolean`
`Line`	`nextLine()`	a whole line, spaces included	`String`
(none, just `next`)	`next()`	one word	`String`

`Int` and `Double` match the Java type name (`int`, `double`). Strings are the odd one out: you do not write `String`, you write `Line` (a whole line) or use `next()` (one word, which you must type by hand because `Type` is part of the placeholder). Other types follow the same idea (`nextLong()`, `hasNextBigInteger()`).

`fdiScanner`: whole program skeleton for the course (package `fondamentidiinformatica.<subpackage>`), with a Scanner named `input`, one prompt and one read. The class name defaults to the file name.

```
package fondamentidiinformatica.subpackage;

import java.util.Scanner;

public class FileName {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("prompt: ");
        type name = input.nextType();

        // code

        input.close();
    }
}
```

Example: prompt `Enter your age`, type `int`, name `age`, `Type` = `Int` gives `int age = input.nextInt();`.

`promptRead`: prints a prompt and reads one value into a new variable. Needs `import java.util.Scanner;` and an existing `input`.

```
System.out.print("prompt: ");
type name = input.nextType();
```

Example: `double`, `price`, `Double` gives `double price = input.nextDouble();`. For a text line: `String`, `city`, `Line` gives `String city = input.nextLine();`.

`readValidated`: asks again until the user types the right kind of value. The `Type` placeholder is used twice, in `hasNext<Type>()` (is the next token of that type?) and in `next<Type>()` (read it), so you type it once.

```
while (!input.hasNextType()) {
    System.out.print("error message: ");
    input.nextLine();
}
type name = input.nextType();
```

Examples: `Type` = `Int` gives `hasNextInt` / `nextInt`; `Type` = `Double` gives `hasNextDouble` / `nextDouble`. The same pattern works for other types, for instance `hasNextBigInteger`. The type name matches Java (`Int`/`int`, `Double`/`double`), except that a String uses `Line` (`hasNextLine` / `nextLine`), which reads a whole line, while `next` reads one word. The `input.nextLine()` inside the loop throws away the wrong input before asking again.

`readNumberThenLine`: reads a number and then a line of text. After `nextInt()` the Enter key is still waiting in the input, so a plain `nextLine()` would return an empty string. The extra `input.nextLine()` consumes that leftover newline. Use it whenever a number read is followed by a line read.

```
type name = input.nextType();
input.nextLine(); // consume the leftover newline
String text = input.nextLine();
```

Example: `int`, `age`, `Int`, then `text` renamed to `fullName`. `Type` is a number type here (`Int`, `Double`), not `Line`.

`readUntilInt`: keeps reading numbers until the user types a stop value (the sentinel), then leaves the loop with `break`. The comparison uses `==`, so it fits numbers and chars, not Strings (use `readUntilString`).

```
while (true) {
    System.out.print("prompt: ");
    type name = input.nextType();

    if (name == sentinel) {
        break;
    }

    // code
}
```

Example: `int`, `n`, `Int`, sentinel `0` gives `int n = input.nextInt();` and `if (n == 0)`. For a double you would write `double`, `Double`, and a sentinel such as `-1`.

`readUntilString`: the same loop for text lines. It compares with `.equals(...)`, never with `==`, because Strings are objects.

```
while (true) {
    System.out.print("prompt: ");
    String name = input.nextLine();

    if (name.equals("sentinel")) {
        break;
    }

    // code
}
```

Example: sentinel `stop` ends the loop when the user types `stop`.

`randomInt`: a random integer from `min` to `min + range - 1`. Example: range `6`, min `1` gives a die roll from 1 to 6.

```
int name = (int) (Math.random() * range) + min;
```

Group: Arrays, matrices and collections

Arrays. `arrayNew` makes an empty array of a given size (all zeros, `false` or `null`); `arrayLiteral` makes one from known values. Delete or add values in the braces as needed.

```
type[] name = new type[size];
type[] name = {value1, value2, value3};
```

Example: `int`, `scores`, `5` gives `int[] scores = new int[5];`.

`minArray` / **`maxArray`**: find the smallest / largest element. The result starts as the first element and the loop compares from index
1. The two snippets are identical except for `<` and `>`.

```
type min = array[0];
for (int i = 1; i < array.length; i++) {
    if (array[i] < min) {
        min = array[i];
    }
}
```

`maxArray` has the variable named `max` and `>` in the comparison. Example: `type` = `int`, `array` = `scores`.

`forMatrix` / **`foreachMatrix`**: walk every cell of a two-dimensional array. The indexed version gives you the row and column numbers `i` and `j` and a spot after each row (for example to print a newline); the foreach version gives the values directly but no indexes.

```
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        // code
    }
    // end of row
}

for (type[] row : matrix) {
    for (type value : row) {
        // code
    }
}
```

Note the tab-stop order of `forMatrix`: the matrix name comes first, then `i`, `j`, the inner code and the end-of-row spot.

`arrayAdd` / **`arrayRemove`**: methods for a fixed-size array plus a counter. The array never grows: `count` says how many slots are really used, and the free slots are at the end. Put them inside your class, where `array` and `count` are fields. Add stores at position

`count` and increments it (it returns `false` if the element is `null` or the array is full). Remove finds the element with `equals`, shifts the following ones left, clears the last used slot and decrements `count` (it returns `false` if not found). Remove works on object arrays (`String`, your own classes), not on `int[]`.

```
boolean methodName(Type element) {
    if (element == null || count >= array.length) {
        return false;
    }
    array[count++] = element;
    return true;
}

boolean methodName(Type element) {
    for (int i = 0; i < count; i++) {
        if (array[i].equals(element)) {
            for (int j = i; j < count - 1; j++) {
                array[j] = array[j + 1];
            }
            array[--count] = null;
            return true;
        }
    }
    return false;
}
```

Example: `methodName` = `addStudent`, `Type` = `Student`, `element` = `student`, `count` = `numStudents`, `array` = `students`. In `arrayRemove` the order is `methodName`, `Type`, `element`, `i`, `count`, `array`, `j`.

Collections. Four declarations. The type names go inside `<>` and must be object types: `Integer` instead of `int`, `Double` instead of `double`, `String`.

Trigger	Expansion	Imports
`hashMap`	`HashMap<KeyType, ValueType> name = new HashMap<>();`	`java.util.HashMap`
`treeMap`	`TreeMap<KeyType, ValueType> name = new TreeMap<>();` (keys kept sorted)	`java.util.TreeMap`
`arrayList`	`List<Type> name = new ArrayList<>();`	`java.util.List`, `java.util.ArrayList`
`hashSet`	`Set<Type> name = new HashSet<>();` (no duplicates)	`java.util.Set`, `java.util.HashSet`

Examples: `HashMap<String, Integer> ages`, `List<String> names`, `Set<Integer> seen`.

`forMapEntry`: loops over a map one entry (key and value) at a time. Needs `import java.util.Map;`. Inside the loop call `entry.getKey()` and `entry.getValue()`.

```
for (Map.Entry<KeyType, ValueType> entry : map.entrySet()) {
    // entry.getKey(), entry.getValue()
}
```

Example: `String`, `Integer`, `entry`, `ages` gives `for (Map.Entry<String, Integer> entry : ages.entrySet())`.

Group: Control flow

Plain loops and conditions. Tab stops go top to bottom (loop header, then body; for `ifelif` each condition and body in turn).

```
for (int i = start; i < end; i++) { /* code */ }        // for
for (Type item : collection) { /* code */ }              // foreach
while (condition) { /* code */ }                         // while
do { /* code */ } while (condition);                     // dowhile
if (condition) { /* code */ }                            // if
if (condition) { /* code */ } else { /* code */ }        // ifelse
if (c1) { /* code */ } else if (c2) { /* code */ } else { /* code */ }  // ifelif
```

(The real expansions are multi-line with one statement per line; they are compressed here.) `for` loops from `start` up to but not including `end`; use `foreach` when you do not need the index. `dowhile` runs the body at least once.

`tern`: ternary conditional assigned to a variable.

```
type name = condition ? valueIfTrue : valueIfFalse;
```

Example: `String`, `label`, `age >= 18`, `"adult"`, `"minor"`.

Switch. Six forms of the same idea; choose by style. `variable` is what you test; the case values are constants (numbers, chars, Strings, enum constants).

Trigger	Style	Use it when
`switchtraditional`	`case v:` with `break;`	the classic form; forgetting `break` makes execution fall through to the next case
`switchmulti`	`case a: case b:`	two values share one body
`switcharrow`	`case v -> statement;`	no `break` needed, no fall-through
`switcharrowmulti`	`case a, b -> statement;`	arrow form with two values per case
`switchyield`	`Type result = switch (...) {...};`	the switch produces a value you assign
`switchyieldblock`	same, with a `{ ... yield ...; }` block	one case needs several statements before giving its value

```
switch (variable) {                     // switchtraditional
    case value1:
        // code
        break;
    case value2:
        // code
        break;
    default:
        // code
        break;
}

switch (variable) {                     // switcharrow
    case value1 -> statement;
    case value2 -> statement;
    default -> statement;
}
```

```
Type result = switch (variable) {         // switchyield
    case value1 -> "result1";
    case value2 -> "result2";
    default -> "defaultResult";
};

Type result = switch (variable) {         // switchyieldblock
    case value1, value2 -> "result1";
    case value3, value4 -> {
        // statements
        yield "result2";
    }
    default -> "defaultResult";
};
```

`switchmulti` and `switcharrowmulti` are the same as the two first forms with `value1`..`value4` (two labels in each case). The switch-expression forms need `Type` to match what the cases return (`String` for the quoted texts; change the quotes if you return numbers) and a `default` case. Arrow switches need Java 14 or later.

Group: Methods

Both are `private static`, the form used in `main` programs where methods sit next to `main` in one class.

```
private static returnType name(type param) {      // staticMethod
    // code
}

private static returnType name(type param) {      // recursive
    if (baseCondition) {
        return baseValue;
    }
    return recursiveCase;
}
```

`recursive` is a method that calls itself: the base case stops the recursion and the last line is the recursive call. Example for a factorial: `int`, `factorial`, `int n`, base `n <= 1`, value `1`, recursive case `n * factorial(n - 1)`. Delete or add parameters as needed.

Group: Classes

`class` / **`publicClass`**: a class with three fields and a constructor that assigns them. `class` is package-private with plain fields; `publicClass` is `public`, with `private final` fields (cannot change after construction) and a `public` constructor. Delete the field lines and constructor parts you do not need; the field names repeat in the constructor, so rename them in one place.

```
public class ClassName {
    private final type field1;
    private final type field2;
    private final type field3;

    public ClassName(type field1, type field2, type field3) {
        this.field1 = field1;
        this.field2 = field2;
        this.field3 = field3;
```

```
    }
}
```

Example: `Student`, `String name`, `int age`. `class` is the same without `public` and `private final`.

`subclass`: a class that extends another and calls its constructor with `super(...)`. Example: `Student`, `Person`, parameters `String name`, arguments `name`.

```
public class SubClass extends SuperClass {
    public SubClass(type param) {
        super(arguments);
    }
}
```

`abstractClass` / **`interface`** / **`implements`**: an abstract class cannot be instantiated and has methods without a body that subclasses must write; an interface lists methods only; `implements` is the class that fulfils an interface (it already includes `@Override`). Use `interface` first, then `implements` in another file with the same interface and method names.

```
public abstract class ClassName {
    // fields and constructor

    public abstract returnType method(type param);
}

public interface InterfaceName {
    returnType method(type param);
}

public class ClassName implements InterfaceName {
    @Override
    public returnType method(type param) {
        // code
    }
}
```

Example: interface `Shape` with `double area()`, class `Circle implements Shape`.

`toString` / **`toStringSuper`**: `toString()` is what `System.out.println(object)` prints. `toString` writes the class name and two fields (delete or copy the `+ ", field=" + this.field` part for other counts); `toStringSuper` is for a subclass and appends one extra field to the text of the superclass.

```
@Override
public String toString() {
    return "ClassName[field1=" + this.field1 + ", field2=" + this.field2 + "]";
}

@Override
public String toString() {
    return super.toString() + ", label: " + this.field;
}
```

`equals` / `hashCode`: they go together. `equals` says when two objects count as equal (same field values rather than same memory address); `hashCode` must give the same number for equal objects, and hash collections (`HashMap`, `HashSet`) rely on that. If you override one, override the other with the same fields. `hashCode` needs `import java.util.Objects;`.

```
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (obj == null || getClass() != obj.getClass()) {
        return false;
    }
    ClassName other = (ClassName) obj;
    return comparison;
}

@Override
public int hashCode() {
    return Objects.hash(field1, field2);
}
```

Example for `comparison`: `name.equals(other.name) && age == other.age` (use `.equals` for objects, `==` for primitives). Then `hashCode` takes `name, age`.

`defaultIfBlank` / `defaultIfBelowMin`: one-line field assignments for a constructor that validates its arguments instead of throwing. The first replaces a `null` or blank String with a default; the second replaces a number below a minimum. They use the constructor parameter named like the field.

```
this.field = field == null || field.isBlank() ? defaultValue : field;   // defaultIfBlank
this.field = field < minimum ? defaultValue : field;                    // defaultIfBelowMin
```

Examples: `this.name = name == null || name.isBlank() ? "unknown" : name;` and `this.age = age < 0 ? 0 : age;`. To reject bad values with an exception instead, use `throwIf`.

`record` / `recordCompact`: a record is a short class for plain data; the compiler writes the constructor, getters (`point.x()`), `equals`, `hashCode` and `toString`. The compact form adds validation code that runs in the constructor (throw an exception there, or normalise a value).

```
public record RecordName(type component1, type component2) {}

public record RecordName(type component) {
    public RecordName {
        // validation
    }
}
```

Example: `public record Point(int x, int y) {};` compact validation: `if (x < 0) throw new IllegalArgumentException("negative");`.

`enum` / `enumFields`: a fixed set of named values. `enumFields` lets each constant carry a value, with a private constructor and a getter.

```
public enum EnumName { CONSTANT1, CONSTANT2, CONSTANT3 }

public enum EnumName {
    CONSTANT1(value1), CONSTANT2(value2);

    private final type field;

    private EnumName(type field) {
        this.field = field;
    }

    public type getField() {
        return field;
    }
}
```

Example: `public enum Size { SMALL(1), LARGE(3); ... private final int weight; ... getWeight() }`.

`instanceOf`: tests the type of an object and gives you a variable of that type in the same step (no cast). Example: `object` = `shape`, `Type` = `Circle`, variable `circle`.

```
if (object instanceof Type variable) {
    // code
}
```

`comparable` / **`comparator`**: both define an order for sorting. `comparable` is the natural order, written inside the class itself, so the class must also say `implements Comparable<ClassName>` (add it by hand; the snippet does not). `comparator` is a separate class for an alternative order, used as `Collections.sort(list, new ByAge())` or `list.sort(new ByAge())`. Both return a negative number, zero or a positive number. `comparator` needs `import java.util.Comparator;`.

```
@Override
public int compareTo(ClassName other) {
    return comparison;
}

public class ComparatorName implements Comparator<Type> {
    @Override
    public int compare(Type first, Type second) {
        return comparison;
    }
}
```

Example for `comparison`: `Integer.compare(age, other.age)` in `compareTo`, and `first.getName().compareTo(second.getName())` in `compare`.

Group: Exceptions

`exception` / **`exceptionData`**: your own checked exception (a method that throws it must declare `throws` or catch it). Type the name once without the word `Exception`: the snippet adds it. The first has a fixed message; the second keeps a value and builds the message from it.

```
public class NameException extends Exception {
    public NameException() {
        super("message");
    }
}

public class NameException extends Exception {
    private final type field;

    public NameException(final type field) {
        super(messageExpression);
        this.field = field;
    }

    public type getField() {
        return field;
    }
}
```

Example: `Name` = `InvalidAge`, message `"Age is not valid"` gives `InvalidAgeException`. For the data version: `int`, `age`, message expression `"Invalid age: " + age`, getter `Age`.

`throwIf`: guard at the start of a method that throws when the condition holds. Common `ExceptionType` values: `IllegalArgumentException` (bad argument), `NullPointerException` (unexpected null); or one of your own exceptions.

```
if (condition) {
    throw new ExceptionType("message");
}
```

Example: `age < 0`, `IllegalArgumentException`, `"age must not be negative"`.

`trycatch` / **`tryfinally`** / **`trywith`**: `try` runs the code, `catch` handles the named exception, `finally` always runs afterwards (cleanup). `trywith` is try-with-resources: the resource declared in the parentheses is closed automatically at the end, with or without an error. Common `ExceptionType` choices: `InputMismatchException`, `NumberFormatException`, `FileNotFoundException`, `IOException`, or just `Exception`.

```
try {
    // code
} catch (ExceptionType e) {
    // handle exception
}

try {
    // code
} catch (ExceptionType e) {
    // handle exception
} finally {
    // cleanup code
}

try (ResourceType resource = initializer) {
    // code
} catch (ExceptionType e) {
    // handle exception
}
```

Example for `trywith`: `Scanner`, `fileScanner`, `new Scanner(new File("data.txt"))`, `FileNotFoundException`. The ready-made file versions are in the next group.

Group: File I/O and serialization

The `twr*` snippets are try-with-resources: the reader or writer is closed automatically at the end of the block, so you never call `close()`. Replace `path` with the file name, relative to where the program runs (for example `data.txt`). Each has a catch clause with the exception the file operation can throw.

`twrScanner` / **`twrPrintWriter`**: read a text file line by line with a Scanner; write to a text file with a PrintWriter. Imports: for reading `java.util.Scanner`, `java.io.File`, `java.io.FileNotFoundException`; for writing `java.io.PrintWriter`, `java.io.FileOutputStream`, `java.io.FileNotFoundException`. The `true` in the writer turns on automatic flushing. `FileOutputStream` overwrites an existing file.

```
try (Scanner fileScanner = new Scanner(new File("path"))) {
    while (fileScanner.hasNextLine()) {
        String line = fileScanner.nextLine();
        // code
    }
} catch (FileNotFoundException e) {
    // handle exception
}

try (PrintWriter writer = new PrintWriter(new FileOutputStream("path"), true)) {
    writer.println(content);
} catch (FileNotFoundException e) {
    // handle exception
}
```

Example: path `data.txt`; content `name + " " + age`. Use `writer.println` in a loop for several lines (put the snippet's `println` line inside your own `for`).

`twrBufferedReader`: reads a file line by line with `BufferedReader.readLine()`, which returns `null` at the end of the file. Imports: `java.io.BufferedReader`, `java.io.FileReader`, `java.io.IOException`. Same job as `twrScanner` with a different class and exception (`IOException`).

```
try (BufferedReader reader = new BufferedReader(new FileReader("path"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        // code
    }
} catch (IOException e) {
    // handle exception
}
```

`twrObjectOut` / **`twrObjectIn`** / **`serialUID`**: serialization saves a whole object to a binary file and reads it back. The class of the object must say `implements Serializable` (`import java.io.Serializable;`), and so must the classes of its fields. `serialUID` adds the version field inside that class; give it any number, change it when the class changes shape. Imports for writing: `java.io.ObjectOutputStream`, `java.io.FileOutputStream`, `java.io.IOException`; for reading: `java.io.ObjectInputStream`,

`java.io.FileInputStream`, `java.io.IOException`. Reading needs a cast to the real type and also catches `ClassNotFoundException`.

```
try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("path"))) {
    out.writeObject(object);
} catch (IOException e) {
    // handle exception
}

try (ObjectInputStream in = new ObjectInputStream(new FileInputStream("path"))) {
    Type name = (Type) in.readObject();
} catch (IOException | ClassNotFoundException e) {
    // handle exception
}

private static final long serialVersionUID = 1L;
```

Example: write `student` to `student.dat`, then read with `Type` = `Student`, `name` = `loaded`.

Group: JavaFX

JavaFX is the graphical interface library of the course.

`fxApp`: skeleton of a JavaFX program: a class extending `Application`, a `start` method that builds the window, and `main` that calls `launch`. Imports: `javafx.application.Application`, `javafx.stage.Stage`, `javafx.scene.Scene`, `javafx.scene.layout.BorderPane`. The `BorderPane` placeholder is the root layout (change it, for example to `VBox`, and add the matching import); width and height are in pixels.

```
public class ClassName extends Application {
    @Override
    public void start(Stage primaryStage) {
        BorderPane root = new BorderPane();
        primaryStage.setTitle("title");
        primaryStage.setScene(new Scene(root, width, height));
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Example: title `My app`, width `400`, height `300`.

`fxProperty`: the usual JavaFX model field: a property plus its getter, setter and `nameProperty()` accessor (used for binding). The snippet only declares the field and the three methods; you must create the property yourself in the constructor, for example `this.name = new SimpleStringProperty(name);`. `PropertyType` is the property class and `valueType` the plain Java type it holds; they go in pairs:

`PropertyType`	`valueType`	Create with
`StringProperty`	`String`	`new SimpleStringProperty(...)`
`IntegerProperty`	`int`	`new SimpleIntegerProperty(...)`

Imports: `javafx.beans.property.StringProperty` (or the property type you chose) and the matching `Simple...Property` class.

```
private final PropertyType name;

public valueType getName() {
    return name.get();
}

public void setName(valueType value) {
    name.set(value);
}

public PropertyType nameProperty() {
    return name;
}
```

Example: `StringProperty`, `title`, `String`, `Title` gives `getTitle()`, `setTitle(...)` and `titleProperty()`.

`fxAlert`: a pop-up dialog that waits for the user to close it. Imports: `javafx.scene.control.Alert`, `javafx.scene.control.Alert.AlertType`. `AlertType` examples: `INFORMATION` (a notice), `ERROR` (something failed); `WARNING` and `CONFIRMATION` also exist.

```
Alert alert = new Alert(AlertType.alertType);
alert.setTitle("title");
alert.setHeaderText("headerText");
alert.setContentText("content");
alert.showAndWait();
```

Example: `INFORMATION`, title `Saved`, header `Done`, content `The file was saved.`. For a confirmation, `showAndWait()` returns the button the user pressed.

`fxOnAction`: runs code when a button is clicked. Example: `button` = `saveButton`, `event` = `e`.

```
button.setOnAction((event) -> {
    // code
});
```

10. Windows while running, testing and debugging

Keys	What it does
`<Ctrl-w>h` `j` `k` `l`	Move between windows: code, the runner or test terminal at the bottom, the report
`<Left>` `<Down>` `<Up>` `<Right>`	The same with arrow keys (normal mode)
`<Esc>` first	In a terminal window press `<Esc>` to get to normal mode, then use a window key. `i` types in the terminal again
`<Space>q`	Save if modified and close the current window (use it in a runner, test or debug terminal)
`<Ctrl-w>o`	Keep only the current window
`<Esc>` in a float	Closes the report float, the Profiles window, `New Name` box and hover floats (section 7 lists `<Esc>` closing floats)

See section 7 (windows) and the cheat sheet in section 2 for the same rows.

11. Troubleshooting

Symptom	Likely cause	Fix
`<Space>j...` shows `Java: jdtls not attached (open nvim inside the Java devShell, or run :DevEnv java)`	`java` is not on PATH (not in the devShell) or jdtls has not finished starting	Quit, `cd` into the project, check `direnv allow` and `which java`, start nvim again; inside the devShell wait about 10 s and check `:LspAttached`
jdtls never attaches	Neovim started before direnv loaded; jdtls crashed	`:LspLog`, `:checkhealth vim.lsp`, `:edit` the file; check `which java`
Tests or debugger do not start, adapter not found	The devShell was not entered since the extensions were linked, or the links broke	Enter the project with direnv once (the shell hook relinks `~/.local/share/nvim/nvim-java/packages/java-debug-adapter/extension` and `java-test/extension`), restart nvim
`<Space>jrr` fails with `declared package "" does not match the expected package`	The file is a Java 25 compact source file (no package line, no top-level type); jdtls cannot run it	Use `<Space>jrc` (Compact source files)
`<Space>jrr` prints nothing	jdtls not finished importing, or the file is not under `src/main/java`, or no `main` method	Wait for the first import, `<Space>jbb`, then `<Space>jrr` again; a project that is its own git root and one nested in a bigger git repo both work
Old errors or deleted classes still shown	Stale jdtls workspace	`<Space>jbc` then Yes (jdtls restarts); if it stays wrong: `rm -r ~/.cache/nvim/jdtls/workspace`
A different, empty workspace appears after starting from another folder	The workspace folder name is a hash of the START folder of Neovim	Always start nvim in the same project folder
Test terminal shows no pass/fail	By design: results are in the report (for `<Space>jtm`: only the `Picked up JAVA_TOOL_OPTIONS` line and `[Process exited 0]`) `<Space>jtr`; no terminal window appears at all after you closed it once: see The test terminal window does not come back after you close it	
`<Space>jtm` warns `cursor is not on a test method`	Cause: no `@Test` above the method, or the cursor is outside a method	Add `@Test`, cursor inside the method; see `cursor is not on a test method`
First test run fails with a download error	Maven offline: JUnit jars not in `~/.m2`. In a shell with an empty `java.configuration.runtimes`; `mvn -o test` stops with `Cannot access central (https://repo.maven.apache.org/maven2) in offline mode and the artifact org.apache.maven.plugins:maven-resources-plugin:jar:3.3.1 has not been downloaded from it before` (10 to 17 s or more) are missing or stale and the server fails silently; other causes exist. Slow answers	Connect once, run `<Space>jtc` again (or `mvn test` in a shell)
`<Space>jj` opens the class, says `No configured runtimes available. argument #1 to 'ipairs'` and stays running after `<Enter>`, no message		Fix the project runtimes (`jdtls-runtimes`); if it shows index errors; steps in When a rename does nothing

Related sections

Section 35 (Java keys), 54 (Java in depth), 36 and 56 (debugging), 19 and 55 (running code, `<Space>rr`), 13 (LSP), 14 (completion), 15 (snippets), 7 (windows), 8 (terminal), 20 (git; the git root decides the jdtls root), 2 (quick reference and cheat sheet).

Chapter 14: Python

79. Python (pyright, ruff, uv, running and debugging)

This section is one walk-through for everything Python in your config: what starts by itself, how to run, format and debug a file, how virtual environments are detected, and what to check when something does not work. Deeper background on the shared parts lives in the sections named in "Related sections" at the end.

What you get

Feature	What it does	Needs
pyright (language server)	Type checking, import resolution, hover (`K`), go to definition (`gd`), rename (`<Space>rn`), references. Mode is `standard`, whole workspace is checked. Its own import sorting is turned off (ruff does that)	`pyright-langserver` on PATH (installed globally by `neovim.nix`, also in the python devShell)
ruff (language server, `ruff server`)	Fast linting (diagnostics), quick fixes and "organise imports" through `<Space>ca`. Formatting is a separate step: `ruff format` through conform.nvim	`ruff` on PATH (installed globally by `neovim.nix`, also in the python and jupyter devShells)
ruff format (formatter, run by conform.nvim)	`:w` formats the file first; `<Space>fm` formats on demand (Visual mode: the selection); after a save that did not format, a background check warns when the file is not formatted	`ruff` on PATH
typos_lsp	Spell checker for identifiers and comments, attaches to every file type (also Python)	`typos-lsp` on PATH (installed globally by `neovim.nix`)
tree-sitter	Syntax highlighting (the `python` parser)	Nothing on Nix systems (parsers come from the nix store)
Snippets	`print` (f-string), `impa`, `main`, `sol` (see Snippets below)	Nothing
uv awareness	In a project with `uv.lock` and no active virtual environment, `<Space>rf` / `<F9>` go through `uv run`	`uv` on PATH (it is: `/run/current-system/sw/bin/uv`)
Statusline label	Shows the active environment as `name (venv)` or `name (conda)` in Python buffers	An activated environment
Format check	After saving, `ruff format --check` runs in the background and warns `<file>: file is not formatted (ruff)` when the save did not format (auto-save, or format on save switched off)	`ruff` on PATH, otherwise silent
Run	`<Space>rf` / `<F9>` (output in the quickfix window) and `<Space>rr` (output in a terminal split)	`python` / `python3` on PATH
Jupyter notebooks (nvjup)	Open `.ipynb` as text, edit cells with `# %%`, run cells, see the output in a split	`uv` (the Windows build, `ipykernel` (the jupyter devShell has it)

Jupyter notebooks

Quick start

1. Start nvim **inside the Python devShell** (a project folder with `.envrc` containing `use_dev_env python`, see "Devshell and tools"). Outside it, only pyright and typos_lsp exist.
2. Open a file: `nvim hello.py`. Wait a second or two. `:LspAttached` lists the running servers; in the python devShell: `pyright`, `ruff`, `typos_lsp`. The statusline shows `pyright (+2)` (the first server and two more).
3. Type a small program, save with `:w`, then run it with `<Space>rf` (or `<F9>`). A 6-line quickfix window opens at the bottom and shows the output.
4. Run it again as a full terminal with `<Space>rr`. A terminal split opens on the LEFT of the code.
5. Format with `<Space>fm` (ruff format). The buffer changes in place (as one undo step, `u` undoes it) and is not saved; saving with `:w` formats too.
6. Debug with `<Space>dp`. The code gets a `▶` marker and a pdb pane opens below it. Step with `<Space>dn` (or `<F10>`), quit with `:GdbDebugStop`. (`<F9>` and `<Space>rf` keep working afterwards.)

Why each tool exists

Tool	Why it is in your setup	How it works
pyright	Python does not check types by itself. Pyright finds wrong argument types, missing imports and typos in attribute names before you run the file	A language server (`pyright-langserver --stdio`). Neovim starts it when you open a `.py` file and it answers questions: errors, hover, definition, rename. Installed globally, so it also works outside a devShell
ruff	Fast linter (unused imports, undefined names, style problems) with automatic fixes and import sorting	`ruff server` is a second language server in the same buffer. Its diagnostics appear next to pyright's, and `<Space>ca` lists its fixes
ruff format	One fixed code style (black-compatible), so you never decide about spaces or quotes	A command-line formatter run by conform.nvim on save and by `<Space>fm`; a background check runs after saves that did not format
uv	Fast Python project and package manager. Projects managed by uv have a `uv.lock` file	The config checks for `uv.lock` and then runs `<Space>rf` / `<F9>` through `uv run`, so they use the project's own environment
venv / conda label	Tells you which environment nvim (and so pyright) was started with	Reads `$VIRTUAL_ENV` / `$CONDA_DEFAULT_ENV`
direnv + devShell	Gives each project its own tools without installing them globally	`.envrc` with `use_dev_env python` loads the nix devShell; start nvim from that folder
nvim-gdb	Step through a script with pdb (Python's built-in debugger) with the current line marked in your code	Starts `python -m pdb file.py` in a terminal pane and talks to it
AsyncRun	Run a script without leaving the editor and read the output in the quickfix window	`<Space>rf` / `<F9>` runs a shell command as a background job and streams its output into quickfix
typos_lsp	Catches misspelt words inside names and comments	A language server for every file type
tree-sitter, aerial, illuminate, treesj, ufo	General tools that also work for Python (highlighting, outline, word highlight, split/join, folding)	See the sections in "Related sections"

Python-specific configuration lives in only a few places:

File	What it holds
`after/ftplugin/python.lua`	indentation, no wrapping, `<Space>rf` / `<F9>`, `<Space>dp`
`after/lsp/pyright.lua`, `after/lsp/ruff.lua`	server settings
`lua/config/lsp.lua`	which servers exist and when they are enabled
`my_snippets/python.snippets`	snippets
`lua/custom-autocmd.lua`	the after-save format check (`ruff format --check`)
`lua/options.lua`	`colorcolumn` 88 for Python

Devshell and tools

Python tools are not installed globally except two. Open nvim in a devShell to get the rest.

Tool	Global (`neovim.nix`)	python devShell	jupyter devShell
`pyright`	yes	yes	no (uses the global one)
`typos-lsp`	yes	no (uses the global one)	no (uses the global one)
`ruff`	yes (formatter and language server)	yes	yes
`black`	no	yes (not used by the config)	no
`uv`	yes, system-wide (`which uv` shows `/run/current-system/sw/bin/uv`; `python3` is also on the system PATH)	not added	not added
`python` / `python3`	system `python3` exists	yes (version depends on the variant)	python 3.13

Templates live in `~/nix/templates/krit/dev-environments/language-specific/`:

Template	What it gives you
`python`	Python, black, flake8, isort, pip, matplotlib, pylint, setuptools, numpy, pyright, ruff, and `venvShellHook` which creates `.venv` in the project folder (`venvDir = ".venv"`). It warns when `.venv` was built with a different Python version than the shell (delete `.venv` and reload)
`jupyter`	poetry, Python 3.13, ruff, ipykernel, pip, `venvShellHook` (also `.venv`). For notebooks in Neovim (nvjup) it also has chafa, ffmpeg, imagemagick, librsvg and, on Linux, the variable `NVJUP_CHROMIUM` (for interactive Plotly/Bokeh charts). No black, no pyright of its own

Variants of the `python` template (flake outputs): `default` = Python 3.15, `py-stable` = 3.13, `py-lts` = 3.12, `py311` = 3.11.

How direnv picks one: the `direnv.nix` helper defines `use_dev_env() { use flake <dev-environments>/language-specific/$1 }`. So the `.envrc` line decides the variant:

`.envrc` content	Result
`use_dev_env python`	`python` template, default output (latest Python)
`use_dev_env "python#py-lts"`	same template, output named after `#`
`use_dev_env "python#py-stable"`	same template, output `py-stable`

Your folders in `~/github-repos/personal/developing-projects/python-projects/`: `python-latest` (`use_dev_env python`), `python-lts` (`use_dev_env "python#py-lts"`), `python-stable` (`use_dev_env "python#py-stable"`). After a new or changed `.envrc` run `direnv allow` once. Then start nvim from that folder so nvim inherits the environment.

When a tool is missing the config does not complain:

Tool missing	What happens
`pyright` or `ruff`	That server is silently not enabled (no warning when opening the file). `:LspStart pyright` or `:LspStart ruff` names the missing program
`ruff` (formatter)	Formatting is skipped silently (no message on `:w` or `<Space>fm`; the language server formats instead when one is attached). The after-save format check stays silent
`uv` (in a uv project)	Not a real case here (uv is installed system-wide). If it were missing, `<Space>rf` / `<F9>` would fail inside the quickfix window
`python` / `python3`	`<Space>rf` / `<F9>` shows the shell error in the quickfix window. `<Space>rr` has no PATH check for Python: with an empty PATH, the terminal shows `bash: line 1: python3: command not found` and no nvim warning appears
`typos-lsp`	Silent

How Python is set up (the real code)

Python has no plugin spec of its own. It is a few small files, quoted here verbatim unless marked abridged. The binaries (`pyright-langserver`, `ruff`, `uv`) are never installed by Neovim: they come from the Nix system or the python devShell, and no Mason is used.

The server entries (lua/config/lsp.lua)

Abridged (the other servers of the table are left out, see "How language servers are registered" in the LSP chapter):

```
-- Servers: configured here (plus after/lsp/<name>.lua), enabled only when the binary exists
---@type table<string, vim.lsp.Config>
local servers = {
  pyright = { cmd = { "pyright-langserver", "--stdio" } },
  ruff = { cmd = { "ruff", "server" } },
```

In plain words:

- Only the command is set here. The settings of each server live in `after/lsp/<name>.lua`, which Neovim merges in by itself.

- A loop at the end of `lua/config/lsp.lua` enables a server only when its binary is on PATH. Outside the python devShell, with pyright or ruff missing, that server silently does not start.

pyright settings (after/lsp/pyright.lua)

```
-- For what diagnostic is enabled in which type checking mode, check doc:
-- https://github.com/microsoft/pyright/blob/main/docs/configuration.md#diagnostic-settings-
defaults
-- Currently, the pyright also has some issues displaying hover documentation:
-- https://www.reddit.com/r/neovim/comments/1gdv1rc/what_is_causeing_the_lsp_hover_docs_to_
looks_like/

local new_capability = {
  textDocument = {
    publishDiagnostics = {
      tagSupport = {
        valueSet = { 2 },
      },
    },
    hover = {
      contentFormat = { "plaintext" },
      dynamicRegistration = true,
    },
  },
}

-- cmd lives in lua/config/lsp.lua (its executable check enables the server)
---@type vim.lsp.Config
return {
  ---@type lspconfig.settings.pyright
  settings = {
    pyright = {
      -- disable import sorting and use Ruff for this
      disableOrganizeImports = true,
      disableTaggedHints = false,
    },
    python = {
      analysis = {
        autoSearchPaths = true,
        diagnosticMode = "workspace",
        typeCheckingMode = "standard",
        useLibraryCodeForTypes = true,
        -- we can this setting below to redefine some diagnostics
        diagnosticSeverityOverrides = {
          deprecateTypingAliases = false,
        },
      },
    },
  },
  capabilities = new_capability,
}
```

In plain words:

- `disableOrganizeImports = true`: ruff owns import sorting, so pyright must not offer a competing "Organize imports" action.
- `typeCheckingMode = "standard"`: the middle level between `basic` and `strict`.
- `diagnosticMode = "workspace"`: pyright reports problems in all files of the project, not only the open ones.

- `useLibraryCodeForTypes = true` and `autoSearchPaths = true`: use the source of installed packages for types when they ship no stubs, and search folders such as `src/` for imports.
- `diagnosticSeverityOverrides.deprecateTypingAliases = false`: no deprecation hints for `typing.List`, `typing.Dict` and similar.
- `capabilities`: `tagSupport = { valueSet = { 2 } }` lets the server mark deprecated code (shown struck through); the `hover` entry asks for plain text instead of Markdown, a work-around for badly rendered hover text (the Reddit link in the file comment describes the problem).

ruff settings (after/lsp/ruff.lua)

```
---@type vim.lsp.Config
return {
  init_options = {
    -- the settings can be found here: https://docs.astral.sh/ruff/editors/settings/
    settings = {
      organizeImports = true,
    },
  },
}
```

In plain words: `organizeImports = true` makes ruff offer the "Organize imports" code action (see "Code actions from ruff"). Everything else comes from the project's `pyproject.toml` / `ruff.toml`.

Which environment: get_py_env (lua/utils.lua)
Verbatim, the decision logic behind the table in "When `<Space>rf` / `<F9>` uses uv":

```
--- Get the current virtual env name ("" if none) and its kind
--- @return string venv_name
--- @return string|nil kind "venv" | "conda" | nil (no env)
function M.get_virtual_env()
  local conda_env = os.getenv("CONDA_DEFAULT_ENV")
  local venv_path = os.getenv("VIRTUAL_ENV")
  if venv_path ~= nil then
    return vim.fn.fnamemodify(venv_path, ":t"), "venv"
  end
  if conda_env ~= nil then
    return conda_env, "conda"
  end
  return "", nil
end

--- Project root for python files
--- @return string|nil
function M.get_proj_root()
  return vim.fs.root(0, { ".git", "pyproject.toml" })
end

--- Python env kind of the current project
--- @return "plain_venv"|"uv"|""|nil (nil = no project root)
function M.get_py_env()
  local project_root = M.get_proj_root()
  if project_root == nil then
    return nil
  end
  if M.get_virtual_env() ~= "" then
```

```
    return "plain_venv"
  end
  if vim.fn.filereadable(vim.fs.joinpath(project_root, "uv.lock")) == 1 then
    return "uv"
  end
  return ""
end
```

In plain words:

- The project root is the nearest folder with `.git` or `pyproject.toml`. No root: `nil`.
- An active environment (`$VIRTUAL_ENV`, or else `$CONDA_DEFAULT_ENV`) always wins: result `"plain_venv"`, and the tools run as plain `python`.
- No active environment and a `uv.lock` in the root: `"uv"`, and the tools run through `uv run`.
- Otherwise `""`: plain commands.

Run keys (after/ftplugin/python.lua)
Verbatim (lines 14-26; the indentation options at the top and the pdb part below are shown elsewhere):

```
-- `:compiler ruff` + `:make`: don't pass `--preview`
vim.g.ruff_makeprg_params = ""

-- in a uv project (uv.lock at the project root) without an activated venv, run tools
through `uv run`
local py_env = utils.get_py_env()

if vim.fn.exists(":AsyncRun") == 2 then
  local py_cmd = (py_env == "uv") and "uv run python" or "python"
  for _, lhs in ipairs({ "<F9>", "<leader>rf" }) do -- <leader>rf: the same without function
keys
    vim.keymap.set("n", lhs, string.format(':<C-U>AsyncRun %s -u "%%"<CR>', py_cmd),
{ buffer = true, silent = true, desc = "run python file" })
  end
end
```

In plain words:

- `py_env` is computed once, when the Python file is opened. That is why activating a venv or creating `uv.lock` afterwards needs `:e!`.
- The run keys exist only when the `:AsyncRun` command exists (the plugin is lazy and registers the command at startup). The `%%` in the format string becomes a literal `%`, which AsyncRun expands to the current file, in double quotes.
- Formatting has no Python-specific key: `<Space>fm` and format on save run `ruff format` through conform.nvim (see Formatting (conform.nvim)).
- `vim.g.ruff_makeprg_params = ""` is what lets `:compiler ruff` + `:make` work without `--preview` (see "Linting from the command line").

AsyncRun (lua/plugin_specs.lua)

```
  -- Asynchronous command execution
  {
    "skywind3000/asyncrun.vim",
    lazy = true,
```

```
    cmd = { "AsyncRun" },
    init = function()
      -- Automatically open quickfix window of 6 line tall after asyncrun starts
      vim.g.asyncrun_open = 6
      if vim.g.is_win then
        -- Command output encoding for Windows
        vim.g.asyncrun_encs = "gbk"
      end
    end,
  },
```

In plain words:

- `cmd = { "AsyncRun" }`: the plugin loads the first time `:AsyncRun` is used, that is, on the first `<F9>` / `<Space>rf`.
- `asyncrun_open = 6` is the "quickfix window 6 lines tall" of the table in "Running code".
- The `gbk` branch matters only on Windows (output encoding of the console); on Linux and macOS nothing is set.

pdb through nvim-gdb (plugin spec and ftplugin)

The plugin spec (`lua/plugin_specs.lua`, verbatim):

```
  -- Debugger plugin
  {
    "sakhnik/nvim-gdb",
    enabled = function()
      return vim.g.is_win or vim.g.is_linux
    end,
    build = { "bash install.sh" },
    cmd = { "GdbStart", "GdbStartLLDB", "GdbStartPDB", "GdbStartBashDB", "GdbStartRR" },
    init = function()
      -- do not let nvim-gdb create its global <leader>dd/dl/dp/db/dr start maps
      -- (they would overwrite the user's <leader>dd / <leader>db / <leader>dp)
      vim.g.nvimgdb_disable_start_keymaps = true
      -- nvim-gdb's eval key is <F9> by default and, at the end of a session, it removed our
<F9> run key in
      -- the buffer: use <leader>dv (Normal: word under cursor, Visual: selection) instead
      vim.g.nvimgdb_config_override = { key_eval = "<space>dv" }
      -- <leader>dp (pdb on the current file) is python buffer-local: after/ftplugin/
python.lua
    end,
    config = function()
      -- nvim-gdb's setup() maps cmdline <c-e> globally (cmake executable picker); keep the
builtin <C-e>
      pcall(vim.keymap.del, "c", "<c-e>")
    end,
  },
```

The Python side (`after/ftplugin/python.lua`, lines 40-68, verbatim):

```
-- <leader>dp: start pdb on the current file (nvim-gdb, lazy-loaded on :GdbStart*; pdb is
the
-- python stdlib module). nvim-gdb is disabled on macOS -> one warning instead.
if vim.fn.exists(":GdbStartPDB") == 2 then
  vim.keymap.set("n", "<leader>dp", [[:<C-U>GdbStartPDB python -m pdb %<CR>]], { buffer =
true, desc = "start pdb on current file (nvim-gdb)" })
  -- debug-session keys without function keys (nvim-gdb's own keys are F4 until, F5
continue, F8 breakpoint,
  -- F10 next, F11 step, F12 finish; they keep working). <leader>dv (eval) is nvim-gdb's
```

```
key_eval, see plugin_specs.lua.
  local function gdb(cmd)
    return function()
      local ok = pcall(vim.cmd, cmd)
      if not ok then
        vim.notify("pdb: no debug session here (start one with <Space>dp)",
vim.log.levels.WARN)
      end
    end
  end
  for lhs, spec in pairs({
    ["<leader>dc"] = { "GdbContinue", "pdb: continue" },
    ["<leader>dn"] = { "GdbNext", "pdb: next line (step over)" },
    ["<leader>ds"] = { "GdbStep", "pdb: step into" },
    ["<leader>df"] = { "GdbFinish", "pdb: finish (run until the function returns)" },
    ["<leader>dB"] = { "GdbBreakpointToggle", "pdb: toggle breakpoint on this line" },
    ["<leader>du"] = { "GdbUntil", "pdb: run until this line" },
  }) do
    vim.keymap.set("n", lhs, gdb(spec[1]), { buffer = true, desc = spec[2] })
  end
else
  vim.keymap.set("n", "<leader>dp", function()
    vim.notify("<leader>dp: nvim-gdb is not available on this platform",
vim.log.levels.WARN)
  end, { buffer = true, desc = "start pdb (needs nvim-gdb)" })
end
```

In plain words:

- `enabled` limits nvim-gdb to Windows and Linux. On macOS the `:GdbStartPDB` command does not exist, so `<Space>dp` shows one warning instead.
- `cmd = { ... }`: the plugin loads on the first `:GdbStart*` command. `build = { "bash install.sh" }` compiles its helper when it is installed.
- Three overrides protect your own keys: `nvimgdb_disable_start_keymaps` stops nvim-gdb from creating its global `<Space>dd/dl/dp/db/dr` (they would replace your `<Space>dd` and `<Space>db`); `key_eval = "<space>dv"` moves its evaluate key off `<F9>` (it used to remove your run key at the end of a session); the `config` function deletes the global command-line `<C-e>` map nvim-gdb adds, so the builtin `<C-e>` keeps working.
- `<Space>dp` runs `:GdbStartPDB python -m pdb %` (plain python, never `uv run`). The `<Space>dc/dn/ds/df/dB/du` keys are thin wrappers: they run the matching `:Gdb...` command and turn the error you would get outside a debug session into one warning. The full key table is in "Debugging with pdb (nvim-gdb)" below.
- There is no debugpy and no nvim-dap setup for Python; nvim-dap is installed only as a dependency of nvim-java.

Running code

Two ways, for different purposes.

Key	Command it runs	Where output goes
`<Space>rf` / `<F9>`	`python -u "<file>"`, or `uv run python -u "<file>"` in a uv project (see below)	Quickfix window, 6 lines tall at the bottom (AsyncRun opens it by itself). The last line says `[Finished in N seconds]` on success and `[Finished in N seconds with code C]` when the exit code C is not 0
`<Space>rr`	`python3 <file>` (shell-escaped)	A new terminal in a vertical split on the LEFT of your code, titled like `term://...:python3 'file.py'`; Claude's panel stays on the right. When the program ends it shows `[Process exited 0]`

Facts that differ between the two:

- `<Space>rf` and `<F9>` are **buffer-local** and the same command: it exists only in Python buffers. `-u` means unbuffered, so `print` output appears while the program runs.
- `<Space>rr` is global and detects the file type. For Python it is always plain `python3 <file>`: **it never uses `uv run`, and has no `-u`**. In a uv project without an active environment use `<Space>rf`.
- Both run the file as saved on disk. Save first (`:w`).
- `<Space>rr` on an unnamed buffer shows one warning (`save the file first`).
- Paths with spaces or special characters are safe in `<Space>rr` (the name is shell-escaped). `<Space>rf` / `<F9>` wraps the name in double quotes, so a double quote or `$` inside a file name would break it.

Example to try both. Save as `hello.py`:

```
import sys, os

def add(a: int, b: int) -> int:
    return a + b

print(add(1, 2))
print(sys.version_info[:2], os.environ.get("VIRTUAL_ENV"), sys.executable)
```

Press `<Space>rf` (or `<F9>`). Expect in the quickfix window: `3`, then the Python version, the venv path (or `None`) and the interpreter path, then `[Finished in 0 seconds]`. A program that crashes ends with `code 1` and the traceback, as in the example `add(1, "two")`:

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
[Finished in 0 seconds with code 1]
```

When `<Space>rf` / `<F9>` uses uv

The choice is made when the Python file is opened (`after/ftplugin/python.lua`), from the project root (nearest `.git` or `pyproject.toml`). Abridged here (`...` stands for the options); the full code is in "How Python is set up (the real code)" above:

```
local py_env = utils.get_py_env()
local py_cmd = (py_env == "uv") and "uv run python" or "python"
vim.keymap.set("n", "<F9>", string.format(':<C-U>AsyncRun %s -u "%%"<CR>', py_cmd), ...)
```

Situation	`<Space>rf` / `<F9>` runs
No project root (no `.git`, no `pyproject.toml`)	`python -u`
An environment is active (`$VIRTUAL_ENV` or `$CONDA_DEFAULT_ENV` set)	`python -u`
`uv.lock` in the project root and no active environment	`uv run python -u` (quickfix title `:AsyncRun uv run python -u "hello.py"`)
Project root without `uv.lock`, no active environment	`python -u`

In a folder with `pyproject.toml` + `uv.lock` and a plain shell, the quickfix title showed `uv run python -u`; inside the python devShell (where `$VIRTUAL_ENV` is the project's `.venv`) it showed plain `python -u`.

Because the check runs at file open, activate the environment (or create `uv.lock`) before opening the file, or reload with `:e!`.

Create a uv test project like this:

```
mkdir uvp && cd uvp
git init
uv init --bare        # creates pyproject.toml
uv lock               # creates uv.lock
nvim hello.py
```

Stopping a running program

Started with	How to stop
`<Space>rf` / `<F9>`	`:AsyncStop` (with a 60-second `time.sleep`: `ps` showed `python -u slow.py` before and nothing after; stronger kill: `:AsyncStop!`)
`<Space>rr`	In the terminal press `<Ctrl-c>`, or delete the terminal buffer with `:bd!` (`ps` showed `python3 slow.py` before `:bd!` and nothing after)

Warning: `<Space>q` on a terminal window only **closes the window**; the running program keeps running hidden. Stop it first. See section 55 and section 8 for terminal navigation.

Formatting and linting

Python is formatted by `ruff format`, run by conform.nvim (see Formatting (conform.nvim)).

Key / command	What it does
`:w`	Formats the file with `ruff format`, then saves it
`<Space>fm`	Formats the buffer in place (Visual mode: only the selection), async, as one undo step. The file is not saved (the buffer is modified, `●` in the tab)
`<Space>fo`	Toggles format on save for all buffers

Try it. Save as `messy.py`:

```
import os, sys
def f( a,b ):
  return a+b
print(f(1,2))
```

Press `<Space>fm`. Expect the buffer to become:

```
import os, sys


def f(a, b):
    return a + b


print(f(1, 2))
```

Save with `:w`: no warning. (`os` and `sys` stay; ruff still shows `E401` and `F401` hints. See "Code actions from ruff".)

Key points:

- **Format on save:** every explicit save (`:w`, `:x`, `ZZ`) formats first. Auto-saves (leaving the buffer, Neovim losing focus) never format.
- **Format check after save:** after every save of a Python file `ruff format --check --quiet <file>` runs in the background. If the file would still change (an auto-save, or format on save switched off with `<Space>fo` / `:FormatDisable`) you get the hint `<file>: file is not formatted (ruff)`. If ruff fails (for example a syntax error) you get `<file>: ruff could not check the file (syntax error?)` plus the first error line. Nothing is changed. If `ruff` is not on PATH the check is silent.
- **Without ruff:** formatting is skipped silently and the language server formats instead when one is attached. `:ConformInfo` shows what will run.
- **Line length:** the `colorcolumn` marker for Python is at 88 (default for other files: 100), the line length of ruff format. A line touching the marker is too long. The formatter does not wrap long strings or comments.
- **Indentation:** `tabstop`, `softtabstop`, `shiftwidth` = 4, `expandtab` on. No wrapping (`wrap` off, `sidescroll` 5, `sidescrolloff` 2).

Code actions from ruff

Put the cursor on the first line of `messy.py` and press `<Space>ca`. Result (with `import os, sys` on line 1) is a list of eight ruff actions (one "Remove unused import" per unused name and one "Disable for this line" per diagnostic, so the exact number depends on how many diagnostics the line has):

```
1. Ruff (E401): Split imports [ruff]
2. Ruff (F401): Remove unused import [ruff]
3. Ruff (F401): Remove unused import [ruff]
4. Ruff (E401): Disable for this line [ruff]
5. Ruff (F401): Disable for this line [ruff]
6. Ruff (F401): Disable for this line [ruff]
```

```
7. Ruff: Fix all auto-fixable problems [ruff]
8. Ruff: Organize imports [ruff]
```

Pick a number with `<CR>`. "Organize imports" sorts and groups the imports (pyright's own version is disabled in `after/lsp/pyright.lua` so there is no conflict). "Fix all" applies every automatic fix, such as removing unused imports.

Linting from the command line (`:compiler ruff`)

The ftplugin sets `vim.g.ruff_makeprg_params = ""` so that Neovim's built-in ruff compiler plugin works without `--preview`:

```
:compiler ruff
:make %
```

runs `ruff check --output-format=concise nav.py` and puts every finding in the quickfix list (then `:copen`, `]q`, `[q`; see section 26). Needs `ruff` on PATH (devShell).

LSP keys

Active in a Python buffer once pyright (and ruff) have attached. Maps are set per buffer and show ONE warning instead of falling through when no server supports the action.

Key	What it does	Server
`gd`	Go to definition (one result jumps, several open the location list)	pyright
`K`	Hover: type and docstring in a floating window (plain text format)	pyright
`<Space>rn`	Rename symbol everywhere	pyright
`<Space>ca`	Code action: quick fixes, organise imports, fix all	ruff (and pyright)
`<Space>gd` / `<Space>gr` / `<Space>gi`	Glance popups: definitions / references / implementations	pyright
`<Space>de` / `<Space>dE`	Next / previous error	diagnostics from pyright and ruff
`<Space>dd`	Show diagnostic detail under the cursor	any
`]d` / `[d`	Next / previous diagnostic (`]D` / `[D` last / first)	any
`<Space>db` / `<Space>dw`	Buffer / whole workspace diagnostics list	any
`<Space>dt`	Toggle diagnostics on and off	any
`<Space>qb` / `<Space>qw`	Put buffer / all diagnostics in the quickfix list	any
`<Ctrl-w>d`	Diagnostics under the cursor in a float	any
`<Space>fm`	Format the buffer (see above)	ruff format via conform.nvim

Commands:

Command	Use
`:LspAttached`	Popup with the servers attached to this buffer
`:LspInfo`	`checkhealth vim.lsp` (configs, enabled servers, problems)
`:LspRestart`	Restart the servers of this buffer (needed after changing a venv or installing a package in some cases)
`:LspStop` / `:LspStart [name]`	Stop / start; `:LspStart` explains a missing program
`:LspLog`	Open the LSP log file
`:LspInlayHints enable` / `disable`	Switch inlay hints globally (off by default)

Who does what:

Task	Server
Type errors, wrong arguments, unknown imports, hover, definitions, rename	pyright (`standard` mode; `deprecateTypingAliases` hint off; `useLibraryCodeForTypes` on)
Style and lint diagnostics, unused imports, fixes	ruff
Import sorting	ruff (`organizeImports = true`); pyright's is disabled
Spelling mistakes in names and comments	typos_lsp

After `:LspInlayHints enable` no inline hints appeared in a small Python file (pyright sends none for this code), so expect little or nothing in Python.

Ruff reads its rules from `pyproject.toml` / `ruff.toml` in the project. Pyright reads `pyrightconfig.json` or `[tool.pyright]` in `pyproject.toml`. See section 13 and section 44 for the LSP basics.

Debugging with pdb (nvim-gdb)

`<Space>dp` (Python buffers only) runs `:GdbStartPDB python -m pdb %` through the nvim-gdb plugin. Linux and Windows only; on macOS you get one warning. In any other file type `<Space>dp` shows one warning (`only in python buffers`). It always uses plain `python -m pdb`, never `uv run`.

Why pdb and not a "real" DAP debugger: nvim-dap is installed only for Java. For Python you use pdb (in the standard library, nothing to install) and nvim-gdb adds the visual part: it marks the current line in your code and gives you function keys and leader keys.

Example to try. Save as `dbg.py`:

```
def square(n):
    r = n * n
    return r


total = 0
for i in range(3):
    total += square(i)
print("total", total)
```

Press `<Space>dp` in this file. After the session starts the cursor is in the pdb terminal pane, so the `<Space>d*` keys are typed into pdb until you move back to the source window (`<Ctrl-\><Ctrl-n>` then `<Ctrl-w>k`). Result: the source window shows the file with a `▶` mark in the sign column on line 1, and below it a terminal pane shows

```
> .../dbg.py(1)<module>()
-> def square(n):
(Pdb)
```

Press `<Space>dn` (or `<F10>`) twice. The `▶` moves to line 6, then line 7, and the terminal shows `n` typed for you each time. `<Space>dc` (or `<F5>`) continues; this small program

finished and printed `total 5`, then pdb says `The program finished and will be restarted` and starts again at line 1.

Keys during the session. The `<Space>d` keys work in Python buffers without function keys; the F-keys are nvim-gdb's own. Outside a debug session the `<Space>d` keys show one warning `pdb: no debug session here (start one with <Space>dp)`. Run them from the code window (go there with `<Ctrl-\><Ctrl-n>` then `<Ctrl-w>k` if you are in the pdb pane):

Key	Action	Command
`<Space>dB` / `<F8>`	Toggle breakpoint on the current line (a `●` appears in the sign column)	`:GdbBreakpointToggle`
`<Space>dc` / `<F5>`	Continue (stops at the next breakpoint: `▶` lands on the `●` line)	`:GdbContinue`
`<Space>dn` / `<F10>`	Next (step over)	`:GdbNext`
`<Space>ds` / `<F11>`	Step (into a call): from `total += square(i)` the `▶` jumped to `def square(n):`	`:GdbStep`
`<Space>df` / `<F12>`	Finish (run until the function returns): the `▶` stopped on `return r`	`:GdbFinish`
`<Space>du` / `<F4>`	Until (pdb: continue until the next line greater than the current one)	`:GdbUntil`
`<Ctrl-p>` / `<Ctrl-n>`	Frame up / down (`<Ctrl-p>` typed `up` in pdb and the `▶` went to the caller line)	`:GdbFrameUp` / `:GdbFrameDown`
`<Space>dv`	Evaluate the word under the cursor (visual mode: the selection); with the cursor on `square` pdb printed `<function square at 0x...>`	`:GdbEvalWord` / `:GdbEvalRange`

Other commands: `:GdbBreakpointClearAll`, `:GdbFrame` (jump to the current line), `:GdbInterrupt`, `:GdbLopenBacktrace`, `:GdbLopenBreakpoints`, `:GdbCreateWatch`.

You can also type plain pdb commands in the terminal pane (`n`, `s`, `c`, `p var`, `l`, `bt`, `q`): go there with `<Ctrl-w>j`, press `i`, type, press `<Enter>`. Leave terminal mode with `<Ctrl-\><Ctrl-n>`.

Quit: `:GdbDebugStop`. The debug layout was in the same tab here (one tab, two windows during the session, one after). Closing the debug windows also ends it.

History (fixed): nvim-gdb used to bind `<F9>` itself (evaluate) and removed it from the code buffer after a session, which also removed your `<F9>` run key until `:e!`. Its evaluate key is now `<Space>dv`, so `<F9>`, `<Space>rf`, `<Space>f` and `<Space>dp` work before, during and after a debug session.

Notes:

- nvim-gdb's own evaluate key was `<F9>`; it is moved to `<Space>dv` (only while a debug session is active), and `<F9>` is no longer the eval key. `<F4>` `<F5>` `<F8>` `<F10>` `<F11>` `<F12>` still work.
- nvim-gdb loads the first time you use a `:GdbStart*` command. Its default start keys `<Space>dd/dl/dp/db/dr` are turned off (`vim.g.nvimgdb_disable_start_keymaps = true`) so

they do not replace your own `<Space>dd` and `<Space>db`; `<Space>dp` is yours, set in the ftplugin.
- A quick alternative without any plugin: add `breakpoint()` in the code and run it with `<Space>rr`; the terminal stops there with a `(Pdb)` prompt.
- Section 56 has the wider debugging picture.

Virtual environments

Statusline label. In Python buffers only, the statusline shows the environment (section B of the lualine bar, section 32):

Environment variable set	Label
`$VIRTUAL_ENV` (any venv, `.venv`, devShell's `venvShellHook`, uv `source .venv/bin/activate`)	`<folder name> (venv)`, for example `.venv (venv)`
only `$CONDA_DEFAULT_ENV`	`<env name> (conda)`
neither	nothing shown

`$VIRTUAL_ENV` wins when both are set. In a uv project with `uv.lock` and no active environment the label stays empty, even though `uv run` will use the project's `.venv`. In the python devShell: the label read `.venv (venv)` (the folder name of `$VIRTUAL_ENV`, with its dot).

How to activate an environment.

Goal	Do this
Use the devShell tools and the project's `.venv`	Enter the project folder (direnv loads the devShell and the `venvShellHook` `.venv`), then start nvim there
Plain venv	`python -m venv .venv`, `source .venv/bin/activate` (fish: `source .venv/bin/activate.fish`), then start nvim in that shell
uv project	Either `source .venv/bin/activate` before starting nvim, or leave it inactive and use `<Space>rf` / `<F9>`, which run through `uv run`
Switch environment	Close nvim, change environment, start nvim again (the shell environment is inherited at start)

How pyright finds packages. In a uv project (`.venv` with packages) opened from the python devShell (whose own `python` lacks them), `import pytokens` gave the error `Import "pytokens" could not be resolved`. After adding a `pyrightconfig.json` with `{"venvPath": ".", "venv": ".venv"}` the error was gone. So the `.venv` folder alone is not enough: pyright uses the `python` on PATH, unless `venvPath` / `venv` say otherwise. Background: Pyright runs as a child of nvim and uses the `python` on PATH. If `VIRTUAL_ENV` was set when nvim started, it resolves imports from that environment. With `autoSearchPaths` it also searches `src/`. To point pyright explicitly, put `venvPath` and `venv` in `pyrightconfig.json` or `[tool.pyright]` in `pyproject.toml`, then `:LspRestart`.

Testing

There is **no test runner support** for Python in this config: no neotest, no vim-test, no pytest or unittest keys. What you can do:

Goal	How
Run all tests	In the terminal split: `:terminal pytest`, or run `pytest` in a shell next to nvim (pytest comes from the project environment; it is not in the python devShell list)
Run a test file	`<Space>rf` (or `<F9>`) or `<Space>rr` on a file that ends with `unittest.main()`, or `:!pytest %`
Debug a failing test	Put `breakpoint()` in the test and run `<Space>rr`, or `pytest --pdb` in a terminal
See failures in quickfix	`:AsyncRun pytest -q` (output opens in the 6-line quickfix window, section 26)

The only plugin that knows about tests is nvim-java (Java only, `<Space>jt...`).

Navigation and text objects

All keys below exist in the keymap dump for Python buffers (or globally).

Key	Mode	What it does
`]]` / `[[`	n, x, o	Next / previous top-level `class` or `def` (Neovim's runtime Python ftplugin; not mapped by your config)
`][` / `[]`	n, x, o	End of the next / previous top-level function or class
`]m` / `[m`	n, x, o	Next / previous `def` or `class`, any nesting (method start)
`]M` / `[M`	n, x, o	Next / previous end of a method
`<Space>t`	n	Toggle the aerial outline (classes, functions, methods)
`]t` / `[t`	n	Next / previous symbol (aerial)
`gS`	n	treesj: toggle split / join of the construct under the cursor (a long call or list to one item per line, and back)
`ii` / `ai`	x, o	Indent scope inside / with its border lines (mini.indentscope): `dii` deletes the body of the block you are in, `vai` selects a whole `if` or `def`
`za`, `zc`, `zo`, `zR`, `zM`, `zr`, `zm`	n	Folds (nvim-ufo: LSP folding ranges, else indent); `<Space>K` previews a closed fold
`gd` then `<Ctrl-o>`	n	Jump to a definition and back
`<Space>gr`	n	References in a Glance popup

Operators combine with the motions: `d]]` deletes up to the next top-level definition, `v]m` selects up to the next method. `ii` depends on indentation, so it works well in Python. Section 21, 37, 47 and 50 have more.

Snippets

Source: `my_snippets/python.snippets` (UltiSnips, 4 snippets) and the shared vim-snippets collection. Type the trigger in insert mode and press `<Ctrl-j>` to expand; `<Ctrl-j>` / `<Ctrl-k>` jump to the next / previous placeholder. The text in quotes after each trigger is its description as the completion menu shows it. "Start of line" snippets expand only at the beginning of a line. Placeholders are shown in tab-stop order.

Trigger	Description	Start of line only
`print`	Print a text with the value of a variable (f-string: the variable name goes inside the braces)	no
`impa`	Import a module under an alias (import FOO as BAR, start of line)	yes
`main`	Main function plus the if **name** == "**main**" guard (start of line)	yes
`sol`	LeetCode style: create a Solution object (needs a class Solution; start of line)	yes

`print`: an f-string print. Type the text, jump, type the name of the variable; it is written inside the braces, so its value appears in the output.

```
print(f"text {variable}")
```

Example: `text` = `total:`, `variable` = `count` gives `print(f"total: {count}")`, which prints `total: 3` when `count` is 3.

`impa`: import under an alias. Both placeholders are upper case words to overwrite.

```
import FOO as BAR
```

Example: `FOO` = `numpy`, `BAR` = `np`.

`main`: the usual entry point. The cursor starts on the `# code` placeholder inside `main`.

```
def main():
    # code


if __name__ == "__main__":
    main()
```

`sol`: the object that coding-challenge sites expect; it needs a `class Solution` in the file.

```
solution = Solution()
```

The snippet menu may also offer vim-snippets entries (`def`, `class`, `ifmain`, ...). Section 15 and section 52 explain the engine.

Jupyter notebooks (nvjup)

Why: a `.ipynb` file is JSON; nvjup lets you edit and run it as cells inside Neovim without converting it.

How: opening an `.ipynb` file shows it as a text view where every cell starts with a marker line (`# %%[nvjup:cell id=... type=code]`), with Tree-sitter colours, outputs under the cells, and a Python process (the "sidecar") that talks to the Jupyter kernel. The file type is `nvjup` and the buffer is not an ordinary file buffer: `:w` writes the notebook (including the outputs and the execution counts). Other plugins of this config stay out of it: no LSP server of this config attaches (nvjup runs its own pyright and ruff for the cells), gitsigns does not attach, auto-save does not write the file and the conform on-save format skips it. A 2.5 MB notebook opened normally.

Config (from `lua/plugin_specs.lua`, shortened):

```
{
  "LumbaBalumba/nvjup",
  lazy = false, -- it must register its BufReadCmd for *.ipynb before such a file is opened
  build = function(plugin) ... end, -- uv sync --frozen in the plugin folder, fail-soft
  opts = {
    keymaps = {
      lsp_type_definition = "gy",
      lsp_rename = "<leader>rn",
      lsp_references = "grr",
      cursor_up_alt = false, cursor_down_alt = false,
      cursor_left_alt = false, cursor_right_alt = false,
    },
  },
},
```

Requirements

What	Where it comes from
`uv`	Global on Neovim's PATH after the next rebuild of the nix configuration (`neovim.nix`). Needed for the build
A Python that can run binary wheels	On nix hosts the build uses uv's own standalone Python 3.13 (not the nix-built Python, which cannot load wheels such as pyzmq: libstdc++ is missing). That needs `nix-ld` with libstdc++, enabled on both NixOS hosts, and internet once, at the first build, to download that Python
`ipykernel`	The kernel's Python needs it: the jupyter devShell has it, or a project `.venv` / `venv` that contains it (uv project: `uv add --dev ipykernel`)
pyright, ruff	Global (nvjup's own LSP for the cells)
chafa, ffmpeg, imagemagick, librsvg, `NVJUP_CHROMIUM`	The jupyter devShell: text-mode images, animations, LaTeX and SVG pictures, interactive Plotly/Bokeh charts
`pdflatex`	NOT provided: LaTeX formulas inside Markdown cells are not drawn as pictures
Awrit, `KITTY_LISTEN_ON`, `google-colab-cli`	Not available (optional): `<Space>nF` (external chart window) and Colab connections do not work

The build never fails hard: without `uv` it warns once ("nvjup: `uv` is not on Neovim's PATH …") and skips; with `uv` it runs `uv sync --frozen` in the plugin folder. Run it by hand with `:Lazy build nvjup` (after a fresh install, after the nix rebuild, or when it printed a warning). `:checkhealth nvjup` lists what is found; with this setup it prints three harmless warnings (no `pdflatex`, no Awrit, no `KITTY_LISTEN_ON`).

Quick start

1. Once: `:Lazy build nvjup` (needs `uv` and internet, see above). Check with `:checkhealth nvjup`.
2. Start Neovim from a jupyter devShell project (a folder whose `.envrc` has `use_dev_env jupyter`, then `direnv allow`) so that `ipykernel` is found: `nvim analysis.ipynb`.
3. Move with `j` / `k` (they cross from one cell into the next) and `]c` / `[c` (next / previous cell).
4. `<Space>nr` runs the current cell and moves to the next one (`<S-CR>` does the same where your terminal sends it); `<C-CR>` runs the current cell only. The output appears under the cell.
5. `<Space>nb` inserts a cell below, `<Space>nd` deletes the current cell, `<Space>nt` cycles its type (code, Markdown, raw).
6. `:w` saves the notebook with the outputs.

`<C-CR>` and `<S-CR>` often do not reach Neovim through tmux or kitty; `<Space>nr` is the reliable key. Running only the current cell without advancing has no `<Space>` key: use `<C-CR>` or `:NvJupRunCurrent`.

Keys (notebook buffers only)

All of nvjup's 71 keymaps are buffer-local: they exist only in a notebook buffer and every one has a description (`:map` or which-key shows them). `<Space>` is the leader and `\` the local leader.

Cells and movement:

Keymap	Mode	What it does
`j` / `k`	n	Move down / up inside the cell; at the edge, cross into the next / previous cell (instead of the global display-line movement)
`h` / `l`	n	Move left / right within the current cell
`]c` / `[c`	n, x, o	Next / previous cell
`]C` / `[C`	n, x, o	Next / previous code cell
`ic` / `ac`	x, o	Inner / around a cell
`<Space>na` / `<Space>nb`	n	Insert a cell above / below
`<Space>nd`	n	Delete the cell
`<Space>nj` / `<Space>nk`	n	Move the cell down / up
`<Space>nq`	n	Split the cell at the cursor
`<Space>nM`	n	Merge the cell with the one below
`<Space>nyy`	n	Duplicate the cell
`<Space>nt`	n	Change the cell type (cycle code, Markdown, raw)
`<Space>nm` / `<Space>ny`	n	Convert to a Markdown cell / to a code cell
`<Space>nz`	n	Collapse or expand the cell source (a Markdown cell shows rendered or as source)
`<Space>nl`	n	Notebook outline (Telescope list of cells)
`<Space>/`	n, x	Comment the line / the selected lines using the cell's language
`<Space>fm`	n	Format the current cell (conform)
`<Space>n`	n	Notebook actions

Running and the kernel:

Keymap	What it does	Command
`<C-CR>`	Run the current cell (normal and insert mode)	`:NvJupRunCurrent`
`<S-CR>` / `<Space>nr`	Run the current cell and advance	`:NvJupRunAndAdvance`
`<Space>nA` / `<Space>nB`	Run the cells above / below	`:NvJupRunAbove` / `:NvJupRunBelow`
`<Space>nR`	Run all cells	`:NvJupRunAll`
`<Space>ns`	Start the kernel	
`<Space>ni`	Interrupt the running cell	`:NvJupKernelInterrupt`
`<Space>nx`	Restart the kernel	`:NvJupKernelRestart` (also `:NvJupKernelRestartRunAll`, `:NvJupKernelShutdown`, `:NvJupKernelStatus`)
`<Space>nS`	Stop the kernel	
`<Space>nv`	Inspect the variables of the live kernel	`:NvJupVariables`

Outputs:

Keymap	What it does	Command
`<Space>nc` / `<Space>nC`	Clear the output of this cell / of all cells	`:NvJupCellClearOutput` / `:NvJupClearAllOutputs`
`<Space>no`	Expand or collapse a long output	`:NvJupCellToggleOutput`
`<Space>np`	Open the full output of the cell	`:NvJupOutputOpen`
`<Space>nL`	Refresh the display	`:NvJupRefresh`
`<Space>nf`	Open an interactive chart (Plotly/Bokeh) in the Neovim view	`:NvJupPlotFocusTui`
`<Space>nF`	Same in an external Awrit window (not available here)	`:NvJupPlotFocus`

Interactive charts and animations need trust first: `:NvJupTrustInteractive` trusts the notebook's active content; `:NvJupTrustRevoke` takes it back and `:NvJupTrustStatus` shows the state. Output of a cell you ran yourself on a local kernel is trusted automatically, and notebook HTML and JavaScript are never executed.

Language server (nvjup's own pyright and ruff for the cells, across all code cells):

Keymap	What it does
`K`	Hover
`gd` / `gD` / `gi`	Definition / declaration / implementation
`gy`	Type definition
`grr`	References
`<Space>rn`	Rename
`<Space>ca`	Code action (n, x)
`<Space>ls`	Signature help
`\ls`	Document symbols
`<C-Space>`	LSP completion (insert mode)

Remote Jupyter: `<Space>nK` (`:NvJupRemoteConnect`) connects to a remote Jupyter server and `<Space>ne` (`:NvJupRemoteFiles`) opens the local/remote file manager.

What differs from the rest of the config

nvjup's own default keys were changed on purpose:

- Rename is `<Space>rn` and references is `grr`, the same keys as everywhere else (nvjup's default was `gr`, which would make Neovim wait for the longer built-in `gr...` keys).
- Type definition is `gy` (nvjup's default `<Space>D` would be a prefix of the `<Space>D...` database keys and make them wait).
- The arrow keys are not remapped, so they keep moving between windows (`<Left>` / `<Right>` / `<Up>` / `<Down>`); `h` `j` `k` `l` are cell-aware.

Inside notebook buffers only, these global keys are replaced by notebook-aware versions: `<Space>ca`, `<Space>rn`, `<Space>fm` and `grr` (same meaning), `j` / `k` (cell-aware movement), and `<Space>ls`: in a notebook it is LSP signature help, not the truth-table "swap operands" key (Truth tables).

Troubleshooting

Symptom	Likely cause	Fix
Nothing highlights errors, `:LspAttached` shows no `pyright`	nvim was not started in a shell where `pyright-langserver` is on PATH	Check `which pyright-langserver`; start nvim in the python devShell or after `direnv allow`; `:LspStart pyright` names the missing program
No `ruff` diagnostics	`ruff` is not on PATH of the shell nvim was started from	Check `which ruff`; start nvim from a shell where it is on PATH (global install, or the python or jupyter devShell)
`Import "xyz" could not be resolved`	Package is not in the Python that pyright sees (no active venv when nvim started, or packages not installed)	Activate the environment BEFORE starting nvim, install the package, then `:LspRestart`; or set `venvPath` / `venv` in `pyrightconfig.json`
Statusline shows no environment	No `$VIRTUAL_ENV` or `$CONDA_DEFAULT_ENV`, or it is a uv project that is not activated	Activate the environment before starting nvim; label only appears in Python buffers
No "not formatted" warning after save, although the file is untidy	`ruff` is not on PATH (check is silent), the save already formatted the file, or the file type is not detected as `python`	`:!which ruff`; `:set ft?`
`<Space>rf` / `<F9>` prints `ModuleNotFoundError`	Wrong interpreter: plain `python` was used (environment not active when the file was opened)	Activate the environment, reopen the file with `:e` so the uv/venv choice is re-evaluated
`<Space>rf` / `<F9>` does nothing visible	Quickfix window closed or scrolled, or `AsyncRun` not loaded	`:copen`; check `:AsyncRun echo hi` works
`<Space>rr` opens nothing	Unnamed buffer (one warning: save the file first) or an unsupported file type	`:w file.py`; check `:set ft?`
`<Space>rr` terminal flashes and closes, or shows `command not found`	`python3` is not on PATH	Start nvim in the devShell
`<Space>rr` output is late	Python buffers stdout when not attached to a tty	Use `<Space>rf` (it uses `-u`) or `print(..., flush=True)`
`<Space>dp` shows one warning	Not a Python buffer, or macOS	Open a `.py` file; on macOS pdb is not available through this key
Pyright hover looks plain	Hover is set to plain text on purpose	None needed
Program keeps running after closing its terminal	`<Space>q` only closes the window	Reopen the buffer with `:ls`, `:b <n>`, `<Ctrl-c>`, or `:bd!`
Opening a .ipynb shows raw JSON	The kernel's Python did not have `ipykernel` on PATH, or no internet at that moment)	Add `ipykernel` to the project's `.venv`
Spelling underlines words that are correct	Type not in the dictionary	Add to `typos.toml` or `_typos.toml`
Starting a cell fails with a missing-kernel error	`ipykernel` or `uv` is not installed	Install `ipykernel` in the project's `.venv`
`:checkhealth nvjup` complains about the sidecar	Neovim's Python is not the one with `pynvim`	Add `pynvim` to the project or to `uv tool install neovim`

Related sections

Section 8 (terminal integration), section 13 (LSP), section 15 (snippets), section 18 (folding), section 19 (code running), section 21 (text objects), section 26 (quickfix), section 32 (statusline), section 36 (debugging), section 37 (aerial), section 41 (file-type settings), section 42 (automatic behaviours), section 43 (toolchain), section 44 (LSP in depth), section 52 (snippets for developers), section 55 (running in depth), section 56 (debugging in depth).

Chapter 15: LaTeX

80. LaTeX (vimtex, texlab, ltex, PDF viewer)

This section covers everything that is specific to LaTeX in this config: what each tool is for, how it works, the keys, and what to do when something fails. Global things (LSP keys, completion menu, windows, spell checking) are only mentioned with a pointer to their own section. Typst and Markdown have their own sections.

The big picture

Writing LaTeX means: a `.tex` source file, a compiler (`latexmk` runs `pdflatex` as many times as needed and runs BibTeX/biber), and a PDF viewer. Each tool in the config covers one part:

Tool	Why it is in the setup	What you use it for
vimtex (plugin)	Knows LaTeX syntax and the compile chain	Compile (`<Space>rf`), view PDF, jump between sections/environments, text objects, change/delete/toggle environments and commands, table of contents, error list
texlab (language server)	Understands the whole project (labels, citations, includes)	Completion of `\ref{` / `\cite{` / commands, diagnostics, hover (`K`), symbols, rename, `:LspTexlabBuild`
ltex_plus (language server)	Grammar and spelling for prose	Underlines mistakes in the text (not in commands) as diagnostics
typos_lsp (language server)	Catches common typos in any file	Same diagnostics channel; it attaches to every file type
UltiSnips snippets	Typing shortcuts	`use`, `eqa`, and 191 math snippets (`mk`, `dm`, symbol names such as `leq`) (file `my_snippets/tex.snippets`)
zathura (PDF viewer)	Light viewer that reloads when the PDF changes and supports forward/inverse search	Shows the result
LaTeX devShell	Provides all the programs	`latex`, `latexmk`, `texlab`, `zathura`, `pandoc`

In a `.tex` buffer the config also sets `textwidth = 120`, `wrap` on and a column marker at 120 (`after/ftplugin/tex.lua`, `lua/options.lua`). The statusline shows `texlab (+2)` when texlab, ltex_plus and typos_lsp are attached.

What it needs (the devShell)

vimtex is **enabled only when `latex` is on PATH**:

The real spec head (`lua/plugin_specs.lua`; the `init` function is shown in pieces under "How vimtex is set up" below):

```
-- LaTeX support: loaded on every platform whenever `latex` is on PATH (on Linux via the
LaTeX devShell)
{
  "lervag/vimtex",
  enabled = function()
    return utils.executable("latex")
  end,
  -- not lazy on purpose: the PDF viewer's Ctrl+click starts a separate headless nvim (no
tex file) that
  -- needs the :VimtexInverseSearch command, which only exists once vimtex is loaded
  lazy = false,
  init = function()
```

In plain words:

- `enabled`: the plugin only exists when `latex` is on PATH, as described above.
- `lazy = false`: vimtex is **not** lazy-loaded on `ft = "tex"`. The PDF viewer's Ctrl+click starts a separate headless Neovim with no `tex` file, and that Neovim needs the `:VimtexInverseSearch` command, which only exists once vimtex is loaded (see "The viewer").
- `init` runs before the plugin loads and sets its options (the next section).

Outside a LaTeX environment the plugin does not load: no `<Space>rf`, no `<F9>`, no `\ll`, no `:Vimtex*` commands. The environment comes from the flake `~/nix/templates/krit/dev-environments/language-specific/latex/flake.nix`. Its packages: `texlive.combined.scheme-full` (gives `latex`, `latexmk`, all packages), `texlab`, `zathura`, `pandoc`, `tectonic`, `latex2html`, `latex2mathml`.

How you enter it	Details
direnv (normal)	The project folder has an `.envrc` with one line: `use_dev_env latex` (see `~/github-repos/personal/developing-projects/latex-projects/.envrc`). Run `direnv allow` once. Then `cd` in and start `nvim` there.
By hand	`nix develop <path-to-flake>` and then `nvim`

Check inside the folder: `which latex latexmk texlab zathura` must print four paths.

Option 1: `:DevEnv latex` in the running Neovim. Type `:DevEnv latex` then `<CR>`. It evaluates the LaTeX flake in the background (about 10 seconds the first time, then cached), adds its programs to `PATH` for this session, enables texlab, loads vimtex and replays the file type for open `.tex` buffers. A notification `DevEnv: latex ready: ...` lists what started. Opening a `.tex` file without `latex` shows a one-time hint, `latex not found on PATH: run :DevEnv latex to enter its devShell`. Details and the list of all devShells: section 43. It affects this session only.

Option 2: start Neovim from inside the devShell. The tools are looked up when Neovim starts; entering the shell with direnv or `nix develop` afterwards does not enable vimtex in an already running Neovim (use Option 1 for that).

Outside the devShell a `.tex` file still gets syntax colours, `ltex_plus`, `typos_lsp`, snippets and buffer/path completion. It gets no vimtex, no texlab, no PDF.

How vimtex is set up (the real code)

All of it is the `init` function of the vimtex spec in `lua/plugin_specs.lua`, quoted verbatim in four pieces plus a short closing block (the spec head is shown in "What it needs"). Each piece is dedented to its own left margin (the original is indented 10 spaces inside `vim.cmd`); only the indentation differs from the file. The VimL part sits inside `vim.cmd([[ ... ]])` and only runs when `latex` is on PATH.

1. Viewer choice and inverse-search helper

```
vim.g.vimtex_view_method = (utils.executable("zathura") and "zathura") or "general"
if utils.executable("zathura") then
  -- always light, whatever the host's zathura theme is (lua/zathura_light.lua)
  vim.g.vimtex_view_zathura_options = "--config-dir=" ..
vim.fn.shellescape(require("zathura_light").config_dir())
end
vim.cmd([[
  if executable('latex')
    " Hacks for inverse search to work semi-automatically,
    function! s:write_server_name() abort
      let nvim_server_file = (has('win32') ? $TEMP : '/tmp') . '/vimtexserver.txt'
      call writefile([v:servername], nvim_server_file)
    endfunction

    augroup vimtex_common
      autocmd!
      autocmd FileType tex call s:write_server_name()
      " buffer-local like the old nmap, via Lua so the map can carry a desc
      autocmd FileType tex lua for _, lhs in ipairs({ "<F9>", "<leader>rf" }) do
vim.keymap.set("n", lhs, "<Plug>(vimtex-compile)", { buffer = true, remap = true, desc =
"LaTeX: start/stop compiling (vimtex)" }) end
    augroup END
```

In plain words:

- `vimtex_view_method` is `zathura` when zathura is installed, otherwise `general` (the system default PDF program). With zathura, `vimtex_view_zathura_options` starts it with a config (in `~/.cache/nvim/zathura-light`) that includes your own zathurarc and overrides only the colors to Catppuccin Latte, so the PDF always opens light even when your desktop and zathura are dark.
- `s:write_server_name` writes Neovim's address (`v:servername`) into `/tmp/vimtexserver.txt` (`%TEMP%` on Windows) every time a `tex` file is opened. The viewer's inverse search (Ctrl+click) uses it to find the right Neovim; see "The viewer".
- The autocommand sets the compile key twice, buffer-local, for `tex` files only: `<F9>` and `<leader>rf` both map to `<Plug>(vimtex-compile)`. `remap = true` is required because the target is a `<Plug>` mapping; it is set from Lua so the key can carry a `desc`.

2. latexmk build directory and the table of contents

```
let g:vimtex_compiler_latexmk = {
      \ 'build_dir' : 'build',
      \ }

" TOC settings
let g:vimtex_toc_config = {
      \ 'name' : 'TOC',
      \ 'layers' : ['content', 'todo', 'include'],
      \ 'resize' : 1,
```

```
      \ 'split_width' : 30,
      \ 'todo_sorted' : 0,
      \ 'show_help' : 1,
      \ 'show_numbers' : 1,
      \ 'mode' : 2,
      \ }
```

In plain words:

- `build_dir = 'build'` is in the config but has no effect: see "Where the output goes" under "Compiling". vimtex's current option is called `out_dir`.
- `g:vimtex_toc_config` configures the `\lt` / `\lT` table of contents (section "Table of contents"): `layers` = what is listed (document content, todo comments, included files); `split_width = 30` = a 30-column window; `resize = 1` = Vim is resized automatically when that vertical window opens (vimtex default 0); `mode = 2` = separate window **and** a location list (1 = window only, 3/4 = location list only); `show_numbers = 1` = section numbers; `show_help = 1` = the key hint lines at the top; `todo_sorted = 0` = todo entries stay in file order. Meanings checked against vimtex's `:help vimtex-toc` upstream text.

3. Viewers on Windows and macOS

```
" Viewer settings for different platforms
if g:is_win
  let g:vimtex_view_general_viewer = 'SumatraPDF'
  let g:vimtex_view_general_options = '-reuse-instance -forward-search @tex @line @pdf'
endif

if g:is_mac
  " let g:vimtex_view_method = "skim"
  let g:vimtex_view_general_viewer = '/Applications/Skim.app/Contents/SharedSupport/
displayline'
  let g:vimtex_view_general_options = '-r @line @pdf @tex'

  augroup vimtex_mac
    autocmd!
    autocmd User VimtexEventCompileSuccess call UpdateSkim()
  augroup END

  " The following code is adapted from https://gist.github.com/skulumani/7ea00478c63193a832a
6d3f2e661a536.
  function! UpdateSkim() abort
    let l:out = b:vimtex.out()
    let l:src_file_path = expand('%:p')
    let l:cmd = [g:vimtex_view_general_viewer, '-r']

    if !empty(system('pgrep Skim'))
      call extend(l:cmd, ['-g'])
    endif

    call jobstart(l:cmd + [line('.'), l:out, l:src_file_path])
  endfunction
endif
```

In plain words:

- Windows (`g:is_win`): SumatraPDF with `-reuse-instance -forward-search @tex @line @pdf` (one window, jumps to the cursor line).

- macOS (`g:is_mac`): Skim through its `displayline` helper (`-r @line @pdf @tex`). `UpdateSkim()` runs after every successful compile (`VimtexEventCompileSuccess`) and re-opens or refreshes Skim at the cursor line; `-g` keeps Skim in the background when it is already running. The function is adapted from the public gist named in the comment. The commented `vimtex_view_method = "skim"` line is a leftover.
- On Linux none of this runs; zathura from piece 1 is used.
- The last four lines (shown below) close the `if executable('latex')`, the `vim.cmd`, the `init` function and the spec.

Closing lines of the spec:

```
      endif
    ]])
  end,
},
```

One more cross-plugin setting belongs to vimtex: `vim.g.matchup_override_vimtex = 1` in the `vim-matchup` spec (same file). It lets vim-matchup take over `%` matching in `tex` files instead of vimtex's own implementation.

Quick start

1. `cd` into the project folder (direnv loads the devShell).
2. Create and open `main.tex`:

```
\documentclass{article}
\begin{document}
\section{Test}
Hello \LaTeX.
\end{document}
```

3. Wait until the statusline shows `texlab (+2)`. `:LspAttached` lists the clients.
4. `<Space>rf` (or `<F9>`) starts continuous compiling. After the first successful compile **zathura opens by itself** with the PDF (see "The viewer").
5. Edit, then `:w`. The compiler notices the saved file, recompiles, zathura reloads. (Auto-save never saves `.tex`; save yourself.)
6. `\lv` shows or re-opens the PDF at the cursor position.
7. `<Space>rf` / `<F9>` again stops compiling; `\lc` removes the auxiliary files.

Compiling

`<Space>rf` and `<F9>` (buffer-local, set by an autocommand for `tex` files; description "LaTeX: start/stop compiling (vimtex)") and `\ll` are the same command, `<Plug>(vimtex-compile)`. It starts `latexmk` in the background in **continuous mode** (recompiles whenever a source file changes); pressing it again stops it. `\` is the local leader: the config sets only `mapleader = <Space>`, so vimtex keeps the backslash.

Where the output goes. `main.pdf`, `main.aux`, `main.log`, `main.bbl` and the other files are written **next to** `main.tex`, not into a `build/` folder. The config contains

```
let g:vimtex_compiler_latexmk = { 'build_dir' : 'build' }
```

but the installed vimtex has no `build_dir` option (its documentation does not mention it; the current name is `out_dir`), so this line has no effect. `:LspTexlabBuild` (texlab's own

one-shot build) also writes `main.pdf` next to the source. If you want a `build/` folder, the option to set is `out_dir`.

Command / key	What it does
`<Space>rf`, `<F9>` or `\ll` (`:VimtexCompile`)	Start or stop continuous compiling
`\lk` (`:VimtexStop`)	Stop. Message "VimTeX: Compiler stopped (name.tex)"
`\lc` (`:VimtexClean`)	Remove auxiliary files; keeps `main.pdf`, `main.bbl`, `main.synctex.gz`. Message "VimTeX: Compiler clean finished"
`\lC`	Clean everything including the PDF
`\lo`	Show the compiler output
`\lg` / `\lG`	Status of this / all compilers
`\lL`	Compile only the selected lines (visual mode too)
`\lq`	Show the vimtex log
`\li` / `\lI`	Info about the vimtex state of this document (short / full)
`\lx` / `\lX`	Reload vimtex / reload its state (use after changing the document structure)
`\ls`	Toggle the main file (`:VimtexToggleMain`)
`\la`	Context menu for the item under the cursor
`:LspTexlabBuild`	texlab's own build, once (produces `main.pdf`). For daily work use `<Space>rf`: it keeps recompiling.

Each of the `\l...` keys above points at the `<Plug>(vimtex-...)` map named in the vimtex documentation.

Errors

When a compile fails vimtex opens the **quickfix list** by itself, titled "VimTeX errors (LaTeX logfile)", and shows "VimTeX: Compilation failed!" (with a file containing `\foobar`; the entry was `Undefined control sequence. \foobar`, with the line number). texlab also puts the problem in the text as a diagnostic (`]d` / `[d`, `<Space>dd`; see the LSP section).

Key / command	What it does
`\le` (`:VimtexErrors`)	Open the error list again
`:cnext` / `:cprev`	Next / previous error; `:cclose` closes the list
`\lo`	Read the raw compiler output

If an error stops the compile, fix it, `:w`: the continuous compiler retries by itself (it does not have to be restarted).

Common messages: "Undefined control sequence" (a command is misspelled or its `\usepackage` is missing), "File `x.sty' not found" (a package is not in texlive; in the full scheme this should not happen)`, ... `"Missing $ inserted" (maths outside ), "Runaway argument" (a {without }`).

Several files (a project with chapters)

Compile always starts from the **main file** (the one with `\documentclass`). When you open a chapter that is loaded with `\input` or `\include`, vimtex has to know the main file. The reliable way is a magic comment on the first lines of the chapter:

```
%! TEX root = ../main.tex
Chapter text.
```

Opening `chap/c1.tex` with this comment gives `b:vimtex.tex` = the full path of `main.tex`, and `<Space>rf` in the chapter built `main.pdf`. Without the comment vimtex tries to find the main file itself, but in my test it picked a different `main.tex` from a neighbouring folder, so always add the comment in chapters. `\ls` toggles between the file and the main file.

The viewer

`vim.g.vimtex_view_method` is `zathura` when `zathura` is on PATH, otherwise `general` (the system default PDF program). Windows uses SumatraPDF and macOS Skim (config lines for those platforms exist in the same spec).

- **A zathura window opens by itself after the first successful compile.** This is vimtex's default (`g:vimtex_view_automatic = 1`), not a bug. Later compiles only refresh the open window. If you do not want it: `:let g:vimtex_view_automatic = 0` (until you quit); you then open the PDF yourself with `\lv`. For a permanent change the line has to go into the config. (Both ways: without the option a zathura window titled with the full path of `main.pdf` opened about 14 seconds after `<Space>rf` in a fresh Neovim; with `view_automatic=0` nothing opened until `\lv`.)
- `\lv` (`:VimtexView`) opens the viewer or, if open, jumps to the place of the cursor (forward search). From a chapter file with `%! TEX root`: zathura opened the PDF of the **main** file (`main.pdf`), and the process got `--synctex-forward 1:1:<path>/chap/c1.tex`, i.e. the cursor position of the chapter.
- **Inverse search** (Ctrl+click in the PDF jumps back to the source): the config writes the address of the running Neovim into `/tmp/vimtexserver.txt` every time a `tex` file is opened (`v:servername`). It is only a helper file; never edit it. With two Neovims open, the last one wins (the file is shared in `/tmp`). Prerequisites: the file exists and holds the same address as the running Neovim (`:echo v:servername`); the zathura process was started by vimtex with the inverse-search callback `-x "nvim --headless -c \"VimtexInverseSearch %{line}:%{column} '%{input}'\""`. Running that same callback in a second Neovim made the first Neovim jump to the clicked line and column. This only works because vimtex is loaded at startup (not only for `tex` files): the callback Neovim has no `tex` file open and needs the `:VimtexInverseSearch` command (found and fixed 2026-10-03; before, the click did nothing and showed no error).

Table of contents

`\lT` toggles, `\lt` opens the vimtex table of contents (`:VimtexTocToggle`, `:VimtexTocOpen`). It is a 30-column window on the left called TOC; the help lines are shown on top (config: `split_width = 30`, `show_help = 1`, layers content, todo, include). It lists sections, labelled equations, `\input` files and `TODO`/`FIXME` comments of the whole project.

Key in the TOC	Action
`<Space>`	Jump to the entry, keep the TOC open
`<Enter>`	Jump and close the TOC
`<Esc>` / `q`	Close
`r`	Refresh
`h`	Toggle the help text
`t`	Toggle sorted TODO list
`s`	Hide / show numbers
`-` / `+`	Show fewer / more section levels
`f` / `F`	Apply / clear a filter
`L`, `I`, …	Switch layers (label, include, …)

Move between the TOC and the text with `<Ctrl-w>h` / `<Ctrl-w>l` or `<Left>` / `<Right>`; the same keys move to the quickfix window (`<Ctrl-w>j` / `<Ctrl-w>k`, `<Down>` / `<Up>`). `<Ctrl-w>c` closes the window you are in.

Moving and editing (vimtex)

Example text for the examples below (all keys act on this exact file):

```
\section{One}
Text with $a+b$ and \textbf{bold} here. See \cite{knuth}.
\begin{equation}
x = 1
\end{equation}
\section{Two}
```

Motions

Key	Action
`]]` / `[[`	Next / previous section start (`\section`, `\begin{document}`, …). From line 1 `]]` went to `\begin{document}`; from the text of One it went to `\section{Two}`
`][` / `[]`	Next / previous section end
`]m` / `[m`	Next / previous `\begin` of an environment
`]M` / `[M`	Next / previous `\end` of an environment
`]n` / `[n`, `]N` / `[N`	Next / previous start / end of a maths zone
`]r` / `[r`, `]R` / `[R`	Frames (beamer)
`]/` / `[/`, `]*` / `[*`	Comment blocks
`K`	In this config **LSP hover** (texlab), not vimtex's package-documentation lookup
`%`	Jumps between `\begin` and `\end` and brackets, but here the map belongs to **matchup** (`<Plug>(matchup-%)`), not vimtex (from `\begin{equation}` it went to the `\end`)

Text objects (use after `d`, `y`, `c`, `v`)

Keys	Object	Result in the example
`ie` / `ae`	Environment: inside / with `\begin..\end`	cursor on `x = 1`: `yie` = `x = 1`, `yae` = whole equation
`ic` / `ac`	Command: name only / whole command	cursor on `textbf`: `yic` = `textbf`, `yac` = `\textbf{bold}`
`id` / `ad`	Delimiter pair: inside / with delimiters	cursor in `{bold}`: `yid` = `bold`, `yad` = `{bold}`
`i$` / `a$`	Maths	**unreliable**: on display maths it gives `x = 1` (`yi$` inside the equation), but on inline `$a+b$` it selected the display equation instead (tree-sitter highlighting replaces vim's syntax for maths), see the note below
`iP` / `aP`	Section	`yaP` on the heading line = from `\section{One}` up to before `\section{Two}`
`im` / `am`	List item	needs an `itemize`/`enumerate` item

`ic` / `ac` also exist in Markdown buffers with a different meaning (a config map for Markdown only); in `.tex` they are vimtex's.

Note on maths detection. Tree-sitter colours `.tex` files here, so Vim's own syntax is off, and vimtex shows a hint at start ("For more info, see :help vimtex-faq-treesitter"). Features that need the syntax groups to know "this is maths" are therefore unreliable for inline maths: `i$` / `a$`, `]n`, and the math insert maps. Environments, commands, delimiters and sections work.

Change, delete, toggle

Keys	Action	Result
`dse`	Delete the surrounding environment	the `\begin..\end` lines vanish, the body stays
`cse`	Change the environment name (prompt "Change surrounding environment: equation", type the new name, Enter)	`equation` became `align` in both places
`tss`	Toggle star of the environment	`equation` became `equation*`
`tse`	Toggle between two environments; default pair only `itemize` / `enumerate` (`g:vimtex_env_toggle_map`)	no change on `equation`
`dsc` / `csc`	Delete / change the surrounding command	`\textbf{bold}` became `bold`
`tsc`	Toggle star of a command	`\section{One}` became `\section*{One}`
`ds$` / `cs$` / `ts$`	Delete / change / toggle maths delimiters (`\[..\]`, `equation`)	on display maths: `ds$` removes the delimiter lines and keeps the body; `cs$` asks for the new delimiter (`\[` turned `equation` into `\[ .. \]`); `ts$` turned `equation` into inline `$x = 1$` and `\[..\]` into `equation`. On inline `$a+b$` nothing happens (see the note on maths detection above)
`tsd` / `tsD`	Toggle `\left..\right` modifiers (`tsD` toggles in the reverse direction)	`$(a+b)$` became `$\left(a+b\right)$`
`tsf`	Toggle fraction `a/b` and `\frac{a}{b}`	`$a/b$` became `$\frac{a}{b}$` and back
`<F6>`	Surround the line (or visual selection) with an environment (vimtex default; no leader alternative is set)	`<Plug>(vimtex-env-surround-line)`
`<F7>`	Create a command from the word (insert and normal mode; vimtex default, no leader alternative)	`<Plug>(vimtex-cmd-create)`
`<F8>`	Add `\left` / `\right` to delimiters (Normal mode; vimtex default, no leader alternative)	`<Plug>(vimtex-delim-add-modifiers)`
`]]` in insert mode	Close the open environment/ delimiter	`<Plug>(vimtex-delim-close)`
`` ` `` + letter in insert mode	Maths shortcuts (`` `a `` = `\alpha`), made by `vimtex#imaps#wrap_math`. `\lm` (`:VimtexImapsList`) opens a "VimTeX imaps" window that lists all of them; close it with `:close`	The maps exist but **did not expand** in the test, even inside `equation`: they only fire when `vimtex#syntax#in_mathzone()` is true, and it returned 0 there (tree-sitter note below). `\lm` itself ran in the same test

These do not clash with the vim-sandwich keys of the config (`sa`, `sd`, `sr`).

Language server (texlab)
texlab attaches to `tex` files when `texlab` is on PATH. It reads the whole project, so it also knows labels and citations of other files.

- **Completion**: commands, environments, `\ref{` labels, `\cite{` keys. The menu opens while you type; `<Ctrl-n>` opens it manually, `<Tab>` / `<Ctrl-n>` move down, `<CR>` confirms only an item you picked, `<Ctrl-e>` or `<Esc>` closes it (see section 14).
- **Diagnostics** (including errors from the build log), hover (`K`), symbols, rename, code actions (`<Space>ca`): the global LSP keys, see the LSP section.
- `:LspTexlabBuild`: one build.

The server entry (`lua/config/lsp.lua`):

```
-- LaTeX (texlab comes from the LaTeX devShell; enabled only when executable)
texlab = { cmd = { "texlab" } },
```

Only the command is given. The server is enabled only when `texlab` is on PATH (the generic rule for all servers), so outside the devShell nothing starts and nothing warns.

Sources for `tex` files (`lua/config/nvim-cmp.lua`):

```
cmp.setup.filetype("tex", {
  sources = {
    { name = "omni" },
    { name = "nvim_lsp" }, -- texlab (LaTeX devShell); no-op when no LSP is attached
    { name = "ultisnips" },
    { name = "buffer",   keyword_length = 2 },
    { name = "path" },
  },
})
```

Duplicates are normal: typing `\sec` and `<Ctrl-n>` shows entries from vimtex (omni) and texlab (nvim_lsp), 26 entries with several "section". Either inserts the same text.

Citations (bibliography)
Put references in a `.bib` file and load it in the document:

```
See \cite{knuth}.
\bibliographystyle{plain}
\bibliography{refs}      % refs.bib next to main.tex
```

with `refs.bib`:

```
@book{knuth, author={Knuth}, title={TAOCP}, year={1968}, publisher={AW}}
```

Typing `\cite{kn` and `<Ctrl-n>` shows `knuth [book] Knuth (1968), "TAOCP"` in the menu. `latexmk` runs BibTeX itself during compile; the first compile after adding a `\cite` may show `[?]` until the next automatic run (`main.bbl` was created by the first run). With biblatex use `\usepackage{biblatex}` and `\addbibresource{refs.bib}`; `latexmk` then runs biber.

Grammar and spelling (ltex_plus)

LanguageTool as a language server. Config (`after/lsp/ltex_plus.lua`):

```
-- LanguageTool grammar/spell checking via ltex-ls-plus (installed globally by nix).
---@type vim.lsp.Config
return {
  filetypes = { "markdown", "tex", "plaintex", "typst", "gitcommit", "text" },
  ---@type lspconfig.settings.ltex
  settings = {
    ltex = {
      -- user's spelllang is en,it,de,fr; LanguageTool checks ONE language per document.
      -- "auto": LanguageTool detects the language of each document (so every subfile of an
Italian
      -- Typst project is its own document and gets checked as Italian). Tested: detection
is correct
      -- even on one-line English/Italian commit messages. Weak spot: mixed-language or very
short
      -- text may be detected wrongly; pin a file with a magic comment (LaTeX: % LTeX:
language=it-IT)
      -- or set language = "en-US" here to go back to English-only.
      language = "auto",
      -- language ids (after get_language_id): this list REPLACES lspconfig's default list
      enabled = { "markdown", "latex", "tex", "plaintex", "typst", "gitcommit", "git-
commit", "plaintext", "text" },
      -- setting ltex.ltex-ls.logLevel: the default ("fine") writes whole documents to
stderr,
      -- i.e. into lsp.log, on every check
      ["ltex-ls"] = { logLevel = "warning" },
    },
  },
}
```

The file is quoted in full (`after/lsp/ltex_plus.lua`). `enabled` lists language ids and **replaces** the plugin default list; `logLevel = "warning"` stops the server from logging whole documents on every check.

- **One language per document, detected automatically (`auto`)**, although `spelllang` is `en,it,de,fr`: an Italian file gets Italian rules, an English one English rules. Before this change the language was fixed to `en-US` and every Italian word was flagged. Detection was right in tests even on one-line commit messages; mixed-language or very short text can be misdetected (use the magic comment below, or set `language = "en-US"` again). LaTeX commands are skipped; only the prose is checked.
- Problems are diagnostics. In my test a misspelled word got **two** underlines, one from `typos` and one from `LTeX`; this is normal.
- **Fix**: `<Space>ca` on the word opens the list:

Entry	Result
`Use 'sentence'`	Replaces the word. Works
`Use 'sen tense'`	Other suggestion
`Add 'sentense' to dictionary`	**Had no effect in my test**: the LTeX diagnostic stayed (the config has no handler for ltex's client commands). Do not rely on it
`Hide false positive`, `Disable rule`	No effect, same as `Add to dictionary` (see section 81)
`sentence`, `Ignore ... in the project` (typos_lsp)	typos fix; "Ignore" is a typos command

- **What works for false positives**: ltex magic comments in the file. `% LTeX: enabled=false` on a line of its own switches ltex off for the file (the LTeX underlines disappeared, typos stayed). `% LTeX: language=de-DE` makes the file checked as German (German messages), which is the way to override a wrong `auto` detection. Remove the line to undo. In Typst the same comments work with `//` (`// LTeX: language=it-IT`, tested), in Markdown with `<!-- LTeX: ... -->`; they apply to the one file only, not to files it includes.
- **Built-in spell checker** (different thing): `<Space>cz` toggles, `]s` / `[s`, `z=`, `zg` (adds to `spell/en.utf-8.add`, a file in the public repo). More in section 31.

Snippets

UltiSnips, file `my_snippets/tex.snippets`. Both are start-of-line snippets (`b`): type the trigger at the beginning of a line, `<Ctrl-j>` expands and jumps forward, `<Ctrl-k>` jumps back. They also appear in the completion menu, which shows the description in quotes below.

Trigger	Description
`use`	\usepackage line: type the package name (start of line)
`eqa`	Numbered equation environment with a label, referenced with \ref{label} (start of line)

`use`: the cursor starts on the `package` placeholder inside the braces.

```
\usepackage{package}
```

Example: `package` = `amsmath` gives `\usepackage{amsmath}`.

`eqa`: an equation environment with a label. The first placeholder is `label` (inside `\label{}`), the second is `content` (the formula on its own indented line). Refer to the equation elsewhere with `\ref{label}` using the same label text.

```
\begin{equation}\label{label}
    content
\end{equation}
```

Example: `label` = `eq:energy`, `content` = `E = mc^2`; later `\ref{eq:energy}` prints the equation number.

Math symbols

Inside math, a symbol is one typed word: you type its name instead of remembering the LaTeX command. This part of the file has 191 snippets: 2 math delimiters and 189 symbols.

How to use them, step by step:

1. Type `mk` for inline math or `dm` for a display math block, then press `<Ctrl-j>` (hold the Ctrl key and press j). `mk` writes `$ $` and puts the cursor between them; `dm` writes `\[`, an indented empty line and `\]` on three lines and puts the cursor on the empty middle line. Typing the `$` characters yourself works too.
2. Inside the math, type a symbol name such as `leq` and press `<Ctrl-j>` (or accept the name in the completion menu). The name is replaced by the symbol and the cursor ends right after it. The symbol snippets expand only while the cursor is inside math: outside math `<Ctrl-j>` does not expand them and only does what it does without a snippet (it starts a new line); the completion menu may still list the names.
3. A snippet with placeholders (for example `frac`) selects the first placeholder; type over it, then press `<Ctrl-j>` to go to the next one. After the last placeholder, `<Ctrl-j>` moves the cursor to the end of the symbol, still inside the math. For `mk`, one more `<Ctrl-j>` then moves it past the closing `$`; for `dm`, past the closing `\]` (the cursor stays on that line). When the math contains only plain symbols (no placeholders), the first `<Ctrl-j>` after you finish typing already leaves the closing delimiter.

After the expansion the cursor is always after the symbol. Word-like commands (`\leq`, `\alpha`, `\infty`, `\cdot`, ...) get one trailing space so the next letter you type does not stick to the command; the space is not shown in the tables. `sub`, `sup`, `inv`, `transpose`, `degree` and `celsius` attach to what you typed before, so they add no space and also expand right after a letter (`x` then `sub` gives `x_{i}`). In the Produces column the words in a placeholder show the text you can type over (for example `\frac{a}{b}`).

Worked example: inline math `x \leq y`.

You type	You get (`<cursor>` marks the cursor)
`mk`, then `<Ctrl-j>`	`$<cursor>$`
`x`	`$x<cursor>$`
a space, then `leq`, then `<Ctrl-j>`	`$x \leq <cursor>$`
`y`	`$x \leq y<cursor>$`
`<Ctrl-j>`	`$x \leq y$<cursor>` (the cursor is after the closing `$`)

Where they work: the check reads the text of the file from the top down to the cursor, so it does not depend on vimtex or on syntax colours. These count as math: text between a pair of `$`, between `$$` and `$$`, between `\(` and `\)`, between `\[` and `\]`, and the body of `equation`, `align`, `gather`, `multline`, `flalign`, `eqnarray`, `displaymath`, `math` and `split` (with or without the `*`). These do not count as math: text outside those places, a `\text{...}`, `\textrm{...}`, `\mbox{...}` group inside math, a line after a `%`

comment sign, a `\$` with a backslash, and the body of `verbatim`, `lstlisting`, `minted` and `comment` environments. A blank line ends an unclosed `$...$`, `$$...$$` or `\(...\)`. The symbol names are not offered straight after a backslash (`\alpha` typed by hand stays as typed). `mk` and `dm` expand in text and in math, but not in a comment or a verbatim environment. The snippets belong to the file type `tex`. `vim.g.tex_flavor = "latex"` (in `lua/globals.lua`) makes every `.tex` file open as `tex`, also one whose first lines have no `\documentclass` (Neovim would otherwise open it as `plaintex`, which gets no snippets).

Switched off in `tex` files: these triggers of the shared vim-snippets collection are removed by a `clearsnippets` line at the top of the math block (`my_snippets/tex.snippets`), so they no longer expand: `cc`, `inn`, `Nn`, `UU`, `uuu`, `nnn`, `HH`, `DD`, `part`, `ooo`, `=>`, `=<`, `==`, `!=`, `<=`, `>=`, `lll`, `xx`, `<!`, `!>`, `srt`, `srto`, `ss`, `__`, `compl`, `conj`, `bar`, `taylor`, `rij`, `pmat`, `bmat`, `lrb`, `lra`, the `invs` suffix snippet (`3invs`), and the word triggers `sin`, `cos`, `arccot`, `cot`, `csc`, `ln`, `log`, `exp`, `star`, `perp` (the math tables below define their own `sin`, `cos`, `ln`, `log`, `exp`, `perp` and the other functions). `sum`, `lim` and `prod` are in the same list; the math tables below define new `sum`, `prod` and `lim`. The environment and structure snippets of vim-snippets (`abs`, `prob`, `prop`, `def`, `lemma`, `fig`, `tab`, `table`, ...) are not touched.

Math delimiters

Trigger	Produces	Meaning
`mk`	`$ $`	Inline math, cursor between the dollar signs
`dm`	`\[`, empty line, `\]`	Display math on its own lines, cursor on the empty line

Relations and operators

Trigger	Produces	Meaning
`neq`	`\neq`	Not equal
`leq`	`\leq`	Less or equal
`geq`	`\geq`	Greater or equal
`ll`	`\ll`	Much less
`gg`	`\gg`	Much greater
`approx`	`\approx`	Approximately equal
`equiv`	`\equiv`	Identical / congruent
`sim`	`\sim`	Similar / distributed as
`simeq`	`\simeq`	Asymptotically equal
`cong`	`\cong`	Congruent
`propto`	`\propto`	Proportional to
`pm`	`\pm`	Plus minus
`mp`	`\mp`	Minus plus
`times`	`\times`	Multiplication cross
`cdot`	`\cdot`	Dot product / multiplication dot
`circ`	`\circ`	Composition
`ast`	`\ast`	Asterisk operator
`oplus`	`\oplus`	Direct sum
`otimes`	`\otimes`	Tensor product
`perp`	`\perp`	Perpendicular
`parallel`	`\parallel`	Parallel
`angle`	`\angle`	Angle
`degree`	`^\circ`	Degree sign
`celsius`	`^\circ\mathrm{C}`	Degrees Celsius
`infinity`	`\infty`	Infinity
`partial`	`\partial`	Partial derivative symbol
`nabla`	`\nabla`	Nabla
`therefore`	`\therefore`	Therefore
`because`	`\because`	Because
`ldots`	`\ldots`	Dots on the line
`cdots`	`\cdots`	Centered dots
`vdots`	`\vdots`	Vertical dots
`ddots`	`\ddots`	Diagonal dots

Logic and sets

Trigger	Produces	Meaning
`implies`	`\implies`	Implies
`iff`	`\iff`	If and only if
`to`	`\to`	Arrow right / maps to
`mapsto`	`\mapsto`	Maps to
`leftarrow`	`\leftarrow`	Arrow left
`forall`	`\forall`	For all
`exists`	`\exists`	Exists
`nexists`	`\nexists`	Does not exist
`neg`	`\neg`	Not
`land`	`\land`	And
`lor`	`\lor`	Or
`in`	`\in`	Element of
`notin`	`\notin`	Not element of
`subset`	`\subset`	Subset
`subseteq`	`\subseteq`	Subset or equal
`supset`	`\supset`	Superset
`cup`	`\cup`	Union
`cap`	`\cap`	Intersection
`setminus`	`\setminus`	Set difference
`emptyset`	`\emptyset`	Empty set
`NN`	`\mathbb{N}`	Natural numbers
`ZZ`	`\mathbb{Z}`	Integers
`QQ`	`\mathbb{Q}`	Rationals
`RR`	`\mathbb{R}`	Reals
`CC`	`\mathbb{C}`	Complex numbers

Greek letters

Trigger	Produces	Meaning
`alpha`	`\alpha`	Greek letter alpha
`beta`	`\beta`	Greek letter beta
`gamma`	`\gamma`	Greek letter gamma
`delta`	`\delta`	Greek letter delta
`epsilon`	`\epsilon`	Greek letter epsilon
`varepsilon`	`\varepsilon`	Greek letter varepsilon
`zeta`	`\zeta`	Greek letter zeta
`eta`	`\eta`	Greek letter eta
`theta`	`\theta`	Greek letter theta
`vartheta`	`\vartheta`	Greek letter vartheta
`iota`	`\iota`	Greek letter iota
`kappa`	`\kappa`	Greek letter kappa
`lambda`	`\lambda`	Greek letter lambda
`mu`	`\mu`	Greek letter mu
`nu`	`\nu`	Greek letter nu
`xi`	`\xi`	Greek letter xi
`pi`	`\pi`	Greek letter pi
`rho`	`\rho`	Greek letter rho
`sigma`	`\sigma`	Greek letter sigma
`tau`	`\tau`	Greek letter tau
`phi`	`\phi`	Greek letter phi
`varphi`	`\varphi`	Greek letter varphi
`chi`	`\chi`	Greek letter chi
`psi`	`\psi`	Greek letter psi
`omega`	`\omega`	Greek letter omega
`Gamma`	`\Gamma`	Greek letter Gamma
`Delta`	`\Delta`	Greek letter Delta
`Theta`	`\Theta`	Greek letter Theta
`Lambda`	`\Lambda`	Greek letter Lambda
`Xi`	`\Xi`	Greek letter Xi
`Pi`	`\Pi`	Greek letter Pi
`Sigma`	`\Sigma`	Greek letter Sigma
`Phi`	`\Phi`	Greek letter Phi
`Psi`	`\Psi`	Greek letter Psi
`Omega`	`\Omega`	Greek letter Omega

Functions, fractions, roots, scripts

Trigger	Produces	Meaning
`frac`	`\frac{a}{b}`	Fraction
`sqrt`	`\sqrt{x}`	Square root
`nroot`	`\sqrt[n]{x}`	N-th root
`absval`	`\left\lvert x \right\rvert`	Absolute value
`norm`	`\left\lVert x \right\rVert`	Norm
`floor`	`\lfloor x \rfloor`	Floor
`ceil`	`\lceil x \rceil`	Ceiling
`sub`	`_{i}`	Subscript
`sup`	`^{n}`	Superscript
`inv`	`^{-1}`	Inverse
`transpose`	`^\top`	Transpose
`sin`	`\sin`	Sin function
`cos`	`\cos`	Cos function
`tan`	`\tan`	Tan function
`ln`	`\ln`	Ln function
`log`	`\log`	Log function
`exp`	`\exp`	Exp function
`arcsin`	`\arcsin`	Arcsin
`arccos`	`\arccos`	Arccos
`arctan`	`\arctan`	Arctan

Calculus and analysis

Trigger	Produces	Meaning
`sum`	`\sum`	sum sign (∑)
`prod`	`\prod`	product sign (∏)
`lim`	`\lim`	limit operator
`int`	`\int`	integral sign (∫)
`limit`	`\lim_{x \to a} f(x)`	Limit
`limsup`	`\limsup_{n \to \infty}`	Limit superior
`liminf`	`\liminf_{n \to \infty}`	Limit inferior
`sumn`	`\sum_{i=1}^{n} a_i`	Sum with bounds
`prodn`	`\prod_{i=1}^{n} a_i`	Product with bounds
`integral`	`\int f(x) \, dx`	Indefinite integral
`defint`	`\int_{a}^{b} f(x) \, dx`	Definite integral
`iint`	`\iint_{D} f \, dA`	Double integral
`iiint`	`\iiint_{V} f \, dV`	Triple integral
`oint`	`\oint_{C} F \cdot d\mathbf{r}`	Closed line integral
`deriv`	`\frac{df}{dx}`	Derivative d/dx
`pderiv`	`\frac{\partial f}{\partial x}`	Partial derivative
`deriv2`	`\frac{d^2 f}{dx^2}`	Second derivative
`grad`	`\nabla f`	Gradient
`divergence`	`\nabla \cdot \mathbf{F}`	Divergence
`curl`	`\nabla \times \mathbf{F}`	Curl
`laplacian`	`\nabla^2 f`	Laplacian
`bigO`	`O\left(n\right)`	Big-O
`seq`	`(a_{n})_{n \in \mathbb{N}}`	Sequence
`epsdelta`	`\forall \varepsilon > 0 \; \exists \delta > 0 :`	Epsilon-delta

Linear algebra

Trigger	Produces	Meaning
`vecarrow`	`\vec{v}`	Vector with arrow
`vecbold`	`\mathbf{v}`	Vector in bold
`hat`	`\hat{x}`	Unit vector / estimator hat
`colvec`	`\begin{pmatrix} a \\ b \\ c \end{pmatrix}`	Column vector
`matrix2`	`\begin{pmatrix} a & b \\ c & d \end{pmatrix}`	2x2 matrix
`matrix3`	`\begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}`	3x3 matrix
`det`	`\det(A)`	Determinant
`detmat`	`\begin{vmatrix} a & b \\ c & d \end{vmatrix}`	Determinant bars
`trace`	`\operatorname{tr}(A)`	Trace
`rank`	`\operatorname{rank}(A)`	Rank
`dim`	`\dim(V)`	Dimension
`kernel`	`\ker(A)`	Kernel
`image`	`\operatorname{im}(A)`	Image
`span`	`\operatorname{span}\{v_1, v_2\}`	Span
`inner`	`\langle u, v \rangle`	Inner product
`cross`	`a \times b`	Cross product
`identity`	`I_{n}`	Identity matrix
`eigen`	`A\mathbf{v} = \lambda \mathbf{v}`	Eigenvalue equation

Statistics and probability

Trigger	Produces	Meaning
`mean`	`\bar{x}`	Sample mean (bar)
`variance`	`\operatorname{Var}(X)`	Variance
`covariance`	`\operatorname{Cov}(X, Y)`	Covariance
`expect`	`\mathbb{E}\left[X\right]`	Expected value
`Prob`	`\mathbb{P}(A)`	Probability
`given`	`\mathbb{P}(A \mid B)`	Conditional probability
`binom`	`\binom{n}{k}`	Binomial coefficient
`normaldist`	`X \sim \mathcal{N}(\mu, \sigma^2)`	Normal distribution
`stddev`	`\sigma`	Standard deviation
`estimator`	`\hat{\theta}`	Estimator hat
`tilde`	`\tilde{x}`	Tilde accent
`chisq`	`\chi^2`	Chi-squared
`zscore`	`z = \frac{x - \mu}{\sigma}`	Z-score
`confint`	`\bar{x} \pm z_{\alpha/2} \frac{\sigma}{\sqrt{n}}`	Confidence interval

Physics

Trigger	Produces	Meaning
`vecF`	`\vec{F}`	Force vector
`dot`	`\dot{x}`	Time derivative (one dot)
`ddot`	`\ddot{x}`	Second time derivative
`hbar`	`\hbar`	Reduced Planck constant
`kB`	`k_{\mathrm{B}}`	Boltzmann constant
`deltaT`	`\Delta T`	Change in temperature
`deltaU`	`\Delta U`	Change in internal energy
`dQ`	`\delta Q`	Heat increment
`dW`	`\delta W`	Work increment
`newton2`	`\vec{F} = m\vec{a}`	Newton's second law
`kinematic`	`v = v_0 + a t`	V = v0 + a t
`workint`	`W = \int \vec{F} \cdot d\vec{r}`	Work as integral
`kinetic`	`E_k = \frac{1}{2} m v^2`	Kinetic energy
`potential`	`E_p = m g h`	Gravitational potential energy
`heatq`	`Q = m c \Delta T`	Heat capacity law
`firstlaw`	`\Delta U = Q - W`	First law of thermodynamics
`idealgas`	`p V = n R T`	Ideal gas law
`entropy`	`\Delta S = \int \frac{\delta Q}{T}`	Entropy change
`conserve`	`E_i = E_f`	Conservation of energy
`momentum`	`\vec{p} = m\vec{v}`	Momentum

Troubleshooting

Problem	Cause and fix
`<Space>rf`, `<F9>` and `\ll` do nothing, `:VimtexCompile` unknown	vimtex not loaded: `latex` not on PATH. Run `:DevEnv latex`, or start Neovim inside the devShell (`which latex`, `direnv allow`). Check `:set ft?` is `tex`
Compile fails, quickfix opened	Read the first entry (`\le`), fix that line, `:w`; the compiler retries. Raw output: `\lo`
No PDF	Look next to `main.tex` (not in `build/`), see "Where the output goes". A fatal error stops the PDF
Viewer did not open	`zathura` not on PATH, or the compile failed, or `g:vimtex_view_automatic` is 0. Try `\lv`
A viewer opened that I did not expect	Normal after the first successful compile; `:let g:vimtex_view_automatic = 0`
"Undefined control sequence"	Misspelled command or missing package: add `\usepackage{...}` (snippet `use`)
Chapter file compiles the wrong document or errors	Add `%! TEX root = ../main.tex` at the top of the chapter
Citation shows `[?]`	`.bib` file not found or first run: compile again; check `\bibliography{refs}`
texlab not in the statusline	`texlab` not on PATH, or still starting. `:LspAttached`, `:checkhealth vim.lsp`
Same completion twice	Normal (vimtex omni and texlab)
`i$` / `]n` / `` ` `` shortcuts do not find inline maths	Tree-sitter highlighting replaces vim syntax, see the note above
Real word marked wrong by ltex	Magic comment, or fix the spelling; "Add to dictionary" does not work here
German/other language text full of errors	`% LTeX: language=de-DE` on its own line
Edits not in the PDF	Auto-save does not save `.tex`: `:w`. Check the compiler is running (`\lg`)
First compile is slow	`latexmk` builds everything once (bibliography, references); later runs are fast. Stop with `<Space>rf` when not needed
Inverse search from zathura jumps nowhere	Quit and restart Neovim (an old session may still have vimtex lazy-loaded); `/tmp/vimtexserver.txt` has the address of the last Neovim that opened a `tex` file, so reopen the file in the Neovim you want

Related sections

Typst (`<Space>tw`), Markdown, 28 (LaTeX and Typst, older summary), 31 (spell checking), the LSP and completion sections (diagnostic and completion keys), snippets (UltiSnips), windows (`<Ctrl-w>` moves).

Chapter 16: Markdown

81. Markdown (writing, preview, footnotes, PDF)

This section covers everything your config does that is specific to `.md` files: what each tool is for, how it works, the exact keys, and what to do when it fails. Global things (diagnostics keys, code actions, the spell keys, `gc` comments) are only mentioned briefly with a pointer. Section 27 is the short key list; this one is the full story.

The tools and why they exist

Tool	Why it is in your setup	What it gives you
marksman (LSP)	Markdown has links and headings that point to each other	Link and heading completion, jump to a heading, symbols. Only for plain `markdown` files
ltex_plus (LSP, LanguageTool)	Prose needs grammar checking, not only spelling	Grammar and spelling problems as diagnostics. Language detected automatically per file (`auto`, so Italian or German notes are not flagged as English)
typos_lsp	Catches common typos in every file type	Diagnostics like `recieve` to `receive`
render-markdown.nvim	Raw Markdown is hard to read	Headings, lists, code blocks, tables, checkboxes drawn nicely inside the buffer
mdkite.nvim (with kitehost.nvim)	See the final look in a browser	Live preview with Mermaid diagrams and math, `<Alt-m>` toggles it
vim-markdownfootnotes	Footnotes need numbering and a jump back	`^^` / `<Space>mf` create a footnote, `@@` / `<Space>mr` return
truth-table.nvim	Digital-logic notes need truth tables and Boolean algebra by hand otherwise	`:TruthTable` builds tables, Karnaugh maps and step-by-step rewrites inside the note; `and@` types `∧` (see Truth tables and Boolean algebra)
`:AddRef` (your own command)	Reference-style links keep the text readable	Adds `[label]: url` at the end of the file
prettier (via conform.nvim: `:w` and `<Space>fm`)	marksman cannot format	Tidies lists, tables, spacing
tabular	Align table columns	`:Tabularize /|`
`ic` / `ac` (your own text objects)	Work on fenced code blocks quickly	`dic`, `yac`, `vic` …
`+` (your own operator)	Make a list from plain lines	`+ip`
`<Space>mb` (your own operator)	Hard line breaks in Markdown	Adds a trailing `\`
`:ToPDF` (your own command)	Share a document as PDF	pandoc plus xelatex, PDF next to the file
smart_comment	Comments in Markdown are HTML comments	`gc` writes `<!-- ... -->` (the filetype's comment string); `gcs` / `gcr` (smart comment) use the language of the fence inside a fenced block

Settings for the Markdown filetype (`after/ftplugin/markdown.vim`):

```
setlocal concealcursor=c
setlocal synmaxcol=3000  " For long Chinese paragraphs
setlocal wrap
```

In a Markdown buffer: `wrap` is on (the global default is off), `textwidth` is 0, `colorcolumn` is 100, `synmaxcol` is 3000, `conceallevel` is 3. `concealcursor` shows as empty: render-markdown.nvim sets its own conceal options on the window after the ftplugin, so the `concealcursor=c` line in the file has no visible effect. Facts that follow from this:

- Long lines wrap at word boundaries (`linebreak` is on globally). Neovim never hard-wraps while you type (no `textwidth`), and prettier keeps your line breaks.
- Syntax colours stop after column 3000 of a very long line.

Quick start

1. Open a file, for example `nvim "my notes.md"`. In normal mode headings, lists, code blocks and tables are drawn nicely. While you type in insert mode the drawing pauses (`render_modes`), and returns after about half a second in normal mode.
2. Press `<Alt-m>`. The browser opens with a live preview (it needs an internet connection, see Preview). Press `<Alt-m>` again to stop it.
3. Press `<Space>fm`. Prettier tidies the file as one undo step (`u` undoes it). The file is not saved to disk. Saving with `:w` formats the same way.
4. Footnote: put the cursor on the last letter of a sentence, press `<Space>mf`. A `[^1]` appears and the cursor jumps to the new `[^1]:` line at the end of the file (in normal mode: press `A` to type the note). Press `@@` to jump back.
5. PDF: save (`:w`) and run `:ToPDF` from a Neovim that was started inside the latex dev shell. Wait 30 seconds or more the first time. `my notes.pdf` appears next to the file.

Requirements

Tool	Where it comes from	Needed for
marksman, prettier, ltex-ls-plus, typos-lsp	Global nix profile (`~/nix/users/krit/common/programs/neovim.nix`)	LSP features, formatting (`:w`, `<Space>fm`), grammar, typos
A web browser and an internet connection	Your system default browser. The preview page loads its Mermaid, KaTeX, markdown-it and highlight.js libraries from CDNs. The preview server itself is Lua inside Neovim: no node, no npm, no build step	`<Alt-m>`
pandoc, xelatex	Only in the latex dev shell (`~/nix/templates/krit/dev-environments/language-specific/latex/flake.nix`: `pandoc` and `texlive.combined.scheme-full`)	`:ToPDF`

Inside the latex dev shell `prettier`, `pandoc`, `xelatex` and `marksman` are all found by Neovim, and the attached LSP clients on a `.md` file are `ltex_plus`, `marksman`, `typos_lsp`. Note that `pandoc` is also found on this system outside the dev shell (it is on the system PATH), but `xelatex` and `prettier` are not found by a plain shell: Neovim must be started from a shell where they are on PATH (the dev shell, or the nix profile that provides them).

If a tool is missing, nothing falls back to a plain Vim key:

- No prettier: formatting is skipped silently (neither `:w` nor `<Space>fm` shows a message) and the file stays as it is.
- No pandoc: `:ToPDF` shows the error `pandoc not found`.
- No xelatex: pandoc runs and fails, one warning `ToPDF: pandoc failed (exit N)`.
- No marksman or ltex-ls-plus: that LSP simply does not start (check with `:checkhealth vim.lsp`).

Try the keys: an example file

Create a scratch file and paste this text. All the examples below refer to it.

````
# Title

Some text here.

```python
x = 1
y = 2
```

* item a
* item b


plain one
plain two

| a | b |
|---|---|
| longer cell | x |

A sentence.
````

Keys

All keys work only in Markdown buffers unless stated. `<Space>` is the leader key.

Keymap	Mode	What it does
`<Alt-m>`	n	Toggle the browser preview (`:MdKite toggle`). In another file type: one warning `Markdown preview: only in markdown buffers`
`<Space>mf`	n	Add a footnote after the character under the cursor (see Footnotes). Elsewhere: one warning
`<Space>mr`	n	Return from the footnote to the text. Elsewhere: one warning
`^^`	n, i	Add a footnote before the character under the cursor (see the off-by-one note below)
`@@`	n, i	Return from the footnote (replaces the macro replay `@@` in Markdown)
`<Space>fm`	n, x	Format the file (Visual mode: the selection) with prettier
`<Space>mb` + motion	n	Add a trailing `\` (hard line break) to the lines of the motion, e.g. `<Space>mbip`
`<Space>mb`	x	Same on a Visual selection
`+` + motion	n	Put `+ ` in front of the lines of the motion: `+ip`
`+`	x	Same on a Visual selection
`ic`, `ac`	o, x	Fenced code block, without or with the fence lines: `vic`, `dic`, `yac`, `cic`
`]]` / `[[`	n, x	Next / previous heading (levels 1 to 5)
`gO`	n	Outline: opens a location list window with one line per heading, indented by level (`Heading One`, `  Heading Two`, `    Heading Three`). `<CR>` jumps to the heading, `:lclose` closes it
`<Space>t`	n	Aerial symbol outline panel; `]t` / `[t` next / previous symbol
`<Space>cz`	n	Toggle spell checking (global key, see Writing quality)
`<Space><Space>`	n	Trailing-space remover, but in Markdown it only warns `markdown: trailing spaces are hard line breaks, not stripped`
`:AddRef <label> <url>`	cmd	Add a reference link at the end of the file
`:Tabularize /|`	cmd	Align table columns
`:ToPDF`	cmd	Export a PDF
`:RenderMarkdown toggle`	cmd	Turn the rendering off / on

Global keys that also work here: `<Space>ca` (code action, for example a ltex_plus fix), `<Space>dd`, `]d`, `[d` (diagnostics), `<Space>rn` renames through marksman (headings and links; marksman advertises rename).

Accepted effects of the footnote maps:

- A single `^` or `@` typed in insert mode appears after 500 ms, because Neovim waits for a possible second key. `^` followed by any other key comes out at once.
- In normal mode `@@` is not "repeat last macro" in Markdown (it is `<Plug>ReturnFromFootnote`). Use `@a` with the register name instead.

Text objects and operators

These live in `after/ftplugin/markdown.vim`. On the example file:

You do (cursor on a line inside the code block)	Result
`dic`	The two code lines are deleted, the lines with the fences stay (an empty block)
`dac`	The whole block with both fences is deleted
`yac` then `p`	The block, fences included, is copied and pasted
`dic` then `u`	Everything is back (one undo step)

You do	Result
`+ip` on `plain one`	Both lines become `+ plain one` and `+ plain two`
`V`, `j`, `+`	The same on the Visual selection
`<Space>mbip` on `plain one`	The lines become `plain one\` and `plain two\`

Notes: `+` keeps existing indentation (`  + text`). `<Space>mb` skips blank lines and lines that already end in `\`. `+` is also a normal Vim key (next line start) that is replaced here in Markdown buffers.

The `+` map is built so a count works (`3+j`): the file explains that a plain `:set` style map would turn the count into a range and give error E481.

Footnotes (vim-markdownfootnotes)

Why: you write the sentence, press a key, write the note, jump back, without scrolling to the end and counting numbers.

How it works: vim-markdownfootnotes inserts `[^N]` in the text and a line `[^N]:` at the end of the file, numbering them in order. Your config adds the keys `^^` and `@@` and removes the plugin's own `<Space>f` and `<Space>r` maps in Markdown, because with Space as leader they swallowed `<Space>f...` typed quickly.

The real code (`after/ftplugin/markdown.vim`, verbatim):

```
" Fix minor issue with footnote, see https://github.com/vim-pandoc/vim-markdownfootnotes/
issues/22
" Also remove the plugin's default <Leader>f insert-mode mapping which
" hijacks Space+f when typed quickly (since leader = space).
if exists(':FootnoteNumber')
  for [s:mode, s:lhs] in [['i', '<Leader>f'], ['n', '<Leader>f'], ['i', '<Leader>r'], ['n',
'<Leader>r']]
    if get(maparg(s:lhs, s:mode, 0, 1), 'buffer', 0)
      execute 'silent! ' . s:mode . 'unmap <buffer> ' . s:lhs
    endif
  endfor

  lua vim.keymap.set("n", "^^", ":<C-U>call markdownfootnotes#VimFootnotes('i')<CR>",
{ buffer = true, silent = true, desc = "markdown: insert footnote" })
  lua vim.keymap.set("i", "^^", "<C-O>:<C-U>call markdownfootnotes#VimFootnotes('i')<CR>",
{ buffer = true, silent = true, desc = "markdown: insert footnote" })
  lua vim.keymap.set("i", "@@", "<Plug>ReturnFromFootnote", { buffer = true, silent = true,
remap = true, desc = "markdown: return from footnote" })
```

```
  lua vim.keymap.set("n", "@@", "<Plug>ReturnFromFootnote", { buffer = true, silent = true,
remap = true, desc = "markdown: return from footnote" })
endif
```

In plain words:

- `exists(':FootnoteNumber')`: everything here runs only when the vim-markdownfootnotes plugin is loaded.
- The loop removes the plugin's buffer-local `<Leader>f` and `<Leader>r` maps (insert and normal mode). With Space as leader they would swallow the start of `<Space>f...` and `<Space>r...` keys typed quickly. The `get(maparg(...), 'buffer', 0)` check makes it unmap only the plugin's buffer-local map, never a global one.
- `^^` (normal and insert) inserts a footnote; `@@` returns from the footnote. They are set from Lua so they get a `desc` for which-key; `remap = true` on `@@` is needed because the right side is a `<Plug>` map.

Flow (normal mode):

1. Cursor on the last letter of `Some text here.` (`$` puts it on the period).
2. `<Space>mf`. The line becomes `Some text here.[^1]` (the mark is inserted after the character under the cursor) and the cursor sits on the new last line `[^1]:` in normal mode.
3. Press `A`, type `My note`, press `<Esc>`.
4. Press `@@` (or `<Space>mr`). The cursor is back at the `[^1]` in the sentence.

Flow (insert mode, the `^^` variant):

```
Mid word done.   (cursor before "word", in insert mode, type ^^)
```

The result is `Mid [^1]word done.` and you are still in insert mode on the note line: type the note, then `<Esc>`, then `@@` to go back. A second footnote becomes `[^2]` and its note is added under `[^1]: ...`.

The off-by-one at the end of a line: in insert mode with the cursor at the very end of a line, `^^` puts the mark one character too early (`Neovim is fas[^1]t`). The reason is that the `<C-O>` command inside the map first moves the cursor onto the last character, and `^^` inserts before that character. Typing `^^` in the middle of a line is exact. At the end of a line, end the sentence in normal mode instead: `<Esc>`, `<Space>mf` (inserts after the last character). In normal mode `^^` inserts before the character under the cursor, `<Space>mf` after it.

Reference links and tables

```
:AddRef docs https://neovim.io/doc
```

Result at the end of the buffer (a blank line, a comment line once, then the definition):

```
<!-- Reference links -->
[docs]: https://neovim.io/doc
```

A second `:AddRef two https://a.b` adds only `[two]: https://a.b` under it (the comment line is not repeated). In the text you write `[the docs][docs]`. The first argument completes from labels already used as `[text][label]` in the file. Label and URL cannot contain spaces; there is no title argument.

Tables: select the table lines and run `:'<,'>Tabularize /|`:

```
| a           | b   |
| ---         | --- |
| longer cell | x   |
```

Tabularize pads the separator row with spaces. `<Space>fm` is the better table aligner: prettier makes the separator row `| ---------- | --- |`.

Checkboxes (`- [ ]`): render-markdown draws them; no key toggles them.

Preview (mdkite.nvim)
Why: render-markdown shows structure in the editor, the browser preview shows the final look (fonts, images, math, diagrams).

How: mdkite.nvim starts a small web server written in Lua inside Neovim (its dependency kitehost.nvim is the HTTP server) and opens your default browser. The page follows the text and the cursor while you edit (instant updates, scroll sync).

Config (from `lua/plugin_specs.lua`):

```lua
{
  "selimacerbas/mdkite.nvim",
  ft = { "markdown" },
  cmd = { "MdKite" },
  dependencies = { { "selimacerbas/kitehost.nvim", lazy = true } },
  opts = {},
},
```

Command	What it does
`:MdKite` or `:MdKite start`	Start the preview of the current buffer and open the browser
`:MdKite stop`	Stop it
`:MdKite refresh`	Force a refresh of the page
`:MdKite toggle`	Start the preview, or stop the one that is running (what `<Alt-m>` runs)

What the page shows: headings, tables and code with syntax colours, relative images next to the file, Mermaid diagrams (fenced ```` ```mermaid ````), KaTeX math (`$E = mc^2$` inline, `$$ ... $$` display), and a light/dark switch in the page header.

- `opts = {}` means the plugin defaults: one preview at a time ("takeover" mode) on port 8421, the browser opens when the preview starts, dark theme.
- The commands exist after the first Markdown buffer is opened, or when you type `:MdKite` (the plugin loads on that command).
- Switching to another buffer does not stop the preview and does not close the browser tab. It stays bound to the buffer it was started in. It ends with `:MdKite stop`, with `<Alt-m>` in a Markdown buffer, or when you quit Neovim.

- `:MdKite toggle` is global (there is only one server). If a preview of buffer A is running and you press `<Alt-m>` in a second Markdown buffer B, that STOPS the preview. To move the preview to B, run `:MdKite start` in B.
- Outside Markdown buffers `<Alt-m>` shows one warning (the real map is buffer-local in `after/ftplugin/markdown.lua`: `vim.keymap.set("n", "<A-m>", "<cmd>MdKite toggle<cr>", { buffer = true, ... })`). The plugin sets no keys of its own.
- The preview page loads Mermaid, KaTeX, markdown-it and highlight.js from CDNs, so it needs internet. Raw HTML written in the previewed file runs in the page, so do not preview a file you do not trust.
- No build step: after a fresh install there is nothing to compile. Node stays installed for other reasons (`:run` on `.js`/`.ts` files), the preview does not use it.

In plain words:

- `ft = { "markdown" }` and `cmd = { "MdKite" }`: the plugin loads for Markdown files or on its command.
- `dependencies`: kitehost.nvim is the server library. It is marked `lazy = true` so it does not start with Neovim (without that, lazy.nvim would load it at startup).

Truth tables and Boolean algebra (truth-table.nvim)

Why: notes on digital logic (computer architecture) are full of truth tables, simplifications such as De Morgan, and Karnaugh maps. truth-table.nvim builds and checks them inside the Markdown note, so you do not draw or verify them by hand.

How: it is loaded shortly after Neovim starts (lazy.nvim `VeryLazy`); its one dependency, cds-lua-fp, comes with it. The tables are GitHub-style pipe tables, so this is for Markdown. The symbols are plain Unicode text, so they can also be pasted into Typst; in LaTeX they need xelatex or lualatex with a font that has them.

Config (from `lua/plugin_specs.lua`, shortened):

```
{
  "cds-io/truth-table.nvim",
  main = "truth-table",
  event = "VeryLazy",
  opts = { abbreviations = false },
  config = function(_, opts)
    require("truth-table").setup(opts)
    -- re-creates the plugin's word@ abbreviations as :iabbrev <buffer>
    -- for the filetypes markdown, tex, plaintex and typst
  end,
},
```

The expression language

Used inside the commands below. Words and symbols both work:

You type	Meaning	Written in the table
`not X`, `!X`, `¬X`	negation	`¬`
`A and B`, `∧`	and	`∧`
`A or B`, `∨`	or	`∨`
`A xor B`, `⊕`	exclusive or	`⊕`
`A implies B`, `->`, `=>`, `→`	implication	`→`
`A iff B`, `=`, `<->`, `⇔`	equivalence	`⇔`
`0`, `1`, `true`, `false`, `⊤`, `⊥`	constants	

There are NO `nand` or `nor` operators (`:TruthTableExpand A nand B` gives a parse error). Write `not (A and B)` and `not (A or B)`. Binary operators group from the left, so use parentheses for `A -> (B -> C)`. A table with N variables has 2^N rows: keep the expressions small.

Commands

Command	What it does
`:TruthTable A B`	Insert a table ABOVE the current line with the variables `A` and `B` (`:TruthTable 3` gives `A B C`)
`:TruthTable A and B | A implies B`	Insert a table from expressions (separated by `|` or a comma): the variables they use plus one computed column per expression
`:.TruthTable` (or a Visual range, `:'<,'>TruthTable`)	Turn the expression on the line (or the selected lines, one expression each) into a table
`:TruthTableExpand A xor B, A and B`	With the cursor in a table, append one computed column per comma-separated expression
`:TruthTableKarnaugh`	With the cursor in a column of a table, insert a Karnaugh map (2 to 4 inputs) and a minimal sum-of-products formula for that column below the table
`:TruthTableDeMorgan`	Preview a De Morgan rewrite of the expression at the cursor (run again to dismiss)
`:TruthTableSimplify`	Preview one simplification (complement, identity, domination, idempotence, absorption, reduction, consensus) and name the law
`:TruthTableFactor`, `:TruthTableDistribute`, `:TruthTableCommute`, `:TruthTableXor`, `:TruthTableUnfold`, `:TruthTableDNF`, `:TruthTableCNF`	Other rewrite previews (factor, distribute, swap operands, recognise exclusive or, unfold `→ ⊕ ⇔` into and/or/not, disjunctive and conjunctive normal form)
`:TruthTableRewrites`	List every rewrite of the expression and write the one you pick as the next step
`:TruthTableApply`	Replace the expression by the pending preview
`:TruthTableApplyStep`	Keep the line and add the pending preview below it as a derivation step: `≡ expression  | by <law>`
`:TruthTableVerify`	Check that every `≡` step of the derivation under the cursor is a real equivalence
`:TruthTableVerify!`	Same, and insert a proof table (one column per step; identical columns mean equivalent)
`:TruthTableEquiv a ∧ b | b ∧ a`	Are these expressions equivalent? (`!` inserts the proof table)
`:TruthTableToggle`, `:TruthTableDropRow`, `:TruthTableDropColumn`	Switch cells between `0/1` and `F/T`, drop the row or column under the cursor
`:TruthTableTutor`	Open the built-in course (see below)
`:TruthTableTutorNext` / `:TruthTableTutorPrev`	Next / previous tutor step

While a rewrite preview is shown, two transient keys exist in that buffer: `<Space>` applies it in place and `<CR>` writes it as a `≡` step. They disappear when the preview is applied, dismissed, or the buffer is edited.

Keys

All in normal mode, global, and each key only does something when its context holds (cursor in a table, on an expression, in a derivation, or with a preview pending); otherwise the command says what is missing. which-key shows the keys that are valid at the cursor under the groups `Truth table` and `Logic rewrite` (the per-key description is deliberately hidden). The prefixes are fixed by the plugin.

Keymap	Context	Same as
`<Space>Tn`	anywhere (also Visual)	`:TruthTable` on the current line or the selected lines; on an empty line it types `:TruthTable` for you
`<Space>Te`	in a table	types `:TruthTableExpand` for you
`<Space>Tt`	in a table	`:TruthTableToggle`
`<Space>Tr` / `<Space>Tc`	in a table	drop row / drop column (`.` repeats)
`<Space>Tk`	in a table	`:TruthTableKarnaugh`
`<Space>T=`	in a table	switch the `≡` / `≢` marks under equivalent columns on or off
`<Space>ld`	on an expression	`:TruthTableDeMorgan`
`<Space>lf` / `<Space>lx`	on an expression	factor / distribute
`<Space>ls` / `<Space>lS`	on an expression	swap with the next / previous operand
`<Space>lo`	on an expression	recognise exclusive or
`<Space>lu`	on an expression	unfold `→`, `⊕`, `⇔`
`<Space>lm` / `<Space>lM`	on an expression	disjunctive / conjunctive normal form
`<Space>lz`	on an expression	`:TruthTableSimplify`
`<Space>ll`	on an expression	`:TruthTableRewrites`
`<Space>lv` / `<Space>lV`	in a derivation	`:TruthTableVerify` / `:TruthTableVerify!`
`<Space>la` / `<Space>lA`	preview pending	`:TruthTableApply` / `:TruthTableApplyStep`

There was no `<Space>T…` or `<Space>l…` key before, so nothing in the global keys clashes. Inside a Jupyter notebook buffer `<Space>ls` is the notebook's LSP signature help instead (see Jupyter notebooks).

Typing the symbols

In insert mode, in Markdown, LaTeX, plain TeX and Typst buffers only, a word followed by `@` and then a space (or any character that is not a letter) turns into the symbol:

Type	Result	Type	Result
`and@`	`∧`	`or@`	`∨`
`not@`	`¬`	`xor@`	`⊕`
`implies@`	`→`	`iff@`	`⇔`
`forall@`	`∀`	`exists@`	`∃`
`true@`	`⊤`	`false@`	`⊥`
`equiv@`	`≡`	`nequiv@`	`≢`

`p and@ q` followed by a space gives `p ∧ q`. The plain word `and` never changes, and `not@a` (a letter right after the `@`) does not expand. The plugin's own version of these abbreviations is global in every buffer; this config switches that off (`abbreviations = false`) and defines them per buffer for the four note filetypes.

Worked lesson: a half adder and a proof

Open a Markdown note and put the cursor on an empty line.

#	What you press or type	What happens
1	`:TruthTable A B` then `<Enter>`, move the cursor into the table (`k`), then `:TruthTableExpand A xor B, A and B`	A table with the four input rows appears above the cursor line. The expand adds the columns `A ⊕ B` (the sum) and `A ∧ B` (the carry): that is a half adder
2	In insert mode type `p and@ q` and a space	The text becomes `p ∧ q`
3	On its own line type `not (A and B) or A` in insert mode, `<Esc>`, then `:TruthTableDeMorgan`	A preview appears at the end of the line: `⇒ ¬A ∨ ¬B ∨ A | by De Morgan`. Run `:TruthTableApplyStep` (or press `<CR>` while the preview shows): the line `≡ ¬A ∨ ¬B ∨ A | by De Morgan` is added below and the cursor moves onto it
4	`:TruthTableSimplify`, then `:TruthTableApplyStep`. Repeat the pair once more	The first step is `≡ 1 ∨ ¬B | by complement` (`A ∨ ¬A` is always true), the second `≡ 1 | by domination`. The whole expression is always true
5	`:TruthTableVerify!` with the cursor in the derivation	The message `every step holds (3 steps)` and a proof table below it: one column per line of the derivation, all four rows identical
6	Make a 4-variable table from one expression: on a line with `(A and B) or (A and not C) or (not A and C and D) or (B and C and D)` run `:.TruthTable` (the line becomes a table), put the cursor on a row of the last column, then `:TruthTableKarnaugh`	A Karnaugh map (rows `AB`, columns `CD` in Gray order) appears below the table, followed by the line `<expression> ≡ (¬A ∧ C ∧ D) ∨ (A ∧ B) ∨ (A ∧ ¬C)`: the minimal formula
7	`:TruthTableTutor`	The built-in course opens in a new tab: lesson on the left, an exercise pane on the right. `:TruthTableTutorNext` (or `]]` inside it) goes on; `:TruthTableTutor!` starts over

The tutor keeps your place in `~/.local/state/nvim/truth-table/tutor.json` (under `stdpath("state")`), so `:TruthTableTutor` in a new Neovim opens where you stopped. The lesson pane shows plain Markdown (the optional vellum.nvim is not installed).

Limits and open points

- prettier (conform) runs on every `:w` and reformats the generated tables: the separator row `|:---:|` becomes `| :-: |` (still centred and valid) and the `\| by` marks of a derivation lose their alignment padding. Cosmetic only.
- `:ToPDF` (pandoc and xelatex) with the default font prints `∧` and `∨` as blank (missing glyphs). Adding `-V mainfont='DejaVu Serif'` to the pandoc command fixes it, but `:ToPDF` has not been changed, so tables with those symbols show blanks in the PDF for now.
- The plugin is young and written with a coding agent. It makes no network calls and runs no shell commands; it writes only the tutor progress file above.

Rendering inside the buffer (render-markdown.nvim)

Why: you read Markdown all day; rendered headings and tables are easier on the eyes, and the raw text is one `<Esc>`-then-`i` away.

The real spec (`lua/plugin_specs.lua`, verbatim, comments included):

```
{
  "MeanderingProgrammer/render-markdown.nvim",
  main = "render-markdown",
  ft = { "markdown" },
  opts = {
    -- 1. Increase update delay (Default is 100ms).
    -- Waits half a second after you stop typing before recalculating graphics.
    debounce = 500,

    -- 2. Strict Mode Limits.
    -- Ensures it ONLY renders in Normal ('n') and Command ('c') mode.
    -- When you enter Insert ('i') mode to type, rendering pauses completely.
    render_modes = { "n", "c" },

    -- 3. Limit processing on huge files.
    -- Stops trying to render if a markdown file is over 1.5MB.
    max_file_size = 1.5,

    -- 4. Anti-conceal tuning.
    -- Anti-conceal hides graphical elements on the exact line your cursor is on.
    -- If the UI still feels slow when moving the cursor up/down, change enabled to `false`.
    anti_conceal = {
      enabled = true,
    },
  },
},
```

In plain words:

- `ft = { "markdown" }`: loaded only when a Markdown file is opened; `main = "render-markdown"` tells lazy.nvim which module to call `setup(opts)` on.
- Only four options are changed from the plugin defaults; everything else (heading icons, table borders, checkbox icons) is the plugin's default. The table below explains each of the four.

Setting	Meaning
`render_modes = n, c`	Rendered only in normal and command mode; insert mode shows the raw text
`debounce = 500`	Waits 0.5 s after typing stops before redrawing
`max_file_size = 1.5`	Files over 1.5 MB are not rendered (and the big-file mode takes over, see below)
`anti_conceal`	The line under the cursor shows raw text so you can edit it

Commands (all run without error; the global state changed `true`, `false`, `true` with two `toggle` calls):

Command	Effect
`:RenderMarkdown toggle`	Rendering off / on in all buffers
`:RenderMarkdown buf_toggle`	Same, this buffer only
`:RenderMarkdown enable` / `disable`	Explicit on / off
`:RenderMarkdown buf_enable` / `buf_disable`	Explicit on / off, this buffer only

An unknown name (for example `:RenderMarkdown bogus`) gives the plugin's error `invalid command - bogus`. `:checkhealth render-markdown` shows the setup.

Big files: above 1.5 MB (or lines averaging over 5000 characters) the file gets the filetype `bigfile`: no tree-sitter, no ftplugin keys (so no `<Alt-m>`, `^^`), and ltex_plus and typos_lsp are not attached. `:set ft=markdown` brings the full mode back (can be slow on huge files).

Formatting with prettier

Why: marksman has no formatting. Markdown is formatted by prettier, run by conform.nvim (see Formatting (conform.nvim)): when you save with `:w`, and on demand with `<Space>fm`.

How: prettier gets the buffer text (so a project `.prettierrc` is honoured), and only the changed lines are written back. Result on an example file:

```
* item a            ->  - item a
* item b                - item b
(two blank lines)   ->  (one blank line)
| a | b |           ->  | a           | b   |
|---|---|               | ----------- | --- |
```

The code block and the `plain one` / `plain two` lines (two lines, no blank between) stay as they are. Also `*emphasis*` becomes `_emphasis_` and a final newline is added (prettier defaults).

- `<Space>fm` changes the buffer as one undo step (`u` restores everything), keeps marks, and does not write the file. In Visual mode only the selection is formatted.
- Saving with `:w` formats first and then writes. Auto-saves (leaving the buffer, Neovim losing focus) never format.
- `<Space>fo` turns format on save off and on again; `:FormatDisable!` + Enter does it for the current buffer only.

- Without prettier on PATH nothing happens and no message appears. Start Neovim from a shell that has prettier (nix profile or dev shell). `:ConformInfo` shows whether prettier is found.

Trailing spaces: two spaces at the end of a line are a Markdown hard line break, so they are never stripped here. The whitespace plugin excludes `markdown` (`trailing_whitespace_exclude_filetypes`), and `<Space><Space>` only warns. The other hard-break form is a trailing backslash; `<Space>mb` adds it. For rewrapping long paragraphs use `gq` after `:set textwidth=80` yourself.

PDF export (`:ToPDF`)

Why: a Markdown note you can send to someone.

How: `plugin/command.vim` starts pandoc as a background job:

```
pandoc --pdf-engine=xelatex --highlight-style=zenburn --table-of-content
  --include-in-header=<config>/resources/head.tex -V fontsize=10pt -V colorlinks
  ... -s "<file>.md" -o "<file>.pdf"
```

- The output is `<same name>.pdf` next to the file; an old PDF is overwritten.
- You get a table of contents, coloured links, zenburn code colours, and your LaTeX header `resources/head.tex`.
- Spaces and special characters in the file name are safe (an argument list, no shell). On `my notes.md`: a valid PDF, no error.
- Save first. An unnamed buffer gives `ToPDF: save the buffer to a file first`, and unsaved edits are not in the PDF.
- The first run took more than 25 seconds. Neovim stays usable. There is no message when it finishes.
- Success is silent and on Linux no viewer opens (a viewer is started only on macOS and Windows): open the PDF yourself.
- Failure: one warning `ToPDF: pandoc failed (exit N)`. The usual reason is that xelatex is missing because Neovim was not started in the latex dev shell. To see the real LaTeX error, run the pandoc command in a terminal.

To use it: `cd` into a folder with the latex shell (direnv), run `nvim "my notes.md"`, then `:ToPDF`.

The real code (`plugin/command.vim`, verbatim):

```
" Convert Markdown file to PDF
command! ToPDF call s:md_to_pdf()

function! s:md_to_pdf() abort
  " check if pandoc is installed
  if executable('pandoc') != 1
    echoerr "pandoc not found"
    return
  endif

  let l:md_path = expand("%:p")
  if l:md_path ==# ''
    echohl WarningMsg | echomsg 'ToPDF: save the buffer to a file first' | echohl None
    return
  endif
```

```
  let l:pdf_path = fnamemodify(l:md_path, ":r") .. ".pdf"

  let l:header_path = stdpath('config') . '/resources/head.tex'

  " argv list: no shell, so spaces, $ and ; in paths are safe
  let l:cmd = ['pandoc', '--pdf-engine=xelatex', '--highlight-style=zenburn', '--table-of-
content',
        \ '--include-in-header=' . l:header_path, '-V', 'fontsize=10pt', '-V', 'colorlinks',
        \ '-V', 'toccolor=NavyBlue', '-V', 'linkcolor=red', '-V', 'urlcolor=teal',
        \ '-V', 'filecolor=magenta', '-s', l:md_path, '-o', l:pdf_path]

  let l:id = jobstart(l:cmd, {'on_exit': function('s:md_to_pdf_done', [l:pdf_path])})

  if l:id == 0 || l:id == -1
    echoerr "Error running command"
  endif
endfunction

" open the PDF after a successful run (mac / windows only, as before)
function! s:md_to_pdf_done(pdf_path, job_id, code, event) abort
  if a:code != 0
    echohl WarningMsg | echomsg 'ToPDF: pandoc failed (exit ' . a:code . ')' | echohl None
    return
  endif
  if g:is_mac
    call jobstart(['open', a:pdf_path])
  elseif g:is_win
    call jobstart(['cmd', '/c', 'start', '', a:pdf_path])
  endif
endfunction
```

In plain words:

- `:ToPDF` is a global command (not Markdown-only), but it works on the current file whatever its type, so use it in Markdown buffers.
- It checks the two things that can be missing (pandoc, a saved file) and shows one message each.
- The command is a list passed to `jobstart`, not a shell string: no quoting problems with spaces or `$` in file names.
- The header file is found through `stdpath('config')`, i.e. `resources/head.tex` inside the Neovim config, so it works wherever the config is deployed.
- `on_exit` runs `s:md_to_pdf_done`: a non-zero exit gives the warning, a zero exit is silent except on macOS (`open`) and Windows (`start`) where the PDF is opened.

Snippets

Source: `my_snippets/markdown.snippets` (208 snippets, UltiSnips). Type the trigger in insert mode in a Markdown buffer and expand it with `<Ctrl-j>` (section 15); `<Ctrl-j>` / `<Ctrl-k>` jump to the next / previous placeholder. The text in quotes after each trigger is its description as the completion menu shows it. Placeholders are shown in tab-stop order; after the last one the cursor leaves the block (`$0`).

Things to know first:

- `img`, `link` and `detail` have the same trigger as entries of the shared vim-snippets collection. The personal ones have priority 0 and the vim-snippets ones −50, so the versions below win.

- The blog-style snippets (`meta`, `more`, `font`, `detail`, `td`) come from a Hexo / Jekyll blog setup. They write plain text or HTML that a static-site generator understands; they do nothing special in a normal Markdown file.
- Several snippets write raw HTML (`<kbd>`, `<p>`, `<font>`, `<details>`, the message boxes). That HTML shows when the Markdown is rendered by a tool that allows HTML (a browser preview, GitHub, a blog); a plain-text reader shows the tags.

Group: Keys

`k1` or `kbd`: "HTML tag for one key (type k1 or kbd)". This trigger is a regular expression (`(k1|kbd)`), so both words expand the same way.

```
<kbd>KEY</kbd>
```

`k2`: "Two keys joined by + as tags". **`k3`**: "Three keys joined by + as tags".

```
<kbd>KEY</kbd> + <kbd>KEY</kbd>
<kbd>KEY</kbd> + <kbd>KEY</kbd> + <kbd>KEY</kbd>
```

Example (`k2`): `Ctrl`, `s` gives `<kbd>Ctrl</kbd> + <kbd>s</kbd>`.

Group: Headings and structure

`h1` ... `h6`: "Heading of level 1 to 6: type h1 ... h6 at the start of a line". The trigger is the regular expression `h([1-6])`, only at the start of a line. After the expansion a small helper replaces the line with that many `#` characters and the placeholder `Section Name`; the cursor goes to the next line afterwards.

```
## Section Name
```

(that is the result for `h2`; `h1` gives one `#`, `h3` three, and so on). Example: `h2`, then `Installation` gives `## Installation`.

`detail`: "Collapsible details block with a clickable summary (start of line; overrides the vim-snippets detail)". The summary text is red and clickable; the content is hidden until the reader clicks it.

```
<details>
<summary><font size="2" color="red">Click to show the code.</font></summary>

content
</details>
```

`meta`: "YAML front matter with title, current date and time, tags and categories (start of line)". The date is filled in by the snippet (current local time with time zone) when it expands.

```
---
title: "title"
date: 2026-01-31 12:00:00+0100
tags: [tag1, tag2]
categories: [category]
---
```

(the date line shows an example value). Example: `title` = `My first post`, `tags` = `vim, notes`.

`more`: "Blog read-more marker ". No placeholder: inserts the marker that Hexo and Jekyll use to cut a post's excerpt.

```
<!--more-->
```

`td`: "tl;dr line (start of line)". A summary line.

```
tl;dr: summary
```

Group: Links, images and text

`link`: "Markdown link `[text](url)` (overrides the vim-snippets link)".

```
[text](url)
```

`rlink`: "Markdown reference link `[text][label]`". The matching definition `[label]: url` is written by hand elsewhere in the file (see "Reference links and tables").

```
[link_text][label]
```

`img`: "Centered image with a width in pixels, written as HTML (overrides the vim-snippets img)". The width `800` is a pixel value you can overwrite.

```
<p align="center">
<img src="URL" width="800">
</p>
```

`font`: "HTML font tag with a color (obsolete in HTML5)". It works in most renderers but is no longer valid HTML.

```
<font color="blue">TEXT</font>
```

`yh`: "Corner brackets for quoting (CJK style)". Inserts the Japanese / Chinese quotation marks and puts the cursor between them.

```
「」
```

Group: Message boxes

`info`, **`warn`**, **`error`**, **`success`**: "Info message box (embeds its own style block; needs network for the Font Awesome 4.2.0 icons from a remote CDN)" (the same words for the other three with their own name). Each snippet writes a `<style>` block with the colours of the box, followed by a `<div>` with an icon, a label and your text. Two honest limits:

- Every one of the four embeds its own copy of the style block, so a page with all four has four `<style>` blocks. They are not shared.
- The style block imports Font Awesome 4.2.0 from a remote address (`maxcdn.bootstrapcdn.com`). The icon only shows when the reader is online and that

address still answers; if it does not load, the box still shows its colours and text, but without the icon.

The `info` expansion in full:

```
<style type="text/css">
@import url('//maxcdn.bootstrapcdn.com/font-awesome/4.2.0/css/font-awesome.min.css');

.info-msg {
    color: #059;
    background-color: #BEF;
    margin: 5px 0;
    margin-bottom: 20px;
    padding: 10px;
    border-radius: 5px 5px 5px 5px;
    border: 2px solid transparent;
    border-color: transparent;
}
</style>

<div class="info-msg">
    <i class="fa fa-info-circle"> Info</i><br>
    info text
</div>
```

The other three have the same layout and differ only in these values:

Trigger	CSS class	Text colour	Background	Icon class and label	Placeholder
`info`	`info-msg`	`#059`	`#BEF`	`fa-info-circle`, Info	`info text`
`warn`	`warning-msg`	`#9F6000`	`#FEEFB3`	`fa-warning`, Warning	`warning text`
`error`	`error-msg`	`#D8000C`	`#FFBABA`	`fa-times-circle`, Error	`error text`
`success`	`success-msg`	`#270`	`#DFF2BF`	`fa-check`, Success	`success text`

Math symbols

Inside math, a symbol is one typed word: you type its name instead of remembering the LaTeX command. The math snippets use LaTeX syntax, which is what Markdown math (`$...$` and `$$...$$`) renders. This part of the file has 191 snippets: 2 math delimiters and 189 symbols.

How to use them, step by step:

1. Type `mk` for inline math or `dm` for a display math block, then press `<Ctrl-j>` (hold the Ctrl key and press j). `mk` writes `$ $` on the line and puts the cursor between them; `dm` writes `$$`, an empty line and `$$` on three lines, then one more empty line below them, and puts the cursor on the empty middle line (the last empty line is where the cursor goes after the final `<Ctrl-j>`). Typing the `$` characters yourself works too.
2. Inside the math, type a symbol name such as `leq` and press `<Ctrl-j>` (or accept the name in the completion menu). The name is replaced by the symbol and the cursor ends right after it. The symbol snippets expand only while the cursor is inside math: outside math `<Ctrl-j>` does not expand them and only does what it does without a snippet (it starts a new line); the completion menu may still list the names.

3. A snippet with placeholders (for example `frac`) selects the first placeholder; type over it, then press `<Ctrl-j>` to go to the next one. After the last placeholder, `<Ctrl-j>` moves the cursor to the end of the symbol, still inside the math. One more `<Ctrl-j>` then moves it past the closing `$` (for `dm`: onto the line below the closing `$$`). When the math contains only plain symbols (no placeholders), the first `<Ctrl-j>` after you finish typing already leaves the closing delimiter.

After the expansion the cursor is always after the symbol. Word-like commands (`\leq`, `\alpha`, `\infty`, `\cdot`, ...) get one trailing space so the next letter you type does not stick to the command; the space is not shown in the tables. `sub`, `sup`, `inv`, `transpose`, `degree` and `celsius` attach to what you typed before, so they add no space and also work in the middle of a word (`xsub` gives `x_{i}`). In the Produces column the words in a placeholder show the text you can type over (for example `\frac{a}{b}`).

Worked example: inline math `x \leq y`.

You type	You get (`<cursor>` marks the cursor)
`mk`, then `<Ctrl-j>`	`$<cursor>$`
`x`	`$x<cursor>$`
a space, then `leq`, then `<Ctrl-j>`	`$x \leq <cursor>$`
`y`	`$x \leq y<cursor>$`
`<Ctrl-j>`	`$x \leq y$<cursor>` (the cursor is after the closing `$`)

Where they work: the math symbols and the delimiters follow the Markdown structure of the file. These count as math: text between a pair of `$`, text after an opening `$` that is not closed yet on that line, and everything between `$$` and `$$` (also over several lines, also when the closing `$$` is not typed yet). These do not count as math: fenced code blocks (backticks or `~~~`), the YAML block at the top of the file, inline code between backticks, a `\$` with a backslash, and a lone dollar sign in prose such as `it costs $5` or `pay $ now`. `mk` and `dm` expand anywhere except in those code places. Limits: inline math that is split over several lines is not recognised; an indented code block (four spaces) is not recognised as code, so a `$` there still starts math; a `$` in front of a word without backticks (for example `$HOME`) is read as the start of math until the end of the line; a `$` followed directly by a digit is read as a price, so math that starts with a digit right after the `$` (for example `$30degree`) is not recognised until other text follows; an unclosed `$$` makes everything below it count as math.

Math delimiters

Trigger	Produces	Meaning
`mk`	`$ $`	Inline math, cursor between the dollar signs
`dm`	`$$`, empty line, `$$`, empty line	Display math on its own lines, cursor on the empty line

Relations and operators

Trigger	Produces	Meaning
`neq`	`\neq`	Not equal
`leq`	`\leq`	Less or equal
`geq`	`\geq`	Greater or equal
`ll`	`\ll`	Much less
`gg`	`\gg`	Much greater
`approx`	`\approx`	Approximately equal
`equiv`	`\equiv`	Identical / congruent
`sim`	`\sim`	Similar / distributed as
`simeq`	`\simeq`	Asymptotically equal
`cong`	`\cong`	Congruent
`propto`	`\propto`	Proportional to
`pm`	`\pm`	Plus minus
`mp`	`\mp`	Minus plus
`times`	`\times`	Multiplication cross
`cdot`	`\cdot`	Dot product / multiplication dot
`circ`	`\circ`	Composition
`ast`	`\ast`	Asterisk operator
`oplus`	`\oplus`	Direct sum
`otimes`	`\otimes`	Tensor product
`perp`	`\perp`	Perpendicular
`parallel`	`\parallel`	Parallel
`angle`	`\angle`	Angle
`degree`	`^\circ`	Degree sign
`celsius`	`^\circ\mathrm{C}`	Degrees Celsius
`infinity`	`\infty`	Infinity
`partial`	`\partial`	Partial derivative symbol
`nabla`	`\nabla`	Nabla
`therefore`	`\therefore`	Therefore
`because`	`\because`	Because
`ldots`	`\ldots`	Dots on the line
`cdots`	`\cdots`	Centered dots
`vdots`	`\vdots`	Vertical dots
`ddots`	`\ddots`	Diagonal dots

Logic and sets

Trigger	Produces	Meaning
`implies`	`\implies`	Implies
`iff`	`\iff`	If and only if
`to`	`\to`	Arrow right / maps to
`mapsto`	`\mapsto`	Maps to
`leftarrow`	`\leftarrow`	Arrow left
`forall`	`\forall`	For all
`exists`	`\exists`	Exists
`nexists`	`\nexists`	Does not exist
`neg`	`\neg`	Not
`land`	`\land`	And
`lor`	`\lor`	Or
`in`	`\in`	Element of
`notin`	`\notin`	Not element of
`subset`	`\subset`	Subset
`subseteq`	`\subseteq`	Subset or equal
`supset`	`\supset`	Superset
`cup`	`\cup`	Union
`cap`	`\cap`	Intersection
`setminus`	`\setminus`	Set difference
`emptyset`	`\emptyset`	Empty set
`NN`	`\mathbb{N}`	Natural numbers
`ZZ`	`\mathbb{Z}`	Integers
`QQ`	`\mathbb{Q}`	Rationals
`RR`	`\mathbb{R}`	Reals
`CC`	`\mathbb{C}`	Complex numbers

Greek letters

Trigger	Produces	Meaning
`alpha`	`\alpha`	Greek letter alpha
`beta`	`\beta`	Greek letter beta
`gamma`	`\gamma`	Greek letter gamma
`delta`	`\delta`	Greek letter delta
`epsilon`	`\epsilon`	Greek letter epsilon
`varepsilon`	`\varepsilon`	Greek letter varepsilon
`zeta`	`\zeta`	Greek letter zeta
`eta`	`\eta`	Greek letter eta
`theta`	`\theta`	Greek letter theta
`vartheta`	`\vartheta`	Greek letter vartheta
`iota`	`\iota`	Greek letter iota
`kappa`	`\kappa`	Greek letter kappa
`lambda`	`\lambda`	Greek letter lambda
`mu`	`\mu`	Greek letter mu
`nu`	`\nu`	Greek letter nu
`xi`	`\xi`	Greek letter xi
`pi`	`\pi`	Greek letter pi
`rho`	`\rho`	Greek letter rho
`sigma`	`\sigma`	Greek letter sigma
`tau`	`\tau`	Greek letter tau
`phi`	`\phi`	Greek letter phi
`varphi`	`\varphi`	Greek letter varphi
`chi`	`\chi`	Greek letter chi
`psi`	`\psi`	Greek letter psi
`omega`	`\omega`	Greek letter omega
`Gamma`	`\Gamma`	Greek letter Gamma
`Delta`	`\Delta`	Greek letter Delta
`Theta`	`\Theta`	Greek letter Theta
`Lambda`	`\Lambda`	Greek letter Lambda
`Xi`	`\Xi`	Greek letter Xi
`Pi`	`\Pi`	Greek letter Pi
`Sigma`	`\Sigma`	Greek letter Sigma
`Phi`	`\Phi`	Greek letter Phi
`Psi`	`\Psi`	Greek letter Psi
`Omega`	`\Omega`	Greek letter Omega

Functions, fractions, roots, scripts

Trigger	Produces	Meaning
`frac`	`\frac{a}{b}`	Fraction
`sqrt`	`\sqrt{x}`	Square root
`nroot`	`\sqrt[n]{x}`	N-th root
`absval`	`\left\lvert x \right\rvert`	Absolute value
`norm`	`\left\lVert x \right\rVert`	Norm
`floor`	`\lfloor x \rfloor`	Floor
`ceil`	`\lceil x \rceil`	Ceiling
`sub`	`_{i}`	Subscript
`sup`	`^{n}`	Superscript
`inv`	`^{-1}`	Inverse
`transpose`	`^\top`	Transpose
`sin`	`\sin`	Sin function
`cos`	`\cos`	Cos function
`tan`	`\tan`	Tan function
`ln`	`\ln`	Ln function
`log`	`\log`	Log function
`exp`	`\exp`	Exp function
`arcsin`	`\arcsin`	Arcsin
`arccos`	`\arccos`	Arccos
`arctan`	`\arctan`	Arctan

Calculus and analysis

Trigger	Produces	Meaning
`sum`	`\sum`	sum sign (∑)
`prod`	`\prod`	product sign (∏)
`lim`	`\lim`	limit operator
`int`	`\int`	integral sign (∫)
`limit`	`\lim_{x \to a} f(x)`	Limit
`limsup`	`\limsup_{n \to \infty}`	Limit superior
`liminf`	`\liminf_{n \to \infty}`	Limit inferior
`sumn`	`\sum_{i=1}^{n} a_i`	Sum with bounds
`prodn`	`\prod_{i=1}^{n} a_i`	Product with bounds
`integral`	`\int f(x) \, dx`	Indefinite integral
`defint`	`\int_{a}^{b} f(x) \, dx`	Definite integral
`iint`	`\iint_{D} f \, dA`	Double integral
`iiint`	`\iiint_{V} f \, dV`	Triple integral
`oint`	`\oint_{C} F \cdot d\mathbf{r}`	Closed line integral
`deriv`	`\frac{df}{dx}`	Derivative d/dx
`pderiv`	`\frac{\partial f}{\partial x}`	Partial derivative
`deriv2`	`\frac{d^2 f}{dx^2}`	Second derivative
`grad`	`\nabla f`	Gradient
`divergence`	`\nabla \cdot \mathbf{F}`	Divergence
`curl`	`\nabla \times \mathbf{F}`	Curl
`laplacian`	`\nabla^2 f`	Laplacian
`bigO`	`O\left(n\right)`	Big-O
`seq`	`(a_{n})_{n \in \mathbb{N}}`	Sequence
`epsdelta`	`\forall \varepsilon > 0 \; \exists \delta > 0 :`	Epsilon-delta

Linear algebra

Trigger	Produces	Meaning
`vecarrow`	`\vec{v}`	Vector with arrow
`vecbold`	`\mathbf{v}`	Vector in bold
`hat`	`\hat{x}`	Unit vector / estimator hat
`colvec`	`\begin{pmatrix} a \\ b \\ c \end{pmatrix}`	Column vector
`matrix2`	`\begin{pmatrix} a & b \\ c & d \end{pmatrix}`	2x2 matrix
`matrix3`	`\begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}`	3x3 matrix
`det`	`\det(A)`	Determinant
`detmat`	`\begin{vmatrix} a & b \\ c & d \end{vmatrix}`	Determinant bars
`trace`	`\operatorname{tr}(A)`	Trace
`rank`	`\operatorname{rank}(A)`	Rank
`dim`	`\dim(V)`	Dimension
`kernel`	`\ker(A)`	Kernel
`image`	`\operatorname{im}(A)`	Image
`span`	`\operatorname{span}\{v_1, v_2\}`	Span
`inner`	`\langle u, v \rangle`	Inner product
`cross`	`a \times b`	Cross product
`identity`	`I_{n}`	Identity matrix
`eigen`	`A\mathbf{v} = \lambda \mathbf{v}`	Eigenvalue equation

Statistics and probability

Trigger	Produces	Meaning
`mean`	`\bar{x}`	Sample mean (bar)
`variance`	`\operatorname{Var}(X)`	Variance
`covariance`	`\operatorname{Cov}(X, Y)`	Covariance
`expect`	`\mathbb{E}\left[X\right]`	Expected value
`Prob`	`\mathbb{P}(A)`	Probability
`given`	`\mathbb{P}(A \mid B)`	Conditional probability
`binom`	`\binom{n}{k}`	Binomial coefficient
`normaldist`	`X \sim \mathcal{N}(\mu, \sigma^2)`	Normal distribution
`stddev`	`\sigma`	Standard deviation
`estimator`	`\hat{\theta}`	Estimator hat
`tilde`	`\tilde{x}`	Tilde accent
`chisq`	`\chi^2`	Chi-squared
`zscore`	`z = \frac{x - \mu}{\sigma}`	Z-score
`confint`	`\bar{x} \pm z_{\alpha/2} \frac{\sigma}{\sqrt{n}}`	Confidence interval

Physics

Trigger	Produces	Meaning
`vecF`	`\vec{F}`	Force vector
`dot`	`\dot{x}`	Time derivative (one dot)
`ddot`	`\ddot{x}`	Second time derivative
`hbar`	`\hbar`	Reduced Planck constant
`kB`	`k_{\mathrm{B}}`	Boltzmann constant
`deltaT`	`\Delta T`	Change in temperature
`deltaU`	`\Delta U`	Change in internal energy
`dQ`	`\delta Q`	Heat increment
`dW`	`\delta W`	Work increment
`newton2`	`\vec{F} = m\vec{a}`	Newton's second law
`kinematic`	`v = v_0 + a t`	V = v0 + a t
`workint`	`W = \int \vec{F} \cdot d\vec{r}`	Work as integral
`kinetic`	`E_k = \frac{1}{2} m v^2`	Kinetic energy
`potential`	`E_p = m g h`	Gravitational potential energy
`heatq`	`Q = m c \Delta T`	Heat capacity law
`firstlaw`	`\Delta U = Q - W`	First law of thermodynamics
`idealgas`	`p V = n R T`	Ideal gas law
`entropy`	`\Delta S = \int \frac{\delta Q}{T}`	Entropy change
`conserve`	`E_i = E_f`	Conservation of energy
`momentum`	`\vec{p} = m\vec{v}`	Momentum

Writing quality

ltex_plus (grammar, LanguageTool)

Why: spell checkers do not see "their" vs "there". ltex_plus checks grammar in prose files: `markdown`, `tex`, `plaintex`, `typst`, `gitcommit`, `text`.

How: it is a Java language server that sends problems as diagnostics.

Config (`after/lsp/ltex_plus.lua`):

```
settings = { ltex = {
  language = "en-US",
  enabled = { "markdown", "latex", "tex", "plaintex", "typst", "gitcommit", "git-commit",
"plaintext", "text" },
  ["ltex-ls"] = { logLevel = "warning" },
} },
```

- The language is fixed to `en-US`: LanguageTool checks one language per file. Italian, German or French text is flagged; change `language` in that file (`"auto"` detects the language, less reliable on short texts) or run `:lsp stop` for the buffer.
- The status line shows `Completed Checking document` and `ltex_plus` after opening a file. The first start is slow.
- Use the diagnostics: `]d` / `[d`, `<Space>dd` (message), `<Space>ca` (fixes).

- No personal dictionary or disabled-rule list is configured in the repo.
- Code actions on an unknown word (`Zorblat`, `<Space>ca`): `Use 'Format'`, `Use 'Combat'`, `Use 'Orbit'`, `Use 'Cobalt'`, `Use 'Oblast'`, `Add 'Zorblat' to dictionary`, `Hide false positive`, `Disable rule`, and `Create a Table of Contents` (from marksman). Choosing a replacement works: the word changes and `u` undoes it.
- `Add ... to dictionary`, `Hide false positive` and `Disable rule` do NOT work in this setup. They are client-side commands (`_ltex.addToDictionary` ...) that Neovim must implement, and your config has no handler; the server answers `Unknown command '_ltex.addToDictionary', ignoring`, nothing is saved, the warning stays. To silence a word use Vim's `zg` (see below), which only affects the Vim spell checker, not ltex_plus; to silence ltex_plus for a word, add it to the `ltex.dictionary` setting in `after/lsp/ltex_plus.lua` yourself.

Vim spell checking

Key	What it does
`<Space>cz`	Toggle `spell`
`]s` / `[s`	Next / previous misspelled word
`z=`	Up to 9 suggestions
`zg`	Add the word to the allowed list
`zw`	Mark the word as wrong

`spelllang` is `en,it,de,fr` together, so a word correct in any of them is fine. `zg` writes to the first list `spell/en.utf-8.add`; `2zg` writes to the second (`it`), `3zg` to `de`, `4zg` to `fr`. The `spell/` folder is in a public repo: do not add private words (see `spell/README.md`).

typos_lsp

Checks common typos in every normal buffer (not help, terminal, quickfix, or start screens). Diagnostics and fixes work as for ltex_plus.

Troubleshooting

Problem	Cause and fix
No rendering, raw `#` and `**` visible	Insert mode (press `<Esc>`), file over 1.5 MB, or rendering toggled off: `:RenderMarkdown enable`. Check `:set ft?` is `markdown`
`<Alt-m>` says "only in markdown buffers"	The buffer is not `markdown`: `:set ft=markdown`
Preview does not open, or the page is blank (no diagrams or math)	No browser, or no internet (the page loads its libraries from CDNs). Check `:MdKite refresh` and that `:Lazy` shows mdkite.nvim installed
Preview tab stays open after I left the file	Intended: the preview keeps running until `:MdKite stop`, `<Alt-m>` in a Markdown buffer, or quitting Neovim
`<Alt-m>` in a second Markdown file stopped the preview instead of showing it	`:MdKite toggle` is global: with a preview running it stops it. Run `:MdKite start` in the second buffer
`and@` does not turn into `∧`	The buffer is not markdown, tex, plaintex or typst, or there is no space (non-letter) after the `@`; check `:iabbrev`
`:TruthTableExpand A nand B` gives a parse error	There is no `nand` / `nor`; write `not (A and B)`
`<Space>fm` or `:w` does not format	prettier is not on PATH (no message is shown): start Neovim from a shell that has prettier (nix profile or dev shell); `:ConformInfo` shows what will run. Or format on save is switched off: `<Space>fo` / `:FormatEnable`
`:ToPDF`: "pandoc not found" / "pandoc failed (exit N)"	Start Neovim inside the latex dev shell. Run the pandoc command in a terminal to see the error
`:ToPDF` does nothing visible	Normal: silent, 30 s or more the first time, no viewer on Linux. Look for the PDF next to the file
`@@` does not replay my macro	In Markdown it means "return from footnote". Use `@a` (register name)
Typing `^` or `@` has a delay	The `^^` / `@@` insert maps wait 500 ms for a second key. Accepted
`^^` puts the mark one letter early	At the end of a line in insert mode (see Footnotes). Use `<Space>mf` in normal mode
`<Space><Space>` does not remove trailing spaces	Intended in Markdown (hard line breaks)
Grammar warnings on Italian or German text	ltex_plus is `en-US` only
`^^` does nothing	The footnote plugin is not loaded: `:echo exists(':FootnoteNumber')` must give `2`
Maps do nothing in a huge file	Big-file mode: `:set ft=markdown`

Related sections

Section 15 and 52 (snippets), section 16 (Code commenting: `gc` in Markdown writes `<!-- -->`; `gcs` / `gcr` use the comment style of the fence language inside a fenced block), section 27 (short Markdown key list), section 28 (LaTeX and Typst: same latex dev shell and ltex_plus), and the sections on spelling, LSP diagnostics and big-file mode.

Chapter 17: Typst

82. Typst (typst.vim, tinymist, watch and preview)

Typst is a modern markup language that compiles to PDF (like LaTeX, but much faster and simpler). This section covers everything that is specific to Typst files (`*.typ`): the plugin, the language server, the compile-and-preview workflow, prose checking and the files that make it work. Global tools (completion, folding, quickfix, spell keys) are only mentioned with a pointer to their own section.

What you get

Piece	What it does	Where it comes from
`kaarmu/typst.vim`	`:TypstWatch`, `:Toc` / `:Toch` / `:Tocv` / `:Toct`, `:make` (compiles once), indentation, `//` comments, Typst syntax colours	plugin, loaded only when `typst` is on PATH
`<Space>tw`	Starts `typst watch` for the current file (recompile on every save, open the PDF)	`after/ftplugin/typst.lua`
`tinymist`	Language server: diagnostics, completion, hover, go to definition, rename, symbols	typst devShell
`typstyle`	The formatter (`<Space>fm`, run by conform.nvim)	installed globally with Neovim, or the typst devShell
`zathura`	PDF viewer that reloads by itself when the PDF changes	typst devShell
`ltex_plus` and `typos_lsp`	Grammar, spelling and typo diagnostics in the prose	global (see "Prose checking")
`my_snippets/typst.snippets`	`mk` / `dm` for inline and display math and 104 math-symbol snippets (`leq`, `integral`, `matrix2`, ...), expanded with `<Ctrl-j>`	personal UltiSnips file (see "Snippets")
Buffer settings	`textwidth=100`, wrap on, colour marker at column 100, indent of 2 spaces	`after/ftplugin/typst.lua`, `lua/options.lua`, typst.vim

Completion has no Typst-specific source: it comes from the language server (section 14). `my_snippets/typst.snippets` adds math snippets (see "Snippets").

Requirements: the Typst devShell

The programs `typst`, `tinymist`, `typstyle` and `zathura` come from a Nix devShell, not from the system. The template is `~/nix/templates/krit/dev-environments/language-specific/typst/flake.nix`; its package list is:

```
packages = with pkgs; [ typst typstyle typstwriter tinymist prettypst utpm zathura ]
```

A project enters it through direnv. The `.envrc` of a Typst project contains one line:

```
use_dev_env typst
```

Then `cd` into the project (run `direnv allow` once per project). Start Neovim from that shell, so it sees the programs. Or, in an already running Neovim, run `:DevEnv typst` (then `<CR>`): it adds the programs to `PATH` for this session, loads typst.vim and enables tinymist (section 43). Check:

```
which typst tinymist typstyle zathura
```

Outside the devShell

What	Result
`typst` not on PATH	The plugin `typst.vim` does not load: no `:TypstWatch`, no `:Toc` (`exists(':TypstWatch')` is 0)
`<Space>tw`	Still mapped (buffer-local); it shows ONE warning: "Typst: typst not found on PATH (open nvim inside the typst devShell)". Nothing starts
`tinymist` not on PATH	No Typst language server. Only `ltex_plus` and `typos_lsp` attach
Filetype	`*.typ` is still recognised as `typst`; prose checking (`ltex_plus`, `typos_lsp`) still works

Quick start

1. In a terminal, enter the project folder (the devShell loads through direnv).
2. Open a file: `nvim main.typ`.
3. Wait a few seconds until the statusline on the right shows `tinymist` (it is the main server; `(+2)` means two more servers: `ltex_plus` and `typos_lsp`). `ltex_plus` attaches last: in a test it was still missing after 14 seconds and present within 40.
4. Press `<Space>tw`. The command line shows `Starting: typst watch  --diagnostic-format short 'main.typ' --open <cache>/zathura-light` (a tiny wrapper that starts zathura) and the PDF opens in zathura, always in light colors.
5. Edit the text and save with `:w` (Typst files are never saved automatically, see "Auto-save" below). The PDF is rebuilt and zathura reloads it.
6. If the document has an error, a quickfix window opens at the bottom with the message. Fix it and save again: the window closes by itself.
7. To stop: quit Neovim (`:qa`). The watcher is a child job and ends with Neovim (no `typst watch` process was left). Closing the zathura window alone does not stop the watcher.

Keys and commands

Key / command	Where	Description
`<Space>tw`	Typst buffers only	Runs `:TypstWatch`: `typst watch --diagnostic-format short <file> --open <viewer>` as a background job. Pressing it again stops the old job and starts a new one (new process id)
`:TypstWatch {args}`	needs typst.vim	Same, with extra `typst` options, e.g. `:TypstWatch --root ..`
`:make`	needs typst.vim	Compiles the file once (`typst compile --diagnostic-format short %`); errors go to the quickfix list (section 26)
`:Toc` / `:Tocv`	needs typst.vim	Table of contents of the `=` headings in a vertical location list on the right; `:Toch` horizontal, `:Toct` in a new tab. Press `Enter` on a line to jump
`<Space>fm`	global	Format the whole file (Visual mode: the selection) with typstyle, run by conform.nvim. Typst files are never auto-saved, so `:w` formats them too (see Formatting (conform.nvim)). Example: `#greet(  "x"  )  #let  y=3` became `#greet("x")  #let y = 3`
`gq`	global	Reformats prose lines to `textwidth` (100); not the Typst formatter
`K`	LSP	Hover: signature and parameters of a function (on `greet` and `text`)
`gd`	LSP	Go to definition: from a function call to its `#let`, from `@intro` to the `<intro>` label
`<Space>rn`	LSP	Rename symbol (`tinymist` supports rename)
`<Space>ca`	LSP	Code action menu at the cursor
`]d` / `[d`, `<Space>dd`	global	Next / previous diagnostic; `<Space>dd` shows a float with the diagnostic of the current line (section 13)
`gcc`	global	Comment line with `//` (typst.vim sets `commentstring` to `// %s`)
`<Space>t`	global	Symbol outline (aerial, section 37); in Typst buffers see the note below
`[t` / `]t`	aerial	Previous / next symbol (heading or `#let`)

Note on `<Space>t`: in Typst buffers `<Space>tw` exists next to the global `<Space>t` (outline). When you press `<Space>t` alone, Neovim waits `timeoutlen` (500 ms) for a possible `w` before it opens the outline. This is expected; press `<Space>t` and wait half a second.

The tinymist server also advertises extra commands (`tinymist.exportPdf`, `tinymist.startDefaultPreview`, `tinymist.exportSvg`, `tinymist.pinMain`, ...). No key is bound to them; this setup uses `typst watch` instead (see next part). The setting `exportPdf = "never"` stops tinymist from writing PDFs on its own.

The language server: tinymist

Why: it understands Typst while you type, without compiling to a PDF. Config (`lua/config/lsp.lua`):

```
tinymist = {
  cmd = { "tinymist" },
  filetypes = { "typst" },
  root_dir = function(bufnr, on_dir)
    local fname = vim.api.nvim_buf_get_name(bufnr)
    local root = vim.fs.dirname(vim.fs.find({ "typst.toml", ".git" }, { path = fname, upward
= true })[1])
      or vim.fs.dirname(fname)
    on_dir(root)
  end,
  on_attach = function(client, bufnr)
    local fname = vim.api.nvim_buf_get_name(bufnr)
    local main = vim.fs.find("main.typ", { path = vim.fs.dirname(fname), upward = true })[1]
    if main and main ~= fname then
      client:exec_cmd({ command = "tinymist.pinMain", arguments = { main } }, { bufnr =
bufnr })
    end
  end,
  settings = {
    exportPdf = "never",
    outputPath = "$root/out/$name",
    formatterMode = "typstyle",
  },
},
```

Setting	Meaning
`root_dir`	The project root is the nearest folder above the file with `typst.toml` (a Typst package) or `.git`; with neither, the folder of the file. Example: a file in a folder without `.git` got that folder as root
`on_attach`	Multi-file projects: when a `main.typ` exists in the file's folder or above it, tinymist is told to use it as the entry file (`tinymist.pinMain`). Without the pin, a section file that `main.typ` `#include`s is checked as its own root, so cross-references to labels defined in other sections show false errors such as `label <sec-03> does not exist`. Per project: no `main.typ` above the file means no pin; opening `main.typ` itself pins nothing
`exportPdf = "never"`	The server does not export PDFs; only `typst watch` does. So there is one PDF writer, not two
`outputPath`	Only used when the server exports (it does not here); it is a leftover default
`formatterMode = "typstyle"`	The server's own formatter is typstyle too; it is used only when conform.nvim finds no `typstyle` program

What it provides, in a real session:

Feature	How to see it
Diagnostics	A broken line (`#greet( 1 , 2 )` with too many arguments) showed `typst:18:unexpected argument` in `vim.diagnostic.get()` and the error sign in the gutter. Source name: `typst`
Completion	38 items for `#set te...` (`array literal`, `cite`, `context expression`, ...) in the completion menu (section 14)
Hover	`K` on a function shows its signature, e.g. `let greet(name: str) = str`
Definition	`gd` on a call or on a `@label`
Symbols	The outline (`<Space>t`) lists headings and `#let` definitions
Formatter	`<Space>fm` and `:w` (typstyle)

Check which servers are attached: `:LspAttached` (popup) or the statusline (right side, first name is the main server `tinymist`, `(+2)` the others). `:LspInfo` shows details; `:LspLog` opens the log (section 13).

Watch, compile and the PDF

`<Space>tw` is defined like this (`after/ftplugin/typst.lua`):

```
if vim.fn.executable("typst") == 1 then
  vim.keymap.set("n", "<leader>tw", "<cmd>TypstWatch<cr>",
    { buffer = true, desc = "Typst: watch & recompile" })
else
  vim.keymap.set("n", "<leader>tw", function()
    vim.notify("Typst: typst not found on PATH (open nvim inside the typst devShell)",
vim.log.levels.WARN)
  end, { buffer = true, desc = "Typst: watch & recompile (needs typst)" })
end
```

The real typst.vim spec (`lua/plugin_specs.lua`, verbatim):

```
-- Typst syntax highlighting, :TypstWatch, and :make support.
-- Requires the `typst` CLI on PATH (add pkgs.typst to your nix env).
{
  "kaarmu/typst.vim",
  enabled = function()
    return utils.executable("typst")
  end,
  ft = { "typst" },
  init = function()
    -- Conceal features are off by default; enable per-user if desired.
    vim.g.typst_conceal       = 0
    vim.g.typst_conceal_math  = 0
    vim.g.typst_conceal_emoji = 0
    -- Folding is off; enable by setting typst_folding = 1 locally.
    vim.g.typst_folding       = 0
    -- Auto-open quickfix window on compile errors.
    vim.g.typst_auto_open_quickfix = 1
    -- zathura goes through a wrapper so it always opens light (lua/zathura_light.lua).
    vim.g.typst_pdf_viewer = vim.env.TYPST_PDF_VIEWER
      or (utils.executable("zathura") and require("zathura_light").wrapper()) or ""
  end,
},
```

In plain words:

- `enabled`: typst.vim only loads when `typst` is on PATH (inside the Typst devShell). Without it there is no `:TypstWatch`, `:Toc` or `:make` support.
- `ft = { "typst" }`: lazy-loaded when a Typst file is opened.
- `typst_conceal`, `typst_conceal_math`, `typst_conceal_emoji` = 0: no concealing; you see the raw source. Set one to 1 locally to hide markup / render math symbols / emoji names.
- `typst_folding = 0`: the plugin's folding is off. Set `vim.g.typst_folding = 1` to get folds by headings.
- `typst_auto_open_quickfix = 1`: a compile error opens the quickfix window by itself.
- `typst_pdf_viewer`: `$TYPST_PDF_VIEWER` if set, else zathura if installed (through the wrapper `~/.cache/nvim/zathura-light`, which runs `zathura` with a config that includes your own zathurarc and overrides only the colors to Catppuccin Latte, so it opens light even when your desktop and zathura are dark), else an empty string (typst's `--open` then uses the system default PDF program).
- Not set: `typst_output_to_tmp` (so the PDF stays next to the source).

The Typst-side filetype file is `after/ftplugin/typst.lua` (verbatim, whole file); it sets the two prose options and the `<Space>tw` key:

```
-- Typst filetype settings and keymaps.
-- Loaded automatically for *.typ buffers by Neovim's after/ftplugin mechanism.

-- Reasonable defaults for prose-heavy markup files.
vim.opt_local.textwidth = 100
vim.opt_local.wrap      = true

-- <leader>tw  - launch typst watch (recompile + open PDF) in background.
-- TypstWatch is provided by kaarmu/typst.vim, which is only enabled when `typst` is on PATH
-- (e.g. inside the typst devShell): same check here. Without typst the key shows ONE
warning
-- (an unmapped key would fall through to <Space>t (aerial) + w).
if vim.fn.executable("typst") == 1 then
  vim.keymap.set("n", "<leader>tw", "<cmd>TypstWatch<cr>",
    { buffer = true, desc = "Typst: watch & recompile" })
else
  vim.keymap.set("n", "<leader>tw", function()
    vim.notify("Typst: typst not found on PATH (open nvim inside the typst devShell)",
vim.log.levels.WARN)
  end, { buffer = true, desc = "Typst: watch & recompile (needs typst)" })
end
```

The `<Space>tw` part of this file is the block above ("Watch, compile and the PDF"); the two `vim.opt_local` lines are the `textwidth` and `wrap` rows in "Filetype settings".

Question	Answer
What runs?	`typst watch --diagnostic-format short '<file>' --open <viewer>` in a background job, started from the current folder of Neovim
Which viewer?	`$TYPST_PDF_VIEWER` if set, else zathura (light, via the wrapper) if installed, else (empty value) `--open` alone, i.e. the system default PDF program
Where is the PDF?	Next to the `.typ` file with the same name (`main.typ` gives `main.pdf`). (`g:typst_output_to_tmp` is not set, so `/tmp/typst_out` is not used)
When does it recompile?	Every time the `.typ` file (or a file it imports) is saved
Why is auto-save off for Typst?	A recompile on every focus change would restart the watcher over and over; save with `:w` (section 42)
Where are compile errors?	In the quickfix list (opens at the bottom, cursor stays in your window). Lines look like `main.typ|19 col 13| error: unexpected argument`. `]q`-style quickfix keys are in section 26
What happens to the PDF on an error?	The old PDF stays (file time unchanged) until the error is fixed
Does a fixed error clear the window?	Yes: the next successful compile empties the list and closes the window
How to stop?	Quit Neovim. A second `<Space>tw` replaces the running watcher. There is no stop command
One watcher only	The plugin keeps one watcher job; watching another file with `<Space>tw` stops the first

To test the viewer by hand without Neovim: `zathura main.pdf` (reloads on change).

Prose checking in Typst

Tool	Result in a `.typ` file
`ltex_plus`	LanguageTool on the text, language detected automatically per file (`auto`: an Italian file is checked as Italian, an English one as English): `'smal': Possible spelling mistake found.`, `Don't put a space before the full stop.`. Source name `LTeX`. Needs `ltex-ls-plus` installed globally
`typos_lsp`	Common typos: `` `smal` should be `small` ``. Source name `typos`
`<Space>cz`	Toggles Vim's own spell checker (section 31); `zg` adds a word to your English list, `2zg` / `3zg` / `4zg` Italian/German/French
`<Space>ca` on a warning	Offers the fix (replace the word)

All three appear as diagnostics together with the `tinymist` ones (use `]d`, `<Space>dd`). Typst code is not skipped: LanguageTool may complain about markup such as `#link(...)`. Both tools never attach to files that are too big (section 42).

Italian (or other non-English) projects: every file of a project (`main.typ` and each subfile) is a separate document for LanguageTool, and the language is detected for each one on its own, so no setting per project is needed. Tested with a `main.typ` plus an included subfile in Italian: no diagnostics on the Italian text, English files still checked. A magic comment `// LTeX: language=it-IT` on a line of its own forces a language, but only for the file that contains it (it does not reach included files). If a file is detected wrongly (mixed languages, very short text), put that comment at its top, or `// LTeX: enabled=false` to switch LTeX off for it.

Filetype settings

Setting	Value	From
`textwidth`	100	`after/ftplugin/typst.lua`
`wrap`	on	`after/ftplugin/typst.lua`
`colorcolumn`	100	`lua/options.lua` (per-language table)
`expandtab`, `shiftwidth`	spaces, 2	typst.vim
`commentstring`	`// %s`	typst.vim
`iskeyword`	letters, digits, `_`, `-` and accents	typst.vim: `ciw` on `my-label` takes the whole word
Folding	off (`typst_folding = 0`; `foldmethod=manual`); folds in general come from nvim-ufo (section 18)	`plugin_specs.lua`
Concealing	off (`typst_conceal*` all 0): you see the raw source	`plugin_specs.lua`
Auto-save	never for Typst files	`plugin_specs.lua` (auto-save condition)

Example document

Save as `main.typ` in a folder with the devShell:

```
#set page(paper: "a5")
#set text(size: 11pt)
#set heading(numbering: "1.")

= Introduction <intro>

This is a smal test document with a typo. See @intro and #link("https://typst.app")[Typst].

#let greet(name) = [Hello, #name!]

#greet("World")

== Math

$ E = m c^2 $

- first item
- second item
```

What to try

Do	Expected result
Open the file, wait 5 seconds	Statusline shows `tinymist (+2)`; signs for line 7 (`smal`)
`<Space>tw`	Message `Starting: typst watch ... --open zathura`; `main.pdf` appears; zathura opens it
Change `a5` to `a4`, `:w`	The PDF is recompiled and zathura shows the bigger page
Add a line `#greet( 1 , 2 )` at the end, `:w`	Quickfix window with `error: unexpected argument`; the error sign on the line; PDF unchanged
Delete that line, `:w`	Quickfix window closes; PDF updated
`gd` on `@intro`	Cursor jumps to the heading line `= Introduction <intro>`
`gd` on `greet` in `#greet("World")`	Cursor jumps to `#let greet(name) = ...`
`K` on `greet`	Floating window with `let greet(name: str) = str`
Type `#set te` and wait	Completion menu with `text` and others
Type `#greet( "x" ) #let y=3` then `<Space>fm`	`#greet("x") #let y = 3`
`:Tocv`	Location list at the right with `Introduction` and `Math`
`<Space>t` and wait	Symbol outline (aerial): `Introduction`, `Math`, `greet`
`:make`	Compiles once; empty quickfix when everything is fine

Snippets

Source: `my_snippets/typst.snippets` (106 snippets, UltiSnips: 2 math delimiters and 104 math symbols). Type the trigger in insert mode in a Typst buffer and expand it with `<Ctrl-j>` (hold Ctrl and press `j`), or accept the entry in the completion menu (section 15). The symbol snippets are offered only while the cursor is inside math (between two `$`). `<Ctrl-j>` jumps to the next placeholder and `<Ctrl-k>` back to the previous one. Triggers are case-sensitive.

Starting flow

1. Type `mk` (inline math) or `dm` (display math, with spaces inside the dollars) and press `<Ctrl-j>`. The dollars appear and the cursor sits inside them.
2. Inside the math, type a symbol trigger such as `leq` and press `<Ctrl-j>` (or accept it in the completion menu). It becomes `lt.eq` followed by a space, and the cursor ends after that space. These snippets are not offered outside math.
3. Keep typing. `<Ctrl-j>` jumps through the placeholders of a structured snippet and, after the last one, past the closing `$`. Typing the closing `$` yourself works too.

Example, step by step:

You type	You get (| is the cursor)
`mk` then `<Ctrl-j>`	`$\|$`
`x leq` then `<Ctrl-j>`	`$x lt.eq \|$`
`y`	`$x lt.eq y\|$`
`<Ctrl-j>`	`$x lt.eq y$|`

Cursor and spacing rules: the cursor always ends after the expansion. Symbol names made of letters (`lt.eq`, `plus.minus`, `arrow.r`, ...) are followed by one space so that the next letter you type does not glue onto the name. `sub`, `sup`, `inv`, `transpose` and `celsius` attach to what you typed before and leave no space. In structured snippets the placeholders are visited from left to right and the final cursor sits right after the whole construct. The default word of a placeholder (for example `x`) is selected: type to replace it, or press `<Ctrl-j>` to keep it.

Where they work and limits

Where they work: the check counts the unescaped `$` characters in the file from the top down to the cursor; an odd number means the cursor is inside math. Comments (`// ...`), a `\$` with a backslash, and raw text (backticks, also code blocks with three backticks) are ignored when counting. `mk` and `dm` expand outside math and outside raw text only. `sub`, `sup`, `inv`, `transpose` and `celsius` also expand in the middle of a word (`xsub`); all other snippets need a space or an operator in front of the trigger (`xneq` does nothing, `x neq` expands).

Limits:

- The completion menu may list the symbol names outside math. Outside math `<Ctrl-j>` does not expand them; it only does what it does without a snippet (it starts a new line).
- `integral` and `sup` are also names of Typst's own completions with another meaning: the language server's `integral` is the bare integral sign, the snippet `integral` writes `integral f(x) dif x`; the language server's `sup` is the supremum operator, the snippet `sup` writes a superscript. Both entries appear in the completion menu; the snippet is the one marked as a snippet.
- Typing the Typst word `limsup` and pressing `<Ctrl-j>` right after it expands the `sup` at its end into a superscript (`lim^(n)`), because `sup` also expands in the middle of a word. Accepting `limsup` from the completion menu is not affected.

Math delimiters

These two work outside math only (not inside a `$ ... $` pair and not inside raw text or code blocks). Spaces inside the dollars make display (block) math in Typst; no spaces make inline math.

Trigger	Produces	Meaning
`mk`	`$$`	Inline math; the cursor is between the two dollars
`dm`	`$  $`	Display math (spaces inside the dollars); the cursor is between the two spaces

Math symbols

104 snippets, offered only inside math. The Produces column shows the text that is inserted (a trailing space after word-like symbols is not shown).

Relations and operators

Trigger	Produces	Meaning
`neq`	`eq.not`	not equal (≠)
`leq`	`lt.eq`	less or equal (≤)
`geq`	`gt.eq`	greater or equal (≥)
`ll`	`lt.double`	much less (≪)
`gg`	`gt.double`	much greater (≫)
`sim`	`tilde.op`	similar, distributed as (∼)
`simeq`	`tilde.eq`	asymptotically equal (≃)
`cong`	`tilde.equiv`	congruent (≅)
`propto`	`prop`	proportional to (∝)
`pm`	`plus.minus`	plus minus (±)
`mp`	`minus.plus`	minus plus (∓)
`cdot`	`dot.op`	dot product, multiplication dot (·)
`circ`	`compose`	composition (∘)
`oplus`	`plus.o`	direct sum (⊕)
`otimes`	`times.o`	tensor product (⊗)
`celsius`	`degree "C"`	degrees Celsius, attaches to the number before (°C)
`ldots`	`dots.h`	dots on the line (…)
`cdots`	`dots.h.c`	centered dots (⋯)
`vdots`	`dots.v`	vertical dots (⋮)
`ddots`	`dots.down`	diagonal dots (⋱)

Logic and sets

Trigger	Produces	Meaning
`implies`	`arrow.r.double`	implies (⇒)
`iff`	`arrow.l.r.double`	if and only if (⇔)
`to`	`arrow.r`	arrow right, maps to (→)
`leftarrow`	`arrow.l`	arrow left (←)
`nexists`	`exists.not`	does not exist (∄)
`neg`	`not`	not (¬)
`land`	`and`	and (∧)
`lor`	`or`	or (∨)
`notin`	`in.not`	not element of (∉)
`subseteq`	`subset.eq`	subset or equal (⊆)
`cup`	`union`	union (∪)
`cap`	`inter`	intersection (∩)
`setminus`	`without`	set difference (\)

Greek letter variants

Trigger	Produces	Meaning
`varepsilon`	`epsilon.alt`	epsilon, variant form (ε)
`vartheta`	`theta.alt`	theta, variant form (ϑ)
`varphi`	`phi.alt`	phi, variant form (φ)

Roots and scripts

Trigger	Produces	Meaning
`nroot`	`root(n, x)`	n-th root
`sub`	`_(i)`	subscript, attaches to what is before
`sup`	`^(n)`	superscript, attaches to what is before
`inv`	`^(-1)`	inverse exponent −1, attaches to what is before
`transpose`	`^(T)`	transpose exponent T, attaches to what is before

Calculus and analysis

Trigger	Produces	Meaning
`limit`	`lim_(x -> a) f(x)`	limit
`sumn`	`sum_(i=1)^(n) a_i`	sum with bounds
`prodn`	`product_(i=1)^(n) a_i`	product with bounds
`integral`	`integral f(x) dif x`	indefinite integral with dif x
`defint`	`integral_(a)^(b) f(x) dif x`	definite integral with bounds
`iint`	`integral.double_(D) f dif A`	double integral
`iiint`	`integral.triple_(V) f dif V`	triple integral
`oint`	`integral.cont_(C) F dot.op dif bold(r)`	closed line integral
`deriv`	`(dif f)/(dif x)`	derivative d f / d x
`pderiv`	`(partial f)/(partial x)`	partial derivative
`deriv2`	`(dif^2 f)/(dif x^2)`	second derivative
`grad`	`nabla f`	gradient
`divergence`	`nabla dot.op bold(F)`	divergence
`curl`	`nabla times bold(F)`	curl
`laplacian`	`nabla^2 f`	Laplacian
`bigO`	`O(n)`	big-O
`seq`	`(a_(n))_( in NN)`	sequence indexed by the naturals
`epsdelta`	`forall epsilon.alt > 0 thick exists delta > 0 :`	epsilon-delta quantifiers

Linear algebra

Trigger	Produces	Meaning
`vecarrow`	`arrow(v)`	vector with arrow
`vecbold`	`bold(v)`	vector in bold
`colvec`	`vec(a, b, c)`	column vector
`matrix2`	`mat(a, b; c, d)`	2x2 matrix
`matrix3`	`mat(a, b, c; d, e, f; g, h, i)`	3x3 matrix
`detmat`	`mat(delim: "|", a, b; c, d)`	determinant bars around a 2x2 matrix
`trace`	`op("tr")(A)`	trace
`rank`	`op("rank")(A)`	rank
`image`	`op("im")(A)`	image of a map
`span`	`op("span")(v_1, v_2)`	span
`inner`	`chevron.l u, v chevron.r`	inner product
`cross`	`a times b`	cross product
`identity`	`I_(n)`	identity matrix
`eigen`	`A bold(v) = lambda bold(v)`	eigenvalue equation

Statistics and probability

Trigger	Produces	Meaning
`mean`	`overline(x)`	sample mean (bar)
`variance`	`op("Var")(X)`	variance
`covariance`	`op("Cov")(X, Y)`	covariance
`expect`	`bb(E)[X]`	expected value
`Prob`	`bb(P)(A)`	probability
`given`	`bb(P)(A | B)`	conditional probability
`normaldist`	`X tilde.op cal(N)(mu, sigma^2)`	normal distribution
`stddev`	`sigma`	standard deviation (σ)
`estimator`	`hat(theta)`	estimator with hat
`chisq`	`chi^2`	chi-squared (χ^2)
`zscore`	`z = (x - mu)/sigma`	z-score
`confint`	`overline(x) plus.minus z_(alpha/2) sigma/sqrt(n)`	confidence interval for the mean

Physics

Trigger	Produces	Meaning
`vecF`	`arrow(F)`	force vector
`ddot`	`dot.double(x)`	second time derivative (double dot)
`hbar`	`planck`	reduced Planck constant (ħ)
`kB`	`k_"B"`	Boltzmann constant
`deltaT`	`Delta T`	change in temperature
`deltaU`	`Delta U`	change in internal energy
`dQ`	`delta Q`	heat increment
`dW`	`delta W`	work increment
`newton2`	`arrow(F) = m arrow(a)`	Newton's second law
`kinematic`	`v = v_0 + a t`	velocity with constant acceleration
`workint`	`W = integral arrow(F) dot.op dif arrow(r)`	work as a line integral
`kinetic`	`E_k = 1/2 m v^2`	kinetic energy
`potential`	`E_p = m g h`	gravitational potential energy
`heatq`	`Q = m c Delta T`	heat from heat capacity
`firstlaw`	`Delta U = Q - W`	first law of thermodynamics
`idealgas`	`p V = n R T`	ideal gas law
`entropy`	`Delta S = integral (delta Q)/T`	entropy change
`conserve`	`E_i = E_f`	conservation of energy
`momentum`	`arrow(p) = m arrow(v)`	momentum

Troubleshooting

Symptom	Cause	Fix
`<Space>tw` shows "Typst: typst not found on PATH"	Neovim was not started inside the devShell	`cd` into the project (direnv), check `which typst`, restart Neovim
`E492: Not an editor command: TypstWatch`	Same: typst.vim did not load without `typst`	Same fix
`label <...> does not exist` in a section file that `main.typ` includes	The section file was checked as its own root	Keep the entry file named `main.typ` in the project folder or above; reopen the file (the pin happens when tinymist attaches); `typst compile main.typ` must be clean
No `tinymist` in the statusline	`tinymist` not on PATH, or file is not `typst` filetype	`which tinymist`; `:set ft?` must say `typst`; `:LspAttached`
`direnv: error ... .envrc is blocked`	The project's `.envrc` is not approved	`direnv allow` once in the project
Watch starts but the PDF does not change	The file has a compile error (old PDF stays), or you did not save	Look at the quickfix window or `:copen`; save with `:w`
Watch compiles the wrong file or cannot find imports	Watcher started from another current folder than the file's	`:cd %:p:h`, then `<Space>tw` again
Viewer does not open	`zathura` missing, or `$TYPST_PDF_VIEWER` points to a missing program	`which zathura`; unset or fix `TYPST_PDF_VIEWER`; open `main.pdf` by hand
Two PDFs or a PDF in `out/`	A server export was enabled by hand	Keep `exportPdf = "never"` in `lua/config/lsp.lua`
Quickfix window did not open	The error has no `file:line:col` (e.g. a missing font warning)	Run `:!typst compile --diagnostic-format short %` and read the output
`<Space>t` feels slow in Typst files	It waits for a possible `w` (`<Space>tw`)	Expected; wait 500 ms, or use `:AerialToggle`
Warnings about prose inside markup	LanguageTool reads Typst source as text	Ignore, or `<Space>cz` / `zg` for words; see section 31
File not saved automatically	By design, Typst files are excluded from auto-save	`:w`

Related sections

Section 13 (LSP), 14 (autocompletion), 15 (snippets), 18 (folding), 26 (quickfix and location list), 27 (Markdown), 28 (LaTeX and Typst overview), 31 (spell checking), 32 (statusline), 37 (symbol outline), 41 (filetype settings), 42 (automatic behaviours), 44 (LSP in depth).

Chapter 18: C++

84. C++ (snippets, compile and run)

This chapter covers the custom C++ snippets (file `my_snippets/cpp.snippets`) and the few things the config does for C++ files. Only what is stated here has been checked in the config; the language server and the toolchain are explained in their own sections, named in "Related sections".

What you get

Feature	What it does	Needs
clangd (language server)	Diagnostics, completion, hover and go to definition for `c` and `cpp` buffers (section 44)	`clangd` on PATH (c-cpp devShell, section 43)
Compile and run (`<Space>rf` or `<F9>`)	Compiles the file with `clang++` (else `g++`), `-Wall -Wextra -std=c++20 -O2`, and runs the binary in a split below. Only mapped when a compiler is on PATH	`clang++` or `g++` (c-cpp devShell)
Run in a terminal (`<Space>rr`)	Compiles with `g++ -Wall -Wextra -std=c++20` and runs the binary that is written next to the source (section 19)	`g++` (c-cpp devShell)
Tree-sitter	The `cpp` parser gives syntax highlighting (section 44 and the tree-sitter section of `07-code.md`)	Parser (from the nix store on Nix systems)
Comment leader	Pressing `o`, `O` or `<Enter>` after a comment line does not continue the comment (`formatoptions` without `o` and `r`)	Nothing
Line-length marker	The coloured column marker sits at column 80 for C++ (section 41)	Nothing
Snippets	See Snippets below	Nothing

Snippets

Source: `my_snippets/cpp.snippets`. Type the trigger in insert mode in a C++ buffer and expand it with `<Ctrl-j>` (section 15); `<Ctrl-j>` / `<Ctrl-k>` jump to the next / previous placeholder. The one-line description is shown by nvim-cmp in the completion menu. A compact trigger list is in `04-completion-snippets.md` ("Other snippets").

Conventions:

- The text in quotes after each trigger below is its description exactly as the completion menu shows it. "Start of line" snippets (UltiSnips option `b`) expand only at the beginning of a line; `w` snippets expand after a word boundary; `ifelse` expands anywhere. `$0` is the final cursor position after the last placeholder.
- `map` and `for` have the same trigger as vim-snippets entries; the personal ones override them, so you get these versions.
- Placeholders are generic English words (`type`, `name`, `condition`); in the code blocks below they are shown in tab-stop order, and a name that appears several times is typed once.
- Only the declaration snippets use unqualified names such as `vector`, so they depend on a `using` line (`bare` or an `inc...` snippet adds it). The print helpers, `cout` and `random` write `std::` in full but still need their `#include` lines, named in their descriptions.

Group: Program skeleton and includes

`bare`: "Barebone C++ program: common includes, using std:: declarations and main" (start of line). A complete starting file; the cursor ends inside `main`, on a `// code` placeholder.

```
#include <iostream>
#include <vector>
#include <string>
#include <map>
#include <unordered_map>
#include <set>
#include <unordered_set>
#include <stack>
#include <queue>
#include <numeric>
#include <algorithm>

using std::cout;
using std::endl;
using std::vector;
using std::string;
using std::map;
using std::unordered_map;
using std::set;
using std::unordered_set;
using std::stack;
using std::queue;
using std::pair;
using std::make_pair;

int main()
{
    // code
    return 0;
}
```

`icd`: "#include directive (start of line)". Type the header name without the brackets.

```
#include <header>
```

Example: `header` = `cstdint` gives `#include <cstdint>`.

`incvec`, **`incmap`**, **`incset`**, **`incqueue`**, **`incstr`**, **`incstack`**: "Include and add using std::vector (start of line)" (and the same for `<map>`, `<set>`, `<queue>`, `<string>`, `<stack>`). They include one header and add the matching `using` line. They exist so the declaration snippets below work without `bare`. The expansion has no placeholder; `incvec` gives:

```
#include <vector>

using std::vector;
```

The others are the same with `<map>` / `map`, `<set>` / `set`, `<queue>` / `queue`, `<string>` / `string`, `<stack>` / `stack`. There is no `inc` snippet for `unordered_map` and `unordered_set`; only `bare` adds their `using` lines.

`sol`: "LeetCode style: create a Solution object (needs a class Solution)". Creates the object that coding-challenge sites expect, then leaves the cursor below it.

```
auto solution = Solution();
```

Group: Declarations

Each ends with `;` and leaves the cursor after it (`$0`). The description of each says what it needs, for example "std::vector declaration (needs using std::vector: bare or incvec)"; `map` adds "overrides the vim-snippets map".

Trigger	Expands to	Needs
`vec`	`vector<type> name;`	`bare` or `incvec`
`map`	`map<keyType, valueType> name;`	`bare` or `incmap`
`umap`	`unordered_map<keyType, valueType> name;`	`bare` (no `inc` snippet)
`set`	`set<type> name;`	`bare` or `incset`
`uset`	`unordered_set<type> name;`	`bare` (no `inc` snippet)
`queue`	`queue<type> name;`	`bare` or `incqueue`
`stack`	`stack<type> name;`	`bare` or `incstack`

Example: `vec`, `int`, `scores` gives `vector<int> scores;`. Example: `map`, `string`, `int`, `count` gives `map<string, int> count;`.

Group: Control flow

`for` ("for loop with init, condition and step (overrides the vim-snippets for)"), **`if`** ("if statement"), **`ifelse`** ("if-else statement"): braces with the body on its own line. After the last placeholder the cursor leaves the block (`$0`).

```
for (init; condition; step) {
    // code
}

if (condition) {
    // code
}

if (condition) {
    // code
```

```
} else {
    // code
}
```

Example for `for`: `int i = 0`, `i < n`, `++i`.

Group: Printing

`cout`: "Print a labelled variable with std::cout (needs #include)".

```
std::cout << "label: " << variable << std::endl;
```

Example: `label` = `total`, `variable` = `sum` gives `std::cout << "total: " << sum << std::endl;`.

`plist`: "print a container as [a, b, c] (needs , ,)". Defines a function template that prints any container (anything with `begin()` / `end()`) as `[a, b, c]` after a description. The description says it needs `<iostream>`, `<string>` and `<iterator>`. Put it above `main`, then call `printList(values, "values");`.

```
template <class T>
void printList(const T& arr, const std::string& desc){
    std::cout << desc << ": [";

    for (auto it = arr.begin(); it != arr.end(); it++){
        std::cout << *it << ((std::next(it) != arr.end()) ? ", " : "");
    }
    std::cout << "]\n";
}
```

`pmat`: "print a vector of vectors, one [a, b, c] row per line (needs , , ,)". The same for a vector of vectors, one `[a, b, c]` row per line. An empty row prints `[]`. Needs `<iostream>`, `<vector>`, `<string>`, `<iterator>`. Call it as `printMat(grid, "grid");`.

```
template <class T>
void printMat(const std::vector<std::vector<T>>& mat, const std::string& desc){
    std::cout << desc << ": " << std::endl;

    for (const auto& row : mat){
        std::cout << "[";
        for (auto it = row.begin(); it != row.end(); it++){
            std::cout << *it << ((std::next(it) != row.end()) ? ", " : "");
        }
        std::cout << "]\n";
    }
}
```

`pqueue`: "print a priority_queue or stack by popping a copy (uses top(); std::queue has front() instead)". Prints a `std::priority_queue` or `std::stack` by popping a copy, so the original stays unchanged. It uses `top()`; a `std::queue` has `front()` instead, so it does not work for `std::queue` without editing that one word. Call it as `printQueue(pq);`.

```
template <class T>
void printQueue(T q){
    while(!q.empty()){
        std::cout << q.top() << " ";
        q.pop();
```

```
    }
    std::cout << '\n';
}
```

Group: Random numbers

`random`: "Function returning a vector of random ints in [low, high] (needs #include and)" (start of line). A function that returns a `vector<int>` of random numbers in a range (both ends inclusive). It needs `#include <random>` and `<vector>`. It has no placeholders.

```
// Generate a random sequence of length len, in range(low, high) (inclusive).
// need to #include<random>
std::vector<int> genRandom(int low, int high, int len){
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<int> distribution(low, high);

    std::vector<int> arr(len, 0);
    for (int i = 0; i != len; ++i){
        arr[i] = distribution(gen);
    }

    return arr;
}
```

Related sections

Section 15 and 52 (snippets), 19 (code running), 41 (filetype settings), 43 (toolchain and devShells), 44 (language server in depth), 78 (Java chapter, section 9 has the same snippet layout).

Chapter 19: Vimscript

85. Vimscript (snippets, source and settings)

This chapter covers the custom Vimscript snippets (file `my_snippets/vim.snippets`) and the few things the config does for `.vim` files. Only what is stated here has been checked in the config; no Vimscript language server is described in this guide.

What you get

Feature	What it does	Needs
Source the file (`<Space>rf` or `<F9>`)	Runs `:source %` on the current file (section 19)	Nothing
`K`	Opens `:help` for the word under the cursor (`keywordprg=:help`)	Nothing
Folding	Folds use `utils#VimFolds` for the fold level and `utils#MyFoldText` for the fold line (section 18)	The `utils` autoload functions of the config
Comment leader	Pressing `o`, `O` or `<Enter>` after a comment line does not continue the comment	Nothing
Line-length marker	The coloured column marker sits at column 80 for Vim script (section 41)	Nothing
Tree-sitter	The `vim` parser gives syntax highlighting (bundled with Neovim)	Nothing
Snippets	See Snippets below	Nothing

These settings live in `after/ftplugin/vim.vim`.

Snippets

Source: `my_snippets/vim.snippets` (2 snippets). Type the trigger in insert mode in a Vim script buffer and expand it with `<Ctrl-j>` (section 15); the text in quotes after each trigger below is its description as the completion menu shows it; `<Ctrl-j>` / `<Ctrl-k>` jump to the next / previous placeholder. Placeholders are shown below in tab-stop order. After the last placeholder the cursor leaves the block (`$0`).

`fun`: "Vimscript function declared with abort (args are comma separated)". A function. The `!` replaces an older function of the same name when you source the file again, and `abort` stops the function at the first error. The name `MyFunc` is preselected. A name for a script-local function starts with `s:`; a function in an autoload file looks like `folder#Name`.

```
function! MyFunc(args) abort
    body
endfunction
```

Example: `MyFunc` replaced by `s:Greet`, `args` by `who`, body `echo "hello " . a:who`.

`aug`: "Autocommand group that clears itself first (EVENT: BufWritePost, FileType, ...; PATTERN: *.vim, ...; start of line)". An autocommand group. The `autocmd!` line empties the group first, so sourcing the file twice does not register the autocommand twice.

```
augroup GROUP_NAME
    autocmd!
    autocmd EVENT PATTERN command
augroup END
```

Example: `GROUP_NAME` replaced by `MyGroup`, `EVENT` by `BufWritePre`, `PATTERN` by `*.txt`, `command` by `%s/\s\+$//e`.

Related sections

Section 15 and 52 (snippets), 18 (folding), 19 (code running), 41 (filetype settings), 78 (Java chapter, section 9 has the same snippet layout).

Chapter 20: Nix

86. Nix (snippets and delib modules)

This chapter covers the custom Nix snippets (file `my_snippets/nix.snippets`, 12 snippets) and the few things the config does for `.nix` files. Only what is stated in "What you get" has been checked in the config. The delib explanation below comes from the session that wrote the snippets, which checked it against the denix source (`lib/configurations/module.nix`) and the NixOS and home-manager option search; it is not re-checked here.

What you get

Feature	What it does	Needs
nixd (language server)	The main language server of `nix` buffers: diagnostics, completion, hover, go to definition (section 44). Started with `--log=error`	`nixd` on PATH (section 44; the nix devShell, section 43, has it)
Format file (`<Space>fm`, and every `:w`)	Runs `nixfmt` through conform.nvim (see Formatting (conform.nvim)). Without `nixfmt` the LSP formatter of nixd runs instead; the formatter command set for nixd is `nixpkgs-fmt`	`nixfmt` on PATH (nix devShell); otherwise `nixd` attached and `nixpkgs-fmt` on PATH
Word highlight	In `.nix` files only the identical word is highlighted (text matching or Tree-sitter), because nixd would mark every package of a `with pkgs; [ ... ]` list (section 3, "Word references")	Nothing
Comments	The smart comment plugin knows Nix: `#` line comments, `/* */` blocks and `'' ''` strings	Nothing
Line-length marker	The coloured column marker sits at column 100 for Nix (section 41)	Nothing
Tree-sitter	Grammars on Nix systems come from the nix store (section 21, section 46)	Nothing extra
Snippets	See Snippets below	Nothing

Not stated here because the config does not show it: a Nix filetype plugin under `after/ftplugin` (there is none), any Nix-specific key, and any use of `statix` by the config. The formatter setting for nixd names `nixpkgs-fmt`; conform.nvim prefers `nixfmt` when it is installed.

The Nix devShell

A Nix devShell already exists and `:DevEnv` already supports it; nothing has to be added to the Nix repo. The template is `~/nix/templates/krit/dev-environments/language-specific/nix/flake.nix` (description "A Nix-flake-based Nix development environment", systems `x86_64-linux`, `aarch64-linux`, `x86_64-darwin`, `aarch64-darwin`). Its default devShell is `pkgs.mkShellNoCC` with this package list:

```
packages = with pkgs; [
  nixd cachix lorri nh niv nixfmt statix vulnix
  haskellPackages.dhall-nix
  inputs.nix-tests.packages.${system}.default
];
```

The last entry is the `nix-tests` flake package (`github:danielefongo/nix-tests`). The guide names the tools and does not describe what each one does, except `nixd` (the language server, see above); section 43 lists the same packages in its devShell table.

Which of these the config uses: `nixd` (`lua/config/lsp.lua`: `cmd = { "nixd", "--log=error" }`, formatter `nixpkgs-fmt`) and `nixfmt` (run by conform.nvim for formatting). The config does not call `statix` or the others.

How to enter it, as for the other languages (section 43):

- **In a running Neovim:** `:DevEnv nix` then `<CR>`. `:DevEnv` lists every sub-folder of its base folder (default `~/nix/templates/krit/dev-environments/language-specific`, changeable with `vim.g.devenv_base` or `$NVIM_DEVENV_BASE`) that holds a `flake.nix`, so `nix` is offered automatically (press `<Tab>` after `:DevEnv `). It adds the programs to `PATH` for this session only.
- **With direnv in a project:** the other chapters use one line in the project's `.envrc`, `use_dev_env <name>`; for this devShell that would be `use_dev_env nix` (same mechanism, `direnv allow` once, start Neovim from that shell).

Honest limit: only the flake file was read; whether it builds was not checked.

What delib is

The author's Nix configuration uses the **denix** module library, called **delib** in the code. These notes explain it in plain words, because the `delibmodule` and `delibalways` snippets create delib modules.

Every `.nix` file under the configured paths is a **delib module**, written like this:

```
delib.module {
  name = ...;
  options = ...;
  <target>.<mode> = ...;
}
```

- `name` is a dotted path into the `myconfig` option tree. `"programs.bat"` means `myconfig.programs.bat`.
- `options` declares what that path contains.
- Each `<target>.<mode>` block is a piece of configuration that is applied only when its condition holds.

Target (what kind of configuration the block produces). Nothing else is valid:

Target	Meaning
`nixos`	System-level NixOS options (ignored on macOS)
`darwin`	System-level nix-darwin options (ignored on NixOS)
`home`	home-manager options for the user (works on both systems)
`myconfig`	Sets values in the repo's own `myconfig.*` option tree, for example constants

Mode (when the block is applied). Nothing else is valid:

Mode	Meaning
`ifEnabled`	Only when `myconfig.<name>.enable` is true
`ifDisabled`	Only when it is false (rare)
`always`	Unconditionally

Arguments of a block. `cfg` is the module's own option subtree, `myconfig.<name>` (so `cfg.enable`). `myconfig` is the whole option tree (read constants such as `myconfig.constants.theme`). Two more arguments exist, `name` and `parent`, but they are rarely needed. The block can be a plain attribute set or a function `{ cfg, myconfig, ... }: { ... }`; take only the arguments you use. `pkgs`, `lib` and `config` also work. `pkgs` is not added to the delib function arguments on purpose: add `pkgs` to the argument set when you need it.

Why `options` exists for `ifEnabled` and `ifDisabled`. delib decides "enabled" by reading `cfg.enable`. If the module declares no `enable` option, `ifEnabled` and `ifDisabled` blocks are silently never applied (no error). So for `ifEnabled` the enable option is required; the shortest form is `options = delib.singleEnableOption <default>;`. For `always` it is not needed. A host switches a module on with `myconfig.programs.bat.enable = true;`.

Examples of the combinations.

- `home.ifEnabled`: a program setting that exists only when the module is switched on, via home-manager, on both NixOS and macOS.
- `nixos.always`: system configuration that must always apply on NixOS with no switch (it decides internally with `lib.mkIf`).
- Others: `darwin.ifEnabled`, `home.always`. One module can contain several blocks, for example `nixos.ifEnabled` and `home.ifEnabled`.

Snippets

Source: `my_snippets/nix.snippets` (12 snippets). Type the trigger in insert mode in a Nix buffer and expand it with `<Ctrl-j>` (section 15); `<Ctrl-j>` / `<Ctrl-k>` jump to the next / previous placeholder. The text in quotes after each trigger below is its description exactly as the completion menu shows it. Every snippet is a "start of line" snippet (UltiSnips option `b`): it expands only at the beginning of a line. Placeholders are shown in tab-stop order; after the last one the cursor leaves the block (`$0`, or the end of the text). A compact trigger list is in `04-completion-snippets.md` ("Other snippets").

Two things apply to all of them:

- The snippets indent with tabs. The repo's Nix files use 2 spaces; the formatter (`<Space>fm`, or saving with `:w`) reindents them.
- Where a snippet uses `pkgs`, it must be in scope (a function argument such as `{ pkgs, ... }:`, or a `let` binding).

Group: Package lists

`homepackages`: "home-manager package list (home.packages; pkgs must be in scope)". The cursor ends inside the list.

```
home.packages = with pkgs; [
    package_names
];
```

`systempackages`: "NixOS system package list (environment.systemPackages; pkgs must be in scope)". The same for the system.

```
environment.systemPackages = with pkgs; [
    package_names
];
```

To remove an arbitrary package from `systemPackages` there is no option: just do not add it. home-manager has no `excludePackages` option.

`excludepackages`: "Exclude default desktop packages on NixOS (desktop: gnome|plasma6|cosmic|cinnamon|budgie|enlightenment|lxqt|mate|pantheon|xfce)". The placeholder `desktop` is the name of the desktop environment.

```
environment.desktop.excludePackages = with pkgs; [
    package_names
];
```

Example: `desktop` = `gnome` gives `environment.gnome.excludePackages = with pkgs; [ ... ];`. The valid desktops are `gnome`, `plasma6`, `cosmic`, `cinnamon`, `budgie`, `enlightenment`, `lxqt`, `mate`, `pantheon` and `xfce`. `services.xserver.excludePackages` is the odd one out and is not covered by the snippet.

Group: delib modules

Read "What delib is" above first. Both snippets produce a complete module and leave a valid file (the closing `};` and `}` are included). The first line `{ delib, ... }:` is the argument set of the file.

`delibmodule`: "delib module with an enable switch (target: nixos|darwin|home|myconfig; mode: ifEnabled|ifDisabled)". A module that has an `enable` option, so an `ifEnabled` block works. Placeholders: `module_path` (the dotted `name`, for example `programs.bat`), `default_enabled` (`true` or `false`), `target` (`nixos`, `darwin`, `home` or `myconfig`), `mode` (`ifEnabled` or `ifDisabled`), then the body.

```
{ delib
, ...
}:
delib.module {
    name = "module_path";
```

```
    options = delib.singleEnableOption default_enabled;

    target.mode =
        { cfg
        , myconfig
        , ...
        }:
        {
            body
        };
}
```

Example: `programs.bat`, `false`, `home`, `ifEnabled` gives `name = "programs.bat";`, `options = delib.singleEnableOption false;` and `home.ifEnabled = ...`. A host then enables it with `myconfig.programs.bat.enable = true;`. The body is the attribute set of home-manager options, for example `programs.bat.enable = true;`.

`delibalways`: "delib module applied unconditionally, no enable switch (target: nixos|darwin|home|myconfig)". A module with no `enable` option; its block is always applied and the mode is fixed to `always`. Placeholders: `module_path`, `target`, then the body. Only `myconfig` is in the argument set; add `cfg`, `pkgs` or `lib` when you need them.

```
{ delib
, ...
}:
delib.module {
    name = "module_path";

    target.always =
        { myconfig
        , ...
        }:
        {
            body
        };
}
```

Example: `target` = `nixos` gives `nixos.always = ...`. Because there is no switch, decide inside the body with `lib.mkIf` when the config depends on a condition.

Group: Language building blocks

`let`: "let ... in block". The body of the `let` first, then the expression after `in`.

```
let
    bindings
in
    expression
```

`mkshell`: "mkShell development environment (packages = tools, shellHook = commands run on entry)". Needs `pkgs` in scope. `packages` is preferred over `buildInputs` for dev-shell tools.

```
pkgs.mkShell {
    packages = with pkgs; [
        tools
    ];
```

```
    shellHook = ''
        commands
    '';
}
```

`mkderiv`: "stdenv.mkDerivation (buildInputs = libraries, nativeBuildInputs = build tools)". Needs `pkgs` in scope. `buildInputs` are libraries, `nativeBuildInputs` are build tools. The install phase is wrapped in `runHook preInstall` and `runHook postInstall`; the last placeholder is between them.

```
pkgs.stdenv.mkDerivation {
    pname = "name";
    version = "version";

    src = source;

    buildInputs = with pkgs; [
        libraries
    ];

    nativeBuildInputs = with pkgs; [
        build_tools
    ];

    installPhase = ''
        runHook preInstall
        install_commands
        runHook postInstall
    '';
}
```

Example: `name` = `hello`, `version` = `1.0`, `source` = `./.`.

`fetchgithub`: "fetchFromGitHub boilerplate (hash: use pkgs.lib.fakeHash first, then paste the hash from the error)". Needs `pkgs` in scope. The hash is **unquoted** `pkgs.lib.fakeHash` on purpose: a quoted hash would be a literal string and fail with "invalid SRI hash". Build once, then copy the real hash from the error message (`got: sha256-...`) and replace `pkgs.lib.fakeHash` with it as a quoted string (an empty `""` also works for the first build).

```
src = pkgs.fetchFromGitHub {
    owner = "owner";
    repo = "repo";
    rev = "commit_or_tag";
    hash = pkgs.lib.fakeHash;
};
```

Example: `owner` = `octocat`, `repo` = `hello-world`, `rev` = `v1.0`.

Group: Files and flakes

`flake`: "Basic flake.nix boilerplate (system: x86_64-linux|aarch64-linux|x86_64-darwin|aarch64-darwin)". Placeholders: `description`, `channel` (a nixpkgs branch such as `nixos-unstable`, or a stable `nixos-YY.MM`), `system` (one of `x86_64-linux`, `aarch64-linux`, `x86_64-darwin`, `aarch64-darwin`). The snippet source writes `\${system}`; the backslash is intentional and the expansion contains a plain `${system}`.

```
{
    description = "description";

    inputs = {
        nixpkgs.url = "github:nixos/nixpkgs?ref=channel";
    };

    outputs = { self, nixpkgs, ... }@inputs:
    let
        system = "system";
        pkgs = nixpkgs.legacyPackages.${system};
    in
    {
        outputs
    };
}
```

Example: `channel` = `nixos-unstable`, `system` = `x86_64-linux`.

`homefile`: "home.file that generates a text file, path relative to $HOME (a literal ${ inside '' '' must be written ''${)". A home-manager file with literal text. The path is relative to `$HOME`; under `~/.config` `xdg.configFile."..."` is the better choice. Inside the indented string a literal `${` must be written `''${`, because `${` otherwise starts an interpolation.

```
home.file."file_path" = {
    text = ''
        content
    '';
};
```

Example: `file_path` = `.config/foo/foo.conf`.

Group: NixOS services

`systemd`: "NixOS systemd service (restart_policy: no|on-success|on-failure|on-abnormal|on-watchdog|on-abort|always)". Placeholders: `service_name`, `description`, `command` (becomes `ExecStart`), `restart_policy`, `user`. The valid restart policies are `no`, `on-success`, `on-failure`, `on-abnormal`, `on-watchdog`, `on-abort` and `always`. The cursor ends after the block.

```
systemd.services.service_name = {
    description = "description";
    wantedBy = [ "multi-user.target" ];
    after = [ "network.target" ];

    serviceConfig = {
        ExecStart = "command";
        Restart = "restart_policy";
        User = "user";
    };
};
```

Example: `service_name` = `backup`, `command` = `/run/current-system/sw/bin/true`, `restart_policy` = `on-failure`, `user` = `root`.

This is NixOS only. Inside a delib module put it in a `nixos.*` block. home-manager user services have a different shape and there is no snippet for them:

```
systemd.user.services.name = {
    Unit = { Description = "..."; };
    Service = { ExecStart = "..."; };
    Install.WantedBy = [ "default.target" ];
};
```

Related sections

Section 15 and 52 (snippets), 3 (word references, in the navigation chapter), 41 (filetype settings), 43 (toolchain and devShells), 44 (language server in depth), 78 (Java chapter, section 9 has the same snippet layout).